

Hybrid AC/DC Module Technical Notebook

Current C++ Architecture, Information Models, Mathematical Models, and Solver Workflows

HACDCPF C++ Module

May 28, 2026

Contents

1	Scope and Documentation Contract	13
1.1	Implemented Runtime Families	13
1.2	Current Boundaries and Removed Legacy APIs	14
1.3	Documentation Principle	14
1.4	Canonical Modeling Pipeline	14
1.5	Important Boundary Between “Implemented” and “Declared”	15
2	Nomenclature	15
2.1	Notation Policy	15
2.2	Index and Qualifier Conventions	16
2.3	Canonical Naming Grammar	16
2.4	Superscript/Subscript Rule	16
2.5	Integration Status Codes	17
3	Current C++ Architecture	17
3.1	Source Layout	17
3.2	Top-Level Public API	18
3.3	Build-Time Optional Features	19
3.4	Algorithm Index	19
4	Engine Solver System	20
4.1	Forwarded Engine Surface	20
4.2	Local Newton Solver Class	21
4.3	OPF Use of Engine Facilities	21
5	Native NLP and Interior-Point Details	22
5.1	Native AC OPF Loop	22
5.2	Parity IPM Loop	22
6	Native IPM Versus Parity IPM	22
7	Symbols, Indexing, and Sign Convention	23
7.1	Electrical Bases	23
7.2	Sets and Indexing	23
7.3	Core AC and DC State Variables	24
7.4	Unified Sign Convention: Bus Injection Positive	24
7.5	Power-Balance Template	25

8	Information Models and Field Dictionaries	25
8.1	Modeling Layers Used by the Code	25
8.2	System Containers and Solver-Level Metadata	26
8.3	Component Inventory	28
8.4	AC Network Elements	31
8.5	Load Models and Demand-Side Components	33
8.6	Generation, Storage, and Aggregation Components	33
8.7	DC and Power-Electronic Components	35
8.8	EnergyRouter Multi-Port Model	36
8.9	VirtualPowerPlant Fleet Aggregation Model	37
8.10	Optional Three-Phase Models	38
9	I/O Contracts, Sheet Dictionaries, and JSON Keys	39
9.1	Serialization Families	39
9.2	Global Parsing and Workbook Rules	40
9.3	Hybrid System JSON Contract	41
9.4	Excel Input Workbook Contract	48
9.5	Compact Power-Flow Workbook Additions	56
9.6	Dedicated Result Workbooks	57
9.7	Current I/O Coverage Gaps That Matter to Engineering Work	61
9.8	Interpretation	62
10	Detailed I/O Field Dictionaries	62
10.1	System Metadata	62
10.2	AC Network and Demand Models	63
10.3	Generation and Storage Models	70
10.4	DC and Hybrid Coordination Models	74
10.5	Cross-Reference	81
11	Output and Report Field Dictionaries	81
11.1	Export Families	81
11.2	Result Semantic Conventions	82
11.3	Solver-Specific Result Notes	85
11.4	Consumer Contracts	87
11.5	Power-Flow Result JSON	89
11.6	Dedicated Power-Flow Result Sheets	90
11.7	Compact Power-Flow Result Columns	96
11.8	Dedicated OPF Sheets	96
11.9	Carbon Result JSON and Workbook Dictionaries	98
11.10	Current Output-Side Gaps	103
12	Power-Flow Models	104
12.1	Scope and Architecture Overview	104
12.2	Mathematical Problem Form	106
12.3	VSC Converter Model	107
12.4	DCDC Converter Model	107
12.5	Jacobian Structure	108
12.6	Hybrid Newton Algorithm	109
12.7	Fast-Decoupled Power Flow (FDPF)	110
12.8	DC-Only Power Flow	111
12.9	Adaptive Islanded Solver	111
12.10	HELM Solver	112

12.11	Error Handling and Diagnostics	112
12.12	Maintainability Roadmap: Code-Duplication Inventory	112
12.13	Numerical Performance (Release Mode)	113
13	Advanced and Fallback Power-Flow Solvers	114
13.1	Continuation Power Flow (CPF) / Voltage Stability	114
13.1.1	Parameterized Power System	114
13.1.2	Predictor–Corrector Continuation	115
13.1.3	CpfOptions Reference	115
13.1.4	CPF Result Structure	116
13.1.5	Algorithm A13: CPF Predictor–Corrector (condensed)	116
13.2	Homotopy Continuation Solver	116
13.2.1	Homotopy Path	116
13.2.2	Step Adaptation	116
13.2.3	Algorithm A14: Homotopy Continuation (condensed)	117
13.3	Distributed Slack Solver	117
13.3.1	Problem Formulation	117
13.3.2	Convergence Criterion	117
13.3.3	Result Structure	117
13.3.4	Algorithm A7: Distributed-Slack PF (condensed)	118
13.4	Newton–Krylov GMRES Fallback (Phase 5)	118
13.4.1	Algorithm A15: Newton–Krylov GMRES (condensed)	118
13.5	Complete RobustNonlinearOptions Reference	118
13.6	PowerFlowOptions Complete Reference	120
14	Optimal Power Flow Models	121
14.1	Problem Form and Runtime Scope	121
14.2	Code Map	121
14.3	Pseudocode Map	122
14.4	Current Status	122
14.5	Top-Level Solver Routing	122
14.6	Fast Dispatch + PF Fallback (AC-Only Path)	122
14.7	Native Primal-Dual OPF Formulation	123
14.7.1	Decision Variables	123
14.7.2	Objective	123
14.7.3	Equality Constraints	123
14.7.4	Bounds and Nonlinear Limits	123
14.7.5	KKT System and Globalization	124
14.8	Parity Full-Space Formulation	124
14.8.1	Decision Variables	124
14.8.2	Objective	124
14.8.3	Equality Constraints	124
14.8.4	Inequality Constraints	125
14.8.5	Primal-Dual IPM System	125
14.9	Component Coverage by Solver Path	126
14.10	AC OPF Result JSON Contract	126
14.11	Fallback Behavior	127
14.12	DC Optimal Power Flow (solve_dc_opf)	127
14.12.1	Decision Variables	127
14.12.2	Network Model	127
14.12.3	Objective Function	127
14.12.4	Constraints	128

14.12.5	Result Audit Fields	128
14.12.6	Solver Backend Selection	129
14.12.7	Locational Marginal Prices	129
14.13	Reactive Power Optimization (<code>solve_rpo</code>)	129
14.13.1	Problem Structure	129
14.13.2	Objective Functions	130
14.13.3	Algorithm: Four-Phase Outer Approximation	130
14.13.4	Inner NLP Solver	130
14.13.5	Result Structure	130
15	Power-Model Builders (AML Layer)	130
15.1	AML Abstraction Layer	131
15.2	LinDistFlow Builder	131
15.2.1	Mathematical Formulation	131
15.2.2	Input/Output Structs	132
15.3	AC OPF Builder	132
15.3.1	Mathematical Formulation	132
15.3.2	Input Structs	133
15.4	Hybrid AC/DC OPF Builder	133
15.4.1	Mathematical Formulation	133
15.4.2	Additional Input Structs	134
15.5	Security-Constrained Unit Commitment (SCUC) Builder	134
15.5.1	Mathematical Formulation	134
15.5.2	Input/Output Structs	134
16	Optimal Network Reconfiguration	135
16.1	Problem Form and Runtime Scope	135
16.2	Code Map	135
16.3	Pseudocode Map	135
16.4	Current Status	136
16.5	Scope of the Current ONR Module	136
16.6	Decision Variables	136
16.7	Net-Injection Model	137
16.8	LinDistFlow MILP	137
16.9	Branch-and-Cut Configuration	138
16.10	Fault-Triggered Reconfiguration Path	138
16.11	Postprocessing and Verification	138
16.12	External Solver Integration (HiGHS)	139
16.13	Current Modeling Boundary	139
17	I/O Layer: Parsers, Case Builders, and JSON Contracts	139
17.1	MATPOWER Parser	139
17.2	JSON I/O	140
17.3	Pre-built Test Networks (<code>case_builders.hpp</code>)	141
18	Test Suite	141
18.1	Test File Overview	141
18.2	Example Test Pattern	143

19 Architecture Assessment and Roadmap	143
19.1 Overall Quality Assessment	143
19.2 Confirmed Strengths	144
19.2.1 Horizontal Infrastructure Layer	144
19.2.2 Monadic Result Handling	144
19.2.3 Backward-Compatible Option Refactoring	144
19.2.4 Explicit Validation Subsystem	145
19.2.5 Projection plus Assembly Separation	145
19.2.6 Centralised Defaults	145
19.3 Remaining Design Risks	145
19.3.1 Analysis Layer May Become Too Broad	145
19.3.2 Three-Phase and Balanced PF Should Not Share One Abstraction	146
19.3.3 Error Handling Policy Needs Explicit Definition	146
19.3.4 Field Dictionary / Serializer Drift	146
19.3.5 Backend Capability Discovery	147
19.4 Module-Level Recommendations	147
19.4.1 Model Layer	147
19.4.2 Validation Layer	147
19.4.3 Projection Layer	148
19.4.4 Assembly Layer	148
19.4.5 Power-Flow Layer	148
19.4.6 OPF Layer	149
19.4.7 I/O Layer	149
19.4.8 Analysis Suite	150
19.5 Recommended Testing Matrix	150
19.6 Recommended CI Pipeline	151
19.7 Dependency Boundary Rules	151
19.7.1 Allowed Dependencies	151
19.7.2 Disallowed Dependencies	151
19.8 Priority Roadmap	151
19.9 Example Clean Public Workflow	152
19.10 Final Recommendation Summary	153
20 Complete Data Model: Enumerations, Defaults, and Error Types	154
20.1 Enumeration Hierarchy	154
20.1.1 Bus Type Enumerations	154
20.1.2 Generator Enumerations	155
20.1.3 Converter and Loss Model Enumerations	155
20.1.4 Load, Storage, and Renewable Enumerations	155
20.1.5 Switching and Grid Enumerations	156
20.2 EnumStrings Utility	156
20.3 Defaults: System-Wide Numerical Constants	157
20.4 Structured Error and Result Types	157
20.4.1 <code>ErrorCode</code> Enumeration	158
20.4.2 <code>Error</code> Descriptor	158
20.4.3 <code>Result<T></code> Monad	158
20.5 Validation Subsystem	159
20.5.1 <code>ValidationReport</code> and <code>ValidationIssue</code>	159
20.5.2 Validation Checks	159

21 Solver Infrastructure: Assembly, Projection, Backend, and Utilities	160
21.1 Assembly Layer	160
21.1.1 SolverData	160
21.1.2 Index Maps	161
21.1.3 Admittance (Y-bus) Builder	162
21.2 Projection Layer	162
21.2.1 Canonical Network Types	162
21.2.2 Projection Functions	163
21.3 Backend Solver Abstractions	164
21.3.1 Linear Solver Backend	164
21.3.2 NLP Solver Backend	164
21.3.3 QP Solver Backend	165
21.4 Thread Pool	165
21.5 Detail Utilities	166
22 Power-Flow Options, Results, and NCP Complementarity	166
22.1 PowerFlowOptions Sub-Structs	166
22.1.1 ConvergenceOptions	166
22.1.2 LinearSolverOptions	166
22.1.3 GlobalizationOptions	167
22.1.4 PVPQSwitchingOptions	167
22.1.5 ConverterModeOptions	167
22.1.6 ZipLoadOptions	167
22.1.7 RuntimeOptions	167
22.2 Power-Flow Result Hierarchy	168
22.2.1 ResidualBreakdown	168
22.2.2 IterationLogEntry	168
22.2.3 SolverDiagnostics	168
22.2.4 SolverProfiling	169
22.2.5 Component Flow Result Types	169
22.2.6 Top-Level Result Types	169
22.3 Fischer-Burmeister NCP Functions	170
22.3.1 Standard Fischer-Burmeister Function	170
22.3.2 PV/PQ Complementarity Equation	170
22.3.3 Smooth (μ -Perturbed) Fischer-Burmeister	171
23 Analysis Suite: Carbon, Production, Lifecycle, Distribution, Reliability, and Time Series	171
23.1 Carbon Emissions Analysis	171
23.1.1 Mathematical Formulation	172
23.1.2 Options and Result Types	172
23.1.3 EmissionsSummary	173
23.1.4 Loss Allocation and Directed Flow Edges	173
23.1.5 Storage State-of-Charge Carbon Intensity	174
23.2 Annual Carbon Analysis	174
23.2.1 Multi-Step Aggregation	174
23.2.2 Storage Carbon State Propagation	175
23.2.3 Green Energy Certificate (GEC) Accounting	175
23.3 Distribution Network Power Flow	176
23.3.1 Three-Phase Newton-Raphson	176
23.3.2 Three-Phase NR Options and Results	176
23.4 Three-Phase Unbalanced Power Flow	177

23.5	PV Array Power Curve Model	177
23.6	Reliability Assessment	177
23.6.1	Options: ReliabilityOptions	178
23.6.2	Results: ReliabilityResult	178
23.6.3	Advanced Result Structs	179
23.6.4	FMEA Options and Results	179
23.6.5	Mathematical Formulation	180
23.6.6	Reference Benchmarks	182
23.7	Resilience Assessment	182
23.7.1	Mathematical Model	182
23.7.2	Algorithm	184
23.7.3	Algorithm Flow Diagram	185
23.7.4	Option and Result Tables	185
23.7.5	Strict Multi-Period MIP Path	186
23.7.6	Numerical Verification	186
23.8	Time-Series Power Flow	187
23.8.1	DC Network Model for Unit Commitment	187
23.8.2	Unit Commitment MILP Formulation	188
23.8.3	Time-Series Data Model	189
23.8.4	Unit Commitment Schedule	189
23.8.5	OPF/PF Cross-Validation	189
23.8.6	Pipeline Options and Result	189
23.8.7	Public API	190
23.9	Annual Production Simulation	190
23.9.1	Mathematical Formulation	190
23.9.2	Decomposition Data Types	191
23.9.3	Per-Step and Per-Block Results	191
23.9.4	Component Annual Statistics	192
23.9.5	Annual Production Simulation Options	192
23.9.6	Annual Production Simulation Result	192
23.10	Lifecycle Simulation	192
23.10.1	Mathematical Formulation	193
23.10.2	Stratified Carbon Attribution (Tier 2)	193
23.10.3	Theoretical Error Bounds (Tier 3)	194
23.10.4	Lifecycle Simulation Options	195
23.10.5	Lifecycle Result Types	195
23.10.6	Capacity Scaling and Comparison	196
23.11	Short-Circuit Analysis (IEC 60909)	196
23.11.1	Voltage Factor	196
23.11.2	Sequence Admittance Matrices	196
23.11.3	Thévenin Impedance and Initial Symmetric Current	197
23.11.4	Peak Current and Kappa Factor	198
23.11.5	Breaking Current	198
23.11.6	Thermal Equivalent Current	198
23.11.7	Voltage at Non-Faulted Buses	198
23.11.8	Options and Result Types	199
23.12	Three-Phase Newton-Raphson Power Flow	199
23.12.1	Mathematical Model	199
23.12.2	Algorithm	201
23.12.3	Flowchart	202
23.12.4	Solver Options and Result Fields	203

23.12.5 Numerical Verification	203
24 Graph Analysis and Reduction Module	203
24.1 Design Motivation	204
24.2 Module Layout	204
24.3 Core Graph Data Structure	205
24.3.1 GraphNode	205
24.3.2 GraphEdge	206
24.3.3 PowerSystemGraph	206
24.4 Topology Analysis	207
24.4.1 Overview	207
24.4.2 TopologyReport Fields	207
24.4.3 Island Validity	207
24.4.4 Diagnostic Codes	208
24.4.5 Algorithms	208
24.5 Reduction Pipeline	208
24.5.1 Reduction Options and Mode	208
24.5.2 Candidate Classification and Reduction Plan	209
24.6 Switch Contraction	209
24.6.1 Rationale	209
24.6.2 Algorithm	210
24.7 Series Reduction	210
24.7.1 Theory	210
24.7.2 Implementation	210
24.8 Pendant Reduction	211
24.8.1 Theory	211
24.9 Kron Reduction (Schur Complement)	211
24.9.1 Theory	211
24.9.2 Fill-In Guard	211
24.9.3 Invertibility Check	212
24.9.4 API	212
24.10 Reduction Mapping (Audit Trail)	212
24.11 Result Recovery	212
24.11.1 Switch Contraction Recovery	213
24.11.2 Series Reduction Recovery	213
24.11.3 Kron Reduction Recovery	213
24.11.4 Pendant Reduction Recovery	213
24.12 Test Coverage	213
24.13 Integration with Downstream Solvers	214
24.14 Numerical Results: Graph-Aware ONR and Reliability	215
24.14.1 ONR Bridge Detection — IEEE 33-bus BW Feeder	215
24.14.2 Reliability Island Pre-Screening — Radial Feeder Contingency	215
25 Three-Stage Fault-Recovery Reliability Analysis	216
25.1 Three Operational Stages	216
25.2 Native Case Construction	217
25.3 Implemented AC Restoration MILP	217
25.4 Hybrid Fallback and Validity Flags	219
25.5 Reliability Index Calculation	219
25.6 C++ API and Result Fields	219
25.7 Regression Test Systems and Observed Results	220

26	Implementation Gap and Stub Audit	221
26.1	Audit Method	221
26.2	High-Risk Items Closed	222
26.3	Other Resolved or Explicitly Bounded Findings	222
26.4	Closed Model-Scope Contracts	224
26.5	Verification Snapshot	225
26.6	Audit Conclusion	225
27	EV Power–Traffic Coupled Simulation: Implemented Model and Solver-Certified Extension	225
27.1	Time Grid, Sets, and Notation	227
27.1.1	Single Time Grid	227
27.1.2	Sets	227
27.1.3	Key Scalar Parameters	227
27.2	Traffic Flow Model	227
27.2.1	Link Capacity and Travel Time (BPR Function)	227
27.2.2	Route Travel Time	228
27.2.3	Generalized Route Cost	228
27.2.4	Implemented SOC Feasibility Filter	228
27.3	Assignment Models	228
27.3.1	Deterministic Shortest Path (<code>DeterministicShortestPath</code>)	228
27.3.2	Logit (<code>Logit</code>)	229
27.3.3	Capacity-Aware Greedy (<code>CapacityAwareGreedy</code>)	229
27.3.4	System-Optimal LP / MILP	229
27.4	System-Optimal LP Formulation	229
27.4.1	Assumptions (Formulation A)	229
27.4.2	Decision Variables	230
27.4.3	Objective	230
27.4.4	Demand Balance	230
27.4.5	Road Capacity	230
27.4.6	Station Charging Capacity	230
27.4.7	Generation Capacity (Optional)	230
27.4.8	MILP Variant	231
27.5	Charging Session Synthesis	231
27.5.1	Smart Charging LP	231
27.6	Station Queue and Occupancy Model	231
27.6.1	Queue Dynamics	232
27.6.2	Service Completion	232
27.6.3	Waiting Time Approximation	232
27.7	Power-System Coupling	232
27.7.1	Station Load Write-Back	232
27.7.2	Optional Power-Flow Validation	232
27.8	Joint Social Welfare Algorithm	233
27.8.1	Sub-Problems	233
27.8.2	Price Update	233
27.8.3	Convergence Criterion	233
27.8.4	Social Welfare Accounting	234
27.8.5	Algorithm Summary	234
27.9	Formulation D: Solver-Certified Dynamic Joint Optimisation	234
27.9.1	Additional Sets and Parameters	234
27.9.2	Decision Variables	235

27.9.3	Dynamic Travel Demand and Departure-Time Choice	235
27.9.4	Blockage-Aware CTM Constraints	236
27.9.5	EV Battery, Charging, and V2G	236
27.9.6	Power-Network Constraints	237
27.9.7	Objective 1: User-Benefit Maximisation	237
27.9.8	Objective 2: Social-Welfare Maximisation	238
27.9.9	Solver-Certified User Equilibrium	238
27.9.10	Problem Class and Solver Selection	239
27.9.11	Solver Algorithm	239
27.10	Implemented Module Structure	240
27.11	Numerical Case Study 1: Two-Route, Five-Vehicle Network	240
27.11.1	Network Parameters	240
27.11.2	Scenario A: Deterministic Shortest Path	240
27.11.3	Scenario B: Capacity-Aware Greedy	241
27.11.4	Scenario C: System-Optimal LP	241
27.12	Numerical Case Study 2: Joint Social Welfare, 2-Bus Grid	241
27.12.1	Network Parameters	241
27.12.2	Scenario 1: Fixed Prices (No OPF Coupling)	242
27.12.3	Scenario 2: LMP-Coupled Joint Optimisation	242
27.13	Numerical Case Study 3: Large-Scale Fleet Routing	243
27.13.1	Setup	243
27.13.2	Scenario A: CapacityAwareGreedy	243
27.13.3	Scenario B: System-Optimal LP	244
27.14	Formulation B: CTM Propagation and Dynamic User Equilibrium	244
27.14.1	Cell Transmission Model	244
27.14.2	Triangular Fundamental Diagram	244
27.14.3	Sending and Receiving Functions	245
27.14.4	Cell Update	245
27.14.5	Commodity-Specific Cell Occupancy	245
27.14.6	Node Model	245
27.14.7	Experienced Link Travel Time	246
27.14.8	Method of Successive Averages (MSA) for DUE	246
27.14.9	Wardrop Relative Gap	246
27.14.10	CTM Time Step vs. Simulation Time Step	247
27.14.11	Formulation B+ Extensions: Session Synthesis, Dispatch, and PoA	247
27.15	Numerical Case Study 4: CTM-DUE Convergence	248
27.15.1	Network	248
27.15.2	Convergence Results	248
27.16	Numerical Case Study 5: Formulation B+ — Session Dispatch, SOC Filtering, and Price-of-Anarchy	248
27.16.1	Scenario B1: Session Synthesis and Smart-Charging Dispatch	248
27.16.2	Scenario B2: SOC Feasibility Filter	249
27.16.3	Scenario B3: Price-of-Anarchy Benchmark	250
27.16.4	Summary of Case Study 5 (Test 8–10 Regression)	250
27.17	Numerical Case Study 6: Formulation C — CTM-DUE + DC-OPF Price Coordination	250
27.17.1	Network	250
27.17.2	Scenario C1: Fixed Prices (No OPF Coupling)	251
27.17.3	Scenario C2: OPF-Coupled Price Feedback	251
27.17.4	Scenario C3: Price-Driven Route Choice	252
27.17.5	Summary of Case Study 6 (Tests C1–C4)	252

27.18	Numerical Case Study 7: Formulation D — Single LP/MILP Assembly	252
27.18.1	Test Network	252
27.18.2	Expected Optimal Solution	253
27.18.3	Test D1: SocialWelfareMax LP — No OPF	253
27.18.4	Test D2: SocialWelfareMax LP + DC-OPF	254
27.18.5	Test D3: UserBenefitMax LP	254
27.18.6	Test D4: Social Welfare $W_D \geq W_C$	254
27.18.7	Test D5: MILP Integer-Flow Mode	254
27.18.8	Test D6: Price Asymmetry Route Shift	254
27.18.9	Summary of Case Study 7 (Tests D1–D7)	254
28	Algorithm Index: Pseudo-Code and Flowcharts	255
28.1	Model Utilities (A1–A2)	255
28.1.1	A1: Rich-Model Projection (<code>project_to_canonical</code>)	255
28.1.2	A2: <code>SolverData</code> Construction	256
28.2	Power-Flow Solvers (A3–A7)	257
28.2.1	A3: Hybrid AC/DC Newton Power Flow (<code>NewtonSolver</code>)	257
28.2.2	A4: Fast-Decoupled Power Flow (<code>FDPFSolver</code>)	258
28.2.3	A5: DC-Only Power Flow (<code>DCSolver</code>)	259
28.2.4	A6: Island-Aware Adaptive Solver (<code>AdaptiveSolver</code>)	260
28.2.5	A7: Distributed-Slack Power Flow	261
28.3	Optimal Power Flow Solvers (A8–A12)	262
28.3.1	A8: AC OPF Top-Level Routing (<code>solve_ac_opf</code>)	262
28.3.2	A9: Parity Full-Space NLP Construction	263
28.3.3	A10: Parity Primal-Dual IPM Solve (<code>parity_ipm</code>)	264
28.3.4	A11: DC Optimal Power Flow (<code>solve_dc_opf</code>)	265
28.3.5	A12: Reactive-Power Optimization (<code>solve_rpo</code>)	266
28.4	Advanced and Fallback Solvers (A13–A15)	267
28.4.1	A13: Continuation Power Flow, Predictor–Corrector (<code>CpfSolver</code>)	267
28.4.2	A14: Homotopy Continuation Fallback (<code>HomotopyContinuationSolver</code>)	268
28.4.3	A15: Newton–Krylov GMRES Fallback (<code>NewtonKrylovSolver</code>)	269
28.5	Power Model Builders (A16–A19)	271
28.5.1	A16: LinDistFlow LP Builder (<code>lindistflow_builder</code>)	271
28.5.2	A17: AML AC OPF Model Builder (<code>acopf_builder</code>)	272
28.5.3	A18: AML Hybrid AC/DC OPF Model Builder (<code>acdcopf_builder</code>)	273
28.5.4	A19: MILP SCUC Builder (<code>scuc_builder</code>)	274
28.6	Network Reconfiguration (A22)	275
28.6.1	A22: Optimal Network Reconfiguration Orchestration (<code>solve_onr</code>)	275
28.7	Time-Series Simulation (A20–A21)	277
28.7.1	A20: Annual Production Simulation (Hierarchical)	277
28.7.2	A21: Multi-Year Lifecycle Simulation	278
28.8	Short-Circuit Analysis (A23)	279
28.8.1	A23: Short-Circuit Z-bus Solver (<code>compute_short_circuit</code>)	279
29	Test Suite: Design, Coverage, and Results	280
29.1	Overview of the New Test Suites	281
29.2	<code>test_acopf_dcopf_crossval</code> : ACOPF/DCOPF Cross-Validation	281
29.3	<code>test_hacdcdss</code> : Model, Power Models, Power Flow, DC OPF	282
29.3.1	Module: Model (tests #56–#59)	282
29.3.2	Module: Power Models (tests #60–#62)	282
29.3.3	Module: Power Flow (tests #63–#68)	282
29.3.4	Module: DC OPF (tests #69–#72)	283

29.4	test_io_json: JSON I/O and MATPOWER Parsing	283
29.4.1	JSON Round-trip Fidelity (tests #73–#76)	283
29.4.2	Safe Monadic API (tests #77–#84)	283
29.4.3	OPF/PF Result Serialisation (tests #85–#86)	283
29.4.4	MATPOWER Parser (tests #87–#106)	283
29.4.5	JPC JSON Compatibility (tests #107–#108)	284
29.5	test_rich_components: 24-Type Component Coverage	284
29.5.1	Component Round-trip Tests (tests #109–#129)	284
29.5.2	System Builder Tests (tests #130–#133)	284
29.6	test_matpower_cases: MATPOWER Convergence Coverage	285
29.6.1	Tier-1: Core Cases (tests #134–#146)	285
29.6.2	Tier-2: Medium Cases (tests #147–#216)	286
29.6.3	Tier-3: Large Cases (tests #172–#180 and #211–#215, large/very_large tag)	287
29.6.4	Safe API and DC OPF (tests #217–#220)	287
29.7	test_validation: Validation API Coverage	288
29.7.1	ValidationReport API (tests #221–#224)	288
29.7.2	Positive and Negative Validation (tests #225–#234)	288
29.7.3	ValidationLevel Behaviour (tests #235–#241)	289
29.7.4	Validate-then-Solve and Converter Guardrails (tests #242–#247)	289
29.7.5	Regression Guardrails (tests #248–#252)	289
29.8	test_solver_capabilities: Compile-time Flags and Audit	289
29.8.1	SolverCapabilities Struct (tests #253–#259)	289
29.8.2	Compile-time Solver Flags (tests #260–#265)	290
29.8.3	verify_opf_result Audit (tests #266–#269)	290
29.9	test_advanced_pf: Island Detection, Distributed Slack, PV/PQ Control	290
29.9.1	Island Detection and Islanded Solving (tests #270–#280)	290
29.9.2	Distributed Slack Buses (tests #281–#290)	290
29.9.3	PV/PQ Conversion and Integration (tests #291–#295)	290
29.10	test_three_phase_pf: Three-Phase Power Flow	291
29.10.1	Data Model (tests #296–#300)	291
29.10.2	Projection-Based Solve (tests #301–#306)	291
29.10.3	Unbalanced, Meshed, and Multi-Source Topologies (tests #307–#313)	291
29.10.4	Transformer Support (test #314)	291
29.10.5	Three-Phase JSON Round-Trip (tests #315–#319)	291
29.11	test_multiport_vpp_er: Multi-Port Energy Router and Virtual Power Plant	291
29.11.1	3-Port Energy Router (tests #320–#323)	292
29.11.2	VF Setpoint Observability (test #323)	292
29.11.3	4-Port Energy Router (tests #324–#327)	292
29.11.4	Multi-Mode VPP (tests #328–#333)	293
29.12	test_topology_crossval: Network Reconfiguration	293
29.12.1	Fault-Reconfiguration Regressions	293
29.12.2	Connectivity Primitives	293
29.12.3	6-Bus Exhaustive vs. MILP Cross-Validation	294
29.12.4	6-Bus Network with DERs	294
29.12.5	33-Bus Benchmark (case33bw)	294
29.13	Quantitative Results and Timing	294
29.13.1	Per-Suite Registration Summary	294
29.13.2	Tier-3 Scaling: Bus Count vs. Solve Time	295
29.13.3	Key Accuracy Metrics	295
29.13.4	Performance Sweep (test #334)	296

29.14	test_crossmodule_integration: Cross-Module Pipeline Tests	296
29.14.1	Pipeline 1: OPF → PF → Carbon Analysis (IEEE 14-bus, test #346) . .	296
29.14.2	Pipeline 2: ONR → DC OPF → PF → Carbon Analysis (IEEE 33-bus BW, test #347)	297
29.14.3	Pipeline Test: Cross-Check Summary	298
29.14.4	Observed Numerical Results	299
29.15	test_short_circuit_crossval: Short-Circuit Z-bus Cross-Validation	299
29.15.1	Mathematical Model	299
29.15.2	Cross-Check Table	300
29.15.3	Observed Numerical Results	300
29.16	test_three_phase_nr: Three-Phase Newton–Raphson Solver	301
29.16.1	PhaseNodeIndexer (tests #355–#357)	301
29.16.2	Solver Convergence and Physical Checks (tests #358–#370)	302
29.17	test_resilience_assessment: Distribution Resilience	302
29.17.1	API and Boundary Checks (tests #371–#372)	302
29.17.2	Fault Behaviour and Load Restoration (tests #373–#377)	302
29.17.3	Infrastructure and Regression Tests (tests #378–#393)	302
29.18	test_graph and test_graph_kron: Graph Module	303
29.19	Summary and Pass Rates	304

1 Scope and Documentation Contract

This notebook is the implementation-facing handbook for the current `hacdcpf` C++ module. It connects mathematical models to the public data structures, preprocessing steps, solver paths, optional build features, and result objects that are present in the repository now. It deliberately separates implemented runtime code from declared stubs and absent/planned APIs.

1.1 Implemented Runtime Families

The current notebook covers these implemented module families:

- the public model layer under `include/hacdcpf/model`;
- canonical projection and solver-data assembly in `src/model` and `src/power_flow`;
- hybrid AC/DC power flow through `solve_power_flow`;
- DC-only, FDPF, linearized AC/DC, adaptive/islanded, distributed-slack, HELM, and continuation PF entry points exposed in the power-flow headers;
- AC OPF through `solve_ac_opf`, including native reduced-space and parity full-space IPM routes;
- DC OPF through `solve_dc_opf`;
- reactive-power optimization through `solve_rpo`;
- carbon emissions analysis through `compute_carbon_analysis` (BFS proportional tracing + matrix solve, with optional PTDF weighting and sector-level attribution);
- time-series power flow pipeline through `run_time_series_power_flow` (multi-scenario, multi-horizon profiles with snapshot solve per time step);
- annual production simulation through `run_annual_production_simulation` (hourly PV/wind dispatch, curtailment accounting, energy yield statistics);

- lifecycle simulation through `run_lifecycle_simulation` (multi-year degradation, replacement scheduling, levelised cost computation);
- JSON, MATPOWER/JPC, case-builder, PF result JSON, OPF result JSON, and carbon result JSON support;
- optional Excel workbook I/O when `HACDCPF_ENABLE_EXCEL=ON`;
- optional OpenDSS bridge source inclusion when `HACDCPF_ENABLE_OPENDSS=ON`.

1.2 Current Boundaries and Removed Legacy APIs

Area	Current code status
<code>analysis::solve_distribution_power_flow</code> / legacy single-phase distribution PF	Removed from the public code surface. Distribution-feeder studies now use the explicit three-phase NR interface in <code>include/hacdcpf/power_flow/distribution_power_flow.hpp</code> : <code>solve_three_phase_nr</code> , <code>PhaseNodeIndexer</code> , and phase-domain Y-bus helpers.
Three-stage MILP reliability	Public API and data types defined in <code>include/hacdcpf/analysis/three_stage_reliability.hpp</code> ; <code>src/reliability/three_stage_reliability.cpp</code> is a native C++ evaluator built on the embedded MIPSolvers engine. The implemented physical optimization is the finite-source AC LinDistFlow restoration MILP; hybrid DC/VSC/DCDC/SOP objects are handled through connectivity/capacity fallback and reported through model-scope/validity fields. No external Julia or Gurobi environment is required.

1.3 Documentation Principle

This notebook follows four rules.

1. Equations are written in the same sign convention as the current C++ implementation.
2. Field explanations are tied to concrete structs, not generic textbook entities.
3. Each component is classified by how it enters a module: direct state, fixed injection, projected equivalent, or not yet consumed.
4. Extension points declared in headers are explicitly separated from functionality that is already wired in source files.

1.4 Canonical Modeling Pipeline

The steady-state solvers do not consume every rich component table directly. Instead, the dominant runtime path is:

1. start from a public `HybridPowerSystem`,
2. call `project_to_canonical_models`,

3. build `SolverData` through `make_solver_data`,
4. assemble residuals, Jacobians, or optimization models for PF and OPF.

The projection stage currently performs the following transformations:

- `FlexibleLoad` $\rightarrow$ equivalent `Load`,
- `AsymmetricLoad` $\rightarrow$ equivalent aggregated `Load`,
- `Transformer2W` and `Transformer3W` $\rightarrow$ one or more equivalent `ACBranch` objects,
- `Switch` $\rightarrow$ equivalent `ACBranch` whose `in_service` flag follows the switch state,
- `CircuitBreaker` $\rightarrow$ equivalent `ACBranch`, followed by zero-impedance bus merging when switches or circuit breakers are present,
- `Charger` $\rightarrow$ accumulation into `ChargingStation` demand,
- `VirtualPowerPlant` and `Microgrid` $\rightarrow$ equivalent `AC StaticGenerator` rows,
- `MobileStorage` $\rightarrow$ equivalent `AC Storage` at its current bus,
- `EnergyRouter` $\rightarrow$ internal DC buses plus VSC/DCDC equivalents,
- optional `ThreePhaseACSystem` $\rightarrow$ canonical single-phase-equivalent AC models when the explicit AC system is absent.

1.5 Important Boundary Between “Implemented” and “Declared”

The parity OPF formulation now instantiates **all** enhanced component decision blocks as run-time variables with full Jacobian and Hessian wiring:

- generator dispatch (P_g, Q_g) ,
- VSC converter power $(P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}})$,
- DCDC converter input power P^{dc} with analytical I^2R loss derivatives,
- DC bus voltages v^{dc} ,
- per-bus load shedding $(\Delta P_d, \Delta Q_d)$ with VOLL penalty,
- curtailable renewable dispatch $(P^{\text{ren}}, Q^{\text{ren}})$ for `RenewableGen` with `curtailable=true` and `PVSystem` with `controllable=true`,
- storage dispatch $(P^{\text{stor}}, Q^{\text{stor}})$ for AC storage and $P^{\text{stor,dc}}$ for DC storage,
- flexible load adjustment P^{flex} .

These are no longer extension hooks: they are wired decision variable blocks with explicit mapping arrays in `parity_formulation.cpp`. The OPF JSON serializer preserves the enhanced dispatch arrays as both legacy bare arrays and structured dispatch objects with identity maps.

2 Nomenclature

2.1 Notation Policy

This notebook uses a strict notation policy consistent with the C++ module:

- scalar variables use italic math symbols, for example P_i, Q_i, V_i ,
- sets use calligraphic symbols, for example $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,
- all semantic qualifiers use upright superscripts via $\cdot$, for example $P_i^{\text{spec}}, P_i^{\text{calc}}, V_k^{\text{dc}}$,
- for variable symbols, subscripts are reserved for ordered indices (bus/component/time) and not for semantic class labels,
- set symbols may use compact domain tags in subscript, e.g., $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,
- model-family and semantic words must stay in superscripts: write $P^{\text{dcdc}}, P^{\text{flex}}, P_r^{\text{rated}}, c_r^{\text{curt}}$, and $\mathcal{R}^{\text{curt}}$.

2.2 Index and Qualifier Conventions

Token	Meaning
i, j	AC bus indices
k, m	DC bus indices
c	converter index (VSC / DCDC)
b	branch index
g	generator index
spec	specified (assembled) quantity from component injections/demands
calc	network-calculated quantity from current state
ac, dc	AC-side / DC-side quantity qualifier
conv	converter-side quantity qualifier
dcdc, flex	DCDC-dispatch / flexible-load variable class qualifier
rated, curt	rated-capacity / curtailment-related qualifier
min, max	lower/upper operating bounds

2.3 Canonical Naming Grammar

Use the canonical symbol template

$$X_{\text{indices}}^{\text{qualifiers}}, \quad (1)$$

with the following rules:

- index block ($\cdot$): ordered indices only ($i, j, k, c, r, t, \dots$),
- qualifier block ($\cdot$): semantic tags only (spec, calc, conv, dcdc, flex, rated, curt, min, max, ...),
- when multiple qualifiers are needed, order them as: model/class $\rightarrow$ side or direction $\rightarrow$ status/bound/cost.

Examples:

$$P_i^{\text{conv,ac}}, P_k^{\text{dcdc,in}}, P_r^{\text{rated}}, c_r^{\text{curt}}. \quad (2)$$

2.4 Superscript/Subscript Rule

Rule	Usage
Subscript (.)	for variable symbols: ordered indices only (i, j, k, c, r, t); set symbols may use domain tags ($\mathcal{N}_{ac}, \mathcal{E}_{dc}$).
Superscript (·)	semantic labels: domain, role, operating mode, variable class, bounds, and status qualifiers.
Forbidden mixed style	avoid semantic words in subscript, such as $P_{dcdc}, P_{flex}, \mathcal{R}_{curt}, P_{rated}, c_{curt}$.
Preferred style	$P^{dcdc}, P^{flex}, \mathcal{R}^{curt}, P^{rated}, c_r^{curt}$.

2.5 Integration Status Codes

The integration matrix uses these codes:

- D: direct model consumption in the module,
- F: fixed-only consumption (no explicit variable),
- P: consumed after projection to canonical equivalents,
- N: not consumed by the module.

3 Current C++ Architecture

This section describes the repository as it exists in the current C++ module. The library target is `hacdcpf`, configured from the top-level `CMakeLists.txt` as a C++20 static library. The build links against `mipsolvers::mipsolvers`; that dependency supplies the shared engine types, solver adapters, and numerical backends used by the OPF and RPO code.

3.1 Source Layout

Directory	Current role
<code>include/hacdcpf/model</code>	Public data structures for AC, DC, converter, switching, three-phase, and result models.
<code>include/hacdcpf/api</code>	High-level public API facade in <code>hacdcpf.hpp</code> .
<code>include/hacdcpf/power_flow</code> and <code>src/power_flow</code>	Hybrid Newton PF, DC PF, FDPF, adaptive/islanded/distributed-slack variants, HELM, continuation, Jacobian assembly, PF utilities, and three-phase distribution NR helpers. The legacy single-phase distribution BFS public API has been removed.
<code>include/hacdcpf/optimal_power_flow</code> and <code>src/optimal_power_flow</code>	AC OPF routing, parity formulation, parity IPM, DC OPF, and reactive-power optimization.

Directory	Current role
<code>include/hacdcpf/power_models</code> and <code>src/power_models</code>	Builder-style models for AC OPF, AC/DC OPF, DC OPF, LinDistFlow, and SCUC experiments.
<code>include/hacdcpf/io</code> and <code>src/io</code>	MATPOWER/JSON/JPC case I/O, optional Excel workbook I/O, optional OpenDSS bridge, and case builders.
<code>include/hacdcpf/carbon_analysis</code> and <code>src/carbon_analysis</code>	Carbon emissions tracing (<code>run_carbon_analysis</code>).
<code>include/hacdcpf/time_series</code> and <code>src/time_series</code>	UC/OPF/PF multi-period time-series pipeline (<code>solve_time_series_pf</code> , <code>solve_unit_commitment</code>); annual production simulation via hierarchical temporal decomposition (<code>solve_annual_production_simulation</code>); multi-year lifecycle simulation with degradation and NPV (<code>run_lifecycle_simulation</code> , <code>run_lifecycle_comparison</code>); shared <code>UCSchedule</code> , <code>TimeSeriesPFResult</code> , and <code>TimeSeriesData</code> types.
<code>include/hacdcpf/reliability</code> and <code>src/reliability</code>	Monte Carlo, frequency-duration, FMEA, and native three-stage reliability assessment with explicit model-scope and validity contracts.
<code>include/hacdcpf/resilience</code> and <code>src/resilience</code>	Distribution resilience assessment with heuristic restoration, storage/MESS dispatch, and the independent AC-only multi-period LinDistFlow MIP path.
<code>include/hacdcpf/engine</code>	Compatibility/forwarding headers into <code>mipsolvers::engine</code> , plus the <code>hacdcpf</code> Newton PF class declaration and the <code>detail/core_compat.hpp</code> backward-compat aliases. There is no <code>src/engine</code> directory in this repository.

3.2 Top-Level Public API

The facade header `include/hacdcpf/api/hacdcpf.hpp` currently exposes:

- PF: `solve_power_flow`, `solve_dc_power_flow`, `solve_power_flow_adaptive`, `solve_power_flow_islanded`, `solve_power_flow_distributed_slack`, `solve_power_flow_distributed_slack_full`, `solve_power_flow_fdpf`, and `solve_ac_dc_power_flow`;
- persistent PF handles: `create_solver_handle`, `reset_solver_handle`, `solve_handle`, `solve_dc_handle`, and `destroy_solver_handle`;
- OPF: `solve_ac_opf`, `solve_dc_opf`, and `solve_rpo`;
- time-series pipeline: `solve_time_series_pf` and `solve_unit_commitment`;
- carbon analysis: `run_carbon_analysis`;
- annual production simulation: `solve_annual_production_simulation`;
- lifecycle simulation: `run_lifecycle_simulation`, `run_lifecycle_comparison`, and `apply_capacity_scaling`;

- reliability: `run_reliability_assessment`;
- resilience: `run_distribution_resilience_assessment`;
- distribution PF: `solve_three_phase_nr` and the header-only `solve_three_phase` wrapper.

Lower-level namespaces expose additional solver objects and helper entry points. The power-flow namespace includes `solve_hybrid`, `solve_hybrid_adaptive`, `solve_ac`, `solve_dc`, `FDPFSolver`, `AdaptiveSolver`, `DistributedSlackSolver`, `HelmSolver`, `CpfSolver`, `HomotopyContinuationSolver`, and the header-only `solve_three_phase` wrapper.

3.3 Build-Time Optional Features

CMake option	Default	Current effect
<code>HACDCPF_BUILD_TESTS</code>	ON	Enables the Catch2 test targets under <code>tests/</code> .
<code>HACDCPF_ENABLE_EXCEL</code>	OFF	Adds <code>src/io/excel_io.cpp</code> ; otherwise <code>HACDCPF_NO_EXCEL</code> is defined and the public Excel functions throw.
<code>HACDCPF_ENABLE_OPENDSS</code>	OFF	Adds <code>src/io/opendss_bridge.cpp</code> ; otherwise OpenDSS bridge functionality is absent from the build.

3.4 Algorithm Index

The notebook uses the following code-aligned algorithm labels.

Label	Main implementation	Purpose
A1	<code>src/model/network_utils.cpp</code>	Rich-model projection to canonical solver inputs.
A2	<code>src/power_flow/solver_data.cpp</code>	<code>SolverData</code> construction, injection aggregation, and matrix build.
A3	<code>src/power_flow/newton_solver.cpp</code>	Hybrid AC/DC Newton power flow.
A4	<code>src/power_flow/fdpf_solver.cpp</code>	AC fast-decoupled power flow.
A5	<code>src/power_flow/dc_solver.cpp</code>	DC-only power flow.
A6	<code>src/power_flow/adaptive_solver.cpp</code>	Island-aware PF orchestration.
A7	<code>src/power_flow/distributed_slack_solver.cpp</code>	Simplified and full distributed-slack PF.
A8	<code>src/optimal_power_flow/ac_opf.cpp</code>	AC OPF top-level routing and fall-back selection.
A9	<code>src/optimal_power_flow/parity_formulation.cpp</code>	Full-space parity NLP construction and derivatives.
A10	<code>src/optimal_power_flow/parity_ipm.cpp</code>	Mehrotra/Gondzio-style parity IPM solve loop.

Label	Main implementation	Purpose
A11	<code>src/optimal_power_flow/dc_opf.cpp</code>	DC OPF QP/LP model build and backend dispatch.
A12	<code>src/optimal_power_flow/reactive_power_opt.cpp</code>	Reactive-power optimization by OA/coordinate search.
A13	<code>src/power_flow/homotopy_continuation.cpp</code>	Continuation Power Flow (CPF) predictor–corrector (§13.1).
A14	<code>src/power_flow/homotopy_continuation.cpp</code>	Homotopy continuation fallback solver (§13.2).
A15	<code>src/power_flow/newton_krylov.cpp</code>	Newton–GMRES Krylov fallback with Schur preconditioner (§13.4).
A16	<code>src/power_models/lindi_stflow_builder.cpp</code>	Linearised DistFlow LP builder (§15.2).
A17	<code>src/power_models/acopf_builder.cpp</code>	AML AC OPF model builder (§15.3).
A18	<code>src/power_models/acdcopf_builder.cpp</code>	AML hybrid AC/DC OPF model builder (§15.4).
A19	<code>src/power_models/scuc_builder.cpp</code>	MILP Security-Constrained Unit Commitment builder (§15.5).
A20	<code>src/time_series/annual_production_sim.cpp</code>	Hierarchical temporal decomposition annual production simulation: L0 annual plan → L1 monthly → L2 weekly UC → L3 daily OPF/PF replay.
A21	<code>src/time_series/lifecycle_simulation.cpp</code>	Multi-year lifecycle simulation with storage SoH degradation, renewable derating, NPV cost roll-up, and Tier-1/2/3 theoretical carbon error bounds.
A22	<code>src/network_reconfiguration/topology_reconfiguration.cpp</code> / <code>topology_analysis.cpp</code>	Network reconfiguration: steady AC ONR with native B&C and PF verification, plus fault-triggered hybrid topology restoration with HiGHS-to-native fallback, structured branch refs, and model-scope/validity flags (§28.6).
A23	<code>src/analysis/short_circuit.cpp</code>	Short-circuit Z-bus analysis: assemble $\mathbf{Y}_{\text{fault}}$, SparseLU factorize, per-bus Thévenin solve, IEC 60909 fault-current and fault-level computation (§28.8).

4 Engine Solver System

The `hacdcpf::engine` namespace in this repository is mainly a compatibility facade over `mipsolvers::engine`. The local headers keep historical include paths stable while importing engine symbols with `using` declarations. Current implementation files for LCQP, dual simplex, branch and cut, and external adapters live in the `mipsolvers` dependency, not under `src/engine` in

this repository.

4.1 Forwarded Engine Surface

Local forwarding header	Imported symbols / role
<code>engine/problem_types.hpp</code>	Problem-class enums, symbolic model structs, LP/QP/MIP/NLP/MINLP model types, <code>SolveStats</code> , and <code>SolveResult</code> .
<code>engine/kernel/ipm/lcqp_solver.hpp</code>	<code>LCQPOptions</code> and <code>NativeLCQPAdapter</code> .
<code>engine/kernel/lp_kernel/dual_simplex.hpp</code>	Simplex options, basis/result types, <code>StandardFormLP</code> , and <code>solve_lp_with_basis</code> .
<code>engine/kernel/linear_algebra/linear_solver.hpp</code>	Sparse solver abstraction and Eigen-based sparse solver factories.
<code>engine/kernel/globalization/globalization.hpp</code>	Dogleg, trust-region update, and pseudo-transient continuation helpers.
<code>engine/solver/external/adapters.hpp</code>	<code>HiGHSAdapter</code> , <code>IpoptAdapter</code> , <code>ScipAdapter</code> , and <code>GurobiAdapter</code> .
<code>engine/branch_and_cut.hpp</code>	Branching and node-selection enums plus B&C statistics/result/option types.

There are no local PDLP or LP-IPM backends documented by the current headers. The backends visible through this compatibility layer are LCQP, dual simplex, HiGHS, Ipopt, SCIP, and Gurobi, subject to the configuration of `mipsolvers` and the optional external packages available at build time.

4.2 Local Newton Solver Class

The one local engine-style solver declaration is `include/hacdcpf/engine/solver/native/nle/newton_solver.hpp`. Its implementation is `src/power_flow/newton_solver.cpp`; it is the hybrid PF Newton class used by the power-flow module. The file is not an engine implementation directory and does not define the optimization-engine LCQP or LP backends.

4.3 OPF Use of Engine Facilities

Consumer	Engine dependency	Current behavior
DC OPF	LCQP, Gurobi, HiGHS, dual simplex	Auto tries <code>NativeQP</code> , then Gurobi, then HiGHS, then native dual simplex. When <code>DCOPFOptions::compute_lmp=true</code> , the solver runs a supporting simplex LP to recover nodal LMPs. Branch congestion duals remain bounded-variable limited and are guarded by <code>DCOPFResult::branch_mu_valid=false</code> .

Consumer	Engine dependency	Current behavior
RPO	<code>BCStats</code> only for result reporting	The current source runs an OA/coordinate-search workflow and reuses B&C statistics fields for counters; it does not call a generic branch-and-bound solve loop.
Parity AC OPF	local parity IPM	Uses the local KKT assembly and Eigen dense/sparse factorizations in <code>parity_ipm.cpp</code> .
Native AC OPF	local reduced-space KKT loop	Uses custom KKT assembly and factorization in <code>ac_opf.cpp</code> .

5 Native NLP and Interior-Point Details

The nonlinear OPF solvers are local to `src/optimal_power_flow`. They do not dispatch through an external NLP engine by default. The AC OPF entry point selects either a native reduced-space primal-dual path or the parity full-space IPM path described in Section 14.

5.1 Native AC OPF Loop

The native path in `ac_opf.cpp` uses variables for non-slack AC angles, PQ voltage magnitudes, generator dispatch, DC voltage magnitudes, and VSC converter powers. Equality rows cover AC balances, DC balances, and VSC energy balance. Bound constraints are handled with barrier terms, while nonlinear branch and converter limits enter through exterior penalty terms. If this path fails and `ACOPFOptions::allow_fallback` is true, the code falls back to the fast dispatch plus AC PF projection route.

5.2 Parity IPM Loop

The parity path builds explicit equality and inequality rows in `parity_formulation.cpp` and solves the resulting full-space NLP in `parity_ipm.cpp`. The solve loop uses a Mehrotra predictor-corrector step with second-order complementarity correction, iterative refinement, and fraction-to-boundary primal/dual step lengths. Current centering is

$$\sigma = \left(\frac{\mu_{aff}}{\mu_{cur} + 10^{-16}} \right)^3 \quad (3)$$

after clamping the ratio to $[0, 1]$. The implementation also applies up to two extra Gondzio corrector solves when enabled by the current loop logic.

The KKT system is solved with Eigen `SparseLU` on sparse systems or `PartialPivLU` on dense systems. There is no separate local NLP/IPM engine source tree beneath `src/engine`.

6 Native IPM Versus Parity IPM

The two AC OPF nonlinear routes differ mainly in variable coverage and in how constraints are represented.

Feature	Native reduced-space path	Parity full-space path
Primary files	<code>ac_opf.cpp</code>	<code>parity_formulation.cpp</code> , <code>parity_ipm.cpp</code>
AC bus variables	non-slack angles and PQ magnitudes	full AC angle and magnitude blocks with bounds
Generator dispatch	P_g, Q_g variables	P_g, Q_g variables
VSC converters	explicit P^{ac}, Q^{ac}, P^{dc} variables and balance rows	explicit P^{ac}, Q^{ac}, P^{dc} variables and balance rows
DC bus voltages	variables, with tight bounds for DC_V buses	variables with DC balance rows and DC branch limits
Load shedding	not modeled	$\Delta P_d, \Delta Q_d$ variables with VOLL penalty
Curtailable renewable / PV	fixed injections	packed decision vectors <code>pren_mw</code> , <code>qren_mvar</code> with identity maps
Storage	fixed injection	AC and DC storage active dispatch packed into <code>pstor_mw</code> ; AC storage also has <code>qstor_mvar</code>
DCDC converters	not modeled as decision variables	input-side <code>pdcdc_mw</code> variables folded into DC bus balance physics
Flexible loads	projected/fixed demand	<code>pflex_mw</code> variables with identity maps
Failure fallback	fast dispatch plus AC PF if allowed	native path if allowed, then native fallback behavior

The parity result stores enhanced component outputs in packed arrays. The maps `ren_map`, `stor_map`, `dcdc_map`, and `flex_map` must be used to join those arrays back to the original component collections. For storage, `source_type=0` denotes AC storage and `source_type=1` denotes DC storage.

One implementation quirk is important for code review: `ConstraintIndex` still allocates `n_dcdc_bal` and `i_dcdc_bal`, and `build_problem` includes those rows in `n_eq_total`. The current equality evaluator leaves the dedicated DCDC rows unpopulated; DCDC physics is represented instead through the DC bus balance rows.

7 Symbols, Indexing, and Sign Convention

7.1 Electrical Bases

$$S_{base} = \text{system apparent-power base (MVA)}, \quad (4)$$

$$V_{base} = \text{nominal line-line voltage base (kV)}, \quad (5)$$

$$Z_{base} = \frac{V_{base}^2}{S_{base}}, \quad (6)$$

$$I_{base} = \frac{S_{base}}{\sqrt{3}V_{base}}. \quad (7)$$

7.2 Sets and Indexing

$$\mathcal{N}_{ac} : \text{AC buses}, \quad \mathcal{E}_{ac} : \text{AC branches}, \quad (8)$$

$$\mathcal{N}_{dc} : \text{DC buses}, \quad \mathcal{E}_{dc} : \text{DC branches}, \quad (9)$$

$$\mathcal{N}_{sl} : \text{AC slack buses}, \quad \mathcal{N}_{PQ} : \text{AC PQ buses}, \quad (10)$$

$$\mathcal{N}_{PV} : \text{AC PV buses}, \quad \mathcal{N}_{sl}^{\text{dc}} : \text{DC voltage-controlled buses}. \quad (11)$$

The public model stores external bus identifiers in each component (`bus`, `from_bus`, `to_bus`). Most solver kernels then remap those identifiers into contiguous zero-based local indices.

7.3 Core AC and DC State Variables

$$V_i = v_i e^{j\theta_i}, \quad (12)$$

$$Y_{ij} = G_{ij} + jB_{ij}, \quad (13)$$

$$V_k^{\text{dc}} = v_k^{\text{dc}}, \quad (14)$$

$$P_k^{\text{dc,calc}} = v_k^{\text{dc}} (G_{dc} v^{\text{dc}})_k. \quad (15)$$

For the main Newton power-flow path, the state vector is

$$x_{PF} = \begin{bmatrix} \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} & v_{\mathcal{N}_{PQ}} & v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}^{\text{dc}}}^{\text{dc}} \end{bmatrix}^{\top}. \quad (16)$$

For the native primal-dual AC OPF path, the decision vector is extended to include generator dispatch and converter operating points:

$$x_{OPF} = \begin{bmatrix} \theta & v & P_g & Q_g & v^{\text{dc}} & P^{\text{conv,ac}} & Q^{\text{conv,ac}} & P^{\text{conv,dc}} \end{bmatrix}^{\top}, \quad (17)$$

with additional load-shedding variables in the parity formulation.

7.4 Unified Sign Convention: Bus Injection Positive

The current C++ module uses one consistent notation in the PF kernels and the notebook adopts the same rule:

Positive active or reactive power at a bus means injection into the network from the component attached to that bus.

Loads therefore store positive demand values in their data model, but they enter power-balance equations with a minus sign.

Quantity / field	Positive value means	Consequence in equations
<code>Generator::pg_mw</code> , <code>qg_mvar</code>	generator injects active/reactive power into its AC bus	added to $P_i^{\text{spec}}, Q_i^{\text{spec}}$.
<code>Load::p_mw</code> , <code>q_mvar</code>	consumption requested from the network	subtracted from $P_i^{\text{spec}}, Q_i^{\text{spec}}$; ZIP dependence scales the subtraction.

Quantity / field	Positive value means	Consequence in equations
<code>Storage::p_mw</code>	discharge to the grid; charging is negative P	treated as generation when positive, load when negative.
<code>Microgrid::p_exchange_mw</code>	export from the microgrid to the host AC grid	added as AC-bus injection in PF; ignored when the microgrid is islanded.
<code>VSCConverter</code> AC-side $P^{\text{conv,ac}}$	converter injects active power into the AC bus	positive AC injection must be balanced by negative DC-bus injection plus losses.
<code>VSCConverter</code> DC-side $P^{\text{conv,dc}}$	converter injects active power into the DC bus	the current code enforces $P^{\text{conv,ac}} + P^{\text{conv,dc}} + P^{\text{loss}} = 0$.
<code>DCDCConverter::p_ref_mw</code>	positive transfer from <code>bus_in</code> toward <code>bus_out</code>	implemented as negative injection at <code>bus_in</code> and positive injection at <code>bus_out</code> .
<code>BranchFlow::pf_mw</code>	active power sent from <code>from_bus</code> to <code>to_bus</code>	reporting only; no sign reversal in stored result.

7.5 Power-Balance Template

With bus-injection-positive notation, the balance equations are written as

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad (18)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad (19)$$

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}. \quad (20)$$

The specified term is assembled by adding all component injections and then subtracting all demand terms. This is the convention followed by the PF residual evaluator, the native AC OPF assembly, and the parity formulation.

8 Information Models and Field Dictionaries

8.1 Modeling Layers Used by the Code

The code base uses three distinct model layers.

1. **Public information model:** the rich structs under `include/hacdcpf/model/` used by I/O, case builders, and external callers.
2. **Canonical steady-state model:** the projected subset generated by `project_to_canonical_models`, where rich components are converted into equivalent loads, branches, or aggregated injections.
3. **Solver assembly model:** `powerflow::SolverData`, which stores contiguous vectors, aggregated per-bus injections, ZIP coefficients, and sparse matrices needed by the PF and OPF kernels.

8.2 System Containers and Solver-Level Metadata

System containers

Struct / group	Fields and meaning	Primary consumers
<code>ACSystem</code>	buses, branches, generators, static_generators, loads, flexible_loads, asymmetric_loads, shunts, storage, renewable_gens, pv_systems, external_grids, transformers_2w, transformers_3w, switches, circuit_breakers, charging_stations, chargers, motors; plus base_mva, freq_hz, name.	PF, OPF, I/O, projection
<code>DCSystem</code>	buses, branches, loads, storage, static_generators, dc_static_generators, pv_arrays, dc_circuit_breakers, plus base_mva and name.	PF, OPF, I/O
<code>HybridPowerSystem</code>	Holds ac, dc, vsc_converters, dcdc_converters, energy_routers, mobile_storage, vpps, microgrids, global base_mva, name, optional three_phase_ac, and optional provenance maps bus_merge_map and branch_expand_map.	all top-level APIs
<code>ThreePhaseACSystem</code>	Optional unbalanced AC subsystem with three-phase buses, lines, transformers, loads, generators, and external grids. It is projected to canonical AC objects when the explicit <code>ACSystem</code> is empty.	projection, PF, I/O

Power-flow options and results

Struct / group	Fields and meaning	Primary consumers
<code>InitialState</code>	vm, va, vdc provide optional warm starts for AC magnitude, AC angle, and DC voltage states.	Newton PF, DC PF
<code>ReactiveLimit</code>	qmin, qmax define reactive limits used by adaptive/PV-PQ logic.	adaptive PF extensions
<code>DistributedSlack</code>	participating_buses, participation_factors, reference_bus, and optional max_participation_p describe how slack mismatch is redistributed.	distributed-slack PF

Struct / group	Fields and meaning	Primary consumers
PowerFlowOptions	Iteration and tolerance controls: <code>max_iter</code> , <code>tol</code> ; robustness controls: line search, regularization, and step clipping; PV/PQ and converter hysteresis parameters; ZIP weights; solver profiling flag; <code>loss_model</code> ; FDPF iteration cap.	PF family
SolverProfiling	Sparse-factorization counters, line-search and regularization statistics, PV/PQ switches, converter-mode switches, timing vectors, and cumulative timing totals.	PF result diagnostics
PowerFlowResult	Solved states <code>vm</code> , <code>va</code> , <code>vdc</code> ; status fields <code>converged</code> , <code>iterations</code> , <code>residual</code> ; and derived reports <code>branch_flows</code> , <code>vsc_transfers</code> , <code>dc_dc_transfers</code> , <code>er_port_transfers</code> .	PF family outputs
AdaptiveSolverResult , IslandedSolveResult , DistributedSlackResult	Variants of PF results that additionally expose detected <code>islands</code> , distributed-slack allocations, and buses hitting slack-sharing limits.	adaptive / islanded / distributed-slack PF

Analysis and OPF options/results

Struct / group	Fields and meaning	Primary consumers
ACOPFOptions	Inner/outer iteration caps, feasibility/stationarity tolerances, barrier schedule, merit-penalty and regularization terms, line-search settings, parallel AC evaluation threads, and path-selection flags <code>enable_primal_dual</code> , <code>use_parity_ipm</code> , <code>allow_fallback</code> .	AC OPF
ACOPFResult	Voltage solution, generator dispatch, optional bus-level load shedding, converter operating points, convergence metrics, status text, and profiling summary.	AC OPF output
DCOPFOptions , DCOPFResult	DC approximation controls, backend selector, branch-limit flag, load-shedding flag, <code>compute_lmp</code> toggle (default <code>true</code> : runs a supporting simplex LP after the primal solve to populate nodal <code>lmp</code> ; branch-dual arrays are guarded by <code>branch_mu_valid</code>), voltage angles, generator dispatch, branch flows, per-bus and total load shedding, <code>solver_chain</code> , <code>objective_model</code> , and solver status.	DC OPF
RPOOptions , RPOResult	Reactive-power objective settings, search limits, voltage/generator before/after vectors, tap and shunt changes, loss metrics, status, and B&C-style statistics fields reused as counters.	RPO

Struct / group	Fields and meaning	Primary consumers
Analysis result types	Core resilience, reliability MC/FMEA, short-circuit, ONR, time-series, and carbon paths are implemented with documented limitations. The legacy distribution power-flow stub/public BFS API has been removed; distribution feeders use the three-phase NR contract. The resilience MIP entry point (<code>run_distribution_resilience_mip_assessment</code>) is a live independent AC-only MILP solver (not a heuristic delegate). FMEA uses <code>ac-only-dcopf</code> for AC-only systems and <code>hybrid-acdc-network-lp</code> for systems with DC/VSC components, with linearized scope exposed through model-scope/limitation fields. The three-stage MILP reliability API (<code>run_three_stage_reliability</code>) is a native C++ MIPSolvers-based evaluator (not an external Julia bridge). See Sections 1.2 and 26.	analysis declarations

`powerflow::SolverData`

Group	Fields and meaning	Primary consumers
Canonical component tables	Copies of projected AC/DC buses, branches, generators, converters, DCDC converters, energy routers, loads, DERs, shunts, switches, charging stations, external grids, VPPs, microgrids, and mobile storage.	PF, native OPF
Aggregated steady-state injections	<code>pg</code> , <code>qg</code> hold per-bus generation after aggregation of generators, static generators, renewables, PV, storage, VPPs, microgrids, and mobile storage.	PF, native OPF
Demand model data	<code>has_component_loads</code> , <code>pd_pu</code> , <code>qd_pu</code> , and per-bus ZIP coefficient vectors describe the demand model after load-table aggregation.	PF, native OPF
Network matrices	<code>ybus</code> and <code>gdc</code> store the canonical AC admittance and DC conductance matrices; <code>build_id</code> is used to validate Jacobian-pattern caching.	PF, native OPF
Bookkeeping	<code>base_mva</code> , <code>loss_model</code> , and global ZIP weights copied from <code>PowerFlowOptions</code> .	PF, native OPF

8.3 Component Inventory

Table 15 enumerates the rich public models currently present in the repository.

Table 15: Public component-model inventory

Struct	Mathematical role	Current steady-state integration mode
ACBus	AC state host, voltage limits, embedded bus demand and shunt	direct
ACBranch	AC series/shunt admittance, tap and phase-shift branch	direct
Transformer2W	named two-winding transformer with short-circuit/tap data	projected to ACBranch
Transformer3W	explicit three-winding transformer with pairwise short-circuit data	projected to three ACBranch equivalents
ExternalGrid	slack source and source-equivalent metadata	direct PF source metadata; richer fault fields are stored but no public SC solver is present
AsynchronousMotor	motor fault-contribution metadata	stored by the model and I/O; no public SC solver currently consumes it
Generator	synchronous generator dispatch and limits	direct
StaticGenerator	fixed/scaled distributed generation	direct fixed injection
RenewableGen	renewable injection with curtailment metadata	fixed injection in PF/native OPF; parity OPF decision variable when curtailable
PVSystem	PV inverter and array metadata	fixed injection in PF/native OPF; parity OPF decision variable when controllable
Load	AC demand with ZIP and motor-fraction metadata	direct
FlexibleLoad	demand-response model	projected to Load for PF; parity OPF has packed flexible-load variables
AsymmetricLoad	per-phase aggregate demand	projected to Load
Shunt	fixed or stepped shunt element	direct fixed shunt in PF; switchable steps are RPO discrete controls
Switch	switching device with electrical/protection data	projected to ACBranch for PF/OPF
CircuitBreaker	high-voltage breaker on an AC bus pair	projected to ACBranch; can trigger zero-impedance bus merging

Struct	Mathematical role	Current steady-state integration mode
ChargingStation	aggregate EV charging demand	direct fixed load
Charger	individual EV charger	aggregated into ChargingStation on projection
Storage	stationary storage power and energy state	fixed injection in PF/native OPF; parity OPF dispatch variable for AC and DC storage
MobileStorage	relocatable storage resource	projected to AC Storage at current bus
VirtualPowerPlant	aggregated DER portfolio	projected to AC StaticGenerator at aggregation bus
Microgrid	PCC exchange model	projected to AC StaticGenerator at aggregation bus/PCC when grid-connected
DCBus	DC voltage host and DC demand host	direct
DCBranch	DC resistive branch	direct
DCLoad	DC active demand	direct
StaticGeneratorDC	typed DC active-power source with profile/control metadata	stored in <code>dc.dc_static_generators</code> ; serialized and available for DC injection assembly where supported
PVArrayDC	DC-side photovoltaic array metadata and setpoint	stored in <code>dc.pv_arrays</code> ; serialized and available for DC injection assembly where supported
DCCircuitBreaker	breaker on a DC bus pair	stored and serialized; no projection equivalent currently runs in <code>network_utils.cpp</code>
VSCConverter	AC/DC coupling with loss model and control mode	direct
DCDCConverter	DC/DC transfer device with efficiency and I^2R conduction loss	direct in PF and parity OPF (decision variable with Jacobian and Hessian)
EnergyRouterPort	AC or DC port injection for a multi-port router	direct fixed injection
EnergyRouter	multi-port converter container	direct through its ports
ThreePhaseACBus	optional unbalanced bus model	projected to ACBus
ThreePhaseACLine	optional unbalanced line model	projected to ACBranch using positive sequence
ThreePhaseTransformer	optional unbalanced transformer model	projected to Transformer2W equivalent

Struct	Mathematical role	Current steady-state integration mode
ThreePhaseLoad	optional unbalanced load model	projected to Load
ThreePhaseGenerator	optional unbalanced generator model	projected to Generator
ThreePhaseExternalGrid	optional unbalanced slack/grid equivalent	projected to ExternalGrid

8.4 AC Network Elements

ACBus

Field group	Detailed explanation	Used by
Identity and topology	index is the external bus identifier referenced by other components; name is purely descriptive; area and zone are retained for partitioning and reporting.	shared model / solver inputs
Steady-state electrical state	bus_type , vm_pu , va_deg , base_kv , vmax_pu , vmin_pu , in_service . These fields initialize the PF state, define slack/PV/PQ behavior, and bound voltages.	PF, OPF
Embedded demand and shunt	pd_mw , qd_mvar store fallback bus demand when no load table is present; gs_mw , bs_mvar store bus shunt terms.	PF, OPF
Resilience / geography	n_customers , importance , latitude , longitude . These are not yet decision variables, but they are useful for planning logic, GIS, or load-priority extensions.	planning / future use

ACBranch

Field group	Detailed explanation	Used by
Connectivity	index , from_bus , to_bus , in_service , name . Bus references use external bus IDs.	PF, OPF
Positive-sequence electrical data	r_pu , x_pu , b_pu , tap , shift_deg . These define the AC π -model and branch-flow reporting.	PF, OPF
Ratings and physical metadata	rate_a_mva , rate_b_mva , rate_c_mva , length_km . rate_a_mva is used directly by OPF thermal limits.	OPF, reporting
Reliability and sequence data	failure_rate , mttr_hr , r0_pu , x0_pu , b0_pu , and transformer-nameplate fields vn_hv_kv , vn_lv_kv , sn_mva . These are stored for planning/fault-model extensions, but no public SC solver is present.	model/I/O metadata

Transformer2W, Transformer3W, Switch, and CircuitBreaker

Struct	Field-level explanation	Current treatment
Transformer2W	Identity/connectivity fields <code>index</code> , <code>name</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>in_service</code> ; nameplate fields <code>sn_mva</code> , <code>vn_hv_kv</code> , <code>vn_lv_kv</code> ; short-circuit fields <code>vk_percent</code> , <code>vkr_percent</code> , <code>pk_kw</code> ; no-load fields <code>pf_e_kw</code> , <code>i0_percent</code> ; tap-changer fields <code>tap_side</code> , <code>tap_pos</code> , <code>tap_min</code> , <code>tap_max</code> , <code>tap_neutral</code> , <code>tap_step_percent</code> ; phase shift <code>shift_deg</code> ; zero-sequence fields <code>z0_percent</code> , <code>x0_r0</code> ; reliability fields <code>mtbf_hours</code> , <code>mttr_hours</code> .	projected to one equivalent ACBranch
Transformer3W	Connectivity <code>hv_bus</code> , <code>mv_bus</code> , <code>lv_bus</code> ; per-winding ratings <code>sn_hv_mva</code> , <code>sn_mv_mva</code> , <code>sn_lv_mva</code> ; pairwise short-circuit data <code>vk_hv_mv_percent</code> , <code>vk_hv_lv_percent</code> , <code>vk_mv_lv_percent</code> and the matching <code>vkr</code> fields; tap data and phase shifts <code>shift_mv_deg</code> , <code>shift_lv_deg</code> .	projected to three equivalent ACBranch objects
Switch	Topology fields <code>bus_from</code> , <code>bus_to</code> , <code>closed</code> ; electrical fields <code>r_contact_ohm</code> , <code>z_ohm</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code> ; protection fields <code>element_type</code> , <code>element_id</code> ; automation fields <code>is_remote</code> , <code>is_automated</code> , <code>t_operation_s</code> ; reliability fields <code>p_sw_fail</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> , <code>t_tp_hr</code> .	projected to equivalent branch with <code>in_service=closed</code>
CircuitBreaker	AC breaker fields <code>index</code> , <code>name</code> , <code>bus_from</code> , <code>bus_to</code> , <code>in_service</code> , <code>breaker_type</code> , <code>closed</code> , <code>z_ohm</code> , <code>rated_voltage_kv</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code> , <code>element_type</code> , <code>element_id</code> .	projected to equivalent branch; if switches or breakers exist, zero-impedance bus merging may run after projection

ExternalGrid and AsynchronousMotor

Struct	Field-level explanation	Current treatment
ExternalGrid	<code>bus</code> , <code>vm_pu</code> , <code>va_deg</code> establish slack voltage targets; <code>s_sc_max_mva</code> , <code>s_sc_min_mva</code> , <code>rx_max</code> , <code>rx_min</code> , <code>r_pu</code> , <code>x_pu</code> , <code>r0_pu</code> , <code>x0_pu</code> , <code>vn_kv</code> , <code>ikq_ka</code> , <code>x_r</code> store source-equivalent/fault metadata; <code>controllable</code> can support dispatch logic.	PF source metadata; additional fields are stored for future analysis

Struct	Field-level explanation	Current treatment
AsynchronousMotor	bus, vn_kv, sn_mva, r_pu, x_pu, x_r, lrc, poles, cos_phi, efficiency, r0_pu, x0_pu. These fields describe possible motor fault contribution, not a steady-state PF state.	model/I/O metadata

8.5 Load Models and Demand-Side Components

Struct	Field-level explanation	Current treatment
Load	Identity fields index, bus, name, in_service; steady-state demand p_mw, q_mvar; model selector model; ZIP coefficients z_percent_p, i_percent_p, p_percent_p, and the matching q fields; demand-response metadata controllable, p_min_mw, cost_mw; load-priority metadata priority, n_customers, profile_id; motor-fraction metadata sn_mva, motor_percent, x_sub_pu, r_sc_pu.	direct in PF/OPF
FlexibleLoad	Baseline demand p_mw, q_mvar; flexibility envelope flex_up_mw, flex_down_mw, flex_duration_h; response timing response_time_s, ramp_rate_mw_min, availability_pct; control_area.	projected to equivalent Load for PF; parity OPF has pflex_mw variables
AsymmetricLoad	Per-phase rated values pa_rated_mw, ..., qc_rated_mvar; per-phase actual values pa_mw, ..., qc_mvar; connection, grounded, scaling; ZIP shares const_z_percent, const_i_percent, const_p_percent.	projected to aggregated Load
DCLoad	p_mw is the direct active demand at a DC bus; controllable, p_min_mw, cost_mw, and profile_id are control metadata for future extensions.	direct in DC PF and hybrid OPF balances
ChargingStation	bus, n_fast, n_slow, num_chargers; capacity fields p_fast_max_kw, p_slow_max_kw, max_power_kw; utilization fields simultaneity_factor, power_factor, utilization_rate; aggregated operating demand p_total_kw, q_total_kvar.	direct fixed load
Charger	station_id links the charger to its parent station; charger_type, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw.	aggregated into ChargingStation during projection

8.6 Generation, Storage, and Aggregation Components

Struct	Field-level explanation	Current treatment
Generator	Dispatch/state fields <code>pg_mw</code> , <code>qg_mvar</code> , <code>vg_pu</code> , <code>is_slack</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; quadratic cost coefficients <code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code> ; unit-commitment metadata <code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code> , <code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code> ; machine/stability data <code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code> , <code>xd_pu</code> , <code>xdp_pu</code> , <code>xdpp_pu</code> , <code>td0p_s</code> , <code>td0pp_s</code> ; SC and environmental metadata <code>ra_pu</code> , <code>vn_kv</code> , <code>cos_phi</code> , <code>xq_pu</code> , <code>xd_xq</code> , <code>x0_pu</code> , <code>r0_pu</code> , emission factors, outage rate, and repair time.	direct
StaticGenerator	Fixed or scaled DER injection <code>p_mw</code> , <code>q_mvar</code> , <code>scaling</code> ; device rating/limits <code>p_rated_mw</code> , <code>sn_mva</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; control metadata <code>controllable</code> , <code>v_ref_pu</code> , <code>k_p</code> , <code>k_q</code> ; SC metadata <code>k</code> , <code>rx</code> .	direct fixed injection
RenewableGen	type, actual output <code>p_mw</code> , <code>q_mvar</code> , rated power <code>p_rated_mw</code> , reactive limits, curtailability flag, curtailment cost, capacity factor, profile pointer, and avoided-emission factor.	fixed injection in PF/native OPF; parity OPF uses <code>pren_mw/qren_mvar</code> for curtailable rows
PVSystem	Actual output <code>p_mw</code> , <code>q_mvar</code> ; inverter limits and control <code>sn_mva</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> , <code>control_mode</code> , <code>controllable</code> , <code>v_ac_set_pu</code> , <code>v_dc_set_pu</code> ; inverter efficiency <code>inverter_eff</code> , <code>loss_percent</code> ; array data <code>num_series</code> , <code>num_parallel</code> , <code>vmpp</code> , <code>impp</code> , <code>voc</code> , <code>isc</code> , <code>alpha_isc</code> , <code>beta_voc</code> ; environmental state <code>irradiance</code> , <code>temperature</code> .	fixed injection in PF/native OPF; parity OPF uses renewable dispatch variables for controllable rows
Storage	Snapshot dispatch <code>p_mw</code> , <code>q_mvar</code> ; power limits <code>p_rated_mw</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; energy state <code>e_rated_mwh</code> , <code>soc_init</code> , <code>soc_min</code> , <code>soc_max</code> ; efficiency <code>eta_charge</code> , <code>eta_discharge</code> , <code>self_discharge_pct</code> ; lifecycle <code>max_cycles</code> , <code>current_cycles</code> , <code>soh</code> , <code>replacement_cost</code> .	fixed injection in PF/native OPF; parity OPF has AC and DC storage dispatch variables
MobileStorage	Reuses storage-style power and energy fields and adds mobility fields <code>is_mobile</code> , <code>status</code> , <code>current_location</code> , <code>target_bus</code> , <code>e_consumption_mwh_km</code> , <code>max_travel_distance_km</code> .	projected to AC Storage at its current bus
VirtualPowerPlant	Fleet counts, aggregated capacities, output fields <code>p_output_mw</code> , <code>q_output_mvar</code> , limits, and ramp rates.	projected to AC StaticGenerator at aggregation bus

Struct	Field-level explanation	Current treatment
Microgrid	PCC/aggregation bus, internal bus list, operating-mode fields <code>operating_mode</code> , <code>islanding_capability</code> , <code>auto_reconnection</code> ; exchange limits and current <code>p_exchange_mw</code> ; aggregate capability metrics and droop/voltage setpoints.	projected to AC <code>StaticGenerator</code> when grid-connected

8.7 DC and Power-Electronic Components

Struct	Field-level explanation	Current treatment
DCBus	<code>index</code> , <code>bus_type</code> , <code>vm_pu</code> , <code>vmax_pu</code> , <code>vmin_pu</code> , <code>pd_mw</code> , <code>in_service</code> , <code>name</code> , <code>base_kv</code> , <code>area</code> , <code>zone</code> , <code>n_customers</code> , <code>importance</code> , <code>is_load</code> , <code>latitude</code> , <code>longitude</code> . <code>pd_mw</code> is used only when the dedicated DC load table is empty.	direct
DCBranch	<code>from_bus</code> , <code>to_bus</code> , <code>r_pu</code> , <code>rate_a_mva</code> , <code>length_km</code> , <code>base_kv</code> , <code>s_max_mva</code> , <code>n_parallel</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> , <code>in_service</code> .	direct
StaticGeneratorDC	<code>index</code> , <code>bus</code> , <code>in_service</code> , <code>name</code> , <code>type</code> , <code>p_set_mw</code> , <code>scaling</code> , <code>profile_id</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>controllable</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> .	stored in <code>dc.dc_static_generators</code> ; serialized as its own DC source family
PVArrayDC	<code>index</code> , <code>bus</code> , <code>in_service</code> , <code>name</code> , <code>p_set_mw</code> , <code>profile_id</code> , array electrical parameters, environmental state, and reliability fields.	stored in <code>dc.pv_arrays</code> ; serialized as its own DC PV family
DCCircuitBreaker	<code>index</code> , <code>name</code> , <code>bus_from</code> , <code>bus_to</code> , <code>in_service</code> , <code>breaker_type</code> , <code>closed</code> , <code>r_ohm</code> , <code>rated_voltage_kv</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code> , <code>element_type</code> , <code>element_id</code> .	stored and serialized; no current projection equivalent
VSCConverter	Connectivity <code>bus_ac</code> , <code>bus_dc</code> ; control mode <code>control_mode</code> ; topology type; setpoints <code>p_set_mw</code> , <code>q_set_mvar</code> , <code>v_dc_set_pu</code> , <code>v_ac_set_pu</code> ; loss parameters <code>eta</code> , <code>loss_percent</code> , <code>loss_mw</code> ; control gains <code>k_vdc</code> , <code>k_p</code> , <code>k_q</code> ; limits and ratings including <code>vn_ac_kv</code> , <code>vn_dc_kv</code> ; controllability, grid-forming and SC-current metadata, and reliability fields.	direct in PF and OPF; some C++ fields are not serialized by the current JSON writer

Struct	Field-level explanation	Current treatment
DCDCConverter	Connectivity <code>bus_in</code> , <code>bus_out</code> ; operating mode <code>control_mode</code> ; references <code>p_ref_mw</code> (output-side referenced), <code>v_ref_pu</code> ; rating fields <code>sn_mva</code> , <code>vn_in_kv</code> , <code>vn_out_kv</code> ; efficiency <code>eta</code> ; conduction-loss resistance <code>r_eq_pu</code> ; droop gain <code>k_droop</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> ; <code>controllable</code> , <code>f_switching_hz</code> , and reliability fields. Sign convention: positive direction is <code>bus_in</code> → <code>bus_out</code> ; PF uses output-referenced <code>p_ref_mw</code> , OPF uses input-referenced P^{in} .	direct in PF and parity OPF; some C++ fields are not serialized by the current JSON writer
EnergyRouterPort	<code>bus</code> , <code>port_type</code> , <code>side</code> , <code>voltage_level_kv</code> , operating injection <code>p_mw</code> , <code>q_mvar</code> , <code>v_pu</code> , efficiency <code>eta</code> ; limits and control metadata.	expanded by projection into internal converter equivalents
EnergyRouter	Container fields <code>router_type</code> , <code>num_ports</code> , <code>ports</code> ; rating/loss fields; <code>control_mode</code> , <code>power_dispatch_strategy</code> , <code>t_scheduled_hr</code> ; device-level limits and reliability.	expanded by projection into internal DC buses, VSCs, and DCDC converters

8.8 EnergyRouter Multi-Port Model

An **EnergyRouter** is a power-electronic converter hub that simultaneously connects to an arbitrary number of AC and DC buses. Each physical terminal is represented by one **EnergyRouterPort** entry in the router's `ports` vector.

Port attributes

Field	Type / default	Description
<code>port_type</code>	<code>ERPortType::AC</code>	AC or DC: determines which bus network this port injects into
<code>side</code>	<code>int\{0\}</code>	Physical winding side: 0 = Side A, 1 = Side B (used by SST/UPFC topologies)
<code>bus</code>	<code>int\{0\}</code>	Index of the connected AC or DC bus
<code>p_mw</code> , <code>q_mvar</code>	<code>double\{0\}</code>	Bus-injection sign convention: positive = power into bus
<code>eta</code>	<code>double\{0.98\}</code>	Per-port conversion efficiency for loss allocation
<code>control_mode</code>	<code>ERControlMode::PQ</code>	Per-port control law: PQ (fixed setpoint), VQ (voltage-droop reactive), or Slack (voltage-forming)
<code>p_set_mw</code> , <code>q_set_mvar</code>	<code>double\{0\}</code>	Active/reactive setpoints when <code>control_mode</code> = PQ

Multi-port power balance

Let port k carry active injection P_k (positive = into bus) and efficiency η_k . The lossless internal balance constraint is:

$$\sum_{k=1}^{N_p} P_k = 0 \quad (\text{lossless ER}). \quad (21)$$

When per-port efficiency $\eta_k < 1$, the actual balance allocates losses to each port proportionally:

$$\sum_{k:P_k < 0} |P_k| = \sum_{k:P_k > 0} \frac{P_k}{\eta_k} \quad (\text{lossy ER, simplified allocation}). \quad (22)$$

In the current projection path, an **EnergyRouter** is expanded into a set of internal DC buses, VSC equivalents for each AC port, and DCDC converter equivalents for each DC port. The internal bus forms the power-exchange node; (21) is enforced implicitly by the solver topology.

Router-level fields

Field	Default	Description
<code>router_type</code>	<code>""</code>	Topology identifier, e.g. "SST" (solid-state transformer), "UPFC" (unified power-flow controller)
<code>num_ports</code>	<code>0</code>	Number of EnergyRouterPort entries in ports
<code>control_mode</code>	<code>""</code>	Router-level dispatch strategy: "autonomous" or "centralized"
<code>power_dispatch_strategy</code>	<code>""</code>	Inter-port power allocation policy: "priority" or "proportional"
<code>p_rated_mw</code>	<code>0</code>	Rated through-power capacity of the device
<code>loss_percent</code>	<code>0</code>	Device-level loss as a percentage of rated power

8.9 VirtualPowerPlant Fleet Aggregation Model

A **VirtualPowerPlant** (VPP) represents a portfolio of distributed energy resources (DERs) co-dispatched as a single grid entity. The struct carries both static fleet composition data and dynamic dispatch outputs.

Fleet composition

Field	Default	Meaning
<code>n_pv_systems</code>	<code>0</code>	Number of rooftop/ground-mounted PV units in portfolio
<code>n_wind_turbines</code>	<code>0</code>	Number of wind turbines
<code>n_battery_systems</code>	<code>0</code>	Number of battery storage units
<code>n_ev_chargers</code>	<code>0</code>	Number of V2G-capable EV chargers
<code>n_controllable_loads</code>	<code>0</code>	Demand-response resources

Field	Default	Meaning
n_chp	0	Combined heat-and-power units
n_biomass, n_thermal_storage, n_hvac, n_industrial	0	Other flexible DER categories
p_pv_sum_mw, p_wind_sum_mw	0	Aggregated renewable capacity
e_storage_sum_mwh	0	Total usable storage energy

Aggregation dispatch model

Let P_i^{DER} be the net active output of DER unit i (positive = generation, negative = consumption). The VPP aggregates to:

$$P^{\text{VPP}} = \sum_{i \in \mathcal{F}} P_i^{\text{DER}} = P^{\text{gen}} - P^{\text{flex,abs}}, \quad (23)$$

where P^{gen} is the total scheduled generation and $P^{\text{flex,abs}}$ is the absorbed flexible load. The field `p_output_mw` stores this aggregated value; `q_output_mvar` carries the corresponding reactive output.

Regulation capability is bounded by:

$$-P^{\text{reg,down}} \leq \Delta P \leq P^{\text{reg,up}}, \quad (24)$$

stored in `p_regulation_down_mw` and `p_regulation_up_mw`.

Projection to grid model

In the current solver path, a `VirtualPowerPlant` is projected to a single AC `StaticGenerator` at `aggregation_bus` with:

$$p_{\text{sg}} = p_{\text{output_mw}}, \quad q_{\text{sg}} = q_{\text{output_mvar}}.$$

Ramp-rate fields (`ramp_up_max_mw_min`, `ramp_down_max_mw_min`) and regulation bounds are available to time-series and OPF formulations but are not yet enforced by the snapshot PF solver.

8.10 Optional Three-Phase Models

Struct	Field-level explanation	Current treatment
<code>ThreePhaseACBus</code>	Per-phase voltages <code>vm_a_pu</code> , <code>va_a_deg</code> , <code>vm_b_pu</code> , ...; per-phase demand and shunts; common voltage limits, area, and zone.	projected to <code>ACBus</code> by phase summation / averaging
<code>ThreePhaseACLine</code>	Positive- and zero-sequence per-km parameters, optional precomputed per-unit values, parallel count, thermal rating, and reliability data.	projected to <code>ACBranch</code> using positive-sequence values

Struct	Field-level explanation	Current treatment
ThreePhaseTransformer	Two-winding transformer with vector group, positive- and zero-sequence short-circuit data, tap changer, and phase shift.	projected to Transformer2W equivalent before branch projection
ThreePhaseLoad	Connection type, grounding flag, per-phase powers, ZIP shares, and motor fraction.	projected to aggregated Load
ThreePhaseGenerator	Per-device dispatch and limits plus sequence impedances <code>xd_pu</code> , <code>xdpp_pu</code> , <code>x2_pu</code> , <code>x0_pu</code> , <code>r0_pu</code> .	projected to Generator
ThreePhaseExternalGrid	Per-sequence grid-equivalent impedances and short-circuit capacities.	projected to ExternalGrid
ThreePhaseBusResult	Per-phase solved voltage and injection report. The current one-line notebook path documents the model, but the main steady-state solvers still operate on projected single-phase equivalents.	reporting / future unbalanced workflows

9 I/O Contracts, Sheet Dictionaries, and JSON Keys

The model dictionaries in the previous section explain the physical meaning of each field. This section documents the external I/O contract actually exposed by the current C++ implementation: JSON path names, Excel sheet names, column names, defaulting rules, aggregated views, and the current serialization gaps.

9.1 Serialization Families

API entry point	Artifact	Direction	Practical behavior
<code>to_json</code> / <code>from_json</code>	one JSON document for <code>HybridPowerSystem</code>	bidirectional	covers the major public model families attached to <code>HybridPowerSystem</code> , including circuit breakers, DC static generators, DC PV arrays, DC circuit breakers, <code>ac.motors</code> , and optional <code>three_phase_ac</code> ; several C++ struct fields are still not serialized.
<code>load_excel</code> / <code>save_excel</code>	public Excel API declared in <code>excel_io.h</code> pp	bidirectional when enabled	without <code>HACDCPF_ENABLED_EXCEL</code> , both inline functions throw; with Excel enabled, the header declares these names, while the current source defines the workbook helpers below under <code>load_xlsx/save_xlsx</code> .

API entry point	Artifact	Direction	Practical behavior
<code>source-only save_xlsx / load_xlsx</code>	engineering workbook with one table per supported component class	bidirectional when compiled	implemented in <code>src/io/excel_io.cpp</code> ; input-oriented Excel template plus derived aggregated views such as <code>ACLoad</code> and <code>DCLoad</code> .
<code>source-only PF workbook helpers</code>	dedicated/compact PF result workbooks	export only	<code>src/io/excel_io.cpp</code> implements result workbook writers, but these helper names are not declared in <code>include/hacdcpf/io/excel_io.hpp</code> .
<code>source-only OPF workbook helpers</code>	dedicated/compact OPF result workbooks	export only	write <code>OPF_*</code> sheets; compact OPF export does not append <code>result_*</code> columns into input tables.
<code>power_flow_result_to_json / save_power_flow_result_json</code>	PF result JSON	export only	serializes convergence summary plus AC/DC bus states, AC/DC branch flows, VSC transfers, and DCDC transfers.
<code>opf_result_to_json / opf_result_from_json</code>	AC OPF result JSON	bidirectional	serializes solver status, solver path, primal solution vectors, enhanced dispatch arrays with identity maps, legacy dispatch arrays, and profiling summary.
<code>dc_opf_result_to_json / dc_opf_result_from_json / save_dc_opf_result_json / load_dc_opf_result_json</code>	DC OPF result JSON	bidirectional	serializes convergence/status, objective, solver name/runtime, <code>solver_chain</code> , <code>objective_model</code> , LMPs, branch-dual vectors with <code>branch_mu_valid</code> , per-bus shedding, generator dispatch, branch flows, and bus angles.
<code>save_carbon_results_xlsx / carbon_result_to_json</code>	carbon-analysis workbook / JSON	export only for Excel, bidirectional for JSON	uses dedicated report-oriented structures, not the main system model.

9.2 Global Parsing and Workbook Rules

Rule	Current implementation behavior
JSON missing keys	Most JSON importers use <code>jget(..., default)</code> ; missing optional keys are silently defaulted. Whole arrays are optional as well.
JSON required keys	Identity and connectivity fields are usually mandatory: for example <code>index</code> , <code>bus</code> , <code>from_bus</code> , <code>to_bus</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>bus_ac</code> , and <code>bus_dc</code> .
JSON enums	Enum values are string-coded: for example <code>PQ/PV/SLACK</code> , <code>DC_V/DC_P</code> , <code>VDC_Q</code> , <code>Wind</code> , <code>FuelCell</code> , <code>Recloser</code> , <code>Islanded</code> .
Excel header row	All Excel tables use row 1 as the header row. Data rows start at row 2.
Excel row termination	Readers stop at the first row with an empty <code>index</code> cell. This means blank rows inside a table terminate import early.
Excel missing sheets	<code>load_xlsx</code> silently skips missing sheets; the corresponding component vector remains empty.
Excel boolean parser	<code>TRUE</code> , <code>T</code> , <code>YES</code> , <code>Y</code> , and <code>1</code> map to true; <code>FALSE</code> , <code>F</code> , <code>NO</code> , <code>N</code> , and <code>0</code> map to false.
Sheet precedence	For AC loads, <code>Load</code> is preferred; <code>ACLoad</code> is used only as a fallback. For DC loads, <code>DCLoadRaw</code> is preferred; <code>DCLoad</code> is only a fallback.
Bus-demand synchronization	After Excel import, <code>sync_bus_demands_from_load_tables(sys)</code> is called so the bus-level embedded demand reflects the explicit load tables.
Aggregated Excel views	<code>ACBus</code> and <code>DCBus</code> written by the Excel exporters are aggregated bus views, not always the raw struct contents; <code>ACLoad</code> and <code>DCLoad</code> are explicitly derived tables.
Empty result sheets in templates	<code>save_xlsx</code> currently pre-creates the <code>Results_*</code> sheets even though it does not fill them. A freshly exported template therefore contains empty result tabs.

9.3 Hybrid System JSON Contract

Top-level JSON object

JSON path	Value type	Current meaning
<code>name</code>	scalar string	top-level <code>HybridPowerSystem::name</code> .
<code>base_mva</code>	scalar number	top-level system base.
<code>ac</code>	object	AC subsystem object with its own name, base, frequency, and AC component arrays.
<code>dc</code>	object	DC subsystem object with its own name, base, and DC component arrays.
<code>vsc_converters</code>	array	top-level VSC array.
<code>dc.dcdc_converters</code>	array	exported DC/DC converter array nested inside the DC subsystem; import also accepts legacy top-level <code>dcdc_converters</code> .

JSON path	Value type	Current meaning
<code>energy_routers</code>	array	top-level multi-port router array; ports are nested inside each router object.
<code>mobile_storage</code>	array	top-level mobile-storage array.
<code>vpps</code>	array	top-level virtual power plant array.
<code>microgrids</code>	array	top-level microgrid array.
<code>three_phase_ac</code>	object	optional unbalanced AC subsystem with its own base, frequency, and three-phase component arrays.

Current JSON caveats

Public model area	Current JSON status
Standalone energy-router port array	not used; ports appear only as nested objects under each <code>energy_routers[i].ports</code> .
Load short-circuit sub-fields	the embedded SC motor-fraction fields <code>sn_mva</code> , <code>motor_percent</code> , <code>x_sub_pu</code> , and <code>r_sc_pu</code> are still not part of the main <code>ac.loads[]</code> JSON contract.
Excel API names	the public header exposes <code>load_excel/save_excel</code> , but the optional source file currently defines <code>load_xlsx/save_xlsx</code> and additional workbook result helpers that are not public-header declarations.
DC bus/branch struct fields	<code>DCBus</code> and <code>DCBranch</code> have more C++ struct fields than the current JSON writer exports; see Section 02b_io_field_dictionaries.

AC-network and demand JSON objects

Model	JSON path	Required keys on import	Other exported keys
ACBus	<code>ac.buses[]</code>	index	<code>bus_type</code> , <code>pd_mw</code> , <code>qd_mvar</code> , <code>vm_pu</code> , <code>va_deg</code> , <code>area</code> , <code>base_kv</code> , <code>vmax_pu</code> , <code>vmin_pu</code> , <code>gs_mw</code> , <code>bs_mvar</code> , <code>zone</code> , <code>in_service</code> , <code>name</code> , <code>n_customers</code> , <code>importance</code> , <code>latitude</code> , <code>longitude</code>
ACBranch	<code>ac.branches[]</code>	index, <code>from_bus</code> , <code>to_bus</code>	<code>r_pu</code> , <code>x_pu</code> , <code>b_pu</code> , <code>tap</code> , <code>shift_deg</code> , <code>rate_a_mva</code> , <code>rate_b_mva</code> , <code>rate_c_mva</code> , <code>in_service</code> , <code>name</code> , <code>length_km</code> , <code>failure_rate</code> , <code>mttr_hr</code>
Load	<code>ac.loads[]</code>	index, bus	<code>in_service</code> , <code>name</code> , <code>p_mw</code> , <code>q_mvar</code> , <code>model</code> , <code>z_percent_p</code> , <code>i_percent_p</code> , <code>p_percent_p</code> , <code>z_percent_q</code> , <code>i_percent_q</code> , <code>p_percent_q</code> , <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> , <code>priority</code> , <code>n_customers</code> , <code>profile_id</code>

Model	JSON path	Required keys on import	Other exported keys
FlexibleLoad	ac.flexible_loads[]	index, bus	in_service, name, p_mw, q_mvar, flex_up_mw, flex_down_mw, flex_duration_h, response_time_s, ramp_rate_mw_min, availability_pct, controllable, priority, control_area
AsymmetricLoad	ac.asymmetric_loads[]	index, bus	in_service, name, connection, grounded, pa_rated_mw, qa_rated_mvar, pb_rated_mw, qb_rated_mvar, pc_rated_mw, qc_rated_mvar, pa_mw, qa_mvar, pb_mw, qb_mvar, pc_mw, qc_mvar, scaling, constant_z_percent, constant_i_percent, constant_p_percent, controllable, priority
Shunt	ac.shunts[]	index, bus	in_service, name, gs_mw, bs_mvar, switchable, n_steps, current_step, bs_per_step
Switch	ac.switches[]	index, bus_from, bus_to	name, in_service, switch_type, closed, r_contact_ohm, z_ohm, i_rated_ka, i_breaking_ka, element_type, element_id, is_remote, is_automated, t_operation_s, p_sw_fail, mtbf_hours, mttr_hours
CircuitBreaker	ac.circuit_breakers[]	index, bus_from, bus_to	name, in_service, breaker_type, closed, z_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id
ChargingStation	ac.charging_stations[]	index, bus	name, location, in_service, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, utilization_rate, p_total_kw, q_total_kvar, mtbf_hours, mttr_hours
Charger	ac.chargers[]	index	name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mttr_hours
Asynchronous Motor	ac.motors[]	index, bus	name, in_service, machine ratings, direct / zero-sequence parameters, locked-rotor data, and efficiency metadata
ExternalGrid	ac.external_grid_s[]	index, bus	name, in_service, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, r_pu, x_pu, r0_pu, x0_pu, vn_kv, controllable
Transformer2W	ac.transformers_2w[]	index, hv_bus, lv_bus	name, std_type, in_service, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vk_r_percent, pk_kw, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, z0_percent, x0_r0, mtbf_hours, mttr_hours

Model	JSON path	Required keys on import	Other exported keys
Transformer3W	ac.transformers_3w[]	index, hv_bus, mv_bus, lv_bus	name, std_type, in_service, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mttr_hours

Generation, storage, and hybrid JSON objects

Model	JSON path	Required keys on import	Other exported keys
Generator	ac.generators[]	index, bus	in_service, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, cost_c2, cost_c1, cost_c0, name, fuel_type, startup_cost, shutdown_cost, min_up_time_hr, min_dn_time_hr, ramp_up_mw_min, ramp_dn_mw_min, mbase_mva, inertia_h, droop_r, xd_pu, xdp_pu, xdpp_pu, td0p_s, td0pp_s, emission_factor_tco2_mwh, nox_factor_kg_mwh, so2_factor_kg_mwh, forced_outage_rate, mttr_hr
StaticGenerator	ac.static_generators[] or dc.static_generators[]	index, bus	in_service, name, sgen_type, p_mw, q_mvar, p_rated_mw, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, scaling, controllable, v_ref_pu, k_p, k_q, k, rx, co2_emission_rate, mtbf_hours, mttr_hours
RenewableGen	ac.renewable_gen_s[]	index, bus	in_service, name, type, p_mw, q_mvar, p_rated_mw, qmax_mvar, qmin_mvar, curtailable, cost_curtail_mwh, capacity_factor, profile_id, emission_offset_tco2_mwh
PVSystem	ac.pv_systems[]	index, bus	in_service, name, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, controllable, v_ac_set_pu, v_dc_set_pu, inverter_eff, loss_percent, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, irradiance, temperature, mtbf_panel_hours, mttr_panel_hours, mtbf_inverter_hours, mttr_inverter_hours

Model	JSON path	Required keys on import	Other exported keys
Storage	ac.storage[] or dc.storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, soc_carbon_intensity_tco2_mwh, eta_charge, eta_discharge, self_discharge_pct, max_cycles, current_cycles, soh, replacement_cost, forced_outage_rate, mttr_hr
DCBus	dc.buses[]	index	bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service, name, latitude, longitude
DCBranch	dc.branches[]	index, from_bus, to_bus	r_pu, rate_a_mva, in_service, name, length_km
VSCConverter	vsc_converters[]	index, bus_ac, bus_dc	in_service, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, loss_mw, k_vdc, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, name, forced_outage_rate, mttr_hr
StaticGeneratorDC	dc.dc_static_generators[]	index, bus	in_service, name, type, p_set_mw, scaling, profile_id, pmax_mw, pmin_mw, controllable, mtbf_hours, mttr_hours, t_scheduled_hr
PVArrayDC	dc.pv_arrays[]	index, bus	in_service, name, p_set_mw, profile_id, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, temperature, irradiance, mtbf_hours, mttr_hours, t_scheduled_hr
DCCircuitBreaker	dc.dc_circuit_breakers[]	index, bus_from, bus_to	name, in_service, breaker_type, closed, r_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id
DCDCConverter	dc.dcdc_converters[]	index, bus_in, bus_out	in_service, name, control_mode, p_ref_mw, v_ref_pu, sn_mva, vn_in_kv, vn_out_kv, eta, r_eq_pu, pmax_mw, pmin_mw, k_droop, mtbf_hours, mttr_hours
EnergyRouterPort	energy_routers[].ports[]	index, bus	name, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service
EnergyRouter	energy_routers[]	index	name, in_service, router_type, num_ports, ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mttr_hours

Model	JSON path	Required keys on import	Other exported keys
MobileStorage	mobile_storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, max_cycles, current_cycles, soh, is_mobile, status, current_location, target_bus, e_consumption_mwh_km, max_travel_distance_km, mtbf_hours, mttr_hours
VirtualPower Plant	vpps[]	index	name, description, pcc_bus, resource-count fields, aggregate capability / output terms, ramp limits, reliability metadata, and in_service; import also accepts legacy aggregation_bus
Microgrid	microgrids[]	index	name, description, in_service, pcc_bus, internal_buses, operating_mode, islanding_capability, auto_reconnection, p_exchange_max_mw, p_exchange_min_mw, p_import_max_mw, p_export_max_mw, p_exchange_mw, total_generation_mw, total_storage_mwh, total_load_mw, capacity_mw, peak_load_mw, f_set_hz, v_set_pu, k_droop, area, mtbf_hours, mttr_hours
ThreePhase ACSystem	three_phase_ac	none	name, base_mva, base_freq_hz, and component arrays for buses, lines, transformers, loads, generators, and external grids
ThreePhaseAC Bus	three_phase_ac.buses[]	index	bus_type, name, base_kv, in_service, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone
ThreePhaseAC Line	three_phase_ac.lines[]	index, from_bus, to_bus	name, in_service, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mttr_hr
ThreePhase Transformer	three_phase_ac.transformers[]	index, hv_bus, lv_bus	name, in_service, nameplate data, leakage and magnetizing terms, zero-sequence data, tap settings, vector group, and reliability metadata
ThreePhaseLoad	three_phase_ac.loads[]	index, bus	name, in_service, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio

Model	JSON path	Required keys on import	Other exported keys
ThreePhase Generator	three_phase_ac.generators[]	index, bus	name, in_service, is_slack, dispatch setpoints / limits, voltage target, machine base, and sequence reactance terms
ThreePhase ExternalGrid	three_phase_ac.external_grids[]	index, bus	name, in_service, voltage target, short-circuit level bounds, R/X bounds, and sequence Thevenin terms

Result JSON objects

JSON artifact	Top-level keys	Current note
PowerFlowResult JSON	converged, iterations, residual, ac_buses, dc_buses, ac_branch_flows, dc_branch_flows, vsc_transfers, dcdc_transfers	export-only helper writes solved bus states, AC/DC branch flow and loading fields, and converter transfer reports.
ACOPFResult JSON	converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status, solver_path, vm, va, vdc, pg_mw, qg_mvar, dpd_mw, dqd_mvar, pac_mw, qac_mvar, enhanced dispatch objects, legacy dispatch arrays, profiling	enhanced dispatch objects are renewable_dispatch, storage_dispatch, dcdc_dispatch, and flex_dispatch; each contains values plus a map of original_index/source_type.
DCOPFResult JSON	converged, iterations, objective, status, solver_name, runtime_sec, solver_chain, objective_model, total_load_shedding_mw, load_shedding_mw, lmp, branch_mu_lower, branch_mu_upper, branch_mu_valid, pg_mw, pf_mw, va	bidirectional helper preserves the current DCOPF audit and primal result fields; branch congestion duals must only be interpreted when branch_mu_valid=true.
CarbonAnalysisResult JSON	tracing_verified, matrix_solved, matrix_residual, load_carbon, dc_load_carbon, branch_carbon, dc_branch_carbon, bus_carbon, dc_bus_carbon, vsc_carbon, dcdc_carbon, storage_carbon, energy_router_carbon, generator_loss_allocation, tracing_summary, matrix_summary	AC/DC load arrays carry the same scalar fields plus nested generator_supply_mw; branch / converter / router arrays expose loss and direction fields; storage_carbon adds charge/discharge-side storage semantics including traced charging intensity.

9.4 Excel Input Workbook Contract

Input-sheet catalog

Sheet	Primary model / load target	Columns	Current note
Info	global metadata	Key, Value	loader recognizes name, base_mva, ac_freq_hz, three_ phase_name, thre e_phase_base_mva, and three_phase_ base_freq_hz.
ACBus	ACBus	index, name, bus_type, pd_mw, qd_mvar, vm_pu, va_deg, base_kv, vmin_pu, vmax_pu, gs_mw, bs _mvar, area, zone, in_service, n_customers, importance, latitude, longitude	write path uses aggregated AC demand, not neces- sarily the raw bus fields.
ACBranch	ACBranch	index, name, from_bus, to_bus, r_pu, x_pu, b_pu, tap, shift_deg, rate_a_mva, rate_b _mva, rate_c_mva, length_km, in_service, failure_rate, mttr_hr	near-1:1 with JSON coverage.
Generator	Generator	index, name, bus, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, fuel_type, mbase_mva, cost_c2, cost_c1, cost_c0, in_service, inertia_h, droop_r, ramp_up_mw_min, ramp_dn_mw_min	Excel omits the deeper machine, emissions, and com- mitment fields that JSON keeps.
Load	Load	index, name, bus, p_mw, q_mvar, model, z_ percent_p, i_percent_p, p_percent_p, z_percent_q, i_percent_q, p_percent_q, co ntrollable, priority, cost_mw, in_service	raw explicit AC load table.
ACLoad	fallback Load import; aggre- gated export view	same columns as Load	written as a derived table that aggre- gates Load, charg- ing stations, charg- ing storage, VPP imports, microgrid imports, and AC router ports.
FlexibleLoad	FlexibleLoad	index, name, bus, p_mw, q_mvar, flex_up_mw, flex_down_mw, flex_duration_h, response_t ime_s, ramp_rate_mw_min, availability_pct, controllable, priority, control_area, in_service	dedicated Excel input sheet now ex- ists; these rows are converted to canoni- cal Load equivalents before PF and also have parity OPF variables.
AsymmetricLoad	AsymmetricLoad	index, name, bus, connection, grounded, pa_rated_mw, qa_rated_mvar, pb_rated_mw, qb_rated_mvar, pc_rated_mw, qc_rated_mvar, pa_mw, qa_mvar, pb_mw, qb_mvar, pc_mw, qc_mvar, scaling, const_z_percent, const_ i_percent, const_p_percent, controllable, priority, in_service	dedicated Excel input sheet now ex- ists; solver-side use still relies on pro- jection to a single Load equivalent.
StaticGenerator	AC StaticGene rator	index, name, bus, sgen_type, p_mw, q_mvar, p_rated_mw, sn_mva, pmax_mw, pmin_mw, qmax _mvar, qmin_mvar, scaling, controllable, in_service	Excel omits control, SC, emissions, and reliability meta- data.

Sheet	Primary model / load target	Columns	Current note
Storage	AC Storage	index, name, bus, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, in_service	Excel omits life-cycle and outage fields.
RenewableGen	RenewableGen	index, name, bus, type, p_mw, q_mvar, p_rate_d_mw, qmax_mvar, qmin_mvar, curtailable, capacity_factor, in_service	Excel omits cost_curtail_mwh, profile_id, and emission-offset metadata.
PVSystem	PVSystem	index, name, bus, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, inverter_eff, in_service	Excel keeps only a narrow operational subset; JSON is much richer.
Shunt	Shunt	index, name, bus, gs_mw, bs_mvar, switchable, n_steps, current_step, bs_per_step, in_service	near-1:1 with JSON coverage.
ExternalGrid	ExternalGrid	index, name, bus, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, controllable, in_service	Excel omits explicit Thevenin impedance fields present in JSON.
Transformer2W	Transformer2W	index, name, hv_bus, lv_bus, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vkr_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, in_service	Excel omits std_type, pk_kw, zero-sequence, and reliability fields.
Transformer3W	Transformer3W	index, name, std_type, hv_bus, mv_bus, lv_bus, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mttr_hours, in_service	dedicated Excel input sheet now exists; solver-side treatment still projects the device into equivalent branches.
ChargingStation	ChargingStation	index, name, bus, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, p_total_kw, q_total_kvar, in_service	Excel omits location, utilization_rate, and reliability fields.
Charger	Charger	index, name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mttr_hours	dedicated Excel input sheet now exists; charger rows are linked to their parent station by station_id.
Asynchronous Motor	Asynchronous Motor	index, name, bus, machine ratings, direct / zero-sequence parameters, SC motor data, and in_service	dedicated Excel input sheet now exists for short-circuit motor data.
Switch	Switch	index, name, bus_from, bus_to, switch_type, closed, in_service	Excel keeps only the topology/status subset.
CircuitBreaker	CircuitBreaker	index, name, bus_from, bus_to, breaker_type, closed, z_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id, in_service	dedicated AC breaker sheet.
DCBus	DCBus	index, name, bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service	write path uses aggregated DC demand.

Sheet	Primary model / load target	Columns	Current note
DCLoad	fallback DCLoad import; aggregated export view	index, name, bus, p_mw, controllable, p_min_mw, cost_mw, in_service	written as derived aggregated DC demand.
DCLoadRaw	raw DCLoad table	same columns as DCLoad	preserves the unaggregated DC load objects and is preferred on import.
DCBranch	DCBranch	index, name, from_bus, to_bus, r_pu, rate_a_mva, length_km, in_service	near-1:1 with JSON coverage.
DCStorage	DC Storage	same columns as Storage	same coverage limits as AC storage.
DCStaticGen	DC StaticGenerator	same columns as StaticGenerator	same coverage limits as AC static generators.
DCStaticGenerator	StaticGeneratorDC	index, name, bus, type, p_set_mw, scaling, profile_id, pmax_mw, pmin_mw, controllable, mtbf_hr, mtrr_hr, t_scheduled_hr, in_service	preferred explicit typed DC source sheet; importer also accepts legacy DCStaticGeneratorDC.
DCPVArray	PVArrayDC	index, name, bus, p_set_mw, profile_id, num_series, num_parallel, vmpp, impv, voc, isc, alpha_isc, beta_voc, temperature, irradiance, mtbf_hr, mtrr_hr, t_scheduled_hr, in_service	typed DC PV array sheet.
DCCircuitBreaker	DCCircuitBreaker	index, name, bus_from, bus_to, breaker_type, closed, r_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id, in_service	dedicated DC breaker sheet.
VSCConverter	VSCConverter	index, name, bus_ac, bus_dc, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, in_service	Excel omits loss_mw, k_vdc, and outage metadata.
DCDCConverter	DCDCConverter	index, name, bus_in, bus_out, control_mode, p_ref_mw, v_ref_pu, sn_mva, eta, pmax_mw, pmin_mw, in_service	Excel omits voltage-rating, droop, resistance, and reliability fields.
EnergyRouter	EnergyRouter	index, name, router_type, num_ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mtrr_hours, in_service	router-level metadata now has a dedicated Excel sheet.
EnergyRouterPort	EnergyRouterPort	router_index, index, name, bus, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service	port rows are matched back to their parent router through router_index.
VPP	VirtualPowerPlant	index, name, pcc_bus, aggregate output / limit snapshot, selected resource counts, and in_service	Excel writes pcc_bus from aggregation_bus; it omits description, EV/load counts, aggregate capability, ramp rates, and reliability.

Sheet	Primary model / load target	Columns	Current note
Microgrid	Microgrid	index, name, pcc_bus, operating_mode, islanding_capability, p_exchange_mw, p_exchange_max_mw, total_generation_mw, total_load_mw, in_service	Excel omits internal_buses and most planning/control metadata.
MobileStorage	MobileStorage	index, name, bus, status, p_mw, q_mvar, p_rated_mw, e_rated_mwh, soc_init, in_service	Excel keeps only a snapshot subset and omits most mobility/limit metadata.
ThreePhaseBus	ThreePhaseAC Bus	index, name, bus_type, base_kv, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone, in_service	explicit unbalanced bus table for the optional three_phase_ac subsystem.
ThreePhaseLine	ThreePhaseAC Line	index, name, from_bus, to_bus, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mttr_hr, in_service	sequence-parameter line sheet for explicit unbalanced input.
ThreePhase Transformer	ThreePhase Transformer	index, name, buses, nameplate data, leakage / zero-sequence terms, tap settings, reliability metadata, and in_service	explicit three-phase transformer input sheet; solver path still projects it to Transformer2W .
ThreePhaseLoad	ThreePhaseLoad	index, name, bus, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio, in_service	explicit unbalanced demand table; projected before main PF/OPF assembly.
ThreePhase Generator	ThreePhase Generator	index, name, bus, slack flag, dispatch setpoints / limits, sequence reactance terms, and in_service	explicit unbalanced generator sheet with sequence impedance metadata.
ThreePhase ExternalGrid	ThreePhase ExternalGrid	index, name, bus, voltage target, SC level bounds, sequence impedance terms, and in_service	explicit three-phase grid-equivalent sheet.

Remaining field-level Excel caveats

Area	Current Excel status
Load short-circuit motor subfields	the embedded SC fields sn_mva , motor_percent , x_sub_pu , and r_sc_pu still do not have dedicated Load/ACLoad workbook columns.
ThreePhaseACSystem metadata container	subsystem metadata is carried in Info keys rather than a dedicated ThreePhaseInfo sheet.

JSON Key / Excel Column Dictionary for the Newly Added Contracts

The next tables are intentionally 1:1 with the current serializer code. The “JSON default” text below means the value used by `from_json` when a key is absent. Excel uses the exact sheet and column names shown here.

Optional three-phase subsystem metadata: `three_phase_ac` and `Info`

Field	Excel sheet / column	JSON default / import rule	Meaning
<code>name</code>	<code>Info[three_phase_name]</code>	default "Three-Phase ACSysstem"	user-visible subsystem name for the optional explicit unbalanced AC network
<code>base_mva</code>	<code>Info[three_phase_base_mva]</code>	default 100.0	per-subsystem power base; independent from top-level <code>base_mva</code>
<code>base_freq_hz</code>	<code>Info[three_phase_base_freq_hz]</code>	default 50.0	nominal subsystem frequency in Hz
<code>buses[]</code>	sheet <code>ThreePhaseBus</code>	missing array / sheet keeps vector empty	explicit three-phase bus family
<code>lines[]</code>	sheet <code>ThreePhaseLine</code>	missing array / sheet keeps vector empty	explicit three-phase line family
<code>transformers[]</code>	sheet <code>ThreePhaseTransformer</code>	missing array / sheet keeps vector empty	explicit three-phase transformer family
<code>loads[]</code>	sheet <code>ThreePhaseLoad</code>	missing array / sheet keeps vector empty	explicit unbalanced load family
<code>generators[]</code>	sheet <code>ThreePhaseGenerator</code>	missing array / sheet keeps vector empty	explicit three-phase generator family
<code>external_grids[]</code>	sheet <code>ThreePhaseExternalGrid</code>	missing array / sheet keeps vector empty	explicit sequence-based grid-equivalent family

Asynchronous motor contract: `ac.motors[]` and sheet `AsynchronousMotor`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable device identifier
<code>bus</code>	<code>bus</code>	required on JSON import	connected AC bus index
<code>name</code>	<code>name</code>	default empty string	display label only
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag; not a solved PF state
<code>vn_kv</code>	<code>vn_kv</code>	default 0.0	rated terminal voltage in kV
<code>sn_mva</code>	<code>sn_mva</code>	default 0.0	rated apparent-power metadata for motor fault-contribution models
<code>r_pu</code>	<code>r_pu</code>	default 0.0	motor equivalent positive-sequence resistance input
<code>x_pu</code>	<code>x_pu</code>	default 0.0	motor equivalent positive-sequence reactance input
<code>x_r</code>	<code>x_r</code>	default 0.0	X/R ratio metadata for short-circuit-style motor models
<code>lrc</code>	<code>lrc</code>	default 0.0	locked-rotor current ratio
<code>poles</code>	<code>poles</code>	default 2	machine pole count
<code>cos_phi</code>	<code>cos_phi</code>	default 0.85	rated power factor

Field	Excel column	JSON default / import rule	Meaning
efficiency	efficiency	default 0.95	rated efficiency factor
r0_pu	r0_pu	default 0.0	zero-sequence resistance input for unsymmetrical faults
x0_pu	x0_pu	default 0.0	zero-sequence reactance input for unsymmetrical faults

Three-phase bus contract: `three_phase_ac.buses[]` and sheet `ThreePhaseBus`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable bus identifier
name	name	default empty string	bus label
bus_type	bus_type	default PQ	bus category using the same string tokens as the AC bus model
base_kv	base_kv	default 0.0	nominal line-to-line bus voltage base
in_service	in_service	default true	availability flag
vm_a_pu, vm_b_pu, vm_c_pu	same columns	default 1.0 each	per-phase initial or specified voltage magnitudes
va_a_deg, va_b_deg, va_c_deg	same columns	defaults 0.0, -120.0, 120.0	phase-angle initialization in degrees for phase order a, b, c
pd_a_mw, pd_b_mw, pd_c_mw	same columns	default 0.0	explicit per-phase active demand before projection
qd_a_mvar, qd_b_mvar, qd_c_mvar	same columns	default 0.0	explicit per-phase reactive demand before projection
gs_a_mw, gs_b_mw, gs_c_mw	same columns	default 0.0	per-phase conductance shunt terms
bs_a_mvar, bs_b_mvar, bs_c_mvar	same columns	default 0.0	per-phase susceptance shunt terms
vmin_pu	vmin_pu	default 0.9	lower per-unit voltage bound
vmax_pu	vmax_pu	default 1.1	upper per-unit voltage bound
area	area	default 1	planning / reporting area tag
zone	zone	default 1	planning / reporting zone tag

Three-phase line contract: `three_phase_ac.lines[]` and sheet `ThreePhaseLine`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable line identifier
from_bus, to_bus	same columns	required on JSON import	line terminal bus indices

Field	Excel column	JSON default / import rule	Meaning
<code>name</code>	<code>name</code>	default empty string	line label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>length_km</code>	<code>length_km</code>	default 0.0	physical route length
<code>parallel</code>	<code>parallel</code>	default 1	number of parallel circuits represented by the row
<code>r1_ohm_per_km</code> , <code>x1_ohm_per_km</code> , <code>c1_nf_per_km</code>	same columns	default 0.0	positive-sequence per-km impedance / capacitance data
<code>r0_ohm_per_km</code> , <code>x0_ohm_per_km</code> , <code>c0_nf_per_km</code>	same columns	default 0.0	zero-sequence per-km impedance / capacitance data
<code>r1_pu</code> , <code>x1_pu</code> , <code>b1_pu</code>	same columns	default 0.0	positive-sequence per-unit branch parameters
<code>r0_pu</code> , <code>x0_pu</code> , <code>b0_pu</code>	same columns	default 0.0	zero-sequence per-unit branch parameters
<code>max_i_ka</code>	<code>max_i_ka</code>	default 0.0	thermal current limit in kA
<code>rate_a_mva</code>	<code>rate_a_mva</code>	default 0.0	apparent-power rating used by planning / screening logic
<code>failure_rate</code>	<code>failure_rate</code>	default 0.0	reliability failure-rate input
<code>mttr_hr</code>	<code>mttr_hr</code>	default 0.0	repair-time input in hours

Three-phase transformer contract: `three_phase_ac.transformers[]` and sheet `ThreePhaseTransformer`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable transformer identifier
<code>hv_bus</code> , <code>lv_bus</code>	same columns	required on JSON import	high-voltage and low-voltage terminal bus indices
<code>name</code>	<code>name</code>	default empty string	transformer label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>sn_mva</code>	<code>sn_mva</code>	default 0.0	rated apparent power
<code>vn_hv_kv</code> , <code>vn_lv_kv</code>	same columns	default 0.0	rated winding voltages
<code>vk_percent</code> , <code>vkr_percent</code>	same columns	default 0.0	short-circuit magnitude and resistive share in percent
<code>pfe_kw</code> , <code>i0_percent</code>	same columns	default 0.0	no-load loss and excitation-current inputs
<code>vector_group</code>	<code>vector_group</code>	default empty string	winding connection / clock notation for zero-sequence path interpretation
<code>vk0_percent</code> , <code>vkr0_percent</code> , <code>mag0_percent</code> , <code>mag0_rx</code> , <code>si0_hv_partial</code>	same columns	defaults 0.0 except <code>si0_hv_partial=0.5</code>	zero-sequence transformer data preserved for SC and future unbalanced workflows
<code>tap_side</code> , <code>tap_pos</code> , <code>tap_min</code> , <code>tap_max</code> , <code>tap_neutral</code>	same columns	default 0	tap-changer placement and discrete tap position meta-data
<code>tap_step_percent</code>	<code>tap_step_percent</code>	default 0.0	tap step size in percent
<code>shift_deg</code>	<code>shift_deg</code>	default 0.0	phase shift angle in degrees
<code>mtbf_hours</code> , <code>mttr_hours</code>	same columns	default 0.0	reliability inputs retained in the explicit contract

Three-phase load contract: `three_phase_ac.loads[]` and sheet `ThreePhaseLoad`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable load identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	load label
in_service	in_service	default true	availability flag
connection	connection	default "wye"	wiring model; current code expects free-form connection text such as wye or delta
grounded	grounded	default true	grounding-path assumption for the connection
p_a_mw, p_b_mw, p_c_mw	same columns	default 0.0	per-phase active demand
q_a_mvar, q_b_mvar, q_c_mvar	same columns	default 0.0	per-phase reactive demand
const_z_percent, const_i_percent	same columns	default 0.0	ZIP-model constant-impedance and constant-current shares
const_p_percent	const_p_percent	default 100.0	ZIP-model constant-power share
motor_percent	motor_percent	default 0.0	embedded motor-fraction share inside the unbalanced load object
lrc_pu	lrc_pu	default 0.0	locked-rotor current ratio for the embedded motor fraction
x_r_ratio	x_r_ratio	default 0.0	X/R ratio for the embedded motor fraction

Three-phase generator contract: `three_phase_ac.generators[]` and sheet `ThreePhaseGenerator`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable generator identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	generator label
in_service	in_service	default true	availability flag
is_slack	is_slack	default false	slack-machine indicator for explicit three-phase studies
p_mw, q_mvar	same columns	default 0.0	current active / reactive injection setpoints
vm_pu	vm_pu	default 1.0	terminal voltage target or initialization
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	capability limits
mbase_mva	mbase_mva	default 0.0	machine MVA base for the sequence reactances

Field	Excel column	JSON default / import rule	Meaning
<code>xd_pu</code> , <code>xdpp_pu</code> , <code>x2_pu</code> , <code>x0_pu</code> , <code>r0_pu</code>	same columns	default 0.0	positive / subtransient / negative / zero-sequence machine impedance set

Three-phase external-grid contract: `three_phase_ac.external_grids[]` and sheet `ThreePhaseExternalGrid`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable external-grid identifier
<code>bus</code>	<code>bus</code>	required on JSON import	connected three-phase bus index
<code>name</code>	<code>name</code>	default empty string	source label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>vm_pu</code>	<code>vm_pu</code>	default 1.0	Thevenin voltage magnitude target
<code>va_deg</code>	<code>va_deg</code>	default 0.0	Thevenin voltage angle in degrees
<code>s_sc_max_mva</code> , <code>s_sc_min_mva</code>	same columns	default 0.0	short-circuit strength limits
<code>rx_max</code> , <code>rx_min</code>	same columns	default 0.0	R/X ratio bounds used in equivalent-source parametrization
<code>r1_pu</code> , <code>x1_pu</code>	same columns	default 0.0	positive-sequence Thevenin impedance
<code>r2_pu</code> , <code>x2_pu</code>	same columns	default 0.0	negative-sequence Thevenin impedance
<code>r0_pu</code> , <code>x0_pu</code>	same columns	default 0.0	zero-sequence Thevenin impedance

9.5 Compact Power-Flow Workbook Additions

Sheet updated by <code>save_results_compact.xlsx</code>	Appended columns	Current note
<code>ACBus</code>	<code>result_vm_pu</code> , <code>result_va_deg</code>	solved AC bus state.
<code>DCBus</code>	<code>result_vdc_pu</code>	solved DC voltage.
<code>ACBranch</code>	<code>result_pf_mw</code> , <code>result_qf_mvar</code> , <code>result_pt_mw</code> , <code>result_qt_mvar</code>	branch terminal powers.
<code>DCBranch</code>	<code>result_v_from_pu</code> , <code>result_v_to_pu</code> , <code>result_i_pu</code> , <code>result_p_from_mw</code> , <code>result_p_to_mw</code> , <code>result_loss_mw</code>	derived from solved DC bus voltages and branch resistance.
<code>Storage</code>	<code>result_vm_pu</code> , <code>result_va_deg</code>	bus-state annotation only; storage dispatch is not recomputed.
<code>DCStorage</code>	<code>result_vdc_pu</code>	bus-state annotation only.

Sheet updated by save_results_c ompact_xlsx	Appended columns	Current note
Load	result_vm_pu, result_va_deg	bus-state annotation on the raw AC load table.
ACLoad	result_vm_pu, result_va_deg	bus-state annotation on the aggregated AC load view.
DCLoad	result_vdc_pu	bus-state annotation on the aggregated DC load view.
ChargingStation	result_vm_pu, result_va_deg	station bus state.
VSCConverter	result_p_ac_mw, result_q_ac_mvar, result_p_dc_mw, result_loss_mw, result_v_ac_pu, result_v_dc_pu	solved converter transfer and terminal voltages.
DCDCConverter	result_p_in_mw, result_p_out_mw, result_loss_mw, result_v_in_pu, result_v_out_pu	solved converter transfer and terminal voltages.
VPP	result_vm_pu, result_va_deg	aggregation-bus annotation only.
Microgrid	result_vm_pu, result_va_deg	PCC-bus annotation only.
MobileStorage	result_vm_pu, result_va_deg	current bus annotation only.
Results_Summary	Key, Value	key-value sheet with converged, iterations, and residual.

9.6 Dedicated Result Workbooks

Dedicated power-flow result sheets

Sheet	Columns	Current note
Results_ACBus	index, name, vm_pu, va_deg	solved AC bus state.
Results_DCBus	index, name, vm_pu	this sheet uses the AC-style column name vm_pu even though the value is the DC solved voltage.
Results_Branch Flow	index, from_bus, to_bus, pf_mw, qf_mvar, pt_mw, qt_mvar	branch flows reported in branch order.
Results_DCBranch	index, name, from_bus, to_bus, in_service, v_from_pu, v_to_pu, i_pu, p_from_mw, p_to_mw, loss_mw	computed from solved voltages and branch resistance.

Sheet	Columns	Current note
Results_VSCConverter	index, name, bus_ac, bus_dc, in_service, control_mode, p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw, p_set_mw, q_set_mvar, v_ac_pu, v_dc_pu, eta, loss_percent	includes both solved transfer and original setpoint metadata.
Results_DCDCConverter	index, name, bus_in, bus_out, in_service, control_mode, p_ref_mw, v_ref_pu, v_in_pu, v_out_pu, eta, p_in_mw, p_out_mw, loss_mw	includes both solved transfer and original reference metadata.
Results_Storage_AC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vm_pu, va_deg	uses soc_init as the reported soc.
Results_Storage_DC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vdc_pu	uses soc_init as the reported soc.
Results_ChargingStation	index, name, bus, in_service, num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate, vm_pu, va_deg	station bus state plus current aggregate demand fields.
Results_Charger	index, name, station_id, station_bus, in_service, charger_type, p_rated_kw, p_ch_max_kw, p_dis_max_kw, eta, v2g_capable, vm_pu, va_deg	charger_type now uses the same string token family as JSON and workbook input sheets.
Results_EnergyRouter	index, name, in_service, router_type, num_ports, p_rated_mw, loss_percent, p_port_sum_mw, q_port_sum_mvar	device-level summary only; solved per-port values live in the port sheet.
Results_EnergyRouterPort	router_index, port_index, port_name, bus, port_type, control_mode, in_service, p_mw, q_mvar, v_solved_pu	flattens nested router ports into one table.
Results_VPP	index, name, aggregation_bus, in_service, p_output_mw, q_output_mvar, pmax_mw, pmin_mw, vm_pu, va_deg	aggregation-bus annotation only.
Results_Microgrid	index, name, pcc_bus, operating_mode, in_service, p_exchange_mw, total_generation_mw, total_load_mw, vm_pu, va_deg, internal_buses	internal_buses is written as one comma-separated string.
Results_MobileStorage	index, name, bus, status, in_service, p_mw, q_mvar, soc, target_bus, vm_pu, va_deg	uses soc_init as the reported soc.
Results_Summary	Key, Value	keys are converged, iterations, and residual.

Result-Sheet Enum and Flag Conventions

Artifact	Field	Output representation	Field note
Results_VSCConverter	control_mode	string token: PQ, VDC_Q, VDC_VAC	copied from the input converter mode; not a solver-selected mode switch
Results_DCDCConverter	control_mode	string token: Voltage, Power, Droop	copied from the input converter mode
Results_Charger	charger_type	string token: AC_L1, AC_L2, DC_Fast	now aligned with JSON and workbook input instead of numeric enum codes
Results_EnergyRouterPort	port_type	string token: AC or DC	physical port side classification
Results_EnergyRouterPort	control_mode	string token: PQ, VF, Droop	copied from the input port control mode
Results_Microgrid	operating_mode	string token: Grid Connected, Islanded, Transition	reported operating mode metadata, not a PF state variable
Results_MobileStorage	status	string token: Stationary, InTransit, Deployed	reported mobility state metadata
Results_Charger	v2g_capable	boolean token: TRUE / FALSE	static capability flag, not current dispatch direction
Results_* device sheets	in_service	boolean token: TRUE / FALSE	copied device availability flag across result workbooks
Results_Summary, OPF_Summary	converged	boolean token: TRUE / FALSE	solver convergence outcome

Dedicated OPF and carbon result sheets

Sheet	Columns	Current note
OPF_Bus	index, name, base_kv, vm_pu, va_deg, dpd_mw, dqd_mvar	bus voltages plus bus-level load shedding vectors.
OPF_Generator	index, name, bus, pg_mw, qg_mvar, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, cost_c2, cost_c1, cost_c0, fuel_type	generator dispatch report.
OPF_Converter	index, name, bus_ac, bus_dc, pac_mw, qac_mvar	only VSC converters are reported here.

Sheet	Columns	Current note
OPF_Summary	Key, Value	keys are converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, and status.
LoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 1.
DCLoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 2.
BusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 3.
DCBusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 4.
BranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 5.
DCBranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 6.
VSCCarbon	converter_index, bus_ac, bus_dc, ac_to_dc, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 7.
DCDCCarbon	converter_index, bus_in, bus_out, input_to_output, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 8.
StorageCarbon	storage_index, bus, is_dc, p_mw, soc, stored_energy_mwh, soc_carbon_intensity_tco2_mwh, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook sheet for explicit storage charge/discharge carbon semantics.
EnergyRouterCarbon	router_index, input_power_mw, output_power_mw, loss_mw, active_input_ports, active_output_ports, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook sheet for router-internal loss allocation reconstructed from port transfers.

Sheet	Columns	Current note
Summary	key-value blocks for tracing and matrix summaries	dedicated carbon workbook summary sheet; contains generation/load/loss totals, balance errors, and flags such as <code>tracing_verified</code> and <code>matrix_solved</code> .

9.7 Current I/O Coverage Gaps That Matter to Engineering Work

Public model area	JSON system file	Excel input workbook	Engineering consequence
ACBus, ACBranch, DCBus, DCBranch	yes	yes	core network objects are available in both I/O families.
Generator, Load, Shunt, ExternalGrid	yes	yes	Excel coverage is narrower than JSON for the richer machine / source metadata.
StaticGenerator, RenewableGen, PVSystem, Storage	yes	yes	Excel exposes only the operational subset; JSON is the better notebook-to-code contract for detailed studies.
Transformer2W	yes	yes	Excel omits zero-sequence and reliability fields.
Transformer3W	yes	yes	JSON and Excel now both carry the explicit 3-winding transformer data, although solver use still goes through projection.
FlexibleLoad, AsymmetricLoad	yes	yes	both I/O families now preserve the rich pre-projection load models.
ChargingStation	yes	yes	input support exists in both families.
Charger	yes	yes	both I/O families now preserve per-charger metadata; the parent station link remains <code>station_id</code> .
DCLoad	yes	yes	the main system JSON and Excel workbook are now aligned for the explicit DC load table.
VSCConverter, DCCDC Converter	yes	yes	Excel coverage is narrower than JSON.
EnergyRouter, EnergyRouterPort	yes	yes	router studies can now use either JSON or Excel input, with ports flattened into a dedicated Excel table.
VPP, Microgrid, MobileStorage	yes	yes	Excel again keeps only a reduced operational subset.
AsynchronousMotor	yes	yes	the SC motor model now has a stable external contract in both families.

Public model area	JSON system file	Excel input workbook	Engineering consequence
ThreePhaseACSystem family	yes	yes	the optional unbalanced subsystem can now round-trip through both JSON and Excel, although main PF/OPF/SC still consume it after projection.

9.8 Interpretation

The current I/O layer is already useful, but it is not a single canonical schema yet.

1. JSON is now the richest external representation for the full current `HybridPowerSystem` container, including `ac.motors` and optional `three_phase_ac`.
2. Excel is an engineering workbook interface with deliberate aggregation and reduced field coverage.
3. The JSON and Excel paths are now aligned for `DCLoad`, `Transformer3W`, `FlexibleLoad`, `AsymmetricLoad`, `Charger`, `EnergyRouter`, `AsynchronousMotor`, and the `ThreePhaseACSystem` family.
4. The remaining gaps are now field-level rather than family-level, especially the SC-specific motor-fraction fields embedded inside `Load` and the lack of dedicated result-workbook contracts for explicit three-phase solved states.

10 Detailed I/O Field Dictionaries

This section expands the per-field contract to the remaining public model families. Section 02a_io_contracts already gives detailed field maps for `AsynchronousMotor`, the optional `three_phase_ac` family, and the result-sheet enum conventions. The tables below complete the same JSON-key / Excel-column dictionary pattern for the other current input models.

When the “Excel sheet / column” entry says `none`, the engineering workbook does not preserve that JSON field directly. When two workbook sheet names are listed, the first is the preferred explicit input table and the second is a fallback or derived view with the same column family.

10.1 System Metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>name</code>	<code>Info[name]</code>	default "HybridAC/DC System"	top-level <code>HybridPowerSystem</code> display name
<code>base_mva</code>	<code>Info[base_mva]</code>	default 100.0	top-level system power base; workbook import also copies it into <code>ac.base_mva</code> and <code>dc.base_mva</code>
<code>ac.name</code>	<code>none</code>	default "ACSystem"	JSON-only AC subsystem label

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>ac.base_mva</code>	<code>none</code>	default 100.0	JSON-only AC subsystem base; workbook uses the shared top-level <code>base_mva</code> instead
<code>ac.freq_hz</code>	<code>Info[ac_freq_hz]</code>	default 50.0	nominal AC frequency in Hz
<code>dc.name</code>	<code>none</code>	default "DCSystem"	JSON-only DC subsystem label
<code>dc.base_mva</code>	<code>none</code>	default 100.0	JSON-only DC subsystem base; workbook again uses the shared top-level <code>base_mva</code>
<code>three_phase_ac.*</code>	<code>Info[three_phase_*]</code> plus dedicated sheets	see Section 02a_io_contracts	optional explicit unbalanced subsystem documented in the earlier detailed dictionary

10.2 AC Network and Demand Models

ACBus : `ac.buses[]` / sheet **ACBus**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>ACBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable bus identifier
<code>name</code>	<code>ACBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>ACBus.bus_type</code>	default PQ	bus category token PQ/PV/SLACK/ISOLATED
<code>pd_mw, qd_mvar</code>	same columns	default 0.0	embedded bus demand snapshot; workbook export may contain synchronized aggregated demand rather than original raw struct values
<code>vm_pu, va_deg</code>	same columns	defaults 1.0, 0.0	initial or specified bus voltage magnitude and angle
<code>base_kv</code>	<code>ACBus.base_kv</code>	default 110.0	nominal bus voltage base
<code>vmin_pu, vmax_pu</code>	same columns	defaults 0.9, 1.1	voltage operating bounds
<code>gs_mw, bs_mvar</code>	same columns	default 0.0	shunt conductance and susceptance embedded at the bus
<code>area, zone</code>	same columns	defaults 1, 1	planning / reporting partitions
<code>in_service</code>	<code>ACBus.in_service</code>	default true	availability flag
<code>n_customers</code>	<code>ACBus.n_customers</code>	default 0	customer-count metadata
<code>importance</code>	<code>ACBus.importance</code>	default 1.0	bus criticality / weighting metadata
<code>latitude, longitude</code>	same columns	default 0.0	geospatial metadata only

ACBranch : `ac.branches[]` / sheet **ACBranch**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>ACBranch.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable branch identifier
<code>from_bus, to_bus</code>	same columns	required on JSON import	branch terminal bus indices
<code>name</code>	<code>ACBranch.name</code>	default empty string	branch label
<code>r_pu, x_pu, b_pu</code>	same columns	default 0.0	series resistance, series reactance, and total shunt charging in per unit
<code>tap</code>	<code>ACBranch.tap</code>	default 1.0	off-nominal tap ratio used in the branch model
<code>shift_deg</code>	<code>ACBranch.shift_deg</code>	default 0.0	phase-shifting angle in degrees
<code>rate_a_mva, rate_b_mva, rate_c_mva</code>	same columns	default 0.0	thermal / planning ratings
<code>length_km</code>	<code>ACBranch.length_km</code>	default 0.0	physical route length metadata
<code>in_service</code>	<code>ACBranch.in_service</code>	default <code>true</code>	availability flag
<code>failure_rate, mttr_hr</code>	same columns	default 0.0	reliability metadata
<code>r0_pu, x0_pu, b0_pu, vn_hv_kv, vn_lv_kv, sn_mva</code>	<code>none</code>	not part of the current JSON / Excel contract	struct-level short-circuit and transformer-nameplate helpers currently stay internal to the in-memory model

Load : `ac.loads[]` / sheet Load (fallback `ACLoad`)

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Load.index</code> or <code>ACLoad.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable load identifier
<code>bus</code>	<code>Load.bus</code> or <code>ACLoad.bus</code>	required on JSON import	connected AC bus index
<code>name</code>	same column	default empty string	load label
<code>in_service</code>	same column	default <code>true</code>	availability flag
<code>p_mw, q_mvar</code>	same columns	default 0.0	active and reactive demand
<code>model</code>	same column	default <code>ConstantPower</code>	ZIP / exponential / constant-power load-model token
<code>z_percent_p, i_percent_p, p_percent_p</code>	same columns	defaults 0.0, 0.0, 100.0	active-power ZIP shares
<code>z_percent_q, i_percent_q, p_percent_q</code>	same columns	defaults 0.0, 0.0, 100.0	reactive-power ZIP shares
<code>controllable</code>	same column	default <code>false</code>	whether shedding / dispatch logic may treat the load as controllable
<code>p_min_mw</code>	<code>none</code>	default 0.0	JSON-only minimum retained active demand; workbook currently omits this field for AC loads
<code>cost_mw</code>	same column	default 0.0	marginal shedding or curtailment penalty

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
priority	same column	default Medium	service-priority token Low/Medium/High/Critical
n_customers	none	default 0	JSON-only customer-count metadata
profile_id	none	default -1	JSON-only profile linkage identifier
sn_mva, motor_percent, x_sub_pu, r_sc_pu	none	not part of the current main JSON contract	short-circuit helper fields remain outside the stable Load I/O contract

FlexibleLoad : ac.flexible_loads[] / sheet FlexibleLoad

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	FlexibleLoad.name	default empty string	load label
in_service	FlexibleLoad.in_service	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	baseline active and reactive demand
flex_up_mw, flex_down_mw	same columns	default 0.0	upward / downward flexibility envelope in MW
flex_duration_h	same column	default 0.0	duration for which the flex envelope can be sustained
response_time_s	same column	default 0.0	response latency in seconds
ramp_rate_mw_min	same column	default 0.0	adjustment ramp-rate limit
availability_pct	same column	default 100.0	expected availability of the flexibility service
controllable	same column	default true	whether the flexibility service is enabled
priority	same column	default Medium	load-service priority token
control_area	same column	default empty string	operator / aggregation area tag

AsymmetricLoad : ac.asymmetric_loads[] / sheet AsymmetricLoad

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	AsymmetricLoad.name	default empty string	load label
in_service	AsymmetricLoad.in_service	default true	availability flag
connection	AsymmetricLoad.connection	default empty string	wiring model string such as wye or delta
grounded	AsymmetricLoad.grounded	default true	grounding-path assumption
paRated_mw, pbRated_mw, pcRated_mw	same columns	default 0.0	rated per-phase active demand

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
qa_rated_mvar, qb_rated_mvar, qc_rated_mvar	same columns	default 0.0	rated per-phase reactive demand
pa_mw, pb_mw, pc_mw	same columns	default 0.0	current per-phase active demand
qa_mvar, qb_mvar, qc_mvar	same columns	default 0.0	current per-phase reactive demand
scaling	AsymmetricLoad.scaling	default 1.0	overall scaling factor applied to the phase values
const_z_percent, const_i_percent	same columns	default 0.0	ZIP constant-impedance and constant-current shares
const_p_percent	same column	default 100.0	ZIP constant-power share
controllable	same column	default false	controllability flag
priority	same column	default Medium	load-service priority token

Shunt : `ac.shunts[]` / sheet **Shunt**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	Shunt.name	default empty string	shunt label
in_service	Shunt.in_service	default true	availability flag
gs_mw, bs_mvar	same columns	default 0.0	fixed shunt conductance and susceptance
switchable	Shunt.switchable	default false	whether the shunt is modeled as stepped / switchable
n_steps	Shunt.n_steps	default 1	number of available discrete shunt steps
current_step	Shunt.current_step	default 1	currently selected shunt step
bs_per_step	Shunt.bs_per_step	default 0.0	susceptance increment per step

ExternalGrid : `ac.external_grids[]` / sheet **ExternalGrid**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	ExternalGrid.name	default empty string	source label
in_service	ExternalGrid.in_service	default true	availability flag
vm_pu, va_deg	same columns	defaults 1.0, 0.0	voltage magnitude and angle target for the slack / Thevenin source
s_sc_max_mva, s_sc_min_mva	same columns	default 0.0	short-circuit strength range

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>rx_max, rx_min</code>	same columns	default 0.0	R/X ratio bounds for equivalent-source construction
<code>r_pu, x_pu</code>	none	default 0.0	JSON-only explicit positive-sequence Thevenin impedance
<code>r0_pu, x0_pu</code>	none	default 0.0	JSON-only zero-sequence source impedance
<code>vn_kv</code>	none	default 0.0	JSON-only nominal source voltage rating
<code>ikq_ka, x_r</code>	none	not part of the current JSON / Excel contract	struct-level IEC 60909 helper fields currently stay internal and are not preserved by either import path
<code>controllable</code>	<code>ExternalGrid.controllable</code>	default true	whether optimization logic may vary the source dispatch

Transformer2W : `ac.transformers_2w[]` / sheet `Transformer2W`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, hv_bus, lv_bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and terminal bus indices
<code>name</code>	<code>Transformer2W.name</code>	default empty string	transformer label
<code>std_type</code>	none	default empty string	JSON-only library / standard type name
<code>in_service</code>	<code>Transformer2W.in_service</code>	default true	availability flag
<code>sn_mva</code>	<code>Transformer2W.sn_mva</code>	default 0.0	rated apparent power
<code>vn_hv_kv, vn_lv_kv</code>	same columns	default 0.0	winding rated voltages
<code>vk_percent, vkr_percent</code>	same columns	default 0.0	short-circuit magnitude and resistive share in percent
<code>pk_kw</code>	none	default 0.0	JSON-only load-loss field retained in the richer transformer contract
<code>pfe_kw, i0_percent</code>	same columns	default 0.0	no-load loss and excitation current
<code>tap_side, tap_pos, tap_min, tap_max, tap_neutral</code>	same columns	default 0	tap-changer placement and discrete tap metadata
<code>tap_step_percent</code>	<code>Transformer2W.tap_step_percent</code>	default 0.0	tap step size in percent
<code>shift_deg</code>	<code>Transformer2W.shift_deg</code>	default 0.0	phase-shift angle in degrees
<code>vector_group</code>	<code>Transformer2W.vector_group</code>	default empty string	winding connection / clock notation
<code>z0_percent, x0_r0</code>	none	default 0.0	JSON-only zero-sequence impedance parameters
<code>mtbf_hours, mtrt_hours</code>	none	default 0.0	JSON-only reliability metadata

Transformer3W : ac.transformers_3w[] / sheet Transformer3W

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, hv_bus, mv_bus, lv_bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and three winding bus indices
name	Transformer3W.name	default empty string	transformer label
std_type	Transformer3W.std_type	default empty string	library / standard type name
in_service	Transformer3W.in_service	default true	availability flag
sn_hv_mva, sn_mv_mva, sn_lv_mva	same columns	default 0.0	winding ratings on the HV / MV / LV sides
vn_hv_kv, vn_mv_kv, vn_lv_kv	same columns	default 0.0	winding rated voltages
vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent	same columns	default 0.0	pairwise short-circuit magnitudes
vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent	same columns	default 0.0	pairwise resistive shares
pfe_kw, i0_percent	same columns	default 0.0	no-load loss and excitation-current data
tap_side, tap_pos	same columns	default 0	tap-changer side and current position
tap_step_percent	Transformer3W.tap_step_percent	default 0.0	tap step size in percent
shift_mv_deg, shift_lv_deg	same columns	default 0.0	phase shifts from HV to MV / LV windings
mtbf_hours, mttr_hours	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

Switch : ac.switches[] / sheet Switch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_from, bus_to	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	Switch.name	default empty string	switch label
in_service	Switch.in_service	default true	availability flag
switch_type	Switch.switch_type	default CircuitBreaker	switching-device token such as Recloser or Fuse
closed	Switch.closed	default true	topology status flag
r_contact_ohm, z_ohm	none	default 0.0	JSON-only detailed contact / arc impedance inputs
i_rated_ka, i_breaking_ka	none	default 0.0	JSON-only current capability ratings
element_type, element_id	none	defaults 0, 0	JSON-only foreign-key reference to the protected element

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>is_remote</code> , <code>is_automated</code>	none	default false	JSON-only automation metadata
<code>t_operation_s</code> , <code>p_sw_fail</code>	none	default 0.0	JSON-only operation timing and failure-probability metadata
<code>mtbf_hours</code> , <code>mttr_hours</code>	none	default 0.0	JSON-only reliability metadata

CircuitBreaker : `ac.circuit_breakers[]` / sheet `CircuitBreaker`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus_from</code> , <code>bus_to</code>	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
<code>name</code>	<code>CircuitBreaker.name</code>	default empty string	breaker label
<code>in_service</code>	<code>CircuitBreaker.in_service</code>	default true	availability flag
<code>breaker_type</code>	<code>CircuitBreaker.breaker_type</code>	default CB	breaker-type token parsed by breaker_type_from_str
<code>closed</code>	<code>CircuitBreaker.closed</code>	default true	topology status flag
<code>z_ohm</code>	<code>CircuitBreaker.z_ohm</code>	default 0.0	AC breaker impedance
<code>rated_voltage_kv</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code>	same columns	default 0.0	voltage and current ratings
<code>element_type</code> , <code>element_id</code>	same columns	defaults empty string, 0	optional protected-element reference

ChargingStation : `ac.charging_stations[]` / sheet `ChargingStation`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
<code>name</code>	<code>ChargingStation.name</code>	default empty string	station label
<code>location</code>	none	default empty string	JSON-only free-form location descriptor
<code>in_service</code>	<code>ChargingStation.in_service</code>	default true	availability flag
<code>n_fast</code> , <code>n_slow</code> , <code>num_chargers</code>	same columns	default 0	counts of charger assets by class and in total
<code>p_fast_max_kw</code> , <code>p_slow_max_kw</code>	same columns	default 0.0	per-charger maximum power by service class
<code>max_power_kw</code>	same column	default 0.0	aggregate station power ceiling
<code>simultaneity_factor</code>	same column	default 1.0	diversification factor used in the aggregate station model
<code>power_factor</code>	same column	default 0.95	station-level reactive-power assumption

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>utilization_rate</code>	<code>none</code>	default 0.0	JSON-only expected utilization metadata
<code>p_total_kw, q_total_kvar</code>	same columns	default 0.0	aggregate present demand of the station
<code>mtbf_hours, mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

Charger : `ac.chargers[]` / sheet **Charger**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Charger.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable charger identifier
<code>name</code>	<code>Charger.name</code>	default empty string	charger label
<code>station_id</code>	<code>Charger.station_id</code>	default 0	parent charging-station foreign key
<code>charger_type</code>	<code>Charger.charger_type</code>	default <code>AC_L2</code>	charger-class token <code>AC_L1/AC_L2/DC_Fast</code>
<code>in_service</code>	<code>Charger.in_service</code>	default <code>true</code>	availability flag
<code>pRated_kw</code>	<code>Charger.pRated_kw</code>	default 0.0	rated charger power
<code>pCh_max_kw, pCh_min_kw</code>	same columns	default 0.0	charging power limits
<code>eta</code>	<code>Charger.eta</code>	default 0.95	charger conversion efficiency
<code>v2g_capable</code>	<code>Charger.v2g_capable</code>	default <code>false</code>	whether reverse power flow is allowed
<code>pDis_max_kw</code>	<code>Charger.pDis_max_kw</code>	default 0.0	maximum discharging power when V2G is enabled
<code>mtbf_hours, mttr_hours</code>	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

10.3 Generation and Storage Models

Generator : `ac.generators[]` / sheet **Generator**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>Generator.name</code>	default empty string	generator label
<code>in_service</code>	<code>Generator.in_service</code>	default <code>true</code>	availability flag
<code>pg_mw, qg_mvar</code>	same columns	default 0.0	active and reactive generation setpoints
<code>vg_pu</code>	<code>Generator.vg_pu</code>	default 1.0	voltage setpoint on the connected bus
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>is_slack</code>	<code>Generator.is_slack</code>	default <code>false</code>	slack-machine flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>fuel_type</code>	<code>Generator.fuel_type</code>	default <code>Unknown</code>	fuel-type token such as <code>Gas</code> , <code>Wind</code> , or <code>Solar</code>
<code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code>	same columns	default 0.0	quadratic production-cost coefficients
<code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code>	<code>none</code>	default 0.0	JSON-only unit-commitment metadata
<code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code>	same columns	default 0.0	ramp-rate limits
<code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code>	same columns	defaults 0.0, 0.0, 0.05	machine base, inertia, and governor droop metadata
<code>xd_pu</code> , <code>xdp_pu</code> , <code>xdpp_pu</code> , <code>td0p_s</code> , <code>td0pp_s</code>	<code>none</code>	default 0.0	JSON-only machine dynamic / short-circuit parameters
<code>ra_pu</code> , <code>vn_kv</code> , <code>cos_phi</code> , <code>xq_pu</code> , <code>xd_xq</code> , <code>x0_pu</code> , <code>r0_pu</code>	<code>none</code>	not part of the current JSON / Excel contract	additional short-circuit parameters exist in the C++ struct but are not yet serialized or workbook-backed
<code>emission_factor_tco2_mwh</code> , <code>nox_factor_kg_mwh</code> , <code>so2_factor_kg_mwh</code>	<code>none</code>	default 0.0	JSON-only emissions metadata
<code>forced_outage_rate</code> , <code>mtr_hr</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

StaticGenerator : `ac.static_generators[]` or `dc.static_generators[]` / sheets
StaticGenerator and **DCStaticGen**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC or DC bus
<code>name</code>	corresponding <code>name</code> column	default empty string	generator label
<code>in_service</code>	corresponding <code>in_service</code> column	default <code>true</code>	availability flag
<code>sgen_type</code>	corresponding <code>sgen_type</code> column	default <code>Other</code>	generator-type token such as <code>Wind</code> , <code>Battery</code> , or <code>FuelCell</code>
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present active / reactive injection
<code>p_rated_mw</code> , <code>sn_mva</code>	same columns	default 0.0	rated active power and apparent power
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>scaling</code>	same column	default 1.0	scalar multiplier for the static generation profile
<code>controllable</code>	same column	default <code>false</code>	controllability flag
<code>v_ref_pu</code> , <code>k_p</code> , <code>k_q</code> , <code>k_rx</code>	<code>none</code>	defaults 1.0, 0.0, 0.0, 1.0, 0.0	JSON-only control and equivalent-source tuning parameters
<code>co2_emission_rate</code>	<code>none</code>	default 0.0	JSON-only emissions metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
mtbf_hours, mttr_hours	none	default 0.0	JSON-only reliability metadata

Storage : `ac.storage[]` or `dc.storage[]` / sheets **Storage** and **DCStorage**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected bus
name	corresponding name column	default empty string	storage label
in_service	corresponding in_service column	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	present active and reactive dispatch
p_rated_mw	same column	default 0.0	rated converter power
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating limits
e_rated_mwh	same column	default 0.0	energy capacity
soc_init, soc_min, soc_max	same columns	defaults 0.5, 0.1, 0.9	initial and admissible state-of-charge values
soc_carbon_intensity_tco2_mwh	same column	default 0.0	explicit carbon intensity of the stored energy state used by carbon-flow analysis during discharge
eta_charge, eta_discharge	same columns	default 0.95	charging and discharging efficiency
self_discharge_pct	none	default 0.0	JSON-only storage leakage metadata
max_cycles, current_cycles	none	defaults 5000, 0	JSON-only lifecycle counters
soh	none	default 1.0	JSON-only state-of-health value
replacement_cost	none	default 0.0	JSON-only replacement-cost metadata
forced_outage_rate, mttr_hr	none	default 0.0	JSON-only reliability metadata

RenewableGen : `ac.renewable_gens[]` / sheet **RenewableGen**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	RenewableGen.name	default empty string	generator label
in_service	RenewableGen.in_service	default true	availability flag
type	RenewableGen.type	default Wind	renewable-type token such as Wind , SolarPV , or Hydro

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
p_mw, q_mvar	same columns	default 0.0	present active and reactive output
p_rated_mw	same column	default 0.0	rated active power
qmax_mvar, qmin_mvar	same columns	default 0.0	reactive capability limits
curtailable	same column	default true	whether curtailment is allowed
cost_curtail_mwh	none	default 0.0	JSON-only curtailment penalty
capacity_factor	RenewableGen.capacity_factor	default 0.3	expected normalized availability
profile_id	none	default -1	JSON-only profile linkage identifier
emission_offset_tco2_mwh	none	default 0.0	JSON-only avoided-emissions metadata

PVSystem : ac.pv_systems[] / sheet PVSystem

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	PVSystem.name	default empty string	PV system label
in_service	PVSystem.in_service	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	present active and reactive output
sn_mva	PVSystem.sn_mva	default 0.0	apparent-power rating
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating capability limits
control_mode	PVSystem.control_mode	default MPPT	inverter control-mode token; one of MPPT, VoltVar, VoltWatt, FixedPF, or Curtailed
controllable	none	default false	JSON-only controllability flag
v_ac_set_pu, v_dc_set_pu	none	default 1.0	JSON-only AC/DC voltage reference targets
inverter_eff	PVSystem.inverter_eff	default 0.97	inverter efficiency
loss_percent	none	default 0.0	JSON-only miscellaneous loss factor
num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc	none	default 0 or 0.0	JSON-only panel-array electrical metadata
irradiance, temperature	none	defaults 1000.0, 25.0	JSON-only environmental inputs
mtbf_panel_hours, mttr_panel_hours, mtbf_inverter_hours, mttr_inverter_hours	none	default 0.0	JSON-only reliability metadata

10.4 DC and Hybrid Coordination Models

DCBus : `dc.buses[]` / sheet **DCBus**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>DCBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable DC bus identifier
<code>name</code>	<code>DCBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>DCBus.bus_type</code>	default <code>DC_P</code>	DC bus-type token <code>DC_P</code> or <code>DC_V</code>
<code>vm_pu</code>	<code>DCBus.vm_pu</code>	default 1.0	DC voltage magnitude in per unit
<code>vmax_pu</code> , <code>vmin_pu</code>	same columns	defaults 1.1, 0.9	DC voltage operating bounds
<code>pd_mw</code>	<code>DCBus.pd_mw</code>	default 0.0	embedded DC demand snapshot; workbook export may reflect synchronized aggregate demand
<code>in_service</code>	<code>DCBus.in_service</code>	default <code>true</code>	availability flag
<code>latitude</code> , <code>longitude</code>	<code>none</code>	defaults 0.0; import also accepts <code>lat/lon</code> aliases	JSON-only coordinates
<code>base_kv</code> , <code>area</code> , <code>zone</code> , <code>n_customers</code> , <code>importance</code> , <code>is_load</code>	<code>none</code>	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCBranch : `dc.branches[]` / sheet **DCBranch**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>from_bus</code> , <code>to_bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and terminal bus indices
<code>name</code>	<code>DCBranch.name</code>	default empty string	branch label
<code>r_pu</code>	<code>DCBranch.r_pu</code>	default 0.0	branch resistance in per unit
<code>rate_a_mva</code>	<code>DCBranch.rate_a_mva</code>	default 0.0	branch rating
<code>length_km</code>	<code>DCBranch.length_km</code>	default 0.0	physical route length metadata
<code>in_service</code>	<code>DCBranch.in_service</code>	default <code>true</code>	availability flag
<code>base_kv</code> , <code>s_max_mva</code> , <code>n_parallel</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code>	<code>none</code>	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCLoad : `dc.loads[]` / sheet **DCLoadRaw** (fallback **DCLoad**)

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected DC bus
name	same column	default empty string	load label
in_service	same column	default true	availability flag
p_mw	same column	default 0.0	active demand
controllable	same column	default false	controllability flag
p_min_mw	same column	default 0.0	minimum retained demand under curtailment
cost_mw	same column	default 0.0	marginal curtailment penalty
profile_id	none	default -1	JSON-only profile linkage identifier

StaticGeneratorDC : dc.dc_static_generators[] / sheet DCStaticGenerator

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected DC bus
name	DCStaticGenerator.name	default empty string	source label
in_service	DCStaticGenerator.in_service	default true	availability flag
type	DCStaticGenerator.type	default empty string	free-form DC source type
p_set_mw	DCStaticGenerator.p_set_mw	default 0.0	active-power setpoint
scaling	DCStaticGenerator.scaling	default 1.0	profile scaling factor
profile_id	DCStaticGenerator.profile_id	default -1	profile linkage identifier
pmax_mw, pmin_mw	same columns	default 0.0	operating limits
controllable	DCStaticGenerator.controllable	default false	controllability flag
mtbf_hours, mttr_hours	Excel columns mtbf_hr , mttr_hr	default 0.0	reliability metadata; Excel uses the hr suffix
t_scheduled_hr	same column	default 0.0	scheduled outage duration metadata

PVArrayDC : dc.pv_arrays[] / sheet DCPVArray

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected DC bus
name	DCPVArray.name	default empty string	DC PV array label
in_service	DCPVArray.in_service	default true	availability flag
p_set_mw	DCPVArray.p_set_mw	default 0.0	active-power setpoint

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>profile_id</code>	<code>DCPVArray.profile_id</code>	default -1	profile linkage identifier
<code>num_series, num_parallel</code>	same columns	default 0	array topology
<code>vmpp, impp, voc, isc</code>	same columns	default 0.0	module electrical parameters
<code>alpha_isc, beta_voc</code>	same columns	default 0.0	temperature coefficients
<code>temperature, irradiance</code>	same columns	defaults 25.0, 1000.0	operating condition inputs
<code>mtbf_hours, mttr_hours</code>	Excel columns <code>mtbf_hr</code> , <code>mttr_hr</code>	default 0.0	reliability metadata; Excel uses the <code>hr</code> suffix
<code>t_scheduled_hr</code>	same column	default 0.0	scheduled outage duration metadata

VSCConverter : `vsc_converters[]` / sheet VSCConverter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus_ac, bus_dc</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and AC/DC terminal bus indices
<code>name</code>	<code>VSCConverter.name</code>	default empty string	converter label
<code>in_service</code>	<code>VSCConverter.in_service</code>	default <code>true</code>	availability flag
<code>control_mode</code>	<code>VSCConverter.control_mode</code>	default PQ	converter-mode token PQ/VD C_Q/VDC_VAC
<code>p_set_mw, q_set_mvar</code>	same columns	default 0.0	AC-side active and reactive setpoints
<code>v_dc_set_pu, v_ac_set_pu</code>	same columns	default 1.0	DC and AC voltage references
<code>eta</code>	<code>VSCConverter.eta</code>	default 0.99	conversion efficiency
<code>loss_percent</code>	<code>VSCConverter.loss_percent</code>	default 0.0	generic loss factor
<code>loss_mw</code>	<code>none</code>	default 0.0	JSON-only fixed loss term
<code>k_vdc</code>	<code>none</code>	default 0.1	JSON-only DC-voltage droop coefficient
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>pRated_mw</code>	<code>VSCConverter.pRated_mw</code>	default 0.0	converter rating
<code>forced_outage_rate, mttr_hr</code>	<code>none</code>	default 0.0	JSON-only reliability metadata
<code>type, vn_ac_kv, vn_dc_kv, k_p, k_q, v_ref_pu, f_ref_hz, controllable, grid-forming / SC fields, t_scheduled_hr</code>	<code>none</code>	not part of current JSON / Excel contract	newer C++ struct fields that are not preserved by either serializer

DCDCConverter : `dc.dcdc_converters[]` / sheet DCDCCConverter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus_in, bus_out</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and input / output DC bus indices
<code>name</code>	<code>DCDCConverter.name</code>	default empty string	converter label
<code>in_service</code>	<code>DCDCConverter.in_service</code>	default <code>true</code>	availability flag
<code>control_mode</code>	<code>DCDCConverter.control_mode</code>	default <code>Voltage</code>	control-mode token <code>Voltage/Power/Droop</code>
<code>p_ref_mw</code>	<code>DCDCConverter.p_ref_mw</code>	default <code>0.0</code>	active-power reference
<code>v_ref_pu</code>	<code>DCDCConverter.v_ref_pu</code>	default <code>1.0</code>	output-voltage reference
<code>sn_mva</code>	<code>DCDCConverter.sn_mva</code>	default <code>0.0</code>	converter rating
<code>vn_in_kv, vn_out_kv</code>	<code>none</code>	default <code>0.0</code>	JSON-only nominal voltage ratings
<code>eta</code>	<code>DCDCConverter.eta</code>	default <code>0.98</code>	conversion efficiency
<code>r_eq_pu</code>	<code>none</code>	default <code>0.0</code>	JSON-only equivalent resistance
<code>pmax_mw, pmin_mw</code>	same columns	default <code>0.0</code>	power limits
<code>k_droop</code>	<code>none</code>	default <code>0.0</code>	JSON-only droop coefficient
<code>mtbf_hours, mttr_hours</code>	<code>none</code>	default <code>0.0</code>	JSON-only reliability meta-data
<code>controllable, f_switching_hz, t_scheduled_hr</code>	<code>none</code>	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCCircuitBreaker : `dc.dc_circuit_breakers[]` / sheet `DCCircuitBreaker`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus_from, bus_to</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and terminal DC bus indices
<code>name</code>	<code>DCCircuitBreaker.name</code>	default empty string	breaker label
<code>in_service</code>	<code>DCCircuitBreaker.in_service</code>	default <code>true</code>	availability flag
<code>breaker_type</code>	<code>DCCircuitBreaker.breaker_type</code>	default <code>CB</code>	breaker-type token parsed by <code>breaker_type_from_str</code>
<code>closed</code>	<code>DCCircuitBreaker.closed</code>	default <code>true</code>	topology status flag
<code>r_ohm</code>	<code>DCCircuitBreaker.r_ohm</code>	default <code>0.0</code>	DC breaker resistance
<code>rated_voltage_kv, iRated_ka, iBreaking_ka</code>	same columns	default <code>0.0</code>	voltage and current ratings
<code>element_type, element_id</code>	same columns	defaults empty string, <code>0</code>	optional protected-element reference

EnergyRouter : `energy_routers[]` / sheet `EnergyRouter`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>EnergyRouter.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable router identifier
<code>name</code>	<code>EnergyRouter.name</code>	default empty string	router label
<code>in_service</code>	<code>EnergyRouter.in_service</code>	default <code>true</code>	availability flag
<code>router_type</code>	<code>EnergyRouter.router_type</code>	default empty string	free-form router classification token
<code>num_ports</code>	<code>EnergyRouter.num_ports</code>	default 0	expected number of ports; workbook import may later recompute it from the separate port table
<code>ports[]</code>	sheet <code>EnergyRouterPort</code>	missing JSON array keeps the vector empty	nested port objects are flattened into a dedicated workbook sheet
<code>p_rated_mw</code>	<code>EnergyRouter.p_rated_mw</code>	default 0.0	total router rating
<code>vn_ac_kv, vn_dc_kv</code>	same columns	default 0.0	nominal AC/DC voltage ratings
<code>loss_percent</code>	<code>EnergyRouter.loss_percent</code>	default 0.0	aggregate conversion loss factor
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	aggregate operating limits
<code>mtbf_hours, mttr_hours</code>	same columns	default 0.0	reliability metadata preserved in both JSON and workbook
<code>control_mode, power_dispatch_strategy, t_scheduled_hr</code>	<code>none</code>	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

EnergyRouterPort : nested `energy_routers[] .ports[]` / sheet `EnergyRouterPort`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>router_index</code>	<code>EnergyRouterPort.router_index</code>	workbook-only join key	parent router identifier used because ports are nested in JSON but flattened in Excel
<code>index</code>	<code>EnergyRouterPort.index</code>	required on JSON import; workbook row also needs <code>router_index</code>	stable port identifier within its router
<code>name</code>	<code>EnergyRouterPort.name</code>	default empty string	port label
<code>bus</code>	<code>EnergyRouterPort.bus</code>	required on JSON import	connected AC or DC bus index
<code>port_type</code>	<code>EnergyRouterPort.port_type</code>	default <code>AC</code>	side-type token <code>AC</code> or <code>DC</code>
<code>voltage_level_kv</code>	<code>EnergyRouterPort.voltage_level_kv</code>	default 0.0	nominal port voltage level
<code>p_mw, q_mvar</code>	same columns	default 0.0	present active and reactive transfer assigned to the port
<code>v_pu</code>	<code>EnergyRouterPort.v_pu</code>	default 1.0	local port voltage state or initialization
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating limits

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>control_mode</code>	<code>EnergyRouterPort.control_mode</code>	default PQ	control-mode token PQ/VF/Droop
<code>p_set_mw, q_set_mvar, v_set_pu</code>	same columns	defaults 0.0, 0.0, 1.0	reference setpoints for the local port controller
<code>in_service</code>	<code>EnergyRouterPort.in_service</code>	default true	availability flag
<code>side, eta</code>	none	not part of current JSON / Excel contract	C++ port fields that are not preserved by either serializer

VirtualPowerPlant : `vpps[]` / sheet VPP

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>VPP.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable VPP identifier
<code>name</code>	<code>VPP.name</code>	default empty string	VPP label
<code>description</code>	none	default empty string	JSON-only free-form description
<code>aggregation_bus</code> / JSON <code>pcc_bus</code>	<code>VPP.pcc_bus</code>	exporter writes <code>pcc_bus</code> ; importer accepts <code>pcc_bus</code> or legacy <code>aggregation_bus</code> ; default 0	reporting / control aggregation bus stored in the C++ <code>aggregation_bus</code> member
<code>in_service</code>	<code>VPP.in_service</code>	default true	availability flag
<code>n_pv_systems, n_wind_turbines, n_battery_systems</code>	same columns	default 0	asset counts preserved in both JSON and workbook
<code>n_ev_chargers, n_controllable_loads</code>	none	default 0	JSON-only aggregation metadata
<code>p_generation_sum_mw, e_storage_sum_mwh, p_load_controllable_mw</code>	none	default 0.0	JSON-only aggregate capability metadata
<code>p_output_mw, q_output_mvar</code>	same columns	default 0.0	present aggregated power output
<code>pmax_mw, pmin_mw</code>	same columns	default 0.0	aggregate operating limits
<code>ramp_up_max_mw_min, ramp_down_max_mw_min</code>	none	default 0.0	JSON-only ramping metadata
<code>mtbf_hours, mttr_hours</code>	none	default 0.0	JSON-only reliability metadata

Microgrid : `microgrids[]` / sheet Microgrid

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Microgrid.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable microgrid identifier
<code>name</code>	<code>Microgrid.name</code>	default empty string	microgrid label
<code>description</code>	<code>none</code>	default empty string	JSON-only free-form description
<code>in_service</code>	<code>Microgrid.in_service</code>	default <code>true</code>	availability flag
<code>pcc_bus</code>	<code>Microgrid.pcc_bus</code>	exporter writes <code>pcc_bus</code> ; importer accepts <code>pcc_bus</code> or legacy <code>aggregation_bus</code> ; default 0	point-of-common-coupling bus index stored in the C++ <code>aggregation_bus</code> member
<code>internal_buses</code>	<code>none</code>	missing JSON array keeps the vector empty	JSON-only explicit internal bus membership list
<code>operating_mode</code>	<code>Microgrid.operating_mode</code>	default <code>GridConnected</code>	operating-mode token; one of <code>GridConnected</code> , <code>Islanded</code> , or <code>Transition</code>
<code>islanding_capability</code>	<code>Microgrid.islanding_capability</code>	default <code>false</code>	whether the microgrid may island
<code>auto_reconnection</code>	<code>none</code>	default <code>false</code>	JSON-only reconnection automation flag
<code>p_exchange_max_mw</code> , <code>p_exchange_min_mw</code>	Excel keeps only <code>p_exchange_max_mw</code>	default 0.0	aggregate exchange limits with the host grid
<code>p_import_max_mw</code> , <code>p_export_max_mw</code>	<code>none</code>	default 0.0	JSON-only directional exchange limits
<code>p_exchange_mw</code>	<code>Microgrid.p_exchange_mw</code>	default 0.0	present net power exchange at the PCC
<code>total_generation_mw</code> , <code>total_load_mw</code>	same columns	default 0.0	current aggregate generation and demand
<code>total_storage_mwh</code> , <code>capacity_mw</code> , <code>peak_load_mw</code>	<code>none</code>	default 0.0	JSON-only planning meta-data
<code>f_set_hz</code> , <code>v_set_pu</code> , <code>k_droop</code>	<code>none</code>	defaults 50.0, 1.0, 0.0	JSON-only islanded-control targets
<code>area</code>	<code>none</code>	default 0	JSON-only area tag
<code>mtbf_hours</code> , <code>mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability meta-data

MobileStorage : `mobile_storage[]` / sheet `MobileStorage`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and current connected bus
<code>name</code>	<code>MobileStorage.name</code>	default empty string	unit label
<code>in_service</code>	<code>MobileStorage.in_service</code>	default <code>true</code>	availability flag
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present dispatch at the current location
<code>p_rated_mw</code>	<code>MobileStorage.p_rated_mw</code>	default 0.0	converter power rating

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	none	default 0.0	JSON-only operating limits
<code>eRated_mwh</code>	<code>MobileStorage.eRated_mwh</code>	default 0.0	energy capacity
<code>soc_init</code>	<code>MobileStorage.soc_init</code>	default 0.5	initial state of charge
<code>soc_min, soc_max</code>	none	defaults 0.1, 0.9	JSON-only admissible state-of-charge bounds
<code>eta_charge, eta_discharge</code>	none	default 0.95	JSON-only efficiency metadata
<code>max_cycles, current_cycles, soh</code>	none	defaults 5000, 0, 1.0	JSON-only lifecycle / health metadata
<code>is_mobile</code>	none	default true	JSON-only flag distinguishing mobile vs. stationary storage representation
<code>status</code>	<code>MobileStorage.status</code>	default <code>Stationary</code>	mobility-state token; one of <code>Stationary</code> , <code>InTransit</code> , or <code>Deployed</code>
<code>current_location</code>	none	default empty string	JSON-only free-form location string
<code>target_bus</code>	none	default 0	JSON-only relocation target
<code>eConsumption_mwh_kWh, max_travel_distance_kWh</code>	none	default 0.0	JSON-only transportation-energy metadata
<code>mtbf_hours, mttr_hours</code>	none	default 0.0	JSON-only reliability metadata

10.5 Cross-Reference

The only remaining detailed dictionaries not repeated in this section are the ones already written in Section 02a_io_contracts:

1. `AsynchronousMotor`
2. the optional `three_phase_ac` metadata container
3. `ThreePhaseBus`, `ThreePhaseLine`, `ThreePhaseTransformer`, `ThreePhaseLoad`, `ThreePhaseGenerator`, and `ThreePhaseExternalGrid`
4. result-sheet enum and flag conventions

11 Output and Report Field Dictionaries

This section applies the same field-level dictionary template to the current export artifacts. Section 02a_io_contracts already catalogs which workbooks and JSON serializers exist; the tables below make the output-side contract explicit at workbook-column and JSON-key level.

11.1 Export Families

Artifact	Export path	Direction	Current note
<code>PowerFlowResult</code> dedicated workbook	source-only workbook helper	export only	writes the <code>Results_*</code> sheets only when optional Excel support is compiled.
<code>PowerFlowResult</code> compact workbook	<code>save_results_compact.xlsx</code>	export only	rewrites input sheets and appends <code>result_*</code> columns to selected tables, plus <code>Results_Summary</code> .

Artifact	Export path	Direction	Current note
PowerFlowResult JSON	power_flow_result_to_json / save_power_flow_result.json	export only	serializes convergence summary plus AC/DC bus states, AC/DC branch flows, VSC transfers, and DC/DC transfers.
ACOPFResult dedicated workbook	save_opf_results.xlsx	export only	writes OPF_* sheets only.
ACOPFResult compact workbook	save_opf_results_compact.xlsx	export only	rewrites input sheets but still emits separate OPF_* sheets; no opf_* columns are appended into input tables.
ACOPFResult JSON	opf_result_to_json / opf_result_from_json	bidirectional	serializes scalar summary, solver_path, bus / generator / converter vectors, enhanced dispatch objects with maps, legacy dispatch arrays, and OPF profiling block.
DCOPFResult JSON	dc_opf_result_to_json / dc_opf_result_from_json / save/load helpers	bidirectional	serializes DCOPF convergence/status, objective, solver audit chain, objective model, LMPs, branch-dual arrays with branch_mu_valid, per-bus shedding, dispatch, branch flows, and bus angles.
CarbonAnalysisResult workbook	save_carbon_results.xlsx	export only	writes AC/DC load and bus sheets, AC/DC branch-loss sheets, converter-loss sheets, storage / EnergyRouter sheets, and block-style Summary.
CarbonAnalysisResult JSON	carbon_result_to_json / carbon_result_from_json	bidirectional	preserves the richer tracing maps and the hybrid AC/DC carrier results that the workbook flattens.
Short-circuit and ONR artifacts	available via module headers; not forwarded from hacdcpf.hpp	hacdcpf/analysis/short_circuit.hpp, hacdcpf/network_reconfiguration/topology_analysis.hpp	Implemented with documented approximation and fallback boundaries; see Section 26. No dedicated workbook serializer exists; results are consumed in-memory or logged. Output field contracts are documented in those module headers.

11.2 Result Semantic Conventions

Semantic aspect	Current rule	Main artifacts	Engineering note
Electrical units	Active power uses MW, reactive power uses Mvar, AC / DC voltage magnitudes use pu, AC voltage angle uses degree, and current stays in pu unless the field name explicitly says ka .	PF / OPF workbooks and JSON	Workbook and JSON field names usually encode the unit directly, for example pg_mw , va_deg , loss_mw , and matrix_residual .
Carbon units	Carbon intensity uses tCO2/MWh ; total emissions use tCO2 .	Carbon workbook and JSON	The JSON / workbook field names use the suffixes _tco2_mwh and _tco2 ; no implicit mass-unit conversion is applied by the exporter.
Vector order	Numerical result vectors follow solver-side component order, not sparse physical ids. vm , va , dpd_mw , and dqd_mvar follow AC-bus order; vdc follows DC-bus order; pg_mw , qg_mvar follow AC-generator order; pac_mw , qac_mvar follow VSC order.	PF / OPF JSON and workbook exports	Consumers should treat workbook rows and JSON vectors as aligned to the source component arrays unless a sheet explicitly says otherwise.
Row identity	Workbook result sheets copy source model index values when a source component exists; Results_BranchFlow.index is currently ACBranch::index .	PF workbooks and PF JSON	For branch-flow joins, use the source component index plus terminal buses; do not assume row number equals component id.
Fallback values	When a solver vector entry is absent or out of range, exporters write deterministic defaults: typically 0.0 for powers, angles, counters, and residual-like scalars, or 1.0 for voltage magnitudes.	PF / OPF workbooks	This keeps workbook shapes stable, but it means a consumer must distinguish “solver wrote a fallback” from “the physical result is exactly zero” using convergence and artifact context.

Semantic aspect	Current rule	Main artifacts	Engineering note
JSON missing keys	JSON importers keep absent arrays or object maps empty and default absent scalar keys to false, zero, or empty string according to the target field type.	OPF / carbon JSON	The JSON contracts are intentionally sparse-friendly; round-trip consumers should not require every optional key to be present.
Echoed vs solved values	Some exported fields are metadata echoes or derived annotations rather than true solver states: storage <code>soc</code> comes from input <code>soc_init</code> ; router port sums come from stored port structs; <code>Results_DC Branch</code> is recomputed from solved voltages and branch resistance.	PF workbooks	Consumers should not infer that every result-sheet field belongs to the nonlinear state vector.
Boolean spelling	PF / OPF workbook booleans use the standard workbook token form produced by <code>bool_str</code> ; carbon <code>Summary</code> uses lowercase <code>true/false</code> .	Result workbooks	Downstream parsers should normalize workbook booleans instead of assuming one literal spelling.
Conditional sheets	Some result sheets are pre-created even when no rows are written. <code>OPF_Converter</code> is the main example.	PF / OPF workbooks	Empty sheets are part of the current export contract and do not by themselves indicate an error.
Object-map semantics	Carbon contribution maps are exported as JSON object maps keyed by internal carbon-source id rather than positional arrays.	Carbon JSON	Map-key order is not semantically meaningful; workbook export intentionally flattens these maps away.

Semantic aspect	Current rule	Main artifacts	Engineering note
Limited analysis outputs	The legacy distribution power-flow stub/public BFS API has been removed; distribution feeders use the three-phase NR result contract. The resilience MIP entry point (<code>run_distribution_resilience_mip_assessment</code>) is a live independent MILP solver. FMEA uses <code>ac-only-dcopf</code> for AC-only cases and <code>hybrid-acdc-net-work-lp</code> for hybrid cases with DC/VSC components. Three-stage MILP reliability (<code>run_three_stage_reliability</code>) is a native C++ MIPSolvers evaluator; no external Julia bridge is required.	analysis names-pace and scope boundary	The serializers listed here describe existing export code paths; see Sections 1.2 and 26 for the up-to-date boundary.

11.3 Solver-Specific Result Notes

Solver path	Result fields with special meaning	Where visible	Current note
Hybrid PF Newton / FDPF / DC PF	<code>converged</code> , <code>iterations</code> , <code>residual</code>	<code>Results_Summary</code> and compact PF summary sheet	The public PF artifacts expose only the compact scalar summary; detailed profiling arrays stay in memory and are not serialized.
Hybrid PF converter transfer postprocessing	<code>p_ac_mw</code> , <code>q_ac_mvar</code> , <code>p_dc_mw</code> , <code>loss_mw</code> , plus DC/DC transfer fields	dedicated and compact PF workbooks	Converter rows are matched by converter <code>index</code> ; when a solved transfer is missing, the exporter falls back to setpoints and static metadata.
Native AC OPF IPM	<code>outer_iterations</code> , <code>profiling.final_barrier_mu</code> , profiling counters	<code>OPF_Summary</code> and <code>ACOPFResult</code> JSON	These fields are meaningful primarily for the native IPM path; they may remain zero-like on simpler fallback paths.

Solver path	Result fields with special meaning	Where visible	Current note
Parity OPF formulation	<code>dpd_mw</code> , <code>dqd_mvar</code>	OPF_Bus and ACOPFResult JSON	The parity path exposes explicit active / reactive load-shedding vectors. On runs without shedding, the vectors may still exist but stay numerically zero.
OPF fallback selection	<code>status</code> , convergence flags, and partial vectors	OPF_Summary, ACOPFResult JSON	Consumers should read <code>status</code> together with <code>converged</code> ; some fallback paths can return structurally valid but less detailed result blocks.
Carbon tracing method	<code>tracing_verified</code> and <code>tracing_summary.*</code>	carbon workbook and JSON	These fields describe graph-tracing consistency, not matrix-solver success.
Carbon matrix method	<code>matrix_solved</code> , <code>matrix_residual</code> , and <code>matrix_summary.*</code>	carbon workbook and JSON	<code>matrix_residual</code> is specific to the matrix solve; it is not directly comparable to PF or OPF residual metrics.
DC OPF	<code>solver_chain</code> , <code>objective_model</code> , <code>lmp</code> , <code>branch_mu_lower</code> / <code>upper</code> , <code>branch_mu_valid</code> , <code>load_shedding_mw</code> , <code>total_load_shedding_mw</code>	DCOPFResult JSON	<code>solver_chain</code> records backend attempts and <code>objective_model</code> records whether the reported objective came from the QP or LP cost model. Per-bus direct isolated-island shedding is preserved in <code>load_shedding_mw</code> ; branch congestion dual arrays are not valid unless <code>branch_mu_valid=true</code> .
Distribution resilience	<code>completed</code> , <code>status</code> , <code>resilience_index</code> , <code>steps[]</code> , <code>total_served_mwh</code> , <code>model_stats</code>	DistributionResilienceResult	The heuristic multi-hour assessment is implemented in <code>src/resilience/resilience_assessment.cpp</code> . The strict <code>MultiPeriodMIPLinDistFlow</code> path is an independent AC-only restoration MILP in <code>src/resilience/resilience_restoration_mip.cpp</code> ; it reports solver status, objective, gap, model scope, and validity flags rather than delegating to the heuristic.

11.4 Consumer Contracts

Results_* workbook family

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
Results_ACBus, Results_DCBus	index	voltage fields are stable; name is descriptive only	consumers should join by index rather than row number even though row order follows the source bus tables.
Entity sheets with explicit ids	index on converter, storage, charging-station, charger, router, VPP, micro-grid, and mobile-storage sheets	identity and meta-data columns are stable; solved voltages / transfer values may fall back to deterministic defaults	additive columns are expected to be backward compatible; consumers should read by header name instead of fixed workbook position.
Results_Energy RouterPort	pair (router_index, port_index)	solved transfer fields may fall back to stored port meta-data; port_name is descriptive only	the flattened port contract is stable at the key-field level, not at the row-order level.
Results_Branch Flow	index copied from ACBranch::index, plus from_bus/to_bus	flow, loss_mw, and loading_pct fields are stable	consumers should still read by header name and tolerate non-contiguous source branch ids.
Results_Summary	Key	current scalar keys are stable; extra keys may appear additively	consumers should parse it as a name/value sheet and must not depend on fixed row numbers.

OPF_* workbook family and ACOPFResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
OPF_Bus	index	vm_pu, va_deg, dpd_mw, and dqd_mvar are the stable solved fields; name and base_kv are echoes	workbook rows align to sys.ac.buses , but consumers should still join by index .
OPF_Generator	index	dispatch fields are stable; cost and fuel_type columns are echoed metadata	unknown future columns should be ignored, and consumers should not assume the sheet contains only solved variables.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
OPF_Converter	index	<code>pac_mw</code> and <code>qac_mvar</code> are stable when the sheet has rows	the sheet may be present but empty when no VSC converters exist, so emptiness is not itself an error condition.
OPF_Summary and JSON scalar block	workbook Key; JSON top-level scalar names	convergence, iteration, objective, residual-like, <code>solver_path</code> , and status fields are stable	summary consumers should tolerate additive keys while continuing to treat existing keys as stable.
ACOPFResult vector and dispatch blocks	AC-bus order for <code>vm</code> , <code>va</code> , <code>vdc</code> , <code>dpd_mw</code> , <code>dqd_mvar</code> ; generator order for <code>pg_mw</code> , <code>qg_mvar</code> ; VSC order for <code>pac_mw</code> , <code>qac_mvar</code> ; map arrays for enhanced dispatch	the profiling object is optional on import; absent arrays stay empty	JSON serializes <code>renewable_dispatch</code> , <code>storage_dispatch</code> , <code>dc_dc_dispatch</code> , <code>flex_dispatch</code> , plus legacy arrays <code>pren_mw</code> , <code>qren_mvar</code> , <code>pstor_mw</code> , <code>qstor_mvar</code> , <code>pdcdc_mw</code> , and <code>pflex_mw</code> .

CarbonAnalysisResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
top-level scalar block	top-level key names	<code>tracing_verified</code> , <code>matrix_solved</code> , and <code>matrix_residual</code> are stable; absent keys default to zero-like values on import	additive scalar keys are backward compatible because the current importer only reads known fields.
<code>load_carbon[]</code> and <code>dc_load_carbon[]</code>	<code>load_index</code>	the explicit scalar fields are stable; <code>generator_supply_mw</code> may be absent and then defaults to an empty map	array order follows producer-side load order, but consumers should join by <code>load_index</code> when combining with other artifacts.
<code>branch_carbon[]</code> and <code>dc_branch_carbon[]</code>	<code>branch_index</code>	the explicit scalar fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	object-map key order is not stable and must not be used as a semantic ordering.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
<code>bus_carbon[]</code> and <code>dc_bus_carbon[]</code>	<code>bus_index</code>	the explicit scalar fields are stable; absent array keeps the vector empty	row order follows producer-side bus order, but <code>bus_index</code> is the durable join key.
<code>vsc_carbon[]</code> and <code>dc_dc_carbon[]</code>	<code>converter_index</code>	scalar direction and power fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	consumers should join on converter index and not rely on array order once projected-canonical filtering is enabled.
<code>storage_carbon[]</code>	pair (<code>storage_index</code> , <code>is_dc</code>)	scalar storage-state fields are stable; <code>source_supply_mw</code> may be absent and then defaults to an empty map	<code>p_mw</code> uses the bus-injection sign convention and determines whether the row should be interpreted as discharge or charging sink.
<code>energy_router_carbon[]</code>	<code>router_index</code>	scalar router loss fields are stable; <code>source_loss_mw</code> may be absent and then defaults to an empty map	consumers should join by router index and not infer semantics from workbook row order.
generator-allocation maps and summary blocks	source-id string for maps; top-level object name for summaries	<code>generator_loss_allocation</code> , <code>tracing_summary</code> , and <code>matrix_summary</code> are stable object names	map keys are JSON strings of internal source ids, and consumers must normalize them back to integer identifiers if a typed downstream schema requires that.

11.5 Power-Flow Result JSON

JSON key / row object	Fields	Population rule	Meaning / note
top-level scalar block	<code>converged</code> , <code>iterations</code> , <code>residual</code>	copied from <code>PowerFlowResult</code>	solver summary for the exported result
<code>ac_buses[]</code>	<code>index</code> , <code>name</code> , <code>vm_pu</code> , <code>va_deg</code>	component identity from <code>sys.ac.buses</code> ; solved values from <code>result.vm/result.va</code> , falling back to stored bus <code>vm_pu/va_deg</code>	solved AC bus states

JSON key / row object	Fields	Population rule	Meaning / note
<code>dc_buses[]</code>	<code>index, name, vm_pu</code>	component identity from <code>sys.dc.buses</code> ; solved value from <code>result.vdc</code> , falling back to stored bus <code>vm_pu</code>	solved DC bus states
<code>ac_branch_flows[]</code>	<code>index, name, from_bus, to_bus, pf_mw, qf_mvar, pt_mw, qt_mvar, loss_mw, loading_pct</code>	<code>index</code> and terminals copied from <code>ACBranch</code> ; flow values copied from <code>result.branch_flows</code> ; loading computed against rating fallback logic	AC branch-flow rows use original branch indices, not row ordinals
<code>dc_branch_flows[]</code>	<code>index, name, from_bus, to_bus, pf_mw, pt_mw, loss_mw, loading_pct</code>	component identity from <code>DCBranch</code> ; flow values from <code>result.dc_branch_flows</code> when present	DC branch-flow rows
<code>vsc_transfers[]</code>	<code>index, bus_ac, bus_dc, p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw</code>	copied from <code>result.vsc_transfers</code> aligned with VSC order	AC/DC converter transfer rows
<code>dcdc_transfers[]</code>	<code>index, bus_in, bus_out, p_in_mw, p_out_mw, loss_mw</code>	copied from <code>result.dcdc_transfers</code> aligned with DC/DC converter order	DC/DC converter transfer rows

11.6 Dedicated Power-Flow Result Sheets

Results_ACBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_ACBus.index</code>	copied from <code>sys.ac.buses[i].index</code>	stable AC bus identifier
<code>name</code>	<code>Results_ACBus.name</code>	copied from <code>sys.ac.buses[i].name</code>	descriptive bus label
<code>vm_pu, va_deg</code>	same columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	solved AC voltage magnitude and angle in bus-vector order

Results_DCBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_DCBus.index</code>	copied from <code>sys.dc.buses[i].index</code>	stable DC bus identifier
<code>name</code>	<code>Results_DCBus.name</code>	copied from <code>sys.dc.buses[i].name</code>	descriptive bus label
<code>vm_pu</code>	<code>Results_DCBus.vm_pu</code>	copied from <code>result.vdc[i]</code> ; out-of-range fallback 0.0	solved DC voltage; the sheet keeps the AC-style column name <code>vm_pu</code>

Results_BranchFlow

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_BranchFlow.index</code>	copied from <code>sys.ac.branches[i].index</code>	source AC branch identifier
<code>from_bus, to_bus</code>	same columns	copied from <code>sys.ac.branches[i]</code> when present; fallback 0	terminal buses of the reported AC branch row
<code>pf_mw, qf_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	sending-end active and reactive power
<code>pt_mw, qt_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	receiving-end active and reactive power
<code>loss_mw</code>	<code>Results_BranchFlow.loss_mw</code>	computed as $\max(\text{pf_mw} + \text{pt_mw}, 0.0)$	exported AC branch loss
<code>loading_pct</code>	<code>Results_BranchFlow.loading_pct</code>	computed from the larger terminal apparent power over <code>rate_a_mva</code> , system base MVA, or AC base MVA fallback	branch loading percentage

Results_DCBranch

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name</code>	corresponding columns	copied from <code>sys.dc.branches[i]</code>	stable identifier and label
<code>from_bus, to_bus</code>	same columns	copied from <code>sys.dc.branches[i]</code>	terminal DC bus indices
<code>in_service</code>	same column	copied from <code>sys.dc.branches[i].in_service</code> via <code>bool_str</code>	availability flag in workbook boolean-token form
<code>v_from_pu, v_to_pu</code>	same columns	looked up from solved <code>result.vdc</code> by branch terminal bus; fallback 1.0	solved DC terminal voltages
<code>i_pu</code>	<code>Results_DCBranch.i_pu</code>	computed as $(v_{\text{from}} - v_{\text{to}})/r_{\text{pu}}$ when in service and <code>r_pu</code> >0; else 0.0	derived branch current on system base
<code>p_from_mw, p_to_mw</code>	same columns	computed from solved terminal voltages, derived current, and system base MVA	terminal powers implied by the solved DC branch state
<code>loss_mw</code>	<code>Results_DCBranch.loss_mw</code>	computed as <code>p_from_mw + p_to_mw</code>	exported branch-loss figure

Field	Workbook sheet / column	Population rule	Meaning / note
loading_pct	Results_DCBranch.loading_pct	computed from terminal active powers over <code>s_max_mva</code> , <code>rate_a_mva</code> , or base MVA fallback	DC branch loading percentage

Results_VSCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus_ac, bus_dc	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable converter identity and terminal buses
in_service, control_mode	same columns	copied from the input converter object	static converter metadata, not a solver-selected mode switch
p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw	same columns	taken from matching <code>result.vsc_transfers[index]</code> when available; fallback uses <code>p_set_mw</code> , <code>q_set_mvar</code> , and stored <code>loss_mw</code>	solved transfer report with deterministic metadata fallback
p_set_mw, q_set_mvar	same columns	copied from <code>sys.vsc_converters[i]</code>	original setpoint metadata retained beside solved transfer values
v_ac_pu, v_dc_pu	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal AC and DC voltage annotations
eta, loss_percent	same columns	copied from the input converter object	static efficiency / loss metadata

Results_DCDCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus_in, bus_out	corresponding columns	copied from <code>sys.dcdc_converters[i]</code>	stable converter identity and terminal buses
in_service, control_mode	same columns	copied from the input converter object	static converter metadata
p_ref_mw, v_ref_pu, eta	same columns	copied from <code>sys.dcdc_converters[i]</code>	original reference and efficiency metadata
v_in_pu, v_out_pu	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal DC voltage annotations
p_in_mw, p_out_mw, loss_mw	same columns	taken from matching <code>result.dcdc_transfers[index]</code> when available; fallback uses <code>p_ref_mw</code> and derived zero-loss default	solved DC/DC transfer report with metadata fallback

Results_Storage_AC

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.ac_storage[i]</code>	stable identity, connected bus, and availability

Field	Workbook sheet / column	Population rule	Meaning / note
p_mw, q_mvar	same columns	copied from <code>sys.ac.storage[i]</code>	current storage dispatch snapshot; not solved from a storage state estimator
soc	Results_Storage_AC.soc	copied from <code>sys.ac.storage[i].soc_init</code>	reported state of charge is the input initial SOC
soh	Results_Storage_AC.soh	copied from <code>sys.ac.storage[i].soh</code>	stored state-of-health metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>bus</code> ; fallback 1.0/0.0	bus-state annotation for the storage location

Results_Storage_DC

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.dc.storage[i]</code>	stable identity, connected bus, and availability
p_mw, q_mvar	same columns	copied from <code>sys.dc.storage[i]</code>	current storage dispatch snapshot
soc	Results_Storage_DC.soc	copied from <code>sys.dc.storage[i].soc_init</code>	reported state of charge is the input initial SOC
soh	Results_Storage_DC.soh	copied from <code>sys.dc.storage[i].soh</code>	stored state-of-health metadata
vdc_pu	Results_Storage_DC.vdc_pu	solved DC voltage looked up by <code>bus</code> ; fallback 1.0	bus-state annotation for the storage location

Results_ChargingStation

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from <code>sys.ac.charging_stations[i]</code>	stable station identity, connected bus, and availability
num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate	same columns	copied from <code>sys.ac.charging_stations[i]</code>	current aggregate station metadata; no per-solver charging dispatch reconstruction is performed here
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>bus</code> ; fallback 1.0/0.0	bus-state annotation for the station location

Results_Charger

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, station_id	corresponding columns	copied from <code>sys.ac.chargers[i]</code>	stable charger identity and parent-station link
station_bus	Results_Charger.station_bus	joined from the parent charging-station map by <code>station_id</code> ; fallback 0	resolved AC bus of the parent charging station

Field	Workbook sheet / column	Population rule	Meaning / note
<code>in_service</code> , <code>charger_type</code> , <code>p_rate_d_kw</code> , <code>p_ch_max_kw</code> , <code>p_dis_max_kw</code> , <code>eta</code> , <code>v2g_capable</code>	same columns	copied from <code>sys.ac.chargers[i]</code>	static charger metadata repeated in the result workbook
<code>vm_pu</code> , <code>va_deg</code>	same columns	solved AC bus voltage looked up by the resolved <code>station_bus</code> ; fallback 1.0/0.0	station-bus voltage annotation, not per-charger internal electrical state

Results_EnergyRouter

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>in_service</code> , <code>router_type</code> , <code>num_ports</code> , <code>pRated_mw</code> , <code>loss_percent</code>	corresponding columns	copied from <code>sys.energy_routers[i]</code>	router metadata preserved in the result workbook
<code>p_port_sum_mw</code> , <code>q_port_sum_mvar</code>	same columns	computed by summing the stored router-port <code>p_mw</code> / <code>q_mvar</code> values	router-level summary over the current port structs; it does not re-sum the solved transfer table

Results_EnergyRouterPort

Field	Workbook sheet / column	Population rule	Meaning / note
<code>router_index</code> , <code>port_index</code> , <code>port_name</code> , <code>bus</code>	corresponding columns	copied from each nested port row under <code>sys.energy_routers[*].ports[*]</code>	flattened router / port identity and connected bus
<code>port_type</code> , <code>control_mode</code> , <code>in_service</code>	same columns	copied from the nested port object	static port metadata
<code>p_mw</code> , <code>q_mvar</code> , <code>v_solved_pu</code>	same columns	taken from the matching solved router-port transfer when available; fallback uses the port's stored <code>p_mw</code> , <code>q_mvar</code> , and solved bus voltage	solved per-port transfer report with deterministic metadata fallback

Results_VPP

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>aggregation_bus</code> , <code>in_service</code>	corresponding columns	copied from <code>sys.vpps[i]</code>	stable VPP identity and aggregation location
<code>p_output_mw</code> , <code>q_output_mvar</code> , <code>pmax_mw</code> , <code>pmin_mw</code>	same columns	copied from <code>sys.vpps[i]</code>	current VPP aggregate output / limits metadata

Field	Workbook sheet / column	Population rule	Meaning / note
vm_pu, va_deg	same columns	solved AC bus voltage looked up by aggregation_bus ; fallback 1.0/0.0	aggregation-bus state annotation

Results_Microgrid

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, pcc_bus, operating_mode, in_service	corresponding columns	copied from sys.microgrids[i]	microgrid identity and current mode metadata
p_exchange_mw, total_generation_mw, total_load_mw	same columns	copied from sys.microgrids[i]	current aggregate exchange / generation / demand snapshot
vm_pu, va_deg	same columns	solved AC bus voltage looked up by pcc_bus ; fallback 1.0/0.0	PCC bus-state annotation
internal_buses	Results_Microgrid.internal_buses	comma-joined string formed from the stored microgrid membership vector	flattened membership list; workbook does not preserve the vector structure directly

Results_MobileStorage

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, status, in_service	corresponding columns	copied from sys.mobile_storage[i]	mobile-storage identity, current location, and mobility state metadata
p_mw, q_mvar	same columns	copied from sys.mobile_storage[i]	current dispatch snapshot
soc	Results_MobileStorage.soc	copied from sys.mobile_storage[i].soc_init	reported state of charge is the input initial SOC
target_bus	Results_MobileStorage.target_bus	copied from sys.mobile_storage[i].target_buses	relocation target metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by the current bus ; fallback 1.0/0.0	current-bus state annotation

Results_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	Results_Summary row with Key=converged	written from result.converged via bool_string	PF convergence flag using workbook boolean-token form
iterations	Results_Summary row with Key=iterations	written from result.iterations	total PF iteration count
residual	Results_Summary row with Key=residual	written from result.residual	final PF residual metric

11.7 Compact Power-Flow Result Columns

Field	Workbook sheet / column	Population rule	Meaning / note
<code>result_vm_pu</code> , <code>result_va_deg</code>	ACBus appended columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> in AC bus order	compact PF solved AC bus state
<code>result_vdc_pu</code>	DCBus appended column	copied from <code>result.vdc</code> <code>[i]</code> in DC bus order	compact PF solved DC bus voltage
<code>result_pf_mw</code> , <code>result_qf_mvar</code> , <code>result_pt_mw</code> , <code>result_qt_mvar</code>	ACBranch appended columns	copied from <code>result.branch</code> <code>flows[i]</code>	compact PF AC branch- flow report
<code>result_v_from_pu</code> , <code>result_v_to_pu</code> , <code>result_i_pu</code> , <code>result_p_from_mw</code> , <code>result_p_to_mw</code> , <code>result_loss_mw</code>	DCBranch appended columns	recomputed from solved DC voltages and branch resistance, same as <code>Results_DCBranch</code>	compact PF DC branch report
<code>result_vm_pu</code> , <code>result_va_deg</code>	Storage, Load, ACLoad, ChargingStation, VPP, Microgrid, MobileStorage appended columns	solved AC voltage looked up by each row's connected bus or aggregation bus	compact PF AC-side bus- state annotation on input tables
<code>result_vdc_pu</code>	DCStorage, DCLoad appended column	solved DC voltage looked up by each row's connected bus	compact PF DC-side bus- state annotation on input tables
<code>result_p_ac_mw</code> , <code>result_q_ac_mvar</code> , <code>result_p_dc_mw</code> , <code>result_loss_mw</code> , <code>result_v_ac_pu</code> , <code>result_v_dc_pu</code>	VSCConverter appended columns	populated from matching <code>result.vsc_transfers[index]</code> plus solved terminal voltages	compact PF converter transfer report
<code>result_p_in_mw</code> , <code>result_p_out_mw</code> , <code>result_loss_mw</code> , <code>result_v_in_pu</code> , <code>result_v_out_pu</code>	DCDCConverter appended columns	populated from matching <code>result.dcdc_transfers[index]</code> plus solved terminal voltages	compact PF DC/DC con- verter report
<code>Results_Summary</code>	separate sheet still written in compact PF export	same key-value rows as the dedicated PF result workbook	compact PF export still carries a standalone sum- mary sheet

11.8 Dedicated OPF Sheets

OPF_Bus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>base_kv</code>	corresponding columns	copied from <code>sys.ac.buses[i]</code>	stable bus identity and base- voltage metadata
<code>vm_pu</code> , <code>va_deg</code>	same columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	optimal AC bus voltage solu- tion
<code>dpd_mw</code> , <code>dqd_mvar</code>	same columns	copied from <code>result.dpd</code> <code>_mw[i]</code> and <code>result.dqd</code> <code>_mvar[i]</code> ; out-of-range fallback 0.0	parity-formulation bus-level active / reactive load shedding

OPF_Generator

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus	corresponding columns	copied from <code>sys.ac.generators[i]</code>	stable generator identity and connected bus
pg_mw, qg_mvar	same columns	copied from <code>result.pg_mw[i]</code> and <code>result.qg_mvar[i]</code> ; fallback to input <code>pg_mw/qg_mvar</code>	optimal generator dispatch
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	copied from <code>sys.ac.generators[i]</code>	generator capability metadata retained beside the optimal point
cost_c2, cost_c1, cost_c0, fuel_type	same columns	copied from <code>sys.ac.generators[i]</code>	cost-model and fuel metadata used for interpretation

OPF_Converter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus_ac, bus_dc	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable VSC identity and terminal buses
pac_mw, qac_mvar	same columns	copied from <code>result.pac_mw[i]</code> and <code>result.qac_mvar[i]</code> ; out-of-range fallback 0.0	optimal AC-side converter operating point

OPF_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	OPF_Summary row with Key=converged	written from <code>result.converged</code> via <code>bool_str</code>	OPF convergence flag
iterations	OPF_Summary row with Key=iterations	written from <code>result.iterations</code>	total inner-iteration count
outer_iterations	OPF_Summary row with Key=outer_iterations	written from <code>result.outer_iterations</code>	barrier / outer-loop iteration count
objective	OPF_Summary row with Key=objective	written from <code>result.objective</code>	final objective value
max_constraint_violation	OPF_Summary row with Key=max_constraint_violation	written from <code>result.max_constraint_violation</code>	worst final feasibility residual
max_stationarity	OPF_Summary row with Key=max_stationarity	written from <code>result.max_stationarity</code>	worst final first-order optimality metric
status	OPF_Summary row with Key=status	written from <code>result.status</code>	textual solver termination status

ACOPFResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status	top-level scalar keys	bidirectional; missing keys default to false, zero, or empty string	OPF scalar summary block

Field	JSON key / path	Export / import rule	Meaning / note
vm, va	top-level arrays	bidirectional; absent array keeps the vector empty	bus-voltage vectors in bus order
pg_mw, qg_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	generator dispatch vectors in generator order
dpd_mw, dqd_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	bus-level parity load-shedding vectors
pac_mw, qac_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	VSC AC-side operating-point vectors
profiling.linear_solver_backend	profiling[linear_solver_backend]	bidirectional; default empty string	linear solver backend name recorded by the OPF run
profiling.analyze_calls, profiling.linear_solve_calls	same nested keys	bidirectional; default 0	OPF linear-solver call counters
factorization_calls	same nested key	bidirectional; default 0	OPF sparse factorization counter
profiling.total_ iterations	profiling[total_ iterations]	bidirectional; default 0	total OPF inner iterations reported by the solver
profiling.accepted_steps	same nested key	bidirectional; default 0	accepted line-search steps
profiling.rejected_steps	same nested key	bidirectional; default 0	rejected line-search steps
profiling.final_barrier_mu	profiling[final_barrier_mu]	bidirectional; default 0.0	final barrier parameter reported by the IPM path
pren_mw, qren_mvar, pstor_mw, qstor_mvar, pdcdc_mw, pflex_mw and companion maps	top-level legacy arrays plus renewable_dispatch, storage_dispatch, dcdc_dispatch, flex_dispatch; workbook sheets OPF_Renewable, OPF_Storage, OPF_DCDC, OPF_FlexLoad	exported/imported in JSON; exported in OPF workbooks when the corresponding vectors are non-empty	enhanced component-dispatch vectors use packed solver-variable order with ComponentRef identity maps, source_type, and component_index metadata

11.9 Carbon Result JSON and Workbook Dictionaries

CarbonAnalysisResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
tracing_verified, matrix_solved	top-level scalar booleans	bidirectional; default false	method-level status flags for tracing and matrix solutions
matrix_residual	top-level scalar number	bidirectional; default 0.0	residual metric from the matrix method
load_carbon[] and dc_load_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	per-load tracing results for AC loads and DC loads respectively
*.load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	nested keys on each AC/DC load row	bidirectional; defaults 0 or 0.0	flat load-level carbon report fields
per-load generator_supply_mw map	nested object map keyed by source id	bidirectional; absent map keeps the per-load map empty	source-contribution map preserved only in JSON form

Field	JSON key / path	Export / import rule	Meaning / note
branch_carbon[] and dc_branch_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	per-branch loss-allocation results for AC branches and DC branches respectively
*.branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	nested keys on each AC/DC branch row	bidirectional; defaults 0 or 0.0	flat branch-loss carbon report fields
per-branch generator_loss_mw map	nested object map keyed by source id	bidirectional; absent map keeps the per-branch map empty	source loss-allocation map preserved only in JSON form
bus_carbon[] and dc_bus_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	matrix-based AC-bus and DC-bus carbon intensities
*.bus_index, carbon_intensity_tco2_mwh	nested keys on each AC/DC bus row	bidirectional; defaults 0 or 0.0	explicit bus-level carbon report fields
vsc_carbon[]	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for VSC links in projected-converter order
vsc_carbon[].converter_index, bus_ac, bus_dc, ac_to_dc, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	nested keys on each VSC row	bidirectional; defaults 0, false, or 0.0	reports actual hybrid transfer direction plus converter loss carbon intensity
dc_dc_carbon[]	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for DCDC links in projected-converter order
dc_dc_carbon[].converter_index, bus_in, bus_out, input_to_output, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	nested keys on each DCDC row	bidirectional; defaults 0, false, or 0.0	reports actual DC/DC transfer direction plus converter loss carbon intensity
storage_carbon[]	top-level array	bidirectional; absent array keeps the vector empty	per-storage carbon report covering both explicit discharge-side sources and traced charging-side sinks
storage_carbon[].storage_index, bus, is_dc, p_mw, soc, stored_energy_mwh, soc_carbon_intensity_tco2_mwh, carbon_intensity_tco2_mwh, total_emissions_tco2	nested keys on each storage row	bidirectional; defaults 0, false, or 0.0	p_mw keeps the bus-injection sign convention; charging rows additionally preserve a source-attribution map in JSON

Field	JSON key / path	Export / import rule	Meaning / note
<code>energy_router_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	per-router internal-loss report reconstructed from port transfer signs and magnitudes
<code>energy_router_carbon[].router_index, input_power_mw, output_power_mw, loss_mw, active_input_ports, active_output_ports, carbon_intensity_tco2_mwh, total_emissions_tco2</code>	nested keys on each router row	bidirectional; defaults 0 or 0.0	router loss is derived from actual port mismatch; JSON preserves the source-loss map
<code>generator_loss_allocation</code> entries	top-level object map keyed by source id	bidirectional; absent map keeps the allocation map empty	total MW of branch and converter losses allocated to each internal source id
<code>tracing_summary.*, matrix_summary.*</code>	nested summary objects	bidirectional; absent object keeps defaults	system-level emissions summary blocks for the tracing and matrix methods
<code>tracing_summary.total_generation_emissions_tco2, total_load_emissions_tco2, total_loss_emissions_tco2, balance_error_tco2, balance_error_pct</code>	same nested keys under both summary objects	bidirectional; default 0.0	emissions-balance totals and consistency metrics

LoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2</code>	corresponding columns	copied from each <code>result.load_carbon[i]</code> row	flat workbook view of the per-load carbon result without the generator-contribution map

DCLoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dc_load_carbon[i]</code> row	flat workbook view of the per-DC-load carbon result without the source-contribution map

BusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>bus_index</code> , <code>carbon_intensity_tco2_mwh</code>	corresponding columns	copied from each <code>result.bus_carbon[i]</code> row	flat workbook view of the per-bus carbon intensity

DCBusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>bus_index</code> , <code>carbon_intensity_tco2_mwh</code>	corresponding columns	copied from each <code>result.dc_bus_carbon[i]</code> row	flat workbook view of the per-DC-bus carbon intensity

BranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>branch_index</code> , <code>from_bus</code> , <code>to_bus</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.branch_carbon[i]</code> row	flat workbook view of the per-branch carbon-loss result without the generator-loss map

DCBranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>branch_index</code> , <code>from_bus</code> , <code>to_bus</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dc_branch_carbon[i]</code> row	flat workbook view of the per-DC-branch carbon-loss result without the source-loss map

VSCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>converter_index</code> , <code>bus_ac</code> , <code>bus_dc</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	stable converter identifier and terminal buses
<code>ac_to_dc</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	actual solved transfer direction, not merely the input setpoint sign
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	carrier input, carrier output, and converter loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

DCDCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>converter_index</code> , <code>bus_in</code> , <code>bus_out</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	stable converter identifier and declared input/output buses
<code>input_to_output</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	actual solved transfer direction relative to the declared <code>bus_in/bus_out</code> pair
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	converter input, converter output, and DCDC loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

StorageCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>storage_index</code> , <code>bus</code>	corresponding columns	copied from each <code>result.storage_carbon[i]</code> row	stable storage identifier and connected bus
<code>is_dc</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	distinguishes AC storage from DC storage when indices overlap across subsystems
<code>p_mw</code>	corresponding column	copied from <code>result.storage_carbon[i].p_mw</code>	signed storage dispatch in bus-injection convention; positive means discharge, negative means charging sink
<code>soc</code> , <code>stored_energy_mwh</code>	corresponding columns	copied from the storage carbon result row	reported storage state snapshot used by the carbon model
<code>soc_carbon_intensity_tco2_mwh</code>	corresponding column	copied from the storage carbon result row	explicit carbon intensity of stored energy used when the unit discharges
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from the storage carbon result row	current-mode carbon intensity and emissions; charging uses traced incoming intensity, discharging uses SOC intensity

EnergyRouterCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>router_index</code>	corresponding column	copied from each <code>result.energy_router_carbon[i]</code> row	stable energy-router identifier
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from the router carbon result row	router-wide input, output, and internally allocated loss reconstructed from solved port transfers
<code>active_input_ports</code> , <code>active_output_ports</code>	corresponding columns	copied from the router carbon result row	number of non-zero sending and receiving ports participating in the router carbon graph
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from the router carbon result row	loss-side carbon intensity and the corresponding router-loss emissions

Summary in the carbon workbook

Field	Workbook sheet / cell block	Population rule	Meaning / note
TracingMethod	Summary[A1]	literal label written at row 1	heading for the tracing summary block
tracing_summary. total_generation_ emissions_tco2	Summary[A2:B2]	key literal plus result .tracing_summary.tot al_generation_emissi ons_tco2	tracing-method generation emissions total
tracing_summary. total_load_emiss ions_tco2	Summary[A3:B3]	key literal plus result .tracing_summary.tot al_load_emissions_tc o2	tracing-method load emis sions total
tracing_summary. total_loss_emiss ions_tco2	Summary[A4:B4]	key literal plus result .tracing_summary.tot al_loss_emissions_tc o2	tracing-method loss emis sions total
tracing_summary. balance_error_tc o2	Summary[A5:B5]	key literal plus result .tracing_summary.bal ance_error_tco2	tracing-method absolute emissions-balance error
tracing_summary. balance_error_pc t	Summary[A6:B6]	key literal plus result .tracing_summary.bal ance_error_pct	tracing-method relative emissions-balance error
tracing_verified	Summary[A7:B7]	key literal plus lower- case string true/false	tracing verification flag; note the lowercase boolean spelling differs from PF / OPF workbooks
MatrixMethod	Summary[A9]	literal label written at row 9	heading for the matrix- summary block
matrix_summary.t otal_generation_ emissions_tco2	Summary[A10:B10]	key literal plus result .matrix_summary.tota l_generation_emissio ns_tco2	matrix-method generation emissions total
matrix_summary.t otal_load_emissi ons_tco2	Summary[A11:B11]	key literal plus result .matrix_summary.tota l_load_emissions_tco2	matrix-method load emis sions total
matrix_summary.t otal_loss_emissi ons_tco2	Summary[A12:B12]	key literal plus result .matrix_summary.tota l_loss_emissions_tco2	matrix-method loss emis sions total
matrix_summary.b alance_error_tco 2	Summary[A13:B13]	key literal plus result .matrix_summary.bala nce_error_tco2	matrix-method absolute emissions-balance error
matrix_summary.b alance_error_pct	Summary[A14:B14]	key literal plus result .matrix_summary.bala nce_error_pct	matrix-method relative emissions-balance error
matrix_solved	Summary[A15:B15]	key literal plus lower- case string true/false	matrix-solver success flag
matrix_residual	Summary[A16:B16]	key literal plus result .matrix_residual	matrix-method residual metric

11.10 Current Output-Side Gaps

Area	Artifact	Status	Engineering consequence
PowerFlowResult JSON import	PF JSON	export only	PF result JSON can be written, but there is no matching public result-from-JSON reader.
PF profiling block	PF workbooks	not exported	the in-memory <code>PowerFlowResult.profil ing</code> structure is not written to workbook output.

Area	Artifact	Status	Engineering consequence
ACOPFResult enhanced dispatch vectors	OPF JSON / workbook	exported with packed semantics	Renewable, storage, DC/DC, and flexible-load dispatch vectors are exported as packed variable arrays with companion <code>ComponentRef</code> identity maps. JSON uses structured <code>renewable_dispatch</code> / <code>storage_dispatch</code> / <code>dc_dc_dispatch</code> / <code>flex_dispatch</code> objects; workbook sheets use <code>var_idx</code> , <code>source_type</code> , and <code>component_index</code> columns.
carbon contribution maps	carbon workbook	not exported	<code>generator_supply_mw</code> , <code>generator_loss_mw</code> , <code>source_supply_mw</code> , <code>source_loss_mw</code> , and <code>generator_loss_allocation</code> are available only in JSON.
OPF_Converter	OPF workbook	conditional population	the sheet is only populated when VSC converters exist; otherwise the sheet may stay empty even though the workbook pre-creates it.
short-circuit outputs	workbook / JSON files	in-memory only	short-circuit result structs and APIs are available in <code>hacdcpf/analysis/short_circuit.hpp</code> , but no workbook or JSON serializer is currently exposed for them.
ONR / topology reconfiguration outputs	workbook / JSON files	in-memory only	<code>solve_optimal_reconfiguration</code> and <code>run_topology_reconfiguration</code> return structured in-memory result objects; no workbook or JSON output contract is currently exposed for these results.

12 Power-Flow Models

12.1 Scope and Architecture Overview

This chapter documents the steady-state hybrid AC/DC power-flow (PF) module in `src/power_flow/`. The legacy engine include path `include/hacdcpf/engine/solver/native/nle/newton_solver.hpp` is a declaration/compatibility entry point for the same Newton class; there is no corresponding `src/engine` implementation directory in this repository.

The facade in `include/hacdcpf/api/hacdcpf.hpp` exposes these public PF entry points:

Public API	Primary implementation / declaration	Scope
<code>solve_power_flow</code>	<code>src/power_flow/hybrid.cpp</code>	Hybrid AC/DC Newton through <code>powerflow::solve_hybrid</code> .
<code>solve_dc_power_flow</code>	<code>src/power_flow/dc.cpp</code> / <code>dc_solver.cpp</code>	DC-only Newton result wrapper.
<code>solve_power_flow_adaptive</code>	<code>src/power_flow/adaptive_solver.cpp</code>	Adaptive island-aware solve through <code>powerflow::solve_hybrid_adaptive</code> .
<code>solve_power_flow_islanded</code>	<code>src/power_flow/adaptive_solver.cpp</code>	Explicit islanded solve wrapper.

Public API	Primary implementation / declaration	Scope
<code>solve_power_flow_distributed_slack</code>	<code>src/power_flow/distributed_slack_solver.cpp</code>	Simplified distributed-slack solve.
<code>solve_power_flow_distributed_slack_full</code>	<code>src/power_flow/distributed_slack_solver.cpp</code>	Full-Jacobian distributed-slack solve.
<code>create_solver_handle, reset_solver_handle, solve_handle, solve_dc_handle, destroy_solver_handle</code>	API handle facade	Reusable solver-handle lifecycle and solve calls.
<code>solve_power_flow_fast_dp</code>	<code>src/power_flow/fast_dp_solver.cpp</code>	Fast-decoupled AC-only PF.
<code>solve_ac_dc_power_flow</code>	<code>src/power_flow/analytic_linearized_pf.cpp</code>	Linearized AC-angle/DC result helper.

Table 123: Public power-flow API family.

The lower-level `hacdcpf::powerflow` namespace additionally declares `solve_ac`, `solve_dc`, `FDPFSolver::solve`, `AdaptiveSolver::solve`, `DistributedSlackSolver` simplified and full-Jacobian solves, `HelmSolver::solve`, `CpfSolver::solve`, and `HomotopyContinuationSolver::solve`. The header-only `three_phase.hpp` wrapper references an analysis-side `solve_three_phase_nr` symbol that is not part of the public `hacdcpf.hpp` facade.

The standard PF paths share canonical preprocessing through model projection, `solver_data.cpp`, and `admittance_builder.cpp`. The main Newton kernel is implemented in `src/power_flow/newton_solver.cpp`; its class is declared through the legacy engine header and re-exported from `include/hacdcpf/power_flow/newton_solver.hpp`.

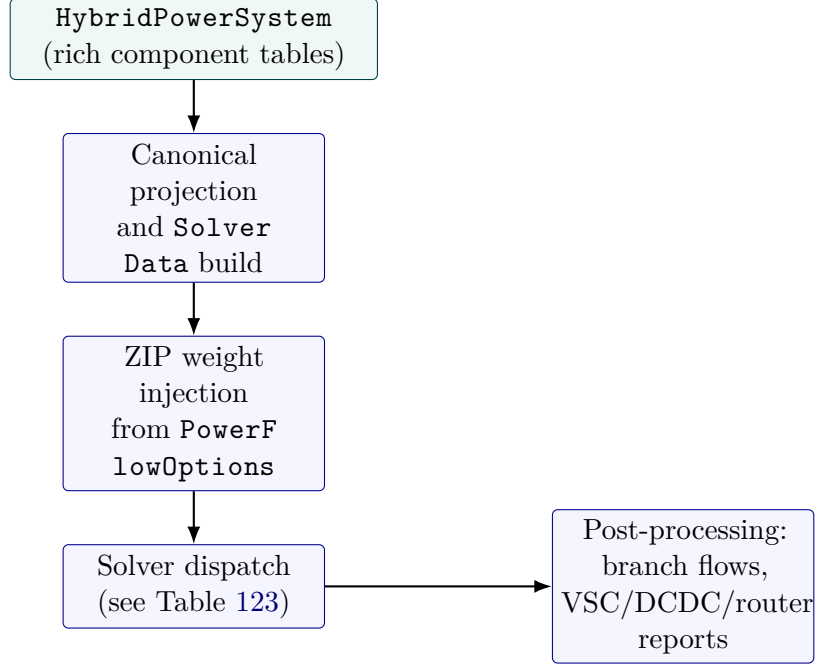


Figure 1: Shared execution pipeline for all power-flow API variants.

12.2 Mathematical Problem Form

The Newton kernel solves the hybrid residual system

$$F(x) = 0, \quad x = \begin{bmatrix} \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \\ v_{\mathcal{N}_{PQ}} \\ v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}}^{\text{dc}} \end{bmatrix}, \quad (25)$$

where $\mathcal{N}_{sl}$ and $\mathcal{N}_{sl}^{\text{dc}}$ are the sets of AC and DC slack buses, and $\mathcal{N}_{PQ}$ is the set of PQ buses.

AC bus mismatch. For each non-slack AC bus i ,

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (26)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad i \in \mathcal{N}_{PQ}, \quad (27)$$

where the calculated nodal injections are

$$P_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \cos \theta_{ij} + B_{ij} \sin \theta_{ij}), \quad (28)$$

$$Q_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \sin \theta_{ij} - B_{ij} \cos \theta_{ij}), \quad (29)$$

with $\theta_{ij} = \theta_i - \theta_j$ and $(G_{ij} + jB_{ij})$ the (i, j) entry of the sparse bus admittance matrix Y_{bus} .

DC bus mismatch. For each non-DC-slack bus k ,

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}, \quad (30)$$

$$P_k^{\text{dc,calc}} = v_k^{\text{dc}} \left(G_{dc} v^{\text{dc}} \right)_k. \quad (31)$$

Specified-injection assembly.

$$P_i^{\text{spec}} = P_i^{\text{g,agg}} - P_i^{\text{load}}(v_i) + P_i^{\text{vsc}}, \quad (32)$$

$$Q_i^{\text{spec}} = Q_i^{\text{g,agg}} - Q_i^{\text{load}}(v_i) + Q_i^{\text{vsc}}, \quad (33)$$

where the aggregated generation vectors are

$$P_i^{\text{g,agg}} = P_i^{\text{gen}} + P_i^{\text{sg}} + P_i^{\text{ren}} + P_i^{\text{pv}} + P_i^{\text{stor}} + P_i^{\text{vpp}} + P_i^{\text{mg}} + P_i^{\text{ms}}, \quad (34)$$

$$Q_i^{\text{g,agg}} = Q_i^{\text{gen}} + Q_i^{\text{sg}} + Q_i^{\text{ren}} + Q_i^{\text{pv}} + Q_i^{\text{stor}} + Q_i^{\text{vpp}} + Q_i^{\text{ms}}. \quad (35)$$

Here P_i^{mg} is microgrid exchange at the PCC bus in `GridConnected` mode.

ZIP demand model. The runtime voltage-dependent load model is

$$P_i^{\text{load}}(v_i) = P_{i,0}(w_{P,0,i} + w_{P,1,i}v_i + w_{P,2,i}v_i^2), \quad (36)$$

$$Q_i^{\text{load}}(v_i) = Q_{i,0}(w_{Q,0,i} + w_{Q,1,i}v_i + w_{Q,2,i}v_i^2). \quad (37)$$

The ZIP weights are populated in one of three assembly cases: (1) explicit Load/charging-station tables present, (2) charging-station tables only, (3) neither table — global `zip_pw/zip_qw` from `PowerFlowOptions` are used.

12.3 VSC Converter Model

The converter model uses bus-injection-positive signs on both AC and DC sides. Three control modes are supported.

Loss model. With active transfer p (p.u.), AC-side proxy current $I^{\text{ac}} = |p|/v^{\text{dc,safe}}$, $v^{\text{dc,safe}} = \max(v^{\text{dc}}, 0.1)$:

$$P^{\text{loss,lin}} = (1 - \eta)|p|, \quad (38)$$

$$P^{\text{loss,cur}} = a + b I^{\text{ac}} + c (I^{\text{ac}})^2, \quad a = \frac{\text{loss_mw}}{S_{\text{base}}}, \quad b = \frac{\text{loss_pct}}{100}, \quad c = 1 - \eta. \quad (39)$$

PQ_MODE.

$$P^{\text{conv,ac}} = P^{\text{set}}, \quad Q^{\text{conv,ac}} = Q^{\text{set}}, \quad P^{\text{conv,dc}} = -(P^{\text{set}} + P^{\text{loss}}). \quad (40)$$

VDC_Q and VDC_VAC.

$$P^{\text{tr}} = k^{\text{vdc}}[(v^{\text{dc}})^2 - (v^{\text{dc,set}})^2], \quad P^{\text{conv,ac}} = P^{\text{tr}} - P^{\text{loss}}, \quad P^{\text{conv,dc}} = -P^{\text{tr}}. \quad (41)$$

In `VDC_VAC` mode the Newton loop enforces $v^{\text{ac}} = v^{\text{ac,set}}$ for the converter AC bus by hard-clamping the trial AC voltage after updates. If `SolverData::enable_augmented_equations` is enabled, that clamp is skipped and the VDC/VAC bus is instead handled by the augmented-equation path in `newton_solver.cpp`.

12.4 DCDC Converter Model

The DCDC converter connects `bus_in` to `bus_out`. Power `p_ref_mw` is output-side referenced.

$$P^{\text{in}} = \begin{cases} P^{\text{ref}}/\eta, & P^{\text{ref}} \geq 0, \\ P^{\text{ref}} \cdot \eta, & P^{\text{ref}} < 0, \end{cases} \quad P^{\text{out}} = P^{\text{ref}} - r_{eq} \frac{(P^{\text{in}})^2}{(v^{\text{dc,in}})^2}. \quad (42)$$

12.5 Jacobian Structure

The linearized Newton system is

$$J \Delta x = F(x), \quad J = \begin{bmatrix} J_{\theta\theta} & J_{\theta V} & J_{\theta V^{dc}} \\ J_{V\theta} & J_{VV} & J_{VV^{dc}} \\ 0 & J_{V^{dc}V} & J_{V^{dc}V^{dc}} \end{bmatrix}. \quad (43)$$

The sparse Jacobian pattern is cached in a `PatternCache` keyed by matrix storage addresses and a monotone `build_id`, and is rebuilt only on topology changes.

AC diagonal entries (with ZIP load corrections).

$$\frac{\partial \Delta P_i}{\partial v_i} = \frac{P_i^{\text{calc}}}{v_i} + G_{ii}v_i - P_{i,0}(w_{P,1,i} + 2w_{P,2,i}v_i), \quad (44)$$

$$\frac{\partial \Delta Q_i}{\partial v_i} = \frac{Q_i^{\text{calc}}}{v_i} - B_{ii}v_i - Q_{i,0}(w_{Q,1,i} + 2w_{Q,2,i}v_i). \quad (45)$$

In the code, v_i is floored at `min_vm_pu` (configurable via `RobustNonlinearOptions`) to prevent zero-division.

DC entries.

$$\frac{\partial P_k^{\text{dc,calc}}}{\partial v_m^{\text{dc}}} = \begin{cases} (G_{dc}v^{\text{dc}})_k + G_{kk}^{\text{dc}}v_k^{\text{dc}}, & m = k, \\ G_{km}^{\text{dc}}v_k^{\text{dc}}, & m \neq k. \end{cases} \quad (46)$$

AC/DC coupling. When `enable_coupled_jacobian` is set, the converter Jacobian blocks $\partial P^{\text{vsc}}/\partial v^{\text{dc}}$, $\partial Q^{\text{vsc}}/\partial v^{\text{dc}}$, and $\partial P^{\text{dc}}/\partial v^{\text{ac}}$ have pattern slots. The current `converter_dc_jacobian_vm_ac` implementation returns zero because the old AC-side converter resistance term was removed, so the $\partial P^{\text{dc}}/\partial v^{\text{ac}}$ block is structurally present but numerically zero today. The coupling entries are pre-registered as `CouplingEntry` records in `JacobianPattern` and updated in-place each iteration without re-analyzing the pattern.

12.6 Hybrid Newton Algorithm

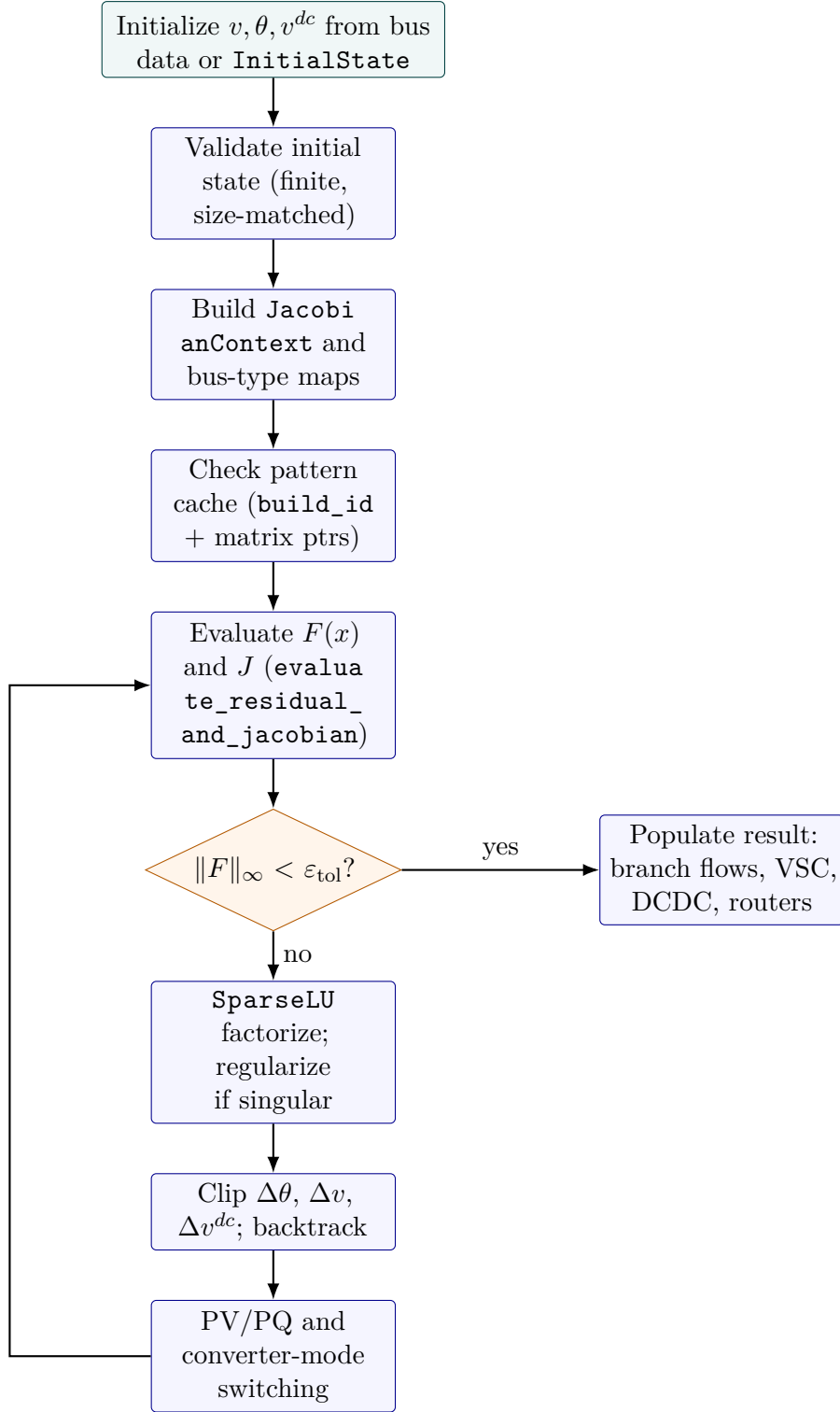


Figure 2: Hybrid AC/DC Newton power-flow algorithm (`NewtonSolver::solve`).

Algorithm PF-1: Hybrid Newton PF

- 1: **procedure** `NEWTONSOLVER::SOLVE`(*data*, *opt*, *init*)
- 2: Validate *init*: check size, NaN, Inf; **throw** `invalid_argument` on failure
- 3: $x \leftarrow$ flat start or *init* values; build `JacobianContext`

```

4:    $x_{ctx}.\text{min\_vm\_pu} \leftarrow \text{opt.robust\_nonlinear.min\_vm\_pu}$ 
5:   for  $k = 1, \dots, \text{opt.max\_iter}$  do
6:      $F, J \leftarrow \text{evaluate\_residual\_and\_jacobian}(x, ctx, pattern)$ 
7:     if  $\|F\|_\infty < \text{opt.tol}$  then break
8:     end if
9:      $\text{SparseLU}(J) \Delta x = F$ ; if singular then regularize and retry
10:    Clip  $\Delta x$ :  $|\Delta \theta| \leq \Delta \theta_{\max}, |\Delta v| \leq \Delta v_{\max}$ 
11:    Backtracking line search; accept best step
12:    PV/PQ hysteresis switching; converter mode switching
13:  end for
14:  if not converged then return a result with converged=false, final iteration count,
    residual, and any available state vectors
15:  Post-process branch, VSC, DCDC, router flows; return result
16: end procedure

```

Robust nonlinear pipeline. The solver implements a five-phase robustification pipeline controlled by `RobustNonlinearOptions`:

1. **Phase 1 – Scaling and diagnostics:** residual-equation scaling, variable scaling, and Jacobian row/column equilibration; condition-number proxy monitor.
2. **Phase 2 – Nonmonotone line search and smooth NCP:** Grippo–Lampariello–Lucidi nonmonotone Armijo search; optional μ -perturbed Fischer–Burmeister complementarity for PV/PQ switching.
3. **Phase 3 – LM trust-region fallback:** Levenberg–Marquardt step $(J^\top J + \lambda I) \Delta x = -J^\top F$ when Newton line search fails; SER-based pseudo-transient continuation.
4. **Phase 4 – Homotopy continuation:** ramp loads and converter setpoints from flat start to target in adaptive λ steps.
5. **Phase 5 – Krylov fallback:** Newton–GMRES with Schur-complement AC/DC block preconditioner (opt-in; not active by default).

12.7 Fast-Decoupled Power Flow (FDPF)

The FDPF solver builds two constant susceptance matrices from Y_{bus} :

$$B' = -\Im \left\{ Y_{bus}^{(R=0, b=0, \text{tap}=1)} \right\}, \quad (47)$$

$$B'' = -\Im \left\{ Y_{bus}^{(\text{full shunts and charging, zero phase shift})} \right\}, \quad (48)$$

and then alternates P- θ and Q-V decoupled solves:

$$B' \Delta \theta = \Delta P / v, \quad (49)$$

$$B'' \Delta v = \Delta Q / v. \quad (50)$$

Both B' and B'' are factorized once via `Eigen::SparseLU`; only the right-hand sides change across iterations. The solver is AC-only (no DC or converter coupling).

Algorithm PF-2: Fast-Decoupled PF

- 1: **procedure** FDPFSOLVER :: SOLVE(*data*, *opt*)
- 2: Build reduced $B'_{\mathcal{N} \setminus \{sl\}}, B''_{\mathcal{N}_{PQ}}$; factorize
- 3: **if** factorization failed **then** return non-converged result

```

4:   end if
5:   for  $k = 1, \dots, \text{opt.fdpf\_max\_iter}$  do
6:       Convert/use dense ybus_dense; compute  $\Delta P/v$ ,  $\Delta Q/v$  with all-pairs loops
7:       if  $\|\Delta P/v\|_\infty < \varepsilon$  and  $\|\Delta Q/v\|_\infty < \varepsilon$  then break
8:       end if
9:        $\Delta \theta \leftarrow (B')^{-1}(\Delta P/v)$ ; update  $\theta$ 
10:       $\Delta v \leftarrow (B'')^{-1}(\Delta Q/v)$ ; update  $v$ 
11:   end for
12: end procedure

```

12.8 DC-Only Power Flow

The DC solver iterates on non-slack DC voltages using dense LU on the small reduced Jacobian:

$$J_{dc} \Delta v^{\text{dc}} = \Delta P^{\text{dc}}, \quad (51)$$

$$J_{dc,kl} = \begin{cases} (G_{dc} v_k^{\text{dc}})_k + G_{kk}^{\text{dc}} v_k^{\text{dc}}, & l = k, \\ G_{kl}^{\text{dc}} v_k^{\text{dc}}, & l \neq k. \end{cases} \quad (52)$$

DC bus injections are assembled from **DCLoad**, DC storage, DC static generators, DC PV arrays, VSC DC-side injections, DCDC transfers, and DC **EnergyRouter** ports. A backtracking line search over $\alpha \in \{1, \frac{1}{2}, \dots\}$ is applied with voltages floored at **kMinVdc** to prevent collapse.

12.9 Adaptive Islanded Solver

The adaptive solver handles multi-island AC/DC topologies that a single global solve cannot address:

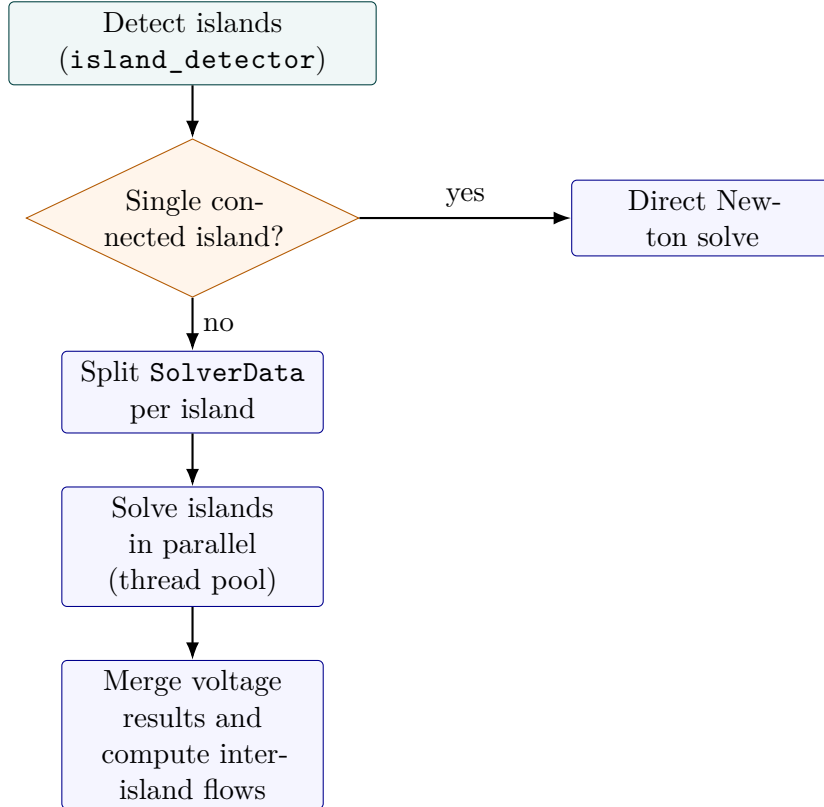


Figure 3: Adaptive islanded power-flow workflow.

12.10 HELM Solver

The HELM (Holomorphic Embedding Load-flow Method) solver constructs a power series in a complex embedding parameter $s \in \mathbb{C}$:

$$V_i(s) = \sum_{n=0}^N a_n^{(i)} s^n. \quad (53)$$

The recursion for order n coefficients follows from the holomorphic embedding of the network equations. Convergence is tested via Padé approximants $[M/M]$ evaluated at $s = 1$:

$$V_i(1) \approx \frac{P_M(1)}{Q_M(1)}. \quad (54)$$

In the current source, HELM is a standalone `HelmSolver::solve` path. `HelmWarmStart::DCPF` and `HelmWarmStart::GaussSeidel` are options consumed inside `HelmSolver::solve`; the hybrid Newton and adaptive solver paths do not call HELM as an automatic fallback.

12.11 Error Handling and Diagnostics

The public result structs do not contain a `SolverFailureReason` enum. `PowerFlowResult` exposes `converged`, `iterations`, `residual`, solved state vectors, flow/transfer vectors, and profiling metadata. `DCPowerFlowResult`, `AdaptiveSolveResult`, `IslandedSolveResult`, and `DistributedSlackResult` use their own compact status fields and nested results.

The solver **throws** `std::invalid_argument` for structural API violations (e.g., size mismatch, NaN in initial state) and **returns a non-converged result** for numerical failures. This two-tier policy separates programming errors from expected numerical difficulties.

12.12 Maintainability Roadmap: Code-Duplication Inventory

All six items D1–D6 from the original inventory were resolved across two refactoring passes (*code-dedup-D1-D5* and *code-dedup-D3-D6*).

D1 — `inf_norm` (DONE). The anonymous-namespace duplicate removed from `jacobian_builder.cpp` and `dc_solver.cpp`. The canonical definition now lives in `include/hacdcpf/power_flow/pf_utils.hpp` as an inline function. Test files `test_powerflow_performance.cpp` updated to use `hacdcpf::powerflow::inf_norm` via a `using` declaration.

D2 — DC injection assembly (DONE). The 80-line DC injection loop extracted into `assemble_dc_injections(data,vm,va,vdc,pdc_spec)`, declared in a new header `include/hacdcpf/power_flow/pf_injection_assembly.hpp` and implemented in `src/power_flow/pf_injection_assembly.cpp`. Both `dc_solver.cpp` and `residual_evaluator.cpp` now call this single canonical function.

D3 — Duplicated `B'` matrix construction (DONE). The anonymous-namespace functions `build_bp` and `build_bpp` were removed from `fdpf_solver.cpp` and replaced with calls to a new utility `build_susceptance_matrix(data,flags)` implemented in `admittance_builder.cpp`. A `BpBuildFlags` struct (declared in `admittance_builder.hpp`) controls all assembly variants: zero resistance, zero line charging, unit tap, zero phase shift, and optional bus shunt susceptance. Convenience factories `make_bp_flags(min_x_pu)` and `make_bpp_flags()` cover the two FDPF use cases.

D4 — DC initial-state warm-start (DONE). The size-guarded copy of `init->vdc` with NaN/Inf validation extracted into `load_dc_initial_state(init,dc_buses,vdc)` in `pf_utils.hpp`. Applied to both `dc_solver.cpp` and `newton_solver.cpp`.

D5 — Bus-type classification loop (DONE). The manual AC bus classification into SLACK/PV/PQ sets extracted into `classify_ac_buses(data)` returning an `AcBusSets` struct (all in `pf_utils.hpp`). Applied to `fdpf_solver.cpp`, `residual_evaluator.cpp`, and the helper in `test_unified_pf_upgrade.cpp`.

D6 — jacobian_builder.cpp file split (DONE). The monolithic `jacobian_builder.cpp` was split into three focused compilation units:

- `jacobian_pattern.cpp` — `build_jacobian_pattern` and its anonymous-namespace helpers (`rc_key`, `add_pattern_position`, `build_entry_to_nz`, `lookup_nz`);
- `ac_kernel.cpp` — `evaluate_ac_kernel_serial`, `evaluate_ac_kernel_parallel`, and `effective_ac_threads`;
- `jacobian_builder.cpp` (retained) — `build_power_spec`, `evaluate_residual_impl`, `evaluate_residual_and_jacobian`, and `evaluate_residual_only`.

The cross-file linkage between `jacobian_builder.cpp` and `ac_kernel.cpp` is provided by an internal header `include/hacdcpf/power_flow/ac_kernel_impl.hpp`.

12.13 Numerical Performance (Release Mode)

Benchmarks were run on 84 MATPOWER cases (`build_rel`, Apple M4, `-O3-DNDEBUG`, single thread) using `build_rel/helm_powerflow_benchmark../data`. Four paths are compared: NR = hybrid Newton, FD = fast-decoupled, HE = HELM standalone, HE+DC = HELM with DC warm start.

Case	<i>N</i>	Branches	NR ms	FD ms	HE ms	HE+DC ms
case14	14	20	0.0	0.3	0.1	0.1
case57	57	80	0.1	2.9	0.6	0.6
case118	118	186	0.4	11.5	1.8	1.9
case300	300	411	0.9	59.5	div	div
case1354pegase	1354	1991	15.2	skip	div	div
case1888rte	1888	2531	10.8	skip	135.8	136.5
case2383wp	2383	2896	3.8	skip	117.6	109.3
case2736sp	2736	3504	5.6	skip	75.2	76.6
case3012wp	3012	3572	35.0	skip	124.3	126.9
case6468rte	6468	9000	31.7	skip	297.7	296.2
case8387pegase	8387	14561	46.7	skip	1776.5	1786.7
case13659pegase	13659	20467	95.9	skip	skip	skip
	10000	12706	21.8	skip	4327.9	4389.7
case_ACTIVSg10k						
	25000	32230	64.3	skip	skip	skip
case_ACTIVSg25k						

Case	N	Branches	NR ms	FD ms	HE ms	HE+DC ms
case_SyntheticUSA	82000	104121	1745.2	skip	skip	skip

Table 124: Power-flow solver timing (release mode, Apple M4, single thread, post D1–D5 refactor). “div” = diverged, “skip” = case excluded from path at this size.

Key observations:

- The hybrid Newton path converges on the 82000-bus USA synthetic system in 1.75s with 17 iterations after the D1–D5 refactoring pass (no regression vs. pre-refactoring baseline).
- Newton solve time scales approximately as $O(N^{1.2})$ in practice for typical sparse transmission systems.
- FDPF is 10–200× slower than Newton on small-to-medium cases due to the current $O(N^2)$ dense Ybus iteration in the power-injection loop; replacing it with a sparse row iteration remains an open performance item.
- HELM converges on most well-conditioned cases under 2000 buses but diverges on stressed systems (case300, case1354pegase); it remains useful as a warm-start provider for Newton on difficult cases.
- The D1–D6 refactoring passes introduced no performance regression: Newton solve times match the pre-refactoring baseline on all benchmark cases.

13 Advanced and Fallback Power-Flow Solvers

This chapter documents the remaining power-flow solver families declared in `include/hacdcpf/power_flow/`: the Continuation Power Flow (`CpfSolver`), the Homotopy Continuation Solver, the Distributed Slack Solver mathematical model, and the Newton–Krylov GMRES fallback. It also provides the complete `RobustNonlinearOptions` and `PowerFlowOptions` reference tables.

13.1 Continuation Power Flow (CPF) / Voltage Stability

The `CpfSolver` traces the P–V curve of a parameterized power system from the base operating point ($\lambda = 0$) toward the voltage-collapse nose point. The implementation lives in `src/power_flow/homotopy_continuation.cpp` (the CPF corrector) and `include/hacdcpf/power_flow/voltage_stability.hpp`. It is exposed at the lower-level `hacdcpf::powerflow` namespace and is not part of the public `hacdcpf.hpp` facade.

13.1.1 Parameterized Power System

The loading direction is described by `CpfDirection`:

$$P_{L,i}(\lambda) = P_{L,i}^0 + \lambda \Delta P_{L,i}, \quad (55)$$

$$Q_{L,i}(\lambda) = Q_{L,i}^0 + \lambda \Delta Q_{L,i}, \quad (56)$$

$$P_{G,i}(\lambda) = P_{G,i}^0 + \lambda \Delta P_{G,i}, \quad (\text{non-slack only}). \quad (57)$$

Here $\lambda = 0$ is the base case; $\lambda = 1$ doubles all loads when using the proportional direction. Two factory methods build standard directions:

- `CpfDirection::proportional(data)` — all buses scale in proportion to their base-case power.
- `CpfDirection::at_buses(data, bus_indices)` — uniform 1 MW/Mvar per bus in the specified list.

13.1.2 Predictor–Corrector Continuation

The predictor step uses a secant (or constant) approximation:

$$x_{\lambda+\Delta\lambda} \approx x_{\lambda} + \Delta\lambda \frac{x_{\lambda} - x_{\lambda-\Delta\lambda}}{\Delta\lambda_{\text{prev}}}. \quad (58)$$

The corrector step solves the parameterized Newton system at the predicted point by running the hybrid Newton PF with the modified injection vector and convergence tolerance `corrector_tol`.

Step adaptation:

$$\Delta\lambda_{\text{new}} = \begin{cases} \min(\text{step_grow} \cdot \Delta\lambda, \text{step_max}), & \text{corrector converged,} \\ \text{step_shrink} \cdot \Delta\lambda, & \text{corrector failed.} \end{cases} \quad (59)$$

If $\Delta\lambda < \text{step_min}$, the solver reports the nose point and stops.

13.1.3 CpfOptions Reference

Field	Default	Meaning
<code>lambda_max</code>	5.0	Maximum loading factor λ to trace.
<code>step_init</code>	0.05	Initial continuation step $\Delta\lambda$.
<code>step_min</code>	1e-5	Minimum step size; stop and declare nose point reached.
<code>step_max</code>	0.20	Maximum step size (prevents skipping over the nose).
<code>step_grow</code>	2.0	Growth factor applied after a successful corrector solve.
<code>step_shrink</code>	0.5	Shrink factor applied after a corrector failure.
<code>corrector_max_iter</code>	50	Max Newton iterations in each corrector call.
<code>corrector_tol</code>	1e-8	Convergence tolerance for the corrector (p.u.).
<code>monitor_bus</code>	-1	Bus index to plot on the P–V curve; -1 = auto-select max demand.
<code>trace_all_buses</code>	false	Store full <code>vm/va</code> vectors at every CPF step.
<code>max_steps</code>	2000	Safety cap on total predictor–corrector steps.
<code>vm_min_pu</code>	0.25	Voltage floor; stop if any bus falls below this (p.u.).
<code>use_secant_predictor</code>	true	Use secant (two-point) predictor; false = constant predictor.

13.1.4 CPF Result Structure

Each tracing step produces a `CpfPoint`:

- `lambda` – continuation parameter value at this step;
- `vm_monitor` – monitor-bus voltage magnitude (p.u.).

The top-level result includes the full trace of `CpfPoint` records, the final converged solution, estimated nose-point λ^* and minimum voltage, and the convergence status.

13.1.5 Algorithm A13: CPF Predictor–Corrector (condensed)

The full pseudo-code and flowchart appear in Section 28.4.1. The condensed form below highlights the inner step logic.

```

1: procedure CPFSOLVER::SOLVE(base, dir, opts)
2:   Solve base case at  $\lambda=0$  via NewtonSolver (A3)
3:    $\lambda \leftarrow 0$ ;  $\Delta\lambda \leftarrow \text{opts.step\_init}$ 
4:   while  $\lambda < \lambda_{\max}$  and  $\text{iter} < \text{max\_steps}$  do
5:     Predictor: secant extrapolation  $\rightarrow x_{\text{pred}}$  at  $\lambda + \Delta\lambda$ 
6:     Corrector: Newton from  $x_{\text{pred}}$  at  $\lambda_{\text{next}}$ 
7:     if converged then
8:       Accept;  $\lambda \leftarrow \lambda_{\text{next}}$ ;  $\Delta\lambda \leftarrow \min(\text{step\_grow} \cdot \Delta\lambda, \text{step\_max})$ 
9:     else
10:       $\Delta\lambda \leftarrow \text{step\_shrink} \cdot \Delta\lambda$ 
11:      if  $\Delta\lambda < \text{step\_min}$  then declare nose; break
12:      end if
13:    end if
14:  end while
15:  return CpfResult{trace, nose  $\lambda^*$ , status}
16: end procedure

```

13.2 Homotopy Continuation Solver

The `HomotopyContinuationSolver` solves the hybrid power flow by gradually ramping “difficult” injections from a flat-start base to the target values. It is invoked as a last-resort fallback when the direct Newton solve fails (Phase 4 of the robust pipeline).

13.2.1 Homotopy Path

The homotopy path scales all active/reactive loads and converter setpoints:

$$P_L(\lambda) = \lambda P_L^{\text{target}}, \quad Q_L(\lambda) = \lambda Q_L^{\text{target}}, \quad (60)$$

$$P_{\text{vsc}}(\lambda) = \lambda P_{\text{vsc}}^{\text{target}}, \quad P_{\text{dcdc}}(\lambda) = \lambda P_{\text{dcdc}}^{\text{target}}. \quad (61)$$

At $\lambda = 0$ the system is trivially solved from the flat start. At $\lambda = 1$ the full target system is solved.

13.2.2 Step Adaptation

$$\text{Success: } \Delta\lambda \leftarrow \min(2\Delta\lambda, \Delta\lambda_{\max}), \quad (62)$$

$$\text{Failure: } \Delta\lambda \leftarrow \frac{\Delta\lambda}{2}; \quad \text{fail if } \Delta\lambda < \Delta\lambda_{\min}. \quad (63)$$

The current implementation records accepted/rejected steps in a `HomotopyState` diagnostic struct and returns both the final `PowerFlowResult` and the state for instrumentation.

13.2.3 Algorithm A14: Homotopy Continuation (condensed)

See Section 28.4.2 for the full pseudo-code and flowchart.

```

1: procedure HOMOTOPYCONTINUATIONSOLVER::SOLVE(sys, opt)
2:   Scale to  $\lambda=0$ ; solve trivially
3:   while  $\lambda < 1$  do
4:      $\lambda_{\text{next}} \leftarrow \min(1, \lambda + \Delta\lambda)$ ; scale trial system
5:     if Newton (A3) from current solution converges then
6:       Accept;  $\lambda \leftarrow \lambda_{\text{next}}$ ;  $\Delta\lambda \leftarrow \min(2\Delta\lambda, \Delta\lambda_{\text{max}})$ 
7:     else
8:        $\Delta\lambda \leftarrow \Delta\lambda/2$ 
9:       if  $\Delta\lambda < \Delta\lambda_{\text{min}}$  then return failed
10:    end if
11:  end while
12:  return PowerFlowResult at  $\lambda=1$ 
13: end procedure

```

13.3 Distributed Slack Solver

The distributed-slack solver distributes the active-power mismatch across a set of participating buses according to participation factors instead of assigning it all to a single slack bus. Two variants are exposed:

- `solve_power_flow_distributed_slack` – simplified form (one extra slack sweep);
- `solve_power_flow_distributed_slack_full` – full-Jacobian distributed-slack, as described below.

13.3.1 Problem Formulation

Let $\alpha_g \geq 0$ be participation factors (normalized to sum to 1) and let P_s be the total system mismatch. Each participating generator g receives an additional dispatch increment:

$$\delta P_g = \alpha_g P_s, \quad (64)$$

subject to individual upper bounds `max_participation_pg` when provided. The reference bus (`reference_bus`) provides the angle reference; it need not be the same as the swing generator.

13.3.2 Convergence Criterion

In the full-Jacobian variant, the distributed-slack equations are appended to the standard Newton residual and solved simultaneously. Convergence is judged on the extended infinity-norm $\|\Delta F\|_\infty < \varepsilon$.

13.3.3 Result Structure

`DistributedSlackResult` extends the standard bus-voltage vectors with:

- `distributed_slack_p` – map from participating bus index to final slack-share allocation (MW);
- `hit_limits` – list of buses that reached `max_participation_p` and were clamped.

13.3.4 Algorithm A7: Distributed-Slack PF (condensed)

See Section 28.2.5 for the full pseudo-code and flowchart.

```

1: procedure SOLVE_DISTRIBUTED_SLACK(sys, cfg, opt)
2:   Run base Newton PF (A3); compute slack-bus mismatch  $\Delta P_s$ 
3:   Normalize participation factors  $\alpha_g$ 
4:   for each participating generator g do
5:      $P_g \leftarrow P_g + \alpha_g \Delta P_s$ ; clamp to  $[P_g^{\min}, P_g^{\max}]$ 
6:   end for
7:   Re-run Newton PF on updated dispatch
8:   return DistributedSlackResult
9: end procedure

```

13.4 Newton–Krylov GMRES Fallback (Phase 5)

Phase 5 of the robust pipeline replaces the direct-factorization linear solve with a GMRES iterative solver inside the Newton loop. The implementation lives in `src/power_flow/newton_krylov.cpp` and is declared in `include/hacdcpf/power_flow/newton_krylov.hpp`. It is opt-in and not active by default (`RobustNonlinearOptions::enable_krylov_fallback`).

At each Newton iteration the augmented linear system

$$(J + \alpha M^{-1}) \Delta x = -F \quad (65)$$

is solved via GMRES with a Schur-complement AC/DC block preconditioner M :

$$M = \begin{bmatrix} J_{ac} & 0 \\ 0 & J_{dc} \end{bmatrix}, \quad (66)$$

where J_{ac} is the AC subsystem Jacobian and J_{dc} is the DC subsystem Jacobian. The Krylov fallback is most effective when the Jacobian is ill-conditioned due to strong AC/DC coupling, but adds per-iteration overhead compared to the direct SparseLU path.

13.4.1 Algorithm A15: Newton–Krylov GMRES (condensed)

See Section 28.4.3 for the full pseudo-code and flowchart.

```

1: procedure NEWTONKRYLOVSOLVER::SOLVE(data, opt)
2:   Compute  $F(x)$ ; form  $J$ ; build Schur preconditioner  $M = \text{blkdiag}(J_{AC}, J_{DC})$ 
3:   while  $\|F\|_{\infty} \geq \varepsilon$  and iter < max do
4:     GMRES(m): solve  $(J + \alpha M^{-1}) \Delta x = -F$  iteratively
5:      $x \leftarrow x + \Delta x$ ; re-evaluate  $F(x)$ 
6:   end while
7:   return NewtonKrylovResult
8: end procedure

```

13.5 Complete RobustNonlinearOptions Reference

Field	Default	Description
Phase 1: Scaling and diagnostics		
<code>enable_residual_scaling</code>	<code>true</code>	Scale residual equations by $\max F_i^{\text{spec}} $.

Field	Default	Description
<code>enable_variable_scaling</code>	<code>true</code>	Scale Newton variables (θ, V, V^{dc}) to $O(1)$.
<code>enable_jacobian_row_col_eq_uilibration</code>	<code>true</code>	Apply $D_F J D_x^{-1}$ row/column equilibration.
<code>enable_condition_monitor</code>	<code>true</code>	Compute ℓ_1 row/col proxy for condition number each iteration.
<code>min_scale</code>	1e-6	Floor for any scale factor.
<code>max_scale</code>	1e6	Ceiling for any scale factor.
<code>min_vm_pu</code>	1e-8	Voltage magnitude floor (pu) in Jacobian $\partial P / \partial V$.
<code>min_branch_x_pu</code>	1e-20	Min branch reactance; below this, branch excluded from FDPF B' .
<code>bad_condition_threshold</code>	1e10	Proxy condition number above which Jacobian is flagged “bad”.
<code>tiny_pivot_threshold</code>	1e-12	Pivot magnitude threshold for informational logging.
Phase 2: Nonmonotone line search		
<code>enable_nonmonotone_linesearch</code>	<code>true</code>	Use GLL nonmonotone Armijo search with sliding window.
<code>nonmonotone_window</code>	5	Sliding window M for nonmonotone reference merit.
<code>armijo_c</code>	1e-4	Armijo sufficient-decrease constant $c_1 \in (0, 1)$.
<code>line_search_beta</code>	0.5	Step-length reduction factor β per backtrack trial.
<code>max_line_search_trials</code>	12	Maximum backtrack trials before step rejection.
Phase 2: Smooth Fischer–Burmeister NCP (off by default)		
<code>enable_smooth_ncp</code>	<code>false</code>	Use μ -perturbed smooth-FB for PV/PQ complementarity.
<code>ncp_mu0</code>	1e-2	Initial smoothing parameter μ_0 .
<code>ncp_mu_min</code>	1e-12	Minimum smoothing parameter (limit of continuation).
<code>ncp_mu_factor</code>	0.1	Fine-phase annealing factor (near-convergence).
<code>ncp_mu_factor_coarse</code>	0.5	Coarse-phase annealing factor (large residual).
<code>ncp_mu_phase_transition</code>	0.1	Fraction of initial residual to switch to fine annealing.
<code>enable_activity_hysteresis</code>	<code>true</code>	Hold PV/PQ active-set for <code>min_active_set_hold_iters</code> iterations.
<code>min_active_set_hold_iters</code>	3	Minimum iterations between consecutive PV/PQ status changes.

Field	Default	Description
<code>q_limit_hysteresis</code>	1e-4	Hysteresis band on Q-limit proximity (pu) before switching.
Phase 3: LM trust-region fallback		
<code>enable_lm_trust_region_fallback</code>	<code>true</code>	Compute LM step $(J^\top J + \lambda I)\Delta x = -J^\top F$ on Newton failure.
<code>lm_lambda0</code>	1e-4	Initial LM damping parameter λ_0 .
<code>lm_lambda_min</code>	1e-12	Minimum LM damping.
<code>lm_lambda_max</code>	1e8	Maximum LM damping.
<code>trust_region_radius0</code>	1.0	Initial trust-region radius.
<code>trust_region_radius_min</code>	1e-6	Minimum radius; below this the LM fallback is abandoned.
<code>trust_region_radius_max</code>	100.0	Maximum trust-region radius.
Phase 3: PTC-SER pseudo-transient continuation		
<code>enable_ptc_ser</code>	<code>true</code>	Use Self-Equalising Residual (SER) exponent for adaptive PTC.
<code>ptc_dt0</code>	0.1	Initial pseudo-timestep δt_0 .
<code>ptc_dt_min</code>	1e-4	Minimum pseudo-timestep (large damping).
<code>ptc_dt_max</code>	1e6	Maximum pseudo-timestep (approaches pure Newton).
Phase 4: Homotopy continuation (see Section 13.2)		
<code>enable_homotopy_fallback</code>	configurable	Trigger <code>HomotopyContinuationSolver</code> when Phase 3 fails.
Phase 5: Krylov fallback (off by default)		
<code>enable_krylov_fallback</code>	<code>false</code>	Replace SparseLU with Newton-GMRES + Schur-complement preconditioner.

13.6 PowerFlowOptions Complete Reference

The complete field-by-field reference for `PowerFlowOptions` and its seven orthogonal sub-structs (`ConvergenceOptions`, `LinearSolverOptions`, `GlobalizationOptions`, `PVPQSwitchingOptions`, `ConverterModeOptions`, `ZipLoadOptions`, `RuntimeOptions`) is in Section 22.1. Table 13.6 below lists only the fields that are directly relevant to the advanced solver paths described in this chapter.

Field	Default	Description
<code>ac_eval_threads</code>	0	Threads for AC Jacobian evaluation; 0 = hardware auto-detect (<code>std::thread::hardware_concurrency()</code>).
<code>globalization</code>	<code>LineSearch</code>	<code>LineSearch</code> , <code>TrustRegion</code> , or <code>PseudoTransient</code> .

Field	Default	Description
<code>trust_region_radius0</code>	1.0	Initial trust-region radius (used when <code>TrustRegion</code> is selected).
<code>trust_region_max</code>	10.0	Maximum trust-region radius.
<code>ptc_delta0</code>	1.0	Initial PTC pseudo-timestep.
<code>ptc_growth</code>	2.0	PTC pseudo-timestep growth factor.
<code>enable_semi_smooth_newton</code>	false	Use NCP-based PV/PQ switching (Fischer-Burmeister).
<code>enable_coupled_jacobian</code>	false	Include cross-coupling Jacobian blocks.
<code>enable_augmented_equations</code>	false	Use augmented-equation path for VDC/VAC bus setpoints.
<code>enable_krylov_fallback</code>	false	Replace SparseLU with Newton-GMRES + Schur-complement preconditioner.
<code>enable_homotopy_fallback</code>	configurable	Trigger <code>HomotopyContinuationSolver</code> when Phase 3 fails.

14 Optimal Power Flow Models

14.1 Problem Form and Runtime Scope

This chapter covers the OPF family built on top of canonical AC/DC network projection. The implemented runtime problem classes are:

- nonlinear AC/DC OPF for the native primal-dual and parity-IPM routes,
- DC OPF with a true-QP model and LP fallback backends,
- reactive-power optimization over discrete taps and shunts by outer-approximation / coordinate-search heuristics.

The runtime solver family is selected from option flags and DC-network presence.

14.2 Code Map

The current implementation is concentrated in:

- `src/optimal_power_flow/ac_opf.cpp` for top-level AC OPF routing and native reduced-space solve orchestration,
- `src/optimal_power_flow/parity_formulation.cpp` and `src/optimal_power_flow/parity_ipm.cpp` for the parity full-space NLP route,
- `src/optimal_power_flow/dc_opf.cpp` for DC OPF model build and backend selection,
- `src/optimal_power_flow/reactive_power_opt.cpp` for the four-phase outer-approximation / coordinate-search workflow,
- `include/hacdcpf/engine` forwarding headers for LCQP, dual simplex, HiGHS, Gurobi, and related engine facilities supplied by the `mipsolvers` dependency.

There is no `src/engine` directory in this repository.

14.3 Pseudocode Map

The relevant code-aligned workflows are:

- Algorithms A8–A10 in Section 3.4 for AC OPF routing, parity-NLP construction, and parity-IPM solve,
- Algorithm A11 for DC OPF,
- Algorithm A12 for reactive-power optimization,
- Section 4 for the forwarding engine surface used by DC OPF and RPO result statistics.

14.4 Current Status

The AC OPF family has multiple implemented routes, but backend coverage differs by path. Parity and native nonlinear routes are the main AC/DC engines; DC OPF builds a specialized QP/LP model; RPO currently runs an OA/coordinate-search workflow and reuses branch-and-cut-style statistics fields for counters.

14.5 Top-Level Solver Routing

The public `solve_ac_opf` path selects among three solver families. A key implementation rule is automatic DC routing: if the case has DC buses or VSC converters and `enable_primal_dual=false`, the options are internally overridden to `enable_primal_dual=true, use_parity_ipm=true`.

Path	Trigger	Current behavior
Fast dispatch + PF fallback (AC-only)	<code>enable_primal_dual=false</code> and no forced DC parity routing	Drops DC and converter subsystems, solves economic dispatch, then runs AC PF projection.
Native primal-dual OPF	<code>enable_primal_dual=true, use_parity_ipm=false</code>	Uses a custom sparse KKT loop with AC equations, DC-bus equations, and converter power-balance equations.
Parity primal-dual IPM	<code>enable_primal_dual=true, use_parity_ipm=true</code> or auto-enabled by DC presence	Builds the parity full-space NLP and solves by Mehrotra-type primal-dual IPM. Optional fallback to native path when enabled.

14.6 Fast Dispatch + PF Fallback (AC-Only Path)

This path is used for fastest feasible dispatch recovery in AC-only operation. It solves a clipped economic dispatch first, then projects via AC PF.

Economic dispatch stage.

$$\min_{P_g} \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}), \quad (67)$$

subject to generator bounds and $\sum_g P_g = P_D$ (clamped to feasible range). For quadratic units, the code uses incremental-cost clipping $P_g = \text{clip}((\lambda - c_1)/(2c_2), P_g^{\min}, P_g^{\max})$.

Feasibility projection stage. The dispatch is passed to AC Newton PF; resulting bus injections are split back across generators at each bus and clamped to individual bounds. If direct projection fails, one reshaped economic-dispatch attempt is executed before declaring failure.

14.7 Native Primal-Dual OPF Formulation

14.7.1 Decision Variables

For the native path, variables are

$$x_{nat} = \left[\theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \quad v_{\mathcal{N}_{PQ}} \quad P_g \quad Q_g \quad v^{dc} \quad P^{\text{conv},ac} \quad Q^{\text{conv},ac} \quad P^{\text{conv},dc} \right]^\top. \quad (68)$$

The implementation includes all DC bus voltages in v^{dc} ; for DC_V buses, tight bounds are built around the setpoint.

14.7.2 Objective

$$\min_x f(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}). \quad (69)$$

14.7.3 Equality Constraints

AC equations. AC rows reuse the PF residual kernel (with converter table disabled in that kernel call), then subtract explicit converter variables:

$$g_{P,i} = \left(P_i^{\text{spec},\text{PF}} - P_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} P_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (70)$$

$$g_{Q,i} = \left(Q_i^{\text{spec},\text{PF}} - Q_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} Q_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{PQ}. \quad (71)$$

DC equations. For each DC bus k :

$$g_k^{dc} = P_{d,k}^{dc} + V_k^{dc} (G_{dc} V^{dc})_k - \sum_{c \in \mathcal{C}_k^{dc}} P_c^{dc} = 0. \quad (72)$$

Converter energy balance. For converter c :

$$g_c^{\text{conv}} = P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (73)$$

$$P_c^{\text{loss}} = a_c + b_c I_c^{\text{ac}} + (1 - \eta_c) (I_c^{\text{ac}})^2, \quad (74)$$

$$I_c^{\text{ac}} = \frac{\sqrt{(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 + \epsilon}}{V_c^{\text{ac}}}, \quad \epsilon = 10^{-6}. \quad (75)$$

14.7.4 Bounds and Nonlinear Limits

Variable bounds are enforced for $\theta, v, P_g, Q_g, v^{dc}, P^{\text{conv},ac}, Q^{\text{conv},ac}, P^{\text{conv},dc}$.

Nonlinear engineering limits are modeled as $h(x) \leq 0$ and handled by exterior quadratic penalty terms inside the merit model:

$$h_{ij}^{\text{from}} = P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (76)$$

$$h_{ij}^{\text{to}} = P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (77)$$

$$h_c^{\text{conv}} = (P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0. \quad (78)$$

14.7.5 KKT System and Globalization

At each inner iteration, the solver builds

$$\begin{bmatrix} H + \delta I & J_g^\top \\ J_g & -\delta I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_x L \\ g(x) \end{bmatrix}, \quad (79)$$

with dynamic regularization δ , line search, trust/restore logic, and bounded step clipping.

A barrier term is applied to variable bounds; nonlinear branch/converter limits enter the merit model through adaptive exterior penalties.

14.8 Parity Full-Space Formulation

14.8.1 Decision Variables

The parity NLP includes AC, DC, converter, shedding, and enhanced DER variables:

$$x_{par} = \begin{bmatrix} \theta, v^{\text{ac}}, P_g, Q_g, v^{\text{dc}}, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}, \\ \Delta P_d, \Delta Q_d, P^{\text{ren}}, Q^{\text{ren}}, P^{\text{stor,ac}}, Q^{\text{stor,ac}}, P^{\text{stor,dc}}, \\ P^{\text{dcdc}}, P^{\text{flex}} \end{bmatrix}^\top. \quad (80)$$

14.8.2 Objective

$$\min_x f_{par}(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}) + \text{VOL}_{\text{L}_{eff}} S_{\text{base}} \sum_i (\Delta P_{d,i} + \Delta Q_{d,i}) \quad (81)$$

$$+ \sum_{r \in \mathcal{R}^{\text{curt}}} \max(0, P_r^{\text{rated}} - P_r) c_r^{\text{curt}}. \quad (82)$$

14.8.3 Equality Constraints

AC full-bus balance (scaled). For each AC bus i :

$$g_{P,i} = \left(P_i^{\text{calc}} - P_i^{\text{gen}} - P_i^{\text{fixed}} - P_i^{\text{ren}} - P_i^{\text{stor}} + P_i^{\text{load}} - \Delta P_{d,i} + P_i^{\text{flex}} - P_i^{\text{conv,ac}} \right) \alpha_P = 0, \quad (83)$$

$$g_{Q,i} = \left(Q_i^{\text{calc}} - Q_i^{\text{gen}} - Q_i^{\text{fixed}} - Q_i^{\text{ren}} - Q_i^{\text{stor}} + Q_i^{\text{load}} - \Delta Q_{d,i} - Q_i^{\text{conv,ac}} \right) \alpha_Q = 0. \quad (84)$$

Here $P_i^{\text{fixed}}, Q_i^{\text{fixed}}$ aggregate non-variable injections (static generators, non-curtailable renewables/PV, VPP, microgrid exchange, mobile storage).

DC balance with DCDC coupling. For each DC bus k :

$$g_k^{\text{dc}} = P_{d,k}^{\text{dc}} + \sum_{m \neq k} g_{km} \left((V_k^{\text{dc}})^2 - V_k^{\text{dc}} V_m^{\text{dc}} \right) - P_k^{\text{conv,dc}} + P_k^{\text{dcdc,in}} - P_k^{\text{dcdc,out}} = 0, \quad (85)$$

where the OPF decision variable is P^{in} (*input-side referenced*, i.e. the power absorbed from `bus_in`). The output power is a deterministic function of P^{in} including both efficiency and $I^2 R$ conduction loss:

$$P^{\text{out}} = \begin{cases} \eta P^{\text{in}} - r_{eq} (P^{\text{in}})^2 / (V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ P^{\text{in}} / \eta - r_{eq} (P^{\text{in}})^2 / (V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (86)$$

The parity `ConstraintIndex` allocates dedicated DCDC balance slots and counts them in the equality total, but the current equality evaluator leaves those dedicated rows unpopulated. The actual DCDC physics is folded directly into the two DC bus balance equations:

$$g_{\text{bus_in}} += P^{\text{in}}, \quad g_{\text{bus_out}} -= P^{\text{out}}. \quad (87)$$

DCDC Jacobian and Hessian. The first-order derivatives of the DCDC coupling terms are:

$$\frac{\partial g_{\text{bus_in}}}{\partial P^{\text{in}}} = 1, \quad \frac{\partial g_{\text{bus_out}}}{\partial P^{\text{in}}} = -\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}}, \quad (88)$$

$$\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}} = \begin{cases} \eta - 2r_{eq}P^{\text{in}}/(V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ 1/\eta - 2r_{eq}P^{\text{in}}/(V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (89)$$

When $r_{eq} > 0$, there is also a cross-derivative with respect to the input-side DC voltage:

$$\frac{\partial g_{\text{bus_out}}}{\partial V_{\text{in}}^{\text{dc}}} = \frac{2r_{eq}(P^{\text{in}})^2}{(V_{\text{in}}^{\text{dc}})^3}. \quad (90)$$

The Hessian contributions from the I^2R loss term are:

$$\frac{\partial^2 g}{\partial (P^{\text{in}})^2} = \frac{2r_{eq}}{(V^{\text{dc}})^2}, \quad \frac{\partial^2 g}{\partial P^{\text{in}} \partial V^{\text{dc}}} = \frac{-4r_{eq}P^{\text{in}}}{(V^{\text{dc}})^3}, \quad \frac{\partial^2 g}{\partial (V^{\text{dc}})^2} = \frac{6r_{eq}(P^{\text{in}})^2}{(V^{\text{dc}})^4}. \quad (91)$$

OPF warm-start convention. Note that the PF uses output-referenced P^{ref} while the OPF uses input-referenced P^{in} . The warm-start path converts between these reference frames when seeding the OPF initial point from a PF result.

Converter balance. For each VSC converter:

$$P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (92)$$

with the same loss model as in the native formulation.

14.8.4 Inequality Constraints

Nonlinear inequalities:

$$P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (93)$$

$$P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (94)$$

$$(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0, \quad (95)$$

$$\left(g_{km}V_k^{\text{dc}}(V_k^{\text{dc}} - V_m^{\text{dc}})\right)^2 - (P_{km}^{\text{max}})^2 \leq 0. \quad (96)$$

Variable bounds are converted to inequality rows:

$$x^{\text{min}} - x \leq 0, \quad x - x^{\text{max}} \leq 0. \quad (97)$$

14.8.5 Primal-Dual IPM System

The parity solver introduces slack $z > 0$ and multipliers $\mu > 0$:

$$g(x) = 0, \quad (98)$$

$$h(x) + z = 0, \quad (99)$$

$$\nabla f(x) + J_g^\top \lambda + J_h^\top \mu = 0, \quad (100)$$

$$z \circ \mu = \gamma \mathbf{1}. \quad (101)$$

A Mehrotra predictor-corrector method is used, with:

1. affine predictor step,

2. adaptive centering parameter $\sigma = (\mu_{aff}/(\mu_{cur} + 10^{-16}))^3$ after clamping the ratio to $[0, 1]$,
3. corrector step with second-order compensation,
4. iterative refinement (2 steps) for conditioning of the KKT solve,
5. fraction-to-boundary step lengths for primal and dual updates.

Implementation note. The current `parity_ipm.cpp` uses the Mehrotra predictor-corrector scheme and implements up to two Gondzio extra correctors. The KKT system is solved using either Eigen `SparseLU` for sparse or `PartialPivLU` for dense factorizations.

Convergence monitors primal infeasibility, dual stationarity, and average complementarity. A best-iterate restoration is applied before final failure.

14.9 Component Coverage by Solver Path

Component / feature	Native primal-dual	Parity IPM
Generator (P_g, Q_g)	decision variable	decision variable
VSCConverter ($P^{\text{conv},ac}, Q^{\text{conv},ac}, P^{\text{conv},dc}$)	decision variable	decision variable
DCBus voltage	decision variable	decision variable
AC branch MVA limits	nonlinear penalty term	nonlinear inequality + slack
DC branch power limits	not explicit in native path	nonlinear inequality + slack
Load shedding ($\Delta P_d, \Delta Q_d$)	not modeled	decision variable with VOLL penalty
Curtailed renewable / controllable PV	not modeled as variable	decision variable
AC storage dispatch	not modeled as variable	decision variable
DC storage dispatch	not modeled as variable	decision variable; packed into <code>OPFResult::pstor_mw</code> with <code>stor_map.source_type</code> identifying AC vs DC storage
DCDC dispatch	not modeled as variable	decision variable + DC coupling
Flexible load dispatch	not modeled as variable	decision variable
StaticGen / fixed DER aggregates	not explicit variable in native objective model	included as fixed injections

14.10 AC OPF Result JSON Contract

`opf_result_to_json` writes the scalar solve summary `converged`, `iterations`, `outer_iterations`, `objective`, `max_constraint_violation`, `max_stationarity`, `status`, and `solver_path`. The `solver_path` token is `NativeAC`, `ParityIPM`, or `Unknown`.

The core vectors are `vm`, `va`, `vdc`, `pg_mw`, `qg_mvar`, `dpg_mw`, `dqg_mvar`, `pac_mw`, and `qac_mvar`. Enhanced dispatch is serialized both as structured objects and as legacy bare arrays:

- `renewable_dispatch`: `pren_mw`, `qren_mvar`, and a map array of `original_index/source_type` records;
- `storage_dispatch`: `pstor_mw`, `qstor_mvar`, and a map array; this packed storage array includes AC and DC storage;
- `dc_dc_dispatch`: `pdc_dc_mw` and a map array;
- `flex_dispatch`: `pflex_mw` and a map array;
- legacy top-level `pren_mw`, `qren_mvar`, `pstor_mw`, `qstor_mvar`, `pdc_dc_mw`, and `pflex_mw` arrays.

The profiling object contains `linear_solver_backend`, `analyze_calls`, `factorization_calls`, `linear_solve_calls`, `total_iterations`, `accepted_steps`, `rejected_steps`, and `final_barrier_mu`.

14.11 Fallback Behavior

1. If parity IPM is selected and fails, and `allow_fallback=true`, the solver retries with `use_parity_ipm=false`.
2. If native primal-dual fails and fallback is allowed, it attempts the fast dispatch + AC PF path.
3. Status strings in `ACOPFResult::status` record which fallback path was taken.

14.12 DC Optimal Power Flow (`solve_dc_opf`)

The DC OPF model solves a linearized economic dispatch problem using the DC power flow approximation. It is independent of the AC OPF paths above and is invoked through `hacdcopf::opf::solve_dc_opf`.

14.12.1 Decision Variables

$$x_{dc} = \begin{bmatrix} \theta_{\mathcal{N} \setminus \{sl\}} & P_g & P_f & \Delta P_d \end{bmatrix}^\top, \quad (102)$$

where $P_{f,ij}$ are branch flow variables (optional, depending on formulation mode) and $\Delta P_{d,i}$ are per-bus load-shedding variables.

14.12.2 Network Model

The DC power flow relationship is:

$$P_{f,ij} = \frac{\theta_i - \theta_j}{x_{ij}}, \quad (103)$$

with the bus susceptance matrix B assembled from branch reactances x_{ij} . The slack bus angle is fixed at $\theta_{sl} = 0$.

14.12.3 Objective Function

The DC OPF builder constructs both an LP model and a QP model. The QP model keeps true quadratic generator costs for QP-capable backends; LP backends use the linearized model.

LP linearized cost.

$$\min \sum_{g \in \mathcal{G}} c_g^{\text{lin}} P_g + \sum_i \text{VOLL}_{\text{eff}} \cdot \Delta P_{d,i} \cdot S_{\text{base}}, \quad (104)$$

where the linearized coefficient is evaluated at the generator midpoint:

$$c_g^{\text{lin}} = c_{1g} + 2c_{2g} \frac{P_g^{\text{min}} + P_g^{\text{max}}}{2}. \quad (105)$$

QP quadratic cost.

$$\min \frac{1}{2} x^\top Q x + c^\top x, \quad (106)$$

where $Q_{gg} = 2c_{2g}S_{\text{base}}^2$ and $c_g = c_{1g}S_{\text{base}}$. This true-QP form is used by `NativeQP` and `Gurobi`. The `HiGHS`-selected path routes the QP through the shared solver engine with `preferred_solver="HiGHS"`; if no QP-capable route succeeds it falls back to a `HiGHS` LP solve with the linearized cost model, and records that choice in `solver_chain` and `objective_model`.

VOLL auto-tuning. The load-shedding penalty is auto-tuned as $1.1 \times$ the maximum generator marginal cost to guarantee load serving priority.

14.12.4 Constraints

Power balance (per bus).

$$\sum_{g \in \mathcal{G}_i} P_g - P_{d,i} + \Delta P_{d,i} = \sum_{j \in \mathcal{N}_i} P_{f,ij}, \quad \forall i. \quad (107)$$

Flow definition.

$$P_{f,ij} - b_{ij}(\theta_i - \theta_j) = 0, \quad \forall (i, j) \in \mathcal{E}. \quad (108)$$

Variable bounds.

$$\theta_{sl} = 0, \quad -\pi \leq \theta_i \leq \pi, \quad i \neq sl, \quad (109)$$

$$P_g^{\text{min}} \leq P_g \leq P_g^{\text{max}}, \quad (110)$$

$$-r_a^{\text{max}} \leq P_{f,ij} \leq r_a^{\text{max}}, \quad (111)$$

$$0 \leq \Delta P_{d,i} \leq \max(P_{d,i}, 0.01/S_{\text{base}}). \quad (112)$$

When a connected component has no online source and cannot enter the algebraic DCOPF block, the implementation directly curtails that component's AC demand. This direct-island curtailment is accumulated into both `DCOPFResult::total_load_shedding_mw` and the matching per-bus entry of `DCOPFResult::load_shedding_mw`. The independent feasibility audit also checks the slack-bus balance, so an otherwise plausible fabricated result cannot pass by hiding a mismatch at the reference bus.

14.12.5 Result Audit Fields

`DCOPFResult` records the numerical solution vectors `pg_mw`, `pf_mw`, `va`, `lmp`, `branch_mu_lower`, `branch_mu_upper`, and `load_shedding_mw`. It also records solver-audit metadata: `solver_chain` stores each backend attempted by the Auto fallback sequence, and `objective_model` is QP or LP according to the cost model used for the reported objective. These fields are part of the bidirectional JSON contract through `dc_opf_result_to_json/dc_opf_result_from_json` and `save_dc_opf_result_json/load_dc_opf_result_json`.

14.12.6 Solver Backend Selection

`DCOPFOptions::solver` defaults to `DCOPFSolverBackend::Auto`. The solver always builds the QP form and then dispatches by backend:

1. **NativeQP** (LCQP solver): true quadratic costs via primal-dual IPM.
2. **Gurobi**: commercial QP solver.
3. **HiGHS**: shared-engine QP route with HiGHS preference, then HiGHS LP fallback using linearized costs.
4. **Native Dual Simplex**: via `solve_lp_with_basis()`.
5. **Auto**: tries `NativeQP`, then Gurobi, then HiGHS, then native dual simplex.

14.12.7 Locational Marginal Prices

LMP extraction is controlled by `DCOPFOptions::compute_lmp` (default `true`). When enabled, the solver runs a supporting simplex LP after the primal solve to recover constraint dual variables; the resulting nodal prices populate `DCOPFResult::lmp`. Set `compute_lmp=false` in high-volume loops (Monte Carlo, FMEA) where price signals are not needed, to skip the dual extraction overhead.

Branch shadow prices (`branch_mu_lower/branch_mu_upper`) are exposed only with an explicit validity flag. The implementation may fill the branch-dual vectors from solver certificates, but callers must treat them as congestion prices only when `DCOPFResult::branch_mu_valid=true`. The current code sets this flag to `false`; zero values therefore mean “not certified by the current extractor”, not necessarily “uncongested”.

Known limitation — congested branch shadow prices: The supporting simplex LP used for dual recovery (`solve_lp_with_basis`) does not handle bounded-variable LP problems. When `include_branch_limits=true` and a branch limit is actively binding (congestion), the native simplex cannot recover correct P_{ij} box duals; `branch_mu_lower/upper` will be unreliable (typically zero) and `branch_mu_valid=false`. LMP nodal prices remain valid. A bounded-variable simplex or an interior-point solver would be required to obtain correct branch shadow prices under congestion.

14.13 Reactive Power Optimization (`solve_rpo`)

The reactive power optimization (RPO) module searches over discrete voltage and reactive-power controls (transformer on-load tap changers and switchable shunt steps) with continuous AC OPF evaluations. The current source implements the OA/coordinate-search/ternary/pairwise refinement workflow below rather than a generic branch-and-bound MINLP solve loop.

14.13.1 Problem Structure

Discrete variables.

- Transformer OLTC tap positions: $\text{tap ratio} = 1 + (\text{tap_pos} - \text{tap_neutral}) \times \text{step_pct}/100$.
- Switchable shunt steps: $B_s = \text{step} \times b_{\text{per_step}}$ (susceptance per step).

Continuous variables. Bus voltages and generator reactive outputs, solved via AC OPF at each discrete configuration.

14.13.2 Objective Functions

Selectable objectives:

$$\text{MinVoltageDeviation: } \sum_i w_{\text{vdev}} (V_i - V_{\text{tgt}})^2, \quad (113)$$

$$\text{MinActiveLoss: } \sum_g P_g - \sum_d P_d, \quad (114)$$

$$\text{Combined: } w_{\text{vdev}} \sum_i (V_i - V_{\text{tgt}})^2 + \left(\sum_g P_g - \sum_d P_d \right). \quad (115)$$

14.13.3 Algorithm: Four-Phase Outer Approximation

Phase 0: Sensitivity ranking. Perturb each discrete variable by ± 1 from the baseline (two NLP solves per variable). Compute $|\Delta \text{obj}|$ for each variable and order by descending sensitivity.

Phase 1: Ternary-search sweep. For each variable in sensitivity order, exploit approximate unimodality of the RPO objective using integer ternary search ($O(\log R)$ evaluations per variable instead of $O(R)$). At each evaluation, generate an outer-approximation cutting plane:

$$f(y) \geq f_k + g_k^\top (y - y_k), \quad (116)$$

where g_k is a finite-difference gradient estimate.

Phase 2: Coordinate descent with OA pruning (up to 3 cycles). Sweep each variable and evaluate only candidates where the OA lower bound $<$ incumbent. Add OA cuts after each NLP evaluation. Terminate if no improvement this cycle. The OA lower bound is:

$$\max_k f_k + g_k^\top (y - y_k). \quad (117)$$

Phase 3: Pairwise neighborhood polishing. Check ± 1 perturbations of every pair of discrete variables. OA cuts prune many candidates before expensive NLP evaluation.

14.13.4 Inner NLP Solver

At each discrete configuration, the AC OPF solver (IPM by default, fallback enabled) is called to solve the continuous sub-problem. Solution caching by discrete configuration key avoids redundant NLP evaluations.

14.13.5 Result Structure

The result includes per-device tap/shunt adjustments, the number of NLP solves, OA cuts added, nodes explored, and the final status. Current source status strings include "Optimal(OA)", "Timelimit(OA)", "Nodelimit(OA)", and "Nofeasibleconfigurationfound(OA)".

15 Power-Model Builders (AML Layer)

The `src/power_models/` subsystem provides four algebraic model builders that construct optimisation or simulation models using the AML expression layer. All builders live under the namespace `hacdcpf::power_models` and are declared in `include/hacdcpf/power_models/`.

15.1 AML Abstraction Layer

The AML layer (`include/hacdcpf/aml/aml.hpp`) is a thin namespace alias:

```
#include <mipsolvers/aml/aml.hpp>
namespace hacdcpf {
    namespace aml = mipsolvers::aml;
}
```

All expression types (`Variable`, `NonlinearExpr`, `Constraint`, etc.) and the `AMLModel` container are inherited from the `mipsolvers` library. The builders populate an `AMLModel` instance and hand it to the solver engine.

15.2 LinDistFlow Builder

15.2.1 Mathematical Formulation

The `LinDistFlowBuilder` constructs a linearised DistFlow LP (Baran & Wu, 1989) for a radial distribution network.

Variables

- P_{ij} (MW) – active power flow on branch (i, j) ;
- Q_{ij} (MVar) – reactive power flow on branch (i, j) ;
- v_i (pu²) – squared voltage magnitude at bus i .

Constraints (per non-root bus j) Active power balance:

$$P_{\pi(j),j} - P_j^{\text{load}} - \sum_{k:(j,k) \in \mathcal{E}} P_{jk} = 0. \quad (118)$$

Reactive power balance:

$$Q_{\pi(j),j} - Q_j^{\text{load}} - \sum_{k:(j,k) \in \mathcal{E}} Q_{jk} = 0. \quad (119)$$

Linearised voltage drop on branch (i, j) with resistance r_{ij} and reactance x_{ij} :

$$v_j = v_i - 2(r_{ij} P_{ij} + x_{ij} Q_{ij}). \quad (120)$$

Thermal limits (VA rating S_{ij}^{max}):

$$P_{ij}^2 + Q_{ij}^2 \leq (S_{ij}^{\text{max}})^2 \quad (\text{relaxed as LP box in current version}). \quad (121)$$

Voltage bounds:

$$v_{\min} \leq v_i \leq v_{\max} \quad \forall i, \quad (122)$$

with defaults $v_{\min} = 0.81$ (pu², i.e. 0.9 pu) and $v_{\max} = 1.21$ (pu², i.e. 1.1 pu).

Objective Minimise total active-power loss (or external generator cost when provided):

$$\min \sum_{(i,j) \in \mathcal{E}} r_{ij} I_{ij}^2 \approx \min \sum_{(i,j) \in \mathcal{E}} r_{ij} \frac{P_{ij}^2 + Q_{ij}^2}{v_i}. \quad (123)$$

In the pure LP (default) the loss objective uses the DistFlow convex-relaxation approximation.

15.2.2 Input/Output Structs

LinDistFlowData field	Type	Meaning
bus_ids	vector<string>	Ordered list of bus IDs (root first).
root_bus	string	Root bus ID (voltage fixed to $v_0 = 1.0$ pu ²).
branches	vector<LinBranchData>	Each branch: id, from_bus, to_bus, r_pu, x_pu, rate_a_mva.
p_load_mw	map<string, double>	Active load by bus ID (MW).
q_load_mvar	map<string, double>	Reactive load by bus ID (MVar).
v_min_pu_sq	double	Squared lower voltage bound; default 0.81.
v_max_pu_sq	double	Squared upper voltage bound; default 1.21.
base_mva	double	System base MVA.

`LinDistFlowResult` provides per-bus squared voltages v_i , per-branch P_{ij} and Q_{ij} , total loss, and the AML solver status.

15.3 AC OPF Builder

15.3.1 Mathematical Formulation

The `ACOPFBuilder` populates an `AMLModel` with the standard MATPOWER AC OPF. All quantities are in per-unit on the system `base_mva`.

Objective

$$\min \sum_g [c_{2,g} (P_g S_b)^2 + c_{1,g} (P_g S_b) + c_{0,g}]. \quad (124)$$

Power balance at bus i

$$\sum_{g \in i} P_g - P_{d,i} - G_{s,i} V_i^2 - \sum_{\ell} (P_{f,\ell} + P_{t,\ell}) = 0, \quad (125)$$

$$\sum_{g \in i} Q_g - Q_{d,i} + B_{s,i} V_i^2 - \sum_{\ell} (Q_{f,\ell} + Q_{t,\ell}) = 0. \quad (126)$$

Branch π -model flows For branch $\ell = (i \rightarrow k)$ with admittance $y_s = 1/(r + jx)$, off-nominal tap τ , phase shift φ and shunt b_c :

$$Y_{ff} = \frac{y_s + jb_c/2}{\tau^2}, \quad Y_{tt} = y_s + jb_c/2, \quad (127)$$

$$Y_{ft} = -\frac{y_s}{\tau e^{-j\varphi}}, \quad Y_{tf} = -\frac{y_s}{\tau e^{j\varphi}}. \quad (128)$$

$$P_{f,\ell} = \Re(Y_{ff}) V_i^2 + \Re(Y_{ft}) V_i V_k \cos(\theta_i - \theta_k) + \Im(Y_{ft}) V_i V_k \sin(\theta_i - \theta_k), \quad (129)$$

$$Q_{f,\ell} = -\Im(Y_{ff}) V_i^2 - \Im(Y_{ft}) V_i V_k \cos(\theta_i - \theta_k) + \Re(Y_{ft}) V_i V_k \sin(\theta_i - \theta_k). \quad (130)$$

(Identical expressions for $P_{t,\ell}$, $Q_{t,\ell}$ with subscripts swapped.)

Branch thermal limits

$$P_{f,\ell}^2 + Q_{f,\ell}^2 \leq (S_\ell^{\max})^2, \quad (131)$$

$$P_{t,\ell}^2 + Q_{t,\ell}^2 \leq (S_\ell^{\max})^2. \quad (132)$$

Voltage/generation limits

$$V_{i,\min} \leq V_i \leq V_{i,\max}, \quad P_{g,\min} \leq P_g \leq P_{g,\max}, \quad Q_{g,\min} \leq Q_g \leq Q_{g,\max}. \quad (133)$$

15.3.2 Input Structs

Struct	Key fields
ACOPFBusData	id, pd_pu, qd_pu, gs_pu, bs_pu, vm_min, vm_max, vm0, va0_rad, is_ref.
ACOPFGenData	id, bus_id, pg_min_pu, pg_max_pu, qg_min_pu, qg_max_pu, cost_c2, cost_c1, cost_c0.
ACOPFBranchData	id, from_bus, to_bus, r_pu, x_pu, bc_pu, tap, shift_deg, rate_a_pu.
ACOPFData	Container: buses, generators, branches, base_mva.

ACOPFBuilderResult exposes the populated AMLModel, variable handles (Pg, Qg, Vm, Va), and the constraint ranges for post-solution dual extraction (LMPs).

15.4 Hybrid AC/DC OPF Builder

The ACDCOPFBuilder extends ACOPFBuilder with DC buses, DC branches, and VSC converter coupling. It inherits all AC OPF data structs and adds the following.

15.4.1 Mathematical Formulation

DC bus voltage bounds

$$V_{k,\min}^{dc} \leq V_k^{dc} \leq V_{k,\max}^{dc} \quad \forall k \in \mathcal{B}^{dc}. \quad (134)$$

DC power balance at bus k

$$\sum_{m \in \delta^+(k)} \frac{V_k^{dc} - V_m^{dc}}{r_{km}} - \sum_{m \in \delta^-(k)} \frac{V_m^{dc} - V_k^{dc}}{r_{mk}} + P_k^{dc, \text{conv}} = 0. \quad (135)$$

VDC-Q converters enforce $V_c^{dc} = V_c^{dc, \text{set}}$ via equality constraints.

VSC converter coupling Per converter c with AC bus i and DC bus k :

$$P_{ac,c} \in [p_{c,\min}, p_{c,\max}]/S_b, \quad (136)$$

$$Q_{ac,c} \in [q_{c,\min}, q_{c,\max}]/S_b. \quad (137)$$

Quadratic loss model:

$$P_{ac,c} + P_{dc,c} + a_c + b_c I_{ac,c} + c_c I_{ac,c}^2 = 0, \quad (138)$$

where $I_{ac,c} = \sqrt{P_{ac,c}^2 + Q_{ac,c}^2 + \varepsilon} / V_i$ and the loss parameters are $a_c = \text{loss_mw}/S_b$, $b_c = \text{loss_percent}/100$, $c_c = 1 - \eta_c$.

15.4.2 Additional Input Structs

Struct	Key fields
ACDCOPFDCBusData	id, vdc_min=0.9, vdc_max=1.1, vdc0=1.0, pd_pu=0.0, is_vdc_slack, vdc_set=1.0.
ACDCOPFDCBranchData	id, from_bus, to_bus, r_pu=0.01.
ACDCOPFConverterData	id, ac_bus_id, dc_bus_id, power bounds (pac_min/max, qac_min/max), warm-start (pac0, qac0), loss coefficients (a_pu, b_loss, c_loss).
ACDCOPFData	AC data via ACOPFData base, plus dc_buses, dc_branches, converters.

15.5 Security-Constrained Unit Commitment (SCUC) Builder

15.5.1 Mathematical Formulation

The `SCUCBuilder` constructs a MILP unit commitment over a multi-period horizon $\mathcal{T}$.

Decision variables

- $p_{g,t} \geq 0$ – dispatch level (MW) of unit g in period t ;
- $u_{g,t} \in \{0, 1\}$ – on/off binary status;
- $s_{g,t} \geq 0$ – start-up cost incurred if unit starts in period t .

Objective

$$\min \sum_{t \in \mathcal{T}} \sum_g [c_g p_{g,t} + S_g s_{g,t} + C_g u_{g,t}]. \quad (139)$$

Constraints System balance:

$$\sum_g p_{g,t} = D_t \quad \forall t. \quad (140)$$

Capacity:

$$P_{g,\min} u_{g,t} \leq p_{g,t} \leq P_{g,\max} u_{g,t} \quad \forall g, t. \quad (141)$$

Ramp limits:

$$p_{g,t} - p_{g,t-1} \leq R_g^{up}, \quad p_{g,t-1} - p_{g,t} \leq R_g^{dn}. \quad (142)$$

Start-up logic:

$$s_{g,t} \geq u_{g,t} - u_{g,t-1} \geq 0. \quad (143)$$

15.5.2 Input/Output Structs

SCUCData field	Type	Meaning
generator_ids	vector<string>	Generator IDs.
period_ids	vector<string>	Period labels (e.g. “H01” ... “H24”).
demand_MW	map<string, double>	System demand per period (MW).
cost_per_MWh	map<string, double>	Energy cost c_g per generator (\$/MWh).
startup_cost	map<string, double>	Start-up cost S_g (\$).
fixed_cost	map<string, double>	No-load cost C_g (\$/h).
pmin_MW	map<string, double>	Minimum stable generation $P_{g,\min}$ (MW).
pmax_MW	map<string, double>	Maximum generation $P_{g,\max}$ (MW).
ramp_up_MW	map<string, double>	Ramp-up limit R_g^{up} (MW/period).
ramp_dn_MW	map<string, double>	Ramp-down limit R_g^{dn} (MW/period).

SCUCResult provides per-generator, per-period schedules $(p_{g,t}, u_{g,t}, s_{g,t})$, total cost, and solver status.

16 Optimal Network Reconfiguration

16.1 Problem Form and Runtime Scope

This chapter documents the current ONR mixed-integer model for radial feeder reconfiguration. The runtime problem is a LinDistFlow-style MILP with binary branch-status variables, feeder-flow variables, squared-voltage variables, and connectivity-flow variables, solved through the native branch-and-cut engine.

16.2 Code Map

The current implementation is concentrated in:

- `src/network_reconfiguration/topology_analysis.cpp` for ONR model build, solve, and post-processing,
- `src/network_reconfiguration/topology_reconfiguration.cpp` for fault-triggered reconfiguration,
- the forwarded MIPSolvers engine for native branch-and-cut, simplex, and external HiGHS/Gurobi adapters.

16.3 Pseudocode Map

The code-aligned workflow summary lives in two places:

- Algorithm A22 in Section 28.6 for the application-level ONR solve flow,
- Section 4 for the solver-kernel mechanics used underneath the ONR MILP.

16.4 Current Status

The ONR route is a live MILP application using the native branch-and-cut stack. Its main current boundary is modeling scope: projected canonical AC components are supported, but higher-level hybrid overlays such as VPP, microgrid, and mobile-storage behavior are not yet injected directly into the ONR MILP.

This chapter distinguishes two runtime paths. The steady ONR path in `topology_analysis.cpp` solves radial feeder reconfiguration for loss reduction. The fault-triggered path in `topology_reconfiguration.cpp` builds a restoration MILP with failed-line availability, source limits, load pickup, and optional PF-style constraints. They share graph and MILP concepts but have different data assembly and fallback behavior.

16.5 Scope of the Current ONR Module

The ONR solver now exposes both `ACSystem` and `HybridPowerSystem` overloads. The `HybridPowerSystem` overload first projects the rich model through `project_to_canonical_models()`, so the current production path can see

- direct AC buses and branches,
- direct AC loads and generators,
- direct static generators, renewable generators, PV systems, storage, and charging stations,
- projected transformers, switches, flexible loads, asymmetric loads, chargers, and optional three-phase reductions,
- still no automatic injection of `VPP`, `Microgrid`, or `MobileStorage` overlays into the ONR MILP.

16.6 Decision Variables

For an AC network with $m = |\mathcal{E}_{ac}|$ branches and $n = |\mathcal{N}_{ac}|$ buses, the MILP uses the variable blocks

$$x = \begin{bmatrix} \alpha & P & Q & v & f & t \end{bmatrix}^\top, \quad (144)$$

where

$$\alpha_b \in \{0, 1\} \quad \text{branch status,} \quad (145)$$

$$P_b, Q_b \in \mathbb{R} \quad \text{branch active/reactive flow,} \quad (146)$$

$$v_i \in \mathbb{R} \quad \text{squared bus voltage magnitude,} \quad (147)$$

$$f_b \in \mathbb{R} \quad \text{single-commodity connectivity flow,} \quad (148)$$

$$t_b \geq 0 \quad \text{auxiliary variable for } |P_b|. \quad (149)$$

16.7 Net-Injection Model

The ONR builder computes per-bus net injections directly from the AC component tables as

$$P_i^{net} = \frac{P_i^{load} + P_i^{cs} - P_i^{gen} - P_i^{sg} - P_i^{ren} - P_i^{pv} - P_i^{stor}}{S_{base}}, \quad (150)$$

$$Q_i^{net} = \frac{Q_i^{load} + Q_i^{cs} - Q_i^{gen} - Q_i^{sg} - Q_i^{ren} - Q_i^{pv} - Q_i^{stor}}{S_{base}}. \quad (151)$$

Storage discharge is therefore treated as generation, and charging stations are additional fixed loads.

16.8 LinDistFlow MILP

Objective. The objective minimizes a linearized active-loss proxy:

$$\min_x \sum_{b=1}^m r_b t_b. \quad (152)$$

Tree cardinality.

$$\sum_{b=1}^m \alpha_b = n - 1. \quad (153)$$

Commodity-flow connectivity. The solver uses a single-commodity-flow spanning-tree formulation. For the root bus,

$$\sum_{b \in \delta^-(root)} f_b - \sum_{b \in \delta^+(root)} f_b = -(n - 1), \quad (154)$$

and for every non-root bus,

$$\sum_{b \in \delta^-(i)} f_b - \sum_{b \in \delta^+(i)} f_b = 1. \quad (155)$$

Capacity of f_b is tied to α_b using big- M logic.

Power-balance constraints. For each non-root bus,

$$\sum_{b \in \delta^-(i)} P_b - \sum_{b \in \delta^+(i)} P_b = P_i^{net}, \quad (156)$$

$$\sum_{b \in \delta^-(i)} Q_b - \sum_{b \in \delta^+(i)} Q_b = Q_i^{net}. \quad (157)$$

Voltage-drop constraints. The current implementation uses the LinDistFlow relation relaxed by big- M :

$$v_j - v_i + 2r_b P_b + 2x_b Q_b \leq M(1 - \alpha_b), \quad (158)$$

$$-(v_j - v_i + 2r_b P_b + 2x_b Q_b) \leq M(1 - \alpha_b). \quad (159)$$

Thermal and absolute-value constraints.

$$|P_b| \leq P_b^{max} \alpha_b, \quad (160)$$

$$|Q_b| \leq Q_b^{max} \alpha_b, \quad (161)$$

$$P_b \leq t_b, \quad (162)$$

$$-P_b \leq t_b. \quad (163)$$

16.9 Branch-and-Cut Configuration

The ONR path solves the MILP through the native branch-and-cut engine with the following current tuning:

- cut family: MIR cuts,
- root cut rounds: 5,
- cuts per round: 20,
- branching: pseudocost,
- node selection: hybrid,
- LP node solver: simplex enabled,
- feasibility pump: enabled.

16.10 Fault-Triggered Reconfiguration Path

`run_topology_reconfiguration()` builds its source set on the unified AC/DC bus index. AC generators, static generators, renewables, PV systems, storage units, `ExternalGrid` rows, and the final SLACK-bus fallback map to AC source nodes. DC static generators, DC-side static generators, DC storage, DC PV arrays, and `DC_V` buses map to DC source nodes. A network with no available AC or DC source returns infeasible before constructing root source variables.

Load demand is additive. Active/reactive demand from explicit `ACSystem::loads` is accumulated first, and direct `ACBus::pd_mw/qd_mvar` demand is then added at the same bus. DC load rows are also mapped onto the hybrid restoration graph when the DC bus is present. The implementation does not treat bus demand and load records as mutually exclusive encodings.

Switchable restoration edges are category-aware in hybrid systems. New code should populate `TopoReconfOptions::switchable_branches` with `BranchRef\{category,index\}` values so AC branches, DC branches, and VSC couplings that share the same numeric `index` remain distinct. The legacy `switchable_branch_ids` vector is retained for compatibility but is interpreted only as AC `ACBranch::index` values. With no explicit switchable list, the implementation keeps the historical automatic detection of normally-open AC/DC tie branches.

If the heuristic warm start fails to produce a feasible restoration plan, the code builds the full MILP fallback. It tries HiGHS when the adapter is available and retries native B&C if HiGHS is unavailable, returns failure, or returns a malformed/wrong-sized solution vector. The stale behavior where a missing heuristic plan directly implied infeasibility is no longer the current runtime path. Solver certificates use exact normalized optimality statuses plus the MIP gap, avoiding substring matches such as “not optimal”. Runtime trace output in this path and the native branch-and-cut fallback honors `verbose=false`, keeping batch and CI runs quiet unless verbose logging is requested.

16.11 Postprocessing and Verification

The steady ONR path in `solve_optimal_reconfiguration()` still applies the returned AC topology to an `ACSystem` and runs Newton PF as a verification step. The fault-triggered hybrid path `run_topology_reconfiguration()` instead performs algebraic post-solve checks on equality rows, inequality rows, binary integrality, and variable bounds before extracting structured `BranchRef` outputs. It does not claim a full post-reconfiguration AC/DC PF or OPF certificate; callers must read `TopoReconfResult::validity.post_power_flow_validated` and `full_hybrid_opf_validated`, both currently false for this path.

16.12 External Solver Integration (HiGHS)

The multi-period topology reconfiguration schedule (`solve_topology_reconfiguration_schedule()`) supports automatic dispatch to external MILP solvers. At runtime the solver selection proceeds as follows:

1. If `enforce_ac_radiality=true` (default) and `tr_solver=Auto`, the engine checks for `GurobiAdapter`, then `HighsAdapter` availability.
2. If an external solver is found, the full multi-period MILP (spanning-tree constraints, `LinDistFlow`, ESS scheduling, switching-cost limits) is built and solved via `solve_multiperiod_onr_highs()`, producing $5m + n$ decision variables per timestep.
3. If no external solver is available, the engine falls back to a stepwise per-timestep solve using the native branch-and-cut engine (`solve_optimal_reconfiguration()`).

HiGHS Adapter Architecture. The `HighsAdapter` invokes the HiGHS CLI executable via the file-based protocol:

1. `write_lp_as_mps()` serialises the `MIPModel` to a temporary MPS file with correct `INTORG/INTEND` markers for integer variable blocks.
2. The adapter constructs a shell command:

```
highs --model_file <tmp>.mps --solution_file <tmp>.sol
```

3. The solution file is parsed via `parse_highs_model_status()` to extract the model status (Optimal/Feasible/Infeasible) and primal variable values.

Executable Resolution. The HiGHS binary is resolved through a three-level fallback:

1. Explicit path passed to the adapter constructor.
2. Environment variable `HACDCPF_HIGHS_EXECUTABLE`.
3. CMake compile-time define `HACDCPF_HIGHS_EXECUTABLE` (set by `find_program()` in `Dependencies.cmake`).

16.13 Current Modeling Boundary

The ONR module is already useful for radial feeder reconfiguration, but its current mathematical scope is intentionally narrower than the main PF/OPF stack. The main structural limitations are:

- no hybrid-side injections from `VPP`, `Microgrid`, or `MobileStorage`,
- no integer co-optimization of DER dispatch and topology in the steady ONR public API,
- fault-triggered hybrid results are graph/`LinDistFlow` topology certificates, not post-solve full AC/DC PF or OPF certificates; the result `model_scope` and validity flags expose this boundary.

17 I/O Layer: Parsers, Case Builders, and JSON Contracts

17.1 MATPOWER Parser

`hacdcpf::io::parse_matpower(path)` reads MATPOWER `.m` case files from disk (`include/hacdcpf/io/matpower_parser.hpp`). The parser understands the `mpc.version~~'2'` format used by MATPOWER 6 and populates a `HybridPowerSystem` object.

Covered table sections

Section	Mapped struct(s)
<code>mpc.bus</code>	<code>ACBus</code> (type, Pd, Qd, Gs, Bs, Vm, Va, Vmin, Vmax).
<code>mpc.gen</code>	<code>Generator</code> (bus, Pg, Qg, Qmax, Qmin, Vg, Pmax, Pmin, status).
<code>mpc.branch</code>	<code>ACBranch</code> (from, to, r, x, b, rateA/B/C, tap, shift, status).
<code>mpc.gencost</code>	Generator cost model: polynomial (type 2) or piecewise-linear (type 1).
<code>mpc.dcline</code>	<code>VSCConverter</code> (basic two-terminal DC; mapped to converter pair).
<code>mpc.bus_name</code>	Populates <code>ACBus::name</code> .

Data directory `data/` ships with 80+ MATPOWER case files covering IEEE 9, 14, 30, 57, 118, 300, and a range of synthetic US/European grids up to 82 000 buses.

17.2 JSON I/O

`include/hacdcpf/io/json_io.hpp` provides the always-built JSON API:

- `to_json(sys)` / `from_json(json)` and `save_json(sys,path)` / `load_json(path)` for `HybridPowerSystem`;
- `try_from_json(json,mode)` and `try_load_json(path,mode)` for exception-free import using `Result<HybridPowerSystem>`;
- `to_jpc_json(sys)` / `from_jpc_json(json)` and `save_jpc_json` / `load_jpc_json` for the MATPOWER-like JPC JSON interchange format;
- `power_flow_result_to_json(sys,result)` and `save_power_flow_result_json(sys,result,path)` for PF results;
- `opf_result_to_json(result)` / `opf_result_from_json(json)` and `save_opf_result_json` / `load_opf_result_json` for AC OPF results;
- `dc_opf_result_to_json(result)` / `dc_opf_result_from_json(json)` and corresponding save/load helpers for DC OPF results;
- `carbon_result_to_json(result)` / `carbon_result_from_json(json)` and save/load helpers for carbon-analysis results;
- `time_series_data_to_json(ts)` / `time_series_data_from_json(json)` for time-series input data.

The on-disk schema is sparse-friendly: optional arrays may be absent, identity and connectivity fields are generally required, and schema compatibility is checked through `check_schema_version`. The detailed key-level contract is maintained in Sections 9 and 11.

Excel I/O remains optional. The public header declares `load_excel/save_excel`; without `HACDCPF_ENABLE_EXCEL` these inline functions throw. When Excel support is compiled, `src/io/excel_io.cpp` also contains source-level workbook helpers such as `load_xlsx`, `save_xlsx`, `save_results_xlsx`, `save_results_compact_xlsx`, `save_opf_results_xlsx`, `save_opf_results_compact_xlsx`, and `save_carbon_results_xlsx`.

17.3 Pre-built Test Networks (`case_builders.hpp`)

`hacdcpf::io` exposes a library of factory functions for use in tests and benchmarks. Most named `_acdc` builders add a DC overlay and VSC links; `build_ac_only_version` strips those components, and `build_market_3bus_toy` is intentionally AC-only for market/DCOPF unit tests.

Function	Description
<code>build_ieee14_acdc()</code>	IEEE 14-bus AC + one DC link.
<code>build_ieee24_3area_acdc()</code>	IEEE 24-bus RTS divided into 3 control areas, interconnected via DC.
<code>build_ieee24_3area_acdc_expanded()</code>	Expanded version with additional DER and storage.
<code>build_ieee118_acdc()</code>	IEEE 118-bus + multi-terminal DC grid.
<code>build_ac_only_version(sys)</code>	Strip all DC components from an existing system.
<code>build_case33bw_acdc()</code>	Baran–Wu 33-bus distribution + DC microgrid.
<code>build_case33mg_acdc()</code>	33-bus microgrid variant with storage.
<code>build_case69_acdc()</code>	69-bus radial distribution + DC feeder.
<code>build_case300_acdc()</code>	IEEE 300-bus + HVDC link.
<code>build_demo_multizone_acdc()</code>	Multi-zone demonstration system.
<code>build_case2000_acdc()</code>	2000-bus synthetic + DC overlay.
<code>build_dist33_microgrid_der()</code>	33-bus distribution with DER and battery storage.
<code>build_comprehensive_hybrid_acdc()</code>	Stress-test system with all component types.
<code>build_market_3bus_toy()</code>	3-bus toy system for market/OPF unit tests.
<code>build_market_5bus_acdc_toy()</code>	5-bus AC/DC toy for hybrid OPF unit tests.

18 Test Suite

All tests use the Catch2 framework (fetched automatically by CMake via `FetchContent`). The authoritative Release test tree used by this notebook is `build_rel`; build and run with:

```
cmake -B build_rel -DCMAKE_BUILD_TYPE=Release -DHACDCPF_BUILD_TESTS=ON
cmake --build build_rel
ctest --test-dir build_rel --output-on-failure
```

18.1 Test File Overview

File	Coverage
<code>test_hacdcpf.cpp</code>	Core public API smoke tests: model construction, PF variants, solver handles, AC/DC OPF, carbon wrappers, and resilience wrapper linkage.
<code>test_all_components_pf.cpp</code>	All-component projection/PF pipeline on the rich model surface.

File	Coverage
<code>test_unified_pf_upgrade.cpp</code>	Unified PF options, coupled Jacobian, augmented equations, semi-smooth Newton, globalization, and default-flag compatibility.
<code>test_acopf_dcopf_crossval.cpp</code>	ACOPF/DCOPF/PF cross-validation plus DCOPF isolated-island, pruning, slack-balance, dispatch-quality, and LMP regressions.
<code>test_enhanced_solver.cpp</code>	Island detection/adaptive solver and distributed-slack participation factors.
<code>test_scaling_regression.cpp</code>	Residual/variable/Jacobian scaling and conditioning guardrails.
<code>test_hacdcoss.cpp</code>	Current <code>hacdcopf</code> model/PF/DCOPF smoke suite, including validation, Y-bus/power-model checks, Newton PF, island/adaptive solver, distributed slack, and DCOPF regressions.
<code>test_io_json.cpp</code>	JSON/JPC round-trips, safe import wrappers, PF/OPF result JSON, MATPOWER parser, and JPC rich-component preservation.
<code>test_rich_components.cpp</code>	Rich component JSON round-trips and builder coverage for transformers, DER, storage, switches, converters, DC components, VPPs, microgrids, and external grids.
<code>test_matpower_cases.cpp</code>	Tier-1/Tier-2/Tier-3 MATPOWER convergence, safe PF wrappers, and DCOPF case regressions.
<code>test_validation.cpp</code>	Validation report API, validation levels, negative structural cases, VSC endpoint checks, and rich AC/DC guardrails.
<code>test_solver_capabilities.cpp</code>	Solver capability flags, always-on solver features, compile-time backend flags, AC/DC OPF availability, and OPF audit helpers.
<code>test_advanced_pf.cpp</code>	Island detection/extraction, islanded/adaptive PF, distributed slack, PV/PQ conversion, and integration checks.
<code>test_three_phase_pf.cpp</code>	Three-phase data model, projection-based solve, unbalanced/meshed/multi-source cases, transformer support, and JSON round-trip.
<code>test_multiport_vpp_er.cpp</code>	Multi-port EnergyRouter controls and multi-mode VPP behaviour.
<code>test_powerflow_performance.cpp</code>	Large-system Release performance sweep over MATPOWER tiers.
<code>test_topology_crossval.cpp</code>	Fault-triggered topology reconfiguration regressions, source/load accounting, structured branch references, legacy ONR fixing, exhaustive enumeration, and 33-bus ONR.
<code>test_crossmodule_integration.cpp</code>	End-to-end OPF→PF→Carbon and ONR→DCOPF→PF→Carbon pipelines.
<code>test_short_circuit_crossval.cpp</code>	Z-bus Thévenin and IEC 60909 short-circuit cross-validation.

File	Coverage
<code>test_three_phase_nr.cpp</code>	Native three-phase Newton–Raphson solver, phase-node indexer, VUF/loss/balance checks, and hybrid wrapper.
<code>test_resilience_assessment.cpp</code>	Distribution resilience, strict MIP entry point, sequential MC, and hybrid FMEA regressions.
<code>test_graph.cpp</code>	Graph topology, switch contraction, series/pendant reduction, same-ID isolation, and result recovery.
<code>test_graph_kron.cpp</code>	Kron Y-bus reduction, voltage recovery, fill-in guard, and approximation disclosure.
<code>test_distribution_pipeline.cpp</code>	Rich distribution CB/switch/VSC pipeline through graph, PF, OPF, short circuit, ONR, and JSON export.
<code>test_three_stage_reliability.cpp</code>	Native C++ three-stage reliability evaluator, AC/hybrid model-scope regressions, finite-source capacity, switch candidates, and VSC transfer limits.

18.2 Example Test Pattern

The typical Catch2 test pattern in this project:

```
TEST_CASE("brief description", "[tag1][tag2]") {
    auto sys = hacdcpf::io::build_ieee14_acdc();
    hacdcpf::PowerFlowOptions opt;
    opt.tol = 1e-8;

    auto result = hacdcpf::solve_power_flow(sys, opt);

    REQUIRE(result.converged);
    REQUIRE_FALSE(result.vm.empty());
    REQUIRE_THAT(result.vm[0],
        Catch::Matchers::WithinAbs(1.0, 0.1));
}
```

Tags used across the suite include `[model]`, `[power_flow]`, `[opf]`, `[integration]`, `[io]`, `[matpower]`, `[validation]`, `[topology]`, `[graph]`, `[resilience]`, `[reliability]`, `[short_circuit]`, `[three_phase]`, `[performance]`, and `[scaling]`. Section 29 gives the current 436-entry CTest registration with exact Release IDs and pass/skip status.

19 Architecture Assessment and Roadmap

This chapter records an external architectural review of the `hacdcpf` package conducted against the updated codebase described in Chapters 20–23. It documents observed strengths, remaining design risks, module-level recommendations, testing strategy, dependency rules, and a prioritised roadmap.

19.1 Overall Quality Assessment

The package architecture now resembles a mature scientific computing library. Table 137 summarises the current design quality by area.

Table 137: Architecture quality by area.

Area	Quality	Comment
Public model design	Strong	Rich component model; clear I/O field dictionaries
Solver layering	Strong	Projection and assembly separation is a major improvement
PF solver design	Strong	Multiple routes and fallback methods well documented
OPF design	Good–Strong	Native, parity-IPM, DC-QP, and reactive routes well separated
I/O contracts	Strong	JSON and Excel field dictionaries are detailed
Error handling	Improved	<code>Result<T></code> , <code>Error</code> , <code>ErrorCode</code> are good foundations
Analysis modules	Promising	Requires careful dependency control
Backend abstraction	Improved	Linear/NLP/QP backend split is the right direction
Documentation	Strong	Implementation-facing handbook is detailed
Testability	Likely improved	Needs evidence through test matrix and CI policy

The overall data-flow pipeline is now clearly separated:

`HybridPowerSystem` ↓ Projection Layer ↓ Canonical Models ↓ Assembly Layer ↓ Flat Solver Data ↓ PF / OPF / Analysis Solvers ↓ Result Objects ↓ I/O and Reports

19.2 Confirmed Strengths

19.2.1 Horizontal Infrastructure Layer

The dedicated solver-infrastructure layer covering assembly, projection, backend abstractions, and the thread pool (Chapter 21) is a major improvement. This separation is essential for long-term maintainability: the rich physical model can evolve without destabilising the numerical kernels.

19.2.2 Monadic Result Handling

The `Result<T>` / `Error` / `ErrorCode` design (Section 20.4) addresses one of the most important concerns in simulation packages: handling invalid input and solver failures predictably. A monadic result type is much better than relying on bare exceptions, booleans, or string messages.

19.2.3 Backward-Compatible Option Refactoring

`PowerFlowOptions::from_parts()` preserves flat API compatibility for existing users while enabling structured internal configuration. The recommended long-term pattern is:

```
struct PowerFlowOptions {
    // Legacy flat fields
    double tolerance;
    int max_iterations;
    bool enable_homotopy;

    static PowerFlowOptions from_parts(
        const ConvergenceOptions& convergence,
        const LinearSolverOptions& linear,
        const GlobalizationOptions& globalization,
        const PVPQSwitchingOptions& pvpq,
        const ConverterModeOptions& converter,
```



```

        const ZipLoadOptions& zip,
        const RuntimeOptions& runtime
    );

    PowerFlowOptionParts to_parts() const; // optional round-trip
};

```

19.2.4 Explicit Validation Subsystem

A dedicated validation layer capable of catching missing slack buses, duplicate IDs, invalid branch endpoints, inconsistent voltage limits, unsupported OPF configurations, and islands without a source gives the package its primary defence against badly formed inputs. The recommended list of checks is given in Section 20.5.

19.2.5 Projection plus Assembly Separation

- **Projection:** rich physical model $\rightarrow$ canonical electrical model.
- **Assembly:** canonical model $\rightarrow$ numerical vectors, matrices, and residual functions.

This boundary allows rich devices (motors, chargers, PV arrays, flexible loads, switches, transformers, VSC and DCDC converters) to evolve independently of solver kernels.

19.2.6 Centralised Defaults

Every default is now traceable to one of: engineering convention, MATPOWER compatibility, package-specific design, safety fallback, or numerical stabilisation. Avoid silent defaults for safety-critical fields; such fields should be required rather than defaulted.

19.3 Remaining Design Risks

19.3.1 Analysis Layer May Become Too Broad

The `analysis/` module currently covers carbon tracing, distribution PF, three-phase unbalanced PF, PV modelling, reliability, resilience, and time-series simulation. These are not all the same type of functionality. Consider explicit subdomains:

```

include/hacdcpf/network_reconfiguration/
    topology_analysis.hpp
    topology_reconfiguration.hpp

```

Dependency rule for analysis Analysis modules may depend on:

- public model and validation API,
- public solver API and result types.

They must not depend on private Newton/IPM internals, assembly private data, or backend private classes.

19.3.2 Three-Phase and Balanced PF Should Not Share One Abstraction

Three-phase unbalanced PF is structurally different from balanced positive-sequence PF. Prefer distinct free functions over one overly generic virtual interface:

```
BalancedPowerFlowResult solve_power_flow(  
    const HybridPowerSystem& sys,  
    const PowerFlowOptions& opt  
);  
  
ThreePhasePowerFlowResult solve_three_phase_power_flow(  
    const ThreePhaseSystem& sys,  
    const ThreePhasePowerFlowOptions& opt  
);
```

A common `SolverSummary` (converged, iterations, max residual, status) can be shared where natural without forcing identical result shapes.

19.3.3 Error Handling Policy Needs Explicit Definition

Now that both `Result<T>` and exceptions are present, define exactly when each mechanism is used. A recommended policy is given in Table 138.

Table 138: Recommended error handling policy.

Situation	Mechanism
Expected input validation failure	<code>Result<T></code>
Solver non-convergence	result object with status
Optional feature unavailable	<code>Result<T></code> consistently
Programmer error / invariant violation	assertion or internal exception
File not found	<code>Result<T></code> for library API
JSON parse failure	<code>Result<T></code> with error code and location
Backend crash / failure	<code>Result<T></code> with backend diagnostic

Provide both styles where helpful:

```
Result<HybridPowerSystem> io::try_load_json(const std::string& path);  
HybridPowerSystem load_json_or_throw(const std::string& path);
```

19.3.4 Field Dictionary / Serializer Drift

Detailed I/O field dictionaries are excellent, but documentation, serializers, and model structs can drift apart over time. Automated tests should verify:

- every documented field exists in the serializer,
- every serializer field is documented,
- JSON round-trip preserves each field,
- each documented default equals the code default.

A single machine-readable field-descriptor registry (see Section 19.4.7) is the preferred long-term solution.

19.3.5 Backend Capability Discovery

Since the package has optional backends (HiGHS, Gurobi, Ipopt, native IPM, etc.), the current public API exposes a runtime capability query:

```
struct SolverCapabilities {
    bool has_sparse_lu;
    bool has_klu;
    bool has_umfpack;
    bool has_highs;
    bool has_gurobi;
    bool has_ipopt;
    bool has_scip;
    bool has_native_ipm;
    bool has_excel;
    bool has_opendss;
    bool supports_quadratic_objective;
    bool supports_integer_variables;
    bool supports_ac_opf;
    bool supports_dc_opf;
    bool supports_three_phase;
};
```

```
SolverCapabilities get_solver_capabilities();
```

This enables portable user code that selects a backend based on compiled-in availability rather than on preprocessor flags alone. Regression tests also check that these flags remain consistent with actual AC/DC OPF solve paths.

19.4 Module-Level Recommendations

19.4.1 Model Layer

Separate persistent fields from runtime fields If any model struct contains both user-input data and runtime caches, split them. Do not mix result fields into input model objects unless explicitly supporting in-place result annotation.

Strong IDs at boundaries At projection boundaries, typed reference structs avoid accidentally connecting a DC converter terminal to an AC bus field:

```
struct ACBusId { std::string value; };
struct DCBusId { std::string value; };
```

Stable enum serialisation Treat enum string names as serialised API. Document whether strings are case-sensitive. Avoid casual renaming once a version ships.

19.4.2 Validation Layer

Use tiered validation levels:

```
enum class ValidationLevel {
    Basic,           // required fields, references, limits
    Electrical,      // topology, slack/reference, physical consistency
    SolverReady,     // checks required by PF/OPF kernels
};
```

```

    Strict          // warnings promoted to errors
};

```

A structured diagnostic shape supports GUIs, APIs, and batch workflows:

```

struct Diagnostic {
    Severity severity;    // Info, Warning, Error
    ErrorCode code;
    std::string message;
    std::string component_type;
    std::string component_id;
    std::string field;
};

```

19.4.3 Projection Layer

Projection report Every projection should optionally return a diagnostic report that records mappings such as:

Source	Canonical
PVSystempv_12	StaticGenerators _g _pv_12
Motormotor_7	EquivalentLoadload_motor_7
ClosedSwitchsw_3	BusMergegroup_2
OpenSwitchsw_4	DisabledBranchbranch_sw_4

Reversible mapping Maintain a `ComponentMapping` record with source type, source ID, canonical type, canonical ID, and participation factor. This is essential for result attribution, carbon tracing, and reporting.

19.4.4 Assembly Layer

Assembly should be *deterministic*: given the same canonical system, it must produce the same ordering, index maps, vector layout, and sparsity pattern. Explicit layout objects make residual code readable:

```

struct PowerFlowVariableLayout {
    std::vector<ACBusIndex> angle_variables;
    std::vector<ACBusIndex> voltage_variables;
    std::vector<DCBusIndex> dc_voltage_variables;
    std::vector<VSCIndex> converter_variables;
};

```

19.4.5 Power-Flow Layer

Standardise solver status Every PF route should return a common summary:

```

struct SolverSummary {
    TerminationStatus status;
    bool converged;
    int iterations;
    double max_mismatch;
    double elapsed_ms;
    std::vector<Diagnostic> diagnostics;
};

```

Visible fallback trace Record every solver method transition in the result:

```
{
  "solver_trace": [
    {"method": "Newton",
     "status": "SingularJacobian", "iterations": 4},
    {"method": "HomotopyContinuation",
     "status": "Converged", "iterations": 12}
  ]
}
```

19.4.6 OPF Layer

Formulation separate from solver For each OPF route maintain the pipeline: model extraction → formulation construction → backend solve → result mapping → post-check.

Independent feasibility audit After every solve, independently check AC/DC power balance, voltage limits, generator and branch limits, converter limits, and objective recomputation. Return a structured `OpfAudit` object.

Explainable infeasibility Provide likely-cause diagnostics instead of a bare “infeasible” status:

- bus has load but no connected source,
- generator `p_min_mw > p_max_mw`,
- voltage upper limit below lower limit,
- converter power exceeds DC grid capacity.

19.4.7 I/O Layer

Format versioning Every exported JSON should include:

```
{
  "format": "hacdcpf.hybrid_power_system",
  "format_version": "1.0",
  "package_version": "x.y.z"
}
```

Import modes

```
enum class ImportMode { Strict, Permissive };
```

In Strict mode unknown columns are errors; in Permissive mode they become warnings.

Field-descriptor registry A machine-readable registry derives the serialiser, documentation table, and tests from one source:

```
constexpr FieldDescriptor ac_bus_fields[] = {
  {"id", Required::Yes, DefaultPolicy::None},
  {"vn_kv", Required::Yes, DefaultPolicy::None},
  {"vm_pu", Required::No, DefaultPolicy::One},
};
```

19.4.8 Analysis Suite

Carbon tracing Expose the tracing method explicitly:

```
enum class CarbonTracingMethod {
    AverageMix, ProportionalSharing,
    MarginalEmission, FlowTracing
};
```

Time-series simulation Avoid mutating the base system in uncontrolled ways. Recommended pattern:

```
struct TimeSeriesScenario {
    HybridPowerSystem base_system;
    std::vector<TimeStepInput> steps;
};
```

19.5 Recommended Testing Matrix

Testing should be organised by layer. Table 139 lists the recommended test groups.

Table 139: Recommended test groups by layer.

Layer	Test groups
Model / validation	ModelDefaultsTest, EnumSerializationTest, ValidationDuplicateIdsTest, ValidationDanglingReferencesTest, ValidationInvalidLimitsTest, ValidationIslandWithoutSlackTest
Projection	ProjectionSwitchMergingTest, ProjectionMotorAsLoadTest, ProjectionPVAsStaticGeneratorTest, ProjectionStorageMappingTest
Assembly	AssemblyVariableOrderingTest, AssemblySparsePatternTest, AssemblyIndexMapTest, AssemblyPerUnitConversionTest
Power flow	PFIEEE14ConvergenceTest, PFHybridTwoBusVSCTest, PFDCGridConvergenceTest, PFIslandedSystemTest, PFHomotopyFallbackTest, PFNewtonKrylovFallbackTest, PFNCPComplementarityTest
OPF	OPFACSmallCaseTest, OPFDCQPBackendTest, OPFParityIPMTest, OPFReactiveTapShuntTest, OPFInfeasibilityDiagnosticTest
I/O	JsonRoundTripAllComponentsTest, MatpowerImportIEEE14Test, UnknownFieldStrictModeTest, MissingOptionalFieldDefaultTest, ResultJsonSchemaTest
Analysis	CarbonTracingKnownCaseTest, DistributionRadialFeederTest, ThreePhaseUnbalancedKnownCaseTest, PVArrayIrradianceCurveTest, ReliabilitySingleOutageTest, TimeSeriesProfileReplayTest

For complementarity variables a and b , each NCP unit test should verify:

$$\phi(a, b) = 0 \iff a \geq 0, b \geq 0, ab = 0. \quad (164)$$

19.6 Recommended CI Pipeline

Stage	Jobs
1	<code>clang-format</code> check
2	<code>clang-tidy</code> selected checks
3	Build matrix: Debug / Release, Excel OFF / ON, backends disabled / enabled
4	Unit tests: model, validation, projection, assembly, I/O
5	Numerical tests: PF, OPF, analysis
6	Sanitizers: AddressSanitizer, UndefinedBehaviorSanitizer
7	Documentation: LaTeX build
Nightly	Large benchmarks, ThreadSanitizer, coverage, performance regression

19.7 Dependency Boundary Rules

19.7.1 Allowed Dependencies

Module	May depend on
<code>api</code>	<code>model</code> , <code>io</code> , <code>validation</code> , <code>power_flow</code> , <code>optimal_power_flow</code> , <code>analysis</code>
<code>io</code>	<code>model</code> , <code>validation</code> , <code>resulttypes</code>
<code>validation</code>	<code>model</code>
<code>projection</code>	<code>model</code> , <code>validation</code>
<code>assembly</code>	canonical/projection output, model enums, utilities
<code>power_flow</code>	<code>assembly</code> , <code>backend</code> , <code>modelresulttypes</code>
<code>optimal_power_flow</code>	<code>assembly</code> , <code>backend</code> , <code>power_models</code>
<code>power_models</code>	AML, canonical model data
<code>analysis</code>	<code>model</code> , <code>resulttypes</code> , <code>publicsolverAPIs</code>
<code>backend</code>	external solver libraries
<code>engine</code>	compatibility/adapters only

19.7.2 Disallowed Dependencies

Module	Must not depend on
<code>model</code>	<code>io</code> , <code>power_flow</code> , <code>optimal_power_flow</code> , <code>analysis</code>
<code>validation</code>	solver internals
<code>power_flow</code>	<code>io</code>
<code>optimal_power_flow</code>	<code>io</code>
<code>assembly</code>	Excel / JSON I/O
<code>backend</code>	model-specific business logic
<code>analysis</code>	private Newton/IPM internals (unless intentionally internal)

These boundaries can be enforced by include-guard audits or a build-system layer-visibility policy.

19.8 Priority Roadmap

Priority 1 — Trustworthiness

1. Add independent PF and OPF audit checks (`OpfAudit`).

2. Add derivative finite-difference tests for Jacobians.
3. Add serialisation completeness tests.
4. Add projection mapping reports.
5. Add solver trace reporting for fallback method transitions.

Priority 2 — User Experience

1. Add strict/permissive import modes.
2. Extend the existing runtime capability discovery API with richer version/path diagnostics for optional external backends.
3. Add better infeasibility diagnostics with probable causes.
4. Add example workflows (see Section 19.9).
5. Publish JSON schema files.

Priority 3 — Maintainability

1. Enforce dependency boundaries via build-system rules.
2. Split large analysis submodules if necessary.
3. Centralise field descriptors into a machine-readable registry.
4. Keep `engine/` as adapter/ façade only.

Priority 4 — Performance and Scalability

1. Benchmark assembly time separately from solve time.
2. Track sparse-matrix nonzero counts across test cases.
3. Add performance regression cases for large networks.
4. Profile three-phase and time-series simulation.
5. Cache symbolic factorisations where safe.

19.9 Example Clean Public Workflow

A mature public API should look like the following. The pattern “load, validate, configure, solve, report” captures the full user journey:

```
#include <hacdcpf/api/hacdcpf.hpp>
#include <hacdcpf/io/json_io.hpp>

int main() {
    // 1. Load with error propagation
    auto system_result = hacdcpf::io::try_load_json("case.json");
    if (!system_result) {
        std::cerr << system_result.error().message << "\n";
        return 1;
    }
}
```



```

    auto system = system_result.value();

    // 2. Validate before solving
    auto report = hacdcpf::validation::validate(
        system, hacdcpf::validation::ValidationLevel::SolverReady);
    if (!report.ok()) {
        std::cerr << report.summary() << "\n";
        return 1;
    }

    // 3. Structured option assembly
    auto options = hacdcpf::PowerFlowOptions::from_parts(
        hacdcpf::ConvergenceOptions{},
        hacdcpf::LinearSolverOptions{},
        hacdcpf::GlobalizationOptions{},
        hacdcpf::PVPQSwitchingOptions{},
        hacdcpf::ConverterModeOptions{},
        hacdcpf::ZipLoadOptions{},
        hacdcpf::RuntimeOptions{}
    );

    // 4. Solve
    auto pf = hacdcpf::solve_power_flow(system, options);
    if (!pf.converged) {
        std::cerr << pf.diagnostics.termination_reason << "\n";
        return 2;
    }

    // 5. Save result
    hacdcpf::io::save_power_flow_result_json(system, pf, "pf_result.json");

    // 6. Post-solve analysis
    auto carbon = hacdcpf::analysis::run_carbon_analysis(system, options);
    hacdcpf::io::save_carbon_result_json(carbon, "carbon.json");

    return 0;
}

```

19.10 Final Recommendation Summary

The package has moved substantially in the right direction. The next most valuable work is not adding more solver features, but improving **trust, diagnostics, and maintainability**:

1. Add independent numerical audit reports for PF and OPF.
2. Add automated serialisation completeness tests.
3. Add projection mapping reports.
4. Extend runtime backend capability discovery with backend version and executable-path diagnostics.
5. Enforce dependency boundary rules.

6. Keep analysis modules modular and subdomain-separated.
7. Improve infeasibility and non-convergence explanations.
8. Build benchmark and regression infrastructure.

The main remaining structural risks are:

- **analysis/** becoming a miscellaneous collection without subdomain separation.
- Documentation and serialisers drifting without a field-descriptor registry.
- Incomplete backend capability discovery forcing compile-time conditional code into user applications.
- Three-phase unbalanced models being over-coupled to balanced PF abstractions.
- OPF infeasibility diagnostics remaining too coarse for user debugging.

If the roadmap in Section 19.8 is followed, the package will be significantly easier to maintain, debug, validate, and adopt by external users.

20 Complete Data Model: Enumerations, Defaults, and Error Types

This chapter provides the complete reference for the lower layers of the `model/` subsystem: the enumeration hierarchy (nine sub-headers under `include/hacdcpf/model/enums/`), the centralised `Defaults` constant struct, the `Result<T>/Error/ErrorCode` monadic result type, and the validation subsystem.

20.1 Enumeration Hierarchy

All domain enumerations live in `include/hacdcpf/model/enums/` and are re-exported by the umbrella header `include/hacdcpf/model/enums.hpp`. Each enum is a scoped `enumclass` in namespace `hacdcpf`.

20.1.1 Bus Type Enumerations

Declared in `model/enums/bus_types.hpp`.

Enum	Enumerator	Meaning
BusType	PQ=1	Constant-power load bus
	PV=2	Voltage-controlled generator bus
	SLACK=3	Swing bus (voltage & angle reference)
	ISOLATED=4	Disconnected bus (removed from solve)
DCBusType	DC_P=1	DC active-power controlled bus
	DC_V=3	DC voltage-controlled bus

20.1.2 Generator Enumerations

Declared in `model/enums/generator_enums.hpp`.

Enum	Enumerator	Meaning
GenCostModel	Polynomial=2	$f(P_g) = c_2 P_g^2 + c_1 P_g + c_0$
	PiecewiseLinear=1	Segmented cost curve
FuelType	Coal, Gas, Oil, Nuclear	Conventional thermal
	Hydro, Wind, Solar	Renewable hydro / wind / PV
	Biomass, Geothermal	Alternative renewables
	Storage	Storage acting as generator
	Unknown=0	Unspecified

20.1.3 Converter and Loss Model Enumerations

Declared in `model/enums/converter_enums.hpp`.

Enum	Enumerator	Meaning
ConverterMode	PQ_MODE	Fixed active + reactive injection
	VDC_Q	DC voltage regulation + fixed Q
	VDC_VAC	DC voltage + AC voltage regulation
LossModelType	Linear	$P_{loss} = a_0 + a_1 I$
	CurrentBased	$P_{loss} = a_0 + a_1 I + a_2 I^2$
DCDCControlMode	Voltage=0	Regulate DC output voltage
	Power=1	Regulate transferred power
	Droop=2	Droop-controlled power sharing
ERPortType	AC=0	AC-side energy-router port
	DC=1	DC-side energy-router port
ERControlMode	PQ=0	Fixed power dispatch
	VF=1	Grid-forming voltage/frequency control
	Droop=2	Droop-based power sharing

20.1.4 Load, Storage, and Renewable Enumerations

Declared in `model/enums/load_enums.hpp`, `storage_enums.hpp`, and `renewable_enums.hpp`.

Enum	Enumerator	Meaning
LoadModel	ConstantPower=0	Standard PQ bus
	ZIP=1	Polynomial: $P = P_0(Z V ^2 + I V + R)$
	Exponential=2	$P = P_0V^\alpha$
LoadPriority	Low=1 to Critical=4	Shedding priority
StorageMode	Idle=0, Charging=1, Discharging=2	Operating mode
MobileStorageStatus	Stationary=0	Not in transit
	InTransit=1	Relocating between sites
	Deployed=2	Operational at new site
RenewableType	Wind=0, SolarPV=1, SolarCSP=2, Hydro=3	Renewable technology
SgenType	PV=0, Wind=1, CHP=2, Diesel=3, FuelCell=4	Distributed generation
PVControlMode	MPPT=0	Max power point tracking
	PQ=1	Fixed P and Q setpoints
	VQ=2	Voltage reactive support
	Curtailed=3	Output curtailment

20.1.5 Switching and Grid Enumerations

Enum	Enumerator	Meaning
SwitchType	CircuitBreaker=0	Full fault-interrupting capability
	Disconnecter=1	No-load isolation
	LoadBreakSwitch=2	Switching under load
	Fuse=3	Overcurrent protection
	Recloser=4	Auto-reclosing
	Sectionalizer=5	Sectionalization
BreakerType	CB=0, LS=1, DS=2	Circuit breaker / load switch / disconnecter
MicrogridMode	GridConnected=0	Connected to utility
	Islanded=1	Autonomous operation
	Transition=2	Mode-change transient
ChargerType	AC_L1=0	Level 1 AC (≤ 3.7 kW)
	AC_L2=1	Level 2 AC (≤ 22 kW)
	DC_Fast=2	DC fast charge (≤ 350 kW)

20.2 EnumStrings Utility

The header `include/hacdcpf/model/enum_strings.hpp` provides header-only inline bidirectional string converters for every enum type. Each type `X` has two functions:

Function	Description
<code>x_str(X) → std::string</code>	Enum value to string
<code>x_from_str(std::string) → X</code>	String to enum (default on unknown)

Converters exist for: `BusType`, `DCBusType`, `ConverterMode`, `FuelType`, `LossModelType`, `LoadModel`, `StorageMode`, `RenewableType`, `SgenType`, `PVControlMode`, `MicrogridMode`, `SwitchType`, and `ChargerType`. They are consumed by `json_io`, `excel_io`, and any future serialisers.

20.3 Defaults: System-Wide Numerical Constants

`include/hacdcpf/model/defaults.hpp` declares a single POD struct `hacdcpf::Defaults` whose static `constexpr` members serve as the single source of truth for all numerical defaults across JSON/Excel import, the MATPOWER parser, model constructors, solver assembly, and tests.

Constant	Value	Units	Description
<code>kBaseMva</code>	100.0	MVA	System base apparent power
<code>kFreqHz</code>	50.0	Hz	Nominal network frequency
<code>kBusVoltage</code>	1.0	pu	Bus voltage magnitude initial value
<code>kBusAngleDeg</code>	0.0	deg	Bus voltage angle initial value
<code>kVoltageMinPu</code>	0.9	pu	AC bus lower voltage bound
<code>kVoltageMaxPu</code>	1.1	pu	AC bus upper voltage bound
<code>kDCVoltageMinPu</code>	0.9	pu	DC bus lower voltage bound
<code>kDCVoltageMaxPu</code>	1.1	pu	DC bus upper voltage bound
<code>kTapRatio</code>	1.0	pu	Transformer tap ratio
<code>kShiftDeg</code>	0.0	deg	Phase-shifting transformer angle
<code>kRateAMva</code>	0.0	MVA	Thermal rating (0 = unlimited)
<code>kGenPmax</code>	10^6	MW	Default generator upper active limit
<code>kGenPmin</code>	0.0	MW	Default generator lower active limit
<code>kGenQmax</code>	10^6	MVAr	Default reactive upper limit
<code>kGenQmin</code>	-10^6	MVAr	Default reactive lower limit
<code>kGenVgPu</code>	1.0	pu	Generator terminal voltage setpoint
<code>kPFTol</code>	10^{-8}	pu	Newton PF convergence tolerance
<code>kPFMaxIter</code>	50	—	Newton PF maximum iterations
<code>kOPFTol</code>	10^{-8}	pu	OPF convergence tolerance
<code>kOPFMaxIter</code>	300	—	OPF maximum iterations
<code>kCostScaleFactor</code>	1.0	\$/h	Objective cost scale factor
<code>kSchemaVersion</code>	"1.0"	—	JSON schema version string
<code>kPackageVersion</code>	"0.5.0"	—	Library release version string

Usage pattern: `usingD=hacdcpf::Defaults;doubletol=D::kPFTol;.`

20.4 Structured Error and Result Types

`include/hacdcpf/model/error.hpp` provides lightweight, exception-free error propagation for batch workflows. It is header-only with no dependencies beyond the C++ standard library.

20.4.1 ErrorCode Enumeration

Code	Value	Description
InvalidInput	1000	Malformed input data
ValidationFailed	1001	Pre-solve validation returned errors
MissingSlackBus	1002	No swing bus defined
DisconnectedNetwork	1003	Network has isolated subsystem
SingularAdmittance	1004	Ybus is numerically singular
NotConverged	2000	Newton solver did not converge
MaxIterationsReached	2001	Iteration limit exceeded
NumericalFailure	2002	Floating-point failure (NaN/Inf)
JacobianSingular	2003	Jacobian is numerically singular
HomotopyFailed	2004	Homotopy path tracking failed
FileNotFound	3000	File path does not exist
ParseError	3001	File format / parsing error
SchemaVersionMismatch	3002	JSON schema version incompatible
NotImplemented	9000	Feature not yet implemented
Unknown	9999	Unclassified error

20.4.2 Error Descriptor

Field	Type	Description
code	ErrorCode	Machine-readable error code
message	std::string	Human-readable description
details	vector<ValidationIssue>	Attached validation issues (optional)

Three factory methods simplify common cases:

- `Error::validation_failed(report)` — attaches a `ValidationReport`.
- `Error::not_converged(reason)` — fills code `NotConverged` with a message.
- `Error::parse_error(path,msg)` — fills code `ParseError`.

20.4.3 Result<T> Monad

`Result<T>` is a `std::variant<T,Error>` wrapper that carries either a success value or an error:

Member	Description
<code>has_value() → bool</code>	True if successful
<code>operatorbool()</code>	Alias for <code>has_value()</code>
<code>value() → T&</code>	Access success payload (UB on error)
<code>operator*(), operator->()</code>	Dereference to payload
<code>error() → Error&</code>	Access error payload
<code>value_or_throw() → T&</code>	Access value or throw <code>std::runtime_error</code>

Typical usage:

```
auto res = safe_solve_power_flow(sys, opt);
if (!res) { std::cerr << res.error().message; return; }
auto& r = res.value();
```

20.5 Validation Subsystem

`include/hacdcpf/validation/` contains two headers that implement static pre-solve network validation.

20.5.1 ValidationReport and ValidationIssue

Declared in `validation_report.hpp`:

Type / Member	Description
<code>Severity::Info</code>	Informational note; solving proceeds
<code>Severity::Warning</code>	Degraded results possible; solving proceeds
<code>Severity::Error</code>	Fatal; system cannot be solved as-is
<code>ValidationIssue::severity</code>	Diagnostic severity level
<code>ValidationIssue::component_type</code>	e.g. "ACBus", "VSCConverter"
<code>ValidationIssue::component_id</code>	External ID / name
<code>ValidationIssue::field</code>	Field name that is problematic
<code>ValidationIssue::message</code>	Human-readable description
<code>ValidationReport::is_valid()</code>	True if no Error -severity issues
<code>ValidationReport::has_errors()</code>	Alias for <code>!is_valid()</code>
<code>ValidationReport::has_warnings()</code>	True if any Warning issues
<code>ValidationReport::count(Severity)</code>	Count of issues at a given severity
<code>ValidationReport::add(s,type,id,field,msg)</code>	Append a new issue

20.5.2 Validation Checks

`validate(constHybridPowerSystem&)` in `validation/validate_system.hpp` / `src/validation/validate_system.cpp` runs 15 static checks and returns a `ValidationReport`:

1. **base_mva > 0** — system base must be positive.
2. **Duplicate AC bus IDs** — no two AC buses share an index.
3. **Duplicate DC bus IDs** — no two DC buses share an index.
4. **AC branch bus references** — `from_bus` and `to_bus` exist.
5. **DC branch bus references** — same, in the DC network.
6. **AC voltage limits** — $V_{min} > 0$, $V_{min} < V_{max}$.
7. **Generator P limits** — $P_{min} \leq P_{max}$.
8. **Generator Q limits** — $Q_{min} \leq Q_{max}$.
9. **Generator bus reference** — generator bus exists in AC network.
10. **Slack bus presence** — at least one `BusType::SLACK` bus.
11. **Transformer tap ratio** — $0.5 < \text{tap} < 2.0$.
12. **VSC converter bus references** — AC and DC bus indices exist.
13. **DC-DC converter bus references** — both DC buses exist.

14. **Non-negative AC branch resistance** — $r_{pu} \geq 0$.

15. **AC/DC base MVA consistency** — subsystem base MVA matches system base.

Any **Error**-severity result from `validate()` will prevent the solver from proceeding. The `Error::validation_failed()` factory method wraps a failed report into the structured error type for API return.

21 Solver Infrastructure: Assembly, Projection, Backend, and Utilities

This chapter documents the horizontal infrastructure layers that sit between the high-level `HybridPowerSystem` model and the numerical solver kernels: the **assembly** layer (flat solver-ready data), the **projection** layer (canonical model expansion and bus merging), the **backend** abstraction interfaces (linear, NLP, and QP solvers), and the **thread pool** utility.

21.1 Assembly Layer

The assembly layer (`include/hacdcpf/assembly/`) converts a `HybridPowerSystem` into flat, pre-computed data structures ready for direct consumption by numerical solvers.

21.1.1 SolverData

`assembly/solver_data.hpp` — namespace `hacdcpf::powerflow`.

`SolverData` is the canonical flat representation of a projected network. It owns all component vectors and the precomputed sparse matrices used by every PF/OPF kernel.

Component vectors

Field	Description
<code>ac_buses</code>	AC bus vector (post-projection)
<code>ac_branches</code>	AC branch vector (incl. expanded transformer equivalents)
<code>dc_buses</code>	DC bus vector
<code>dc_branches</code>	DC branch vector
<code>converters</code>	VSC AC/DC converters
<code>dcdc_converters</code>	DC-DC converters
<code>energy_routers</code>	Multi-port energy routers
<code>generators</code>	Synchronous and asynchronous generators
<code>loads</code>	AC constant-power and ZIP loads
<code>flexible_loads</code>	Demand-response flexible loads
<code>static_generators</code>	Distributed generation (inverter-coupled)
<code>renewable_gens</code>	Wind and solar renewable generation
<code>pv_systems</code>	PV array systems (MPPT model)
<code>storage_units</code>	AC battery / flywheel storage
<code>dc_loads</code>	DC bus loads
<code>dc_storage</code>	DC bus storage
<code>dc_static_generators</code>	DC-connected distributed generation
<code>dc_pv_arrays</code>	DC PV arrays
<code>shunts</code>	Fixed and switchable shunts

Field	Description
charging_stations	EV charging stations
chargers	Individual EV chargers
external_grids	External grid equivalents
switches	Switching devices (CB, DS, LBS, etc.)
vpps	Virtual power plants
microgrids	Microgrid aggregates
mobile_storage	Mobile storage units

Numerical parameters and flags

Field	Default	Description
base_mva	100.0	System base MVA
loss_model	Linear	VSC converter loss model type
zip_pw[3]	{1,0,0}	ZIP P weights (Z,I,P fractions)
zip_qw[3]	{1,0,0}	ZIP Q weights
has_component_loads	false	True after ZIP load aggregation
pd_pu, qd_pu	—	Aggregated bus P/Q load (pu)
bus_zip_*	—	Per-bus ZIP weight vectors (Z/I/P, P and Q)
enable_coupled_jacobian	false	Coupled AC/DC Jacobian
enable_augmented_equations	false	VDC/VAC augmented rows
enable_semi_smooth_newton	false	Fischer-Burmeister PV/PQ
nep_mu	0.0	Smooth-NCP smoothing parameter
pg, qg	—	Aggregated bus generator P/Q (pu)
ybus	—	Complex admittance matrix $\mathbf{Y}_{bus}$ (sparse)
gdc	—	DC conductance matrix $\mathbf{G}_{dc}$ (sparse)
build_id	0	Monotone counter incremented on each rebuild
bus_merge_map	nullopt	Optional zero-impedance bus merge map

Factory and utility functions

Function	Description
make_solver_data(sys)	Build from HybridPowerSystem (copy)
make_solver_data(sys&&)	Build from rvalue (move-efficient)
make_solver_data_projected(projected&&)	Build from already-projected system
aggregate_generation(data)	Sum generator injections onto bus vectors
aggregate_load_demand(data)	Sum load demands onto bus vectors
rebuild_matrices(data)	Recompute $\mathbf{Y}_{bus}$ and $\mathbf{G}_{dc}$

21.1.2 Index Maps

assembly/index_map.hpp maps between external (1-based user-facing) and internal (0-based solver) bus and branch indices.

Type	Description
<code>BusIndexMap::ext_to_pos</code>	<code>unordered_map<int,int></code> : external $\rightarrow$ internal
<code>BusIndexMap::pos_to_ext</code>	<code>vector<int></code> : internal $\rightarrow$ external
<code>BranchIndexMap</code>	Same pattern for branches
<code>SystemIndexMap</code>	Combined AC/DC bus and branch maps + optional merge map pointer

21.1.3 Admittance (Y-bus) Builder

`assembly/ybus_builder.hpp` provides three matrix builders:

Functions

Function	Returns
<code>build_admittance_matrix(data)</code>	Full complex $\mathbf{Y}_{bus}$ for AC Newton PF
<code>build_dc_conductance(data)</code>	Real $\mathbf{G}_{dc}$ for DC PF
<code>build_susceptance_matrix(data,flags)</code>	Real $\mathbf{B}'$ or $\mathbf{B}''$ for FDPF

BpBuildFlags (used with `build_susceptance_matrix`):

Flag	Default	Effect
<code>zero_resistance</code>	false	Set series $R = 0$ (builds $\mathbf{B}'$ for XB-FDPF)
<code>zero_charging</code>	false	Ignore line charging b_{pu}
<code>unit_tap</code>	false	Treat tap magnitude as 1
<code>zero_phase_shift</code>	false	Zero all transformer phase shifts
<code>add_bus_shunts</code>	false	Add bus shunt susceptance $-B_s$ (builds $\mathbf{B}''$)
<code>min_x_pu</code>	0.0	Skip branches where $ x_{pu} <$ this threshold

Two preset builders are provided:

- `make_bp_flags(min_x=1e-6)` — builds $\mathbf{B}'$ for the FDXB variant.
- `make_bpp_flags()` — builds $\mathbf{B}''$ for the FDBX variant.

21.2 Projection Layer

The projection layer (`include/hacdcpf/projection/`) converts a *rich* `HybridPowerSystem` (with transformers, switches, complex converters) into a *canonical* form ready for solver assembly.

21.2.1 Canonical Network Types

Declared in `projection/canonical_network.hpp`:

Type	Description
<code>BranchOriginType</code>	Transformer2W, Transformer3W, Switch
<code>BranchExpandEntry</code>	Records which branch was synthesised from which rich element
<code>BranchExpandMap</code>	Collection of <code>BranchExpandEntry</code> records

Type	Description
<code>BusMergeMap::ext_to_int</code>	External bus index $\rightarrow$ merged internal position
<code>BusMergeMap::int_to_ext</code>	Internal position $\rightarrow$ representative external index
<code>BusMergeMap::groups</code>	Per-representative list of merged external buses
<code>BusMergeMap::dead_bus_indices</code>	External indices of dead-island buses stripped
<code>BusMergeMap::branch_orig_to_proj</code>	Original branch pos $\rightarrow$ projected pos (-1 = dead)
<code>BusMergeMap::has_merges()</code>	True if any zero-impedance merging occurred
<code>BusMergeMap::has_dead_buses()</code>	True if any dead islands were stripped
<code>BusMergeMap::identity(n)</code>	Constructs no-op (identity) mapping for n buses

21.2.2 Projection Functions

Declared in `projection/project_to_canonical.hpp`:

Count queries (read-only)

Function	Returns
<code>n_ac_buses(sys), n_dc_buses(sys), n_buses(sys)</code>	Bus counts
<code>n_ac_branches(sys), n_dc_branches(sys), n_branches(sys)</code>	Branch counts
<code>n_generators(sys), n_loads(sys), n_shunts(sys)</code>	Component counts
<code>n_storage(sys), n_renewable_gens(sys), n_converters(sys)</code>	Component counts

Aggregation queries

Function	Description
<code>total_gen_capacity_mw(sys)</code>	Sum of all generator P_{max}
<code>total_gen_pmin_mw(sys)</code>	Sum of all generator P_{min}
<code>total_load_p_mw(sys) / q_mvar</code>	Total AC active / reactive load
<code>total_renewable_capacity_mw(sys)</code>	Total renewable nameplate capacity
<code>total_storage_capacity_mwh(sys)</code>	Total storage energy capacity
<code>total_storage_power_mw(sys)</code>	Total storage peak power
<code>total_system_inertia_mws(sys)</code>	Total rotational inertia
<code>total_emission_tco2_h(sys)</code>	Total CO ₂ emission rate

Topology helpers

- `ac_adjacency_list(sys)` — adjacency list of AC buses.
- `generators_per_bus(sys)` — generator count per bus.
- `print_summary(sys, os) / summary_string(sys)` — formatted system summary.

Projection pipeline `project_to_canonical_models(sys)` performs:

1. Expand `Transformer2W` / `Transformer3W` into equivalent π -model `ACBranch` entries.
2. Expand closed `Switch` devices into zero-impedance branches.
3. Populate `BranchExpandMap` for result un-projection.

Two overloads accept either `const` reference (copy) or `rvalue` (move-efficient).

Zero-impedance bus merging `merge_zero_impedance_buses(sys)` merges buses connected by branches with $|Z| < \text{kBusMergeZThreshold} = 10^{-4}$ pu into equivalence classes, choosing one representative per class. The resulting `BusMergeMap` allows `unproject_bus_vector(merged, map)` to expand solver voltage solutions back to the original bus indexing, returning 0.0 for dead-island buses.

`strip_dead_islands(sys)` removes buses with no generation path (neither direct generators nor grid connections).

21.3 Backend Solver Abstractions

The `include/hacdcpf/backend/` headers expose thin adapter namespaces and abstract interfaces for the three solver categories used by the numerical kernels.

21.3.1 Linear Solver Backend

`backend/linear_solver.hpp` re-exports into `hacdcpf::backend` and `hacdcpf::powerflow` (for backward compatibility):

- `SparseLinearSolver` — abstract base class for sparse LU-type solvers.
- `EigenSparseLUSolver` — Eigen SparseLU concrete implementation.
- `make_default_sparse_solver()` — factory returning an `EigenSparseLUSolver`.

These are defined in `engine/kernel/linear_algebra/linear_solver.hpp` and exposed here for layer-agnostic consumption.

21.3.2 NLP Solver Backend

`backend/nlp_solver.hpp` — namespace `hacdcpf::backend`.

Type / Field	Default	Description
NLPStatus		
<code>Optimal</code>	—	Global/local optimum found
<code>LocalOptimal</code>	—	Local optimum, not proven global
<code>Infeasible</code>	—	Problem is infeasible
<code>Unbounded</code>	—	Objective is unbounded below
<code>MaxIterations</code>	—	Iteration limit reached
<code>NumericalFailure</code>	—	NaN/Inf or pivot breakdown
NLPResult		
<code>status</code>	<code>Unknown</code>	Solve outcome
<code>x</code>	—	Primal solution vector
<code>lambda</code>	—	Equality constraint multipliers
<code>mu</code>	—	Inequality constraint multipliers
<code>objective</code>	0.0	Optimal objective value
<code>iterations</code>	0	Inner iteration count
<code>primal_infeasibility</code>	0.0	$\ \mathbf{g}(x^*)\ _\infty$
<code>dual_infeasibility</code>	0.0	$\ \nabla_x L\ _\infty$
<code>complementarity</code>	0.0	KKT complementarity residual

Type / Field	Default	Description
NLPOptions		
feasibility_tol	10^{-6}	Constraint feasibility tolerance
optimality_tol	10^{-6}	KKT optimality tolerance
max_iterations	300	Maximum inner iterations
verbose	false	Print solver progress

21.3.3 QP Solver Backend

backend/qp_solver.hpp — namespace `hacdcpf::backend`.

The IQPSolver abstract class solves:

$$\min_x \frac{1}{2}x^\top Hx + c^\top x \quad \text{s.t.} \quad Ax \leq b, \quad l \leq x \leq u. \quad (165)$$

Type / Field	Default	Description
QPStatus		
Optimal	—	Optimal solution found
Infeasible, Unbounded	—	Problem degenerate
MaxIterations, NumericalFailure	—	Solver limits
QPResult		
primal	—	Solution x^*
dual	—	Constraint multipliers
objective	0.0	Optimal objective
iterations	0	Simplex / interior-point iterations
QPOptions		
feasibility_tol	10^{-8}	Primal feasibility tolerance
optimality_tol	10^{-8}	Dual optimality tolerance
max_iterations	10000	Iteration cap

21.4 Thread Pool

include/hacdcpf/util/thread_pool.hpp provides a lightweight fixed-size worker pool for parallelising the AC Jacobian evaluation kernel. It exposes three core operations:

Member	Description
ThreadPool(num_threads)	Construct; 0 uses <code>hardware_concurrency()</code>
submit(callable)	Enqueue a task; returns <code>std::future</code>
parallel_for(n, fn)	Execute <code>fn(i)</code> for $i \in [0, n)$ across workers
ThreadPool::global()	Process-wide singleton (created on first call)

The thread count is controlled by `PowerFlowOptions::ac_eval_threads`; setting it to 1 (the default) disables parallelism and uses the calling thread. The pool is persistent across solve calls (no spin-up overhead per solve).

21.5 Detail Utilities

`include/hacdcpf/detail/` contains internal implementation helpers not part of the public API:

Header	Contents
<code>logging.hpp</code>	<code>hacdcpf::detail::log(level,msg)</code> debug logger
<code>string_utils.hpp</code>	String trimming, splitting, and formatting helpers
<code>core_compat.hpp</code>	C++17/20 portability shims (<code>std::span</code> polyfill, etc.)
<code>internal_helpers.hpp</code>	Miscellaneous internal utilities (range checks, etc.)

These headers are consumed only by `src/` translation units. External users should not include them directly; the public API encapsulates all necessary behaviour.

22 Power-Flow Options, Results, and NCP Complementarity

This chapter provides the complete reference tables for `PowerFlowOptions` and its seven sub-structs, all power-flow result types, and the Fischer-Burmeister NCP functions used by the semi-smooth Newton solver.

22.1 PowerFlowOptions Sub-Structs

`PowerFlowOptions` (`include/hacdcpf/power_flow/power_flow_options.hpp`) is a flat layout struct for backward compatibility; it can also be assembled from seven orthogonal sub-structs via `PowerFlowOptions::from_parts()`.

22.1.1 ConvergenceOptions

Field	Default	Units	Description
<code>max_iter</code>	50	—	Newton max iterations (<code>Defaults::kPFMaxIter</code>)
<code>tol</code>	10^{-8}	pu	Newton convergence tolerance
<code>fdpf_max_iter</code>	1000	—	FDPF inner iteration limit

22.1.2 LinearSolverOptions

Field	Default	Units	Description
<code>max_line_search_steps</code>	10	—	Backtracking step budget
<code>max_regularization_steps</code>	8	—	Tikhonov regularisation attempts
<code>regularization_lambda0</code>	10^{-6}	—	Initial regularisation strength
<code>regularization_growth</code>	10.0	—	Regularisation growth factor
<code>max_delta_va_rad</code>	1.50	rad	Clamp on angle step $ \Delta\theta $
<code>max_delta_vm_pu</code>	0.50	pu	Clamp on voltage step $ \Delta V $
<code>max_delta_vdc_pu</code>	0.50	pu	Clamp on DC voltage step $ \Delta V_{dc} $
<code>enable_coupled_jacobian</code>	false	—	Coupled AC/DC Newton block
<code>enable_augmented_equations</code>	false	—	Augmented VDC/VAC control rows
<code>enable_semi_smooth_newton</code>	false	—	Fischer-Burmeister PV/PQ

22.1.3 GlobalizationOptions

Field	Default	Units	Description
strategy	LineSearch	—	LineSearch TrustRegion PseudoTransient
trust_region_radius0	1.0	—	Initial trust-region radius
trust_region_max	10.0	—	Maximum trust-region radius
ptc_delta0	1.0	—	Initial PTC pseudo-timestep
ptc_growth	2.0	—	PTC timestep growth factor

22.1.4 PVPQSwitchingOptions

Field	Default	Units	Description
pv_q_hysteresis_pu	0.01	pu	Q-limit hysteresis band for PV→PQ switch
pv_recover_vm_tol_pu	0.01	pu	Voltage recovery tolerance for PQ→PV
enable_pv_pq_conversion	true	—	Allow dynamic PV/PQ bus switching
enable_auto_swing_selection	true	—	Automatically select slack bus if missing

22.1.5 ConverterModeOptions

Field	Default	Units	Description
converter_vdc_switch_high_pu	0.03	pu	VSC mode switch high threshold
converter_vdc_switch_low_pu	0.01	pu	VSC mode switch low threshold
mode_hysteresis_iters	2	—	Min iterations between mode changes
enable_converter_mode_switching	true	—	Dynamic VSC mode changes
loss_model	Linear	—	VSC loss model type

22.1.6 ZipLoadOptions

Field	Default	Units	Description
pw[3]	{1, 0, 0}	—	P model weights: {Z fraction, I fraction, P fraction}
qw[3]	{1, 0, 0}	—	Q model weights: {Z fraction, I fraction, P fraction}

The ZIP load model computes:

$$P_i = P_i^0 (z_P |V_i|^2 + i_P |V_i| + p_P), \quad (166)$$

$$Q_i = Q_i^0 (z_Q |V_i|^2 + i_Q |V_i| + p_Q). \quad (167)$$

Default weights {1, 0, 0} recover the constant-power (PQ) model.

22.1.7 RuntimeOptions

Field	Default	Units	Description
ac_eval_threads	0	—	Parallel threads for Jacobian evaluation; 0 = auto-detect (hardware)
enable_solver_profiling	false	—	Collect per-call timing counters
enable_iteration_log	false	—	Store per-iteration log entries
verbose	false	—	Print convergence progress to stdout

22.2 Power-Flow Result Hierarchy

All result types are declared in `include/hacdcpf/power_flow/power_flow_result.hpp`.

22.2.1 ResidualBreakdown

Reports per-equation group ℓ_∞ norms at a single iteration:

Field	Description
<code>ac_p_norm</code>	$\ \Delta P_{AC}\ _\infty$
<code>ac_q_norm</code>	$\ \Delta Q_{AC}\ _\infty$
<code>dc_p_norm</code>	$\ \Delta P_{DC}\ _\infty$
<code>converter_p_norm</code>	$\ \Delta P_{conv}\ _\infty$
<code>converter_q_norm</code>	$\ \Delta Q_{conv}\ _\infty$
<code>control_eq_norm</code>	$\ \Delta f_{ctrl}\ _\infty$ (augmented equations)
<code>total_norm()</code>	max of the above six norms

22.2.2 IterationLogEntry

Field	Default	Description
<code>iteration</code>	0	Newton iteration index
<code>residual_norm</code>	0.0	ℓ_∞ residual at this step
<code>step_norm</code>	0.0	ℓ_∞ Newton step size
<code>damping_factor</code>	1.0	Line-search damping α
<code>jacobian_refactorized</code>	false	True if Jacobian was refactorised
<code>lm_fallback_used</code>	false	True if LM step replaced Newton step
<code>breakdown</code>	—	Per-group <code>ResidualBreakdown</code>

22.2.3 SolverDiagnostics

High-level convergence summary stored in every `PowerFlowResult`:

Field	Default	Description
<code>converged</code>	false	Final convergence flag
<code>termination_reason</code>	—	Human-readable reason string
<code>iterations</code>	0	Total Newton iterations
<code>initial_mismatch_norm</code>	0.0	Residual norm at iteration 0
<code>final_mismatch_norm</code>	0.0	Residual norm at convergence
<code>max_p_mismatch_pu</code>	0.0	Worst active power mismatch
<code>max_q_mismatch_pu</code>	0.0	Worst reactive power mismatch
<code>max_i_violation_pu</code>	0.0	Worst current limit violation
<code>condition_estimate</code>	0.0	Jacobian ℓ_1 condition proxy
<code>final_breakdown</code>	—	Per-group residuals at final step
<code>iteration_log</code>	{}	Per-step log (populated if enabled)
<code>warnings</code>	{}	Textual warnings from the solver

22.2.4 SolverProfiling

Detailed performance counters (populated when `enable_solver_profiling=true`):

Field	Default	Description
<code>linear_solver_backend</code>	""	Name of sparse solver used
<code>ac_eval_threads</code>	1	Thread count for Jacobian eval
<code>jacobian_pattern_rebuilds</code>	0	Sparsity pattern rebuilds
<code>jacobian_analyze_calls</code>	0	Symbolic factorisation calls
<code>factorization_calls</code>	0	Numeric factorisation calls
<code>linear_solve_calls</code>	0	Back-substitution calls
<code>regularization_attempts</code>	0	Tikhonov regularisation invocations
<code>line_search_evaluations</code>	0	Total line-search residual evaluations
<code>rejected_steps</code>	0	Steps discarded by line search
<code>pv_to_pq_switches</code>	0	PV→PQ bus type transitions
<code>pq_to_pv_switches</code>	0	PQ→PV bus type recoveries
<code>converter_mode_switches</code>	0	VSC control mode transitions
<code>residual_by_iter</code>	{}	Residual norm per iteration
<code>eval_jacobian_ms_total</code>	0.0	Total Jacobian wall time (ms)
<code>linear_solve_ms_total</code>	0.0	Total linear solve wall time (ms)
<code>line_search_ms_total</code>	0.0	Total line-search wall time (ms)
<code>raw_residual_norm</code>	0.0	Unscaled residual norm
<code>scaled_residual_norm</code>	0.0	Scaled residual norm (convergence criterion)
<code>condition_estimate</code>	0.0	Proxy Jacobian condition number
<code>regularization_count</code>	0	Successful regularisations applied

22.2.5 Component Flow Result Types

Struct	Fields	Description
BranchFlow	<code>index, from_bus, to_bus</code>	AC branch identifiers
	<code>pf_mw, qf_mvar</code>	From-end apparent power flow
	<code>pt_mw, qt_mvar</code>	To-end apparent power flow
VSCTransfer	<code>index, bus_ac, bus_dc</code>	Converter identifiers
	<code>p_ac_mw, q_ac_mvar</code>	AC terminal power
	<code>p_dc_mw, loss_mw</code>	DC terminal power and loss
DCDCTransfer	<code>index, bus_in, bus_out</code>	DC-DC converter identifiers
	<code>p_in_mw, p_out_mw, loss_mw</code>	Terminal powers and loss
Trafo3WFlow	<code>hv_bus, mv_bus, lv_bus</code>	Three-winding transformer buses
	<code>p/qatHV, MV, LV</code>	Terminal power flows
	<code>loss_mw, loading_pct, rate_mva</code>	Loss and loading metrics
ERPortTransfer	<code>router_index, port_index, bus</code>	Energy-router port
	<code>is_ac, p_mw, q_mvar, v_pu</code>	Port type and power

22.2.6 Top-Level Result Types

Type	Key Fields	Use Case
PowerFlowResult	vm, va, vdc (bus voltages) converged, iterations, residual profiling, diagnostics branch_flows, vsc_transfers dcdc_transfers, trafo3w_flows er_port_transfers	Primary AC/DC PF output Convergence summary Performance and diagnostics Component-level flows Transformer and DC-DC flows Energy-router port flows
DCPowerFlowResult	vdc, converged, iterations, residual	Linearised DC-only PF
AdaptiveSolveResult	vm, va, vdc, islands, branch_flows	Island-decomposed adaptive solve
IslandedSolveResult	vm, va, vdc, islands	Pure island detection result
DistributedSlackResult	vm, va, vdc distributed_slack_p hit_limits	Distributed slack solve Per-bus slack participation ΔP Buses that hit reactive limits

22.3 Fischer-Burmeister NCP Functions

`include/hacdcpf/power_flow/ncp_functions.hpp` provides the complementarity functions used by the semi-smooth Newton PV/PQ handling.

22.3.1 Standard Fischer-Burmeister Function

The smoothed Fischer-Burmeister (FB) NCP function is:

$$\text{FB}(a, b) = \sqrt{a^2 + b^2 + \varepsilon} - a - b, \quad \varepsilon = 10^{-14}. \quad (168)$$

$$\text{FB}(a, b) = 0 \iff a \geq 0, b \geq 0, ab = 0.$$

Its partial derivatives are:

$$\frac{\partial \text{FB}}{\partial a} = \frac{a}{\sqrt{a^2 + b^2 + \varepsilon}} - 1, \quad \frac{\partial \text{FB}}{\partial b} = \frac{b}{\sqrt{a^2 + b^2 + \varepsilon}} - 1. \quad (169)$$

22.3.2 PV/PQ Complementarity Equation

For a generator bus with reactive limits $[Q_{\min}, Q_{\max}]$ and voltage setpoint $\bar{V}$:

$$F_{NCP} = \text{FB}(Q_{\max} - Q_g, V - \bar{V}) + \text{FB}(Q_g - Q_{\min}, \bar{V} - V). \quad (170)$$

$F_{NCP} = 0$ enforces three complementary conditions simultaneously:

- $Q_{\min} < Q_g < Q_{\max}$: voltage regulation $V = \bar{V}$ (PV).
- $Q_g = Q_{\max}$: $V \leq \bar{V}$ (reactive limit, switch to PQ).
- $Q_g = Q_{\min}$: $V \geq \bar{V}$ (lower limit, switch to PQ).

Jacobian w.r.t. (Q_g, V) :

$$\frac{\partial F_{NCP}}{\partial Q_g} = -\partial_a \text{FB}_1 + \partial_a \text{FB}_2, \quad \frac{\partial F_{NCP}}{\partial V} = \partial_b \text{FB}_1 - \partial_b \text{FB}_2. \quad (171)$$

22.3.3 Smooth (μ -Perturbed) Fischer-Burmeister

Phase 2 of `RobustNonlinearOptions` uses a μ -perturbed smooth FB:

$$\varphi_\mu(a, b) = \sqrt{a^2 + b^2 + 2\mu} - a - b, \quad \mu > 0. \quad (172)$$

As $\mu \rightarrow 0$, this recovers the standard near-zero FB. μ is annealed from `ncp_mu0` to `ncp_mu_min` as the residual decreases, using a two-phase schedule:

$$\mu_{k+1} = \begin{cases} \text{ncp_mu_factor_coarse} \cdot \mu_k & \text{if } \|F\| > \theta \cdot \|F_0\|, \\ \text{ncp_mu_factor} \cdot \mu_k & \text{otherwise (fine phase).} \end{cases} \quad (173)$$

where $\theta = \text{ncp_mu_phase_transition}$ (default 0.1).

The smooth PV/PQ NCP residual and its Jacobian follow the same structure as the standard version but with `smooth_fb` replacing `fischer_burmeister`.

Activity hysteresis (`enable_activity_hysteresis=true`) prevents PV/PQ chattering by requiring `min_active_set_hold_iters` = 3 iterations to elapse between consecutive active-set changes.

23 Analysis Suite: Carbon, Production, Lifecycle, Distribution, Reliability, and Time Series

This chapter documents the analytical capability modules of the `hacdcpf` library. Following a module restructuring, each family of analytical algorithms now lives in its own peer directory rather than a shared `analysis/` facade. The current layout is:

Module directory	Contents
<code>include/hacdcpf/carbon_analysis / src/carbon_analysis</code>	Carbon emissions analysis (<code>compute_carbon</code>)
<code>include/hacdcpf/time_series / src/time_series</code>	UC/OPF/PF multi-period pipeline, annual p
<code>include/hacdcpf/reliability / src/reliability</code>	Monte Carlo reliability assessment
<code>include/hacdcpf/resilience / src/resilience</code>	Distribution resilience assessment
<code>include/hacdcpf/power_flow / src/power_flow</code>	Core Newton PF solvers plus the three-phases

All modules are declared in namespace `hacdcpf::analysis` (or the top-level `hacdcpf` namespace for shared data types) and exposed through `include/hacdcpf/api/hacdcpf.hpp`.

23.1 Carbon Emissions Analysis

Declared in `carbon_analysis/carbon_analysis.hpp`, implemented in `src/carbon_analysis/carbon_analysis.cpp`.

The primary entry point is `compute_carbon_analysis`, which has two overloads:

// Overload 1: supply an already-computed power-flow result.

```
CarbonAnalysisResult compute_carbon_analysis(
    const HybridPowerSystem& sys,
    const PowerFlowResult& pf_result,
    const CarbonAnalysisOptions& opt = {});
```

// Overload 2: run power flow internally, then analyse.

```
CarbonAnalysisResult compute_carbon_analysis(
    const HybridPowerSystem& sys,
    const PowerFlowOptions& pf_opt = {},
    const CarbonAnalysisOptions& ca_opt = {});
```

A backward-compatible inline wrapper `run_carbon_analysis(sys, opt)` calls overload 2 with default `PowerFlowOptions`.

`compute_carbon_analysis` performs power-flow-based emissions attribution via two independent methods:

1. **Proportional tracing** — Kirchhoff-based upstream tracing that allocates each generator's output to loads and losses in proportion to power flow direction.
2. **Nodal matrix method** — Builds and solves a linear system $\mathbf{A} \mathbf{c} = \mathbf{q}$ where c_i is the carbon intensity at bus i (tCO₂/MWh).

23.1.1 Mathematical Formulation

Proportional Tracing. Let $\mathcal{G}_i$ denote generators injecting into bus i , and let $\mathcal{U}(i) = \{j : P_{j \rightarrow i} > 0\}$ be the set of upstream buses with active inflow. The total inflow at bus i is:

$$T_i = \sum_{g \in \mathcal{G}_i} P_{g,i} + \sum_{j \in \mathcal{U}(i)} P_{j \rightarrow i}. \quad (174)$$

The carbon intensity at bus i is computed by the upstream tracing rule:

$$c_i = \frac{\sum_{g \in \mathcal{G}_i} e_g P_{g,i} + \sum_{j \in \mathcal{U}(i)} c_j P_{j \rightarrow i}}{T_i}, \quad (175)$$

where e_g (tCO₂/MWh) is the emission factor of generator g . The system is solved by topological ordering from generator buses outward. Mass-balance verification requires:

$$\left| \sum_g e_g P_g - \sum_{i \in \mathcal{L}} c_i P_i^L - \sum_{(i,j)} c_i P_{ij}^{loss} \right| \leq \varepsilon_{bal}. \quad (176)$$

Nodal Matrix Method. Equations (174)–(175) can be recast as a linear system $\mathbf{A} \mathbf{c} = \mathbf{q}$ with:

$$A_{ii} = T_i = \sum_{g \in \mathcal{G}_i} P_{g,i} + \sum_{j \in \mathcal{U}(i)} P_{j \rightarrow i}, \quad (177)$$

$$A_{ij} = -P_{j \rightarrow i} \quad (j \neq i, j \in \mathcal{U}(i)), \quad (178)$$

$$q_i = \sum_{g \in \mathcal{G}_i} e_g P_{g,i}. \quad (179)$$

The matrix $\mathbf{A}$ is diagonally dominant and its solution $\mathbf{c}$ is identical to the tracing result when $\mathbf{A}$ is non-singular. The residual $\|\mathbf{A} \mathbf{c} - \mathbf{q}\|_\infty$ is stored in `CarbonAnalysisResult::matrix_residual`.

23.1.2 Options and Result Types

Field (<code>CarbonAnalysisOptions</code>)	Default	Description
<code>min_contribution_mw</code>	1e-6	Minimum generator contribution to track (MW)
<code>verification_tol</code>	1e-4	Relative balance tolerance for tracing verification
<code>max_tracing_depth</code>	0	BFS depth limit (0 = unlimited)
<code>loss_allocation_alpha</code>	0.5	Loss split: α at sending end, $(1 - \alpha)$ at receiving end
<code>regularization_eps</code>	1e-8	Diagonal regulariser for near-zero pivot buses
<code>verbose</code>	false	Emit debug-level logging during analysis

CarbonAnalysisResult aggregates per-component results:

Field	Description
tracing_verified	Tracing mass-balance check passed
matrix_solved	Matrix method converged
matrix_residual	Residual of the linear system
load_carbon	Per-AC-load: demand, carbon intensity, total emissions, supply mix
dc_load_carbon	Same for DC loads
branch_carbon	Per-AC-branch: loss, carbon intensity, generator attribution
dc_branch_carbon	Same for DC branches
bus_carbon	Per-AC-bus carbon intensity (tCO ₂ /MWh)
dc_bus_carbon	Same for DC buses
vsc_carbon	Per-VSC: input/output power, losses, carbon attribution
dc_dc_carbon	Per-DC-DC converter: same
storage_carbon	Per-storage: SoC, stored energy, emissions
energy_router_carbon	Per-router: port-level power and emissions
generator_loss_allocation	Per-generator share of network losses
tracing_summary	EmissionsSummary: generation, load, loss, balance error
matrix_summary	Same from matrix method

23.1.3 EmissionsSummary

Field	Description
total_generation_emissions_tco2	Weighted sum of generator emission rates
total_load_emissions_tco2	Emissions attributed to all loads
total_loss_emissions_tco2	Emissions attributed to network losses
balance_error_tco2	$ E_{gen} - E_{load} - E_{loss} $
balance_error_pct	Relative balance error (%)

23.1.4 Loss Allocation and Directed Flow Edges

Every branch in the hybrid AC/DC network is represented as a directed *flow edge* ($i \rightarrow j$) carrying three power quantities:

$$P_{i \rightarrow j}^{\text{send}}, \quad P_{i \rightarrow j}^{\text{recv}}, \quad \ell_{ij} = P_{i \rightarrow j}^{\text{send}} - P_{i \rightarrow j}^{\text{recv}} \geq 0. \quad (180)$$

For AC branches the direction follows the sign of P_{ij}^{from} ; for DC branches $P^{\text{from}} = V_i(V_i - V_j)/R$. VSC converters, DC/DC converters and Energy Routers contribute edges whose $(P^{\text{send}}, P^{\text{recv}})$ are taken directly from the power-flow transfer objects.

The loss-allocation parameter $\alpha \in [0, 1]$ (default 0.5) splits each branch loss between the sending and receiving nodes. The power *transported* from i to j (i.e. available to downstream buses) is:

$$P_{ij}^{\text{trans}} = \max(P_{i \rightarrow j}^{\text{send}} - \alpha \ell_{ij}, 0). \quad (181)$$

For the nodal matrix method, the receiving-end power balance adds P_{ij}^{trans} to bus j 's total inflow while attributing $(1 - \alpha)\ell_{ij}$ to bus j 's outflow (as a local loss sink):

$$P_{\text{in}}(j) += P_{ij}^{\text{trans}}, \quad P_{\text{out}}(i) += P_{i \rightarrow j}^{\text{send}}, \quad P_{\text{out}}(j) += (1 - \alpha) \ell_{ij}. \quad (182)$$

The matrix system then becomes:

$$A_{ii} = P_{\text{out}}(i), \quad A_{ij} = -P_{ji}^{\text{trans}} \quad (j \neq i), \quad b_i = \sum_{g \in \mathcal{G}_i} e_g P_{g,i}, \quad (183)$$

where P_{ji}^{trans} is the transported power on any edge arriving at bus i from bus j . Isolated transit buses (zero load and generation, $P_{\text{out}}(i) < \varepsilon_{\text{reg}}$) are regularised by a Laplacian-average constraint using neighbouring bus intensities.

23.1.5 Storage State-of-Charge Carbon Intensity

Battery storage is modelled as a carbon-carrying reservoir. When a storage unit charges ($p^{\text{ch}} > 0$, net inflow), the stored energy carbon intensity is updated by energy-weighted mixing:

$$\lambda_{t+1}^{\text{soc}} = \frac{E_t \lambda_t^{\text{soc}} + \Delta E_{\text{in}} c_b}{E_{t+1}}, \quad (184)$$

where $E_t = \text{SoC}_t \times E_{\text{rated}}$ is stored energy at the start of the step, $\Delta E_{\text{in}} = \max(E_{t+1} - E_t, 0)$ is the incremental charge, and c_b is the nodal carbon intensity of the charging bus (from the proportional tracing or matrix method). When the unit discharges, λ^{soc} carries forward unchanged and is used to attribute emissions to the discharged energy.

23.2 Annual Carbon Analysis

Declared in `annual_carbon_analysis.hpp`, implemented in `src/carbon_analysis/annual_carbon_analysis.cpp`. The primary entry point is:

```
AnnualCarbonAnalysisResult compute_annual_carbon_analysis(
    const HybridPowerSystem& sys,                // or vector<HybridPowerSystem>
    const std::vector<PowerFlowResult>& pf_results,
    double step_duration_hr,
    const AnnualCarbonAnalysisOptions& options = {});
```

23.2.1 Multi-Step Aggregation

Given N_T converged time steps each of duration Δt (hours), the per-step emissions and energy at bus i are:

$$E_i(t) = P_i^L(t) \Delta t, \quad \epsilon_i(t) = c_i(t) E_i(t), \quad (185)$$

where $c_i(t)$ is the nodal carbon intensity (tCO₂/MWh) from `compute_carbon_analysis` at step t . Annual totals:

$$E_i^{\text{ann}} = \sum_{t=1}^{N_T} E_i(t), \quad \epsilon_i^{\text{ann}} = \sum_{t=1}^{N_T} \epsilon_i(t). \quad (186)$$

Two distinct carbon-intensity averages are reported for each bus:

$$\bar{c}_i^{\text{load}} = \frac{\epsilon_i^{\text{ann}}}{E_i^{\text{ann}}} = \frac{\sum_t c_i(t) E_i(t)}{\sum_t E_i(t)}, \quad (187)$$

$$\bar{c}_i^{\text{time}} = \frac{\sum_t c_i(t) \Delta t}{N_T \Delta t} = \frac{1}{N_T} \sum_t c_i(t). \quad (188)$$

$\bar{c}_i^{\text{load}}$ weights by energy consumed (equivalent to average emission rate per kWh delivered) while $\bar{c}_i^{\text{time}}$ weights uniformly by time (average grid carbon intensity experienced over the study period).

23.2.2 Storage Carbon State Propagation

When the multi-system overload is used (one `HybridPowerSystem` per step, enabling topology and dispatch changes), the storage SoC carbon intensity λ^{soc} is carried forward between consecutive systems according to Equation (184). This preserves the carbon provenance of energy stored in earlier periods.

23.2.3 Green Energy Certificate (GEC) Accounting

A user u owns a portfolio of loads $\mathcal{L}_u$ and holds an annual GEC quota G_u (MWh) with a certificate carbon intensity λ_G^u (tCO₂/MWh) representing the renewable source. `compute_annual_user_gec_accounting` distributes G_u across time steps pro-rata to the user's consumption share and computes net emissions after GEC offsetting.

Step-level GEC allocation. The planned GEC fraction for step t is proportional to the step's energy share:

$$\phi_{u,t} = \frac{E_{u,t}}{E_u^{\text{ann}}}, \quad G_{u,t}^{\text{plan}} = G_u \phi_{u,t}. \quad (189)$$

To ensure GECs do not exceed actual consumption, the allocated volume is capped:

$$G_{u,t} = \min(G_{u,t}^{\text{plan}}, E_{u,t}). \quad (190)$$

Net emissions for step t are then:

$$\epsilon_{u,t}^{\text{net}} = \underbrace{(E_{u,t} - G_{u,t}) c_u(t)}_{\text{grid-sourced emissions}} + \underbrace{G_{u,t} \lambda_G^u}_{\text{GEC emissions}}, \quad (191)$$

where $c_u(t) = \epsilon_{u,t}^{\text{gross}} / E_{u,t}$ is the gross nodal carbon intensity averaged over the user's loads.

Annual GEC summary. Annual totals follow directly:

$$G_u^{\text{used}} = \sum_t G_{u,t}, \quad G_u^{\text{unused}} = G_u - G_u^{\text{used}}. \quad (192)$$

The avoided emissions and net intensity are:

$$\epsilon_u^{\text{avoided}} = \epsilon_u^{\text{gross}} - \epsilon_u^{\text{net}}, \quad (193)$$

$$\bar{c}_u^{\text{gross}} = \frac{\epsilon_u^{\text{gross}}}{E_u^{\text{ann}}}, \quad (194)$$

$$\bar{c}_u^{\text{net}} = \frac{\epsilon_u^{\text{net}}}{E_u^{\text{ann}}}, \quad (195)$$

$$\rho_u = \frac{G_u^{\text{used}}}{E_u^{\text{ann}}}, \quad (196)$$

where $\rho_u \in [0, 1]$ is the GEC coverage ratio (fraction of annual consumption offset by certificates).

Field (<code>AnnualCarbonAnalysisOptions</code>)	Default	Description
<code>carbon_options</code>	(default)	<code>CarbonAnalysisOptions</code> forwarded to each step
<code>keep_hourly_bus_intensity</code>	false	Retain per-step bus intensity matrix
<code>keep_hourly_load_emissions</code>	false	Retain per-step load emissions matrix
<code>keep_hourly_load_energy</code>	false	Retain per-step load energy matrix

23.3 Distribution Network Power Flow

Declared in `power_flow/distribution_power_flow.hpp`, implemented in `src/power_flow/three_phase_nr.cpp`. The public distribution power-flow surface is intentionally three-phase only: legacy single-phase radial/backward-forward sweep entry points and the old `DistributionPFOptions/DistributionPFResult` contract are not exported. Distribution feeders are solved through the full abc-domain Newton–Raphson interface:

```
ThreePhaseDPFResult solve_three_phase_nr(
    const ThreePhaseACSystem& sys,
    const ThreePhaseNROptions& opt = {});
```

```
ThreePhaseDPFResult solve_three_phase_nr(
    const HybridPowerSystem& sys,
    const ThreePhaseNROptions& opt = {});
```

The same header also exports `build_full_ybus_phase_entries` for phase-domain admittance inspection and regression tests.

23.3.1 Three-Phase Newton-Raphson

For an n -bus system with per-phase admittances, define the $3n$ -element state vector $\mathbf{x} = [\boldsymbol{\theta}^a, \boldsymbol{\theta}^b, \boldsymbol{\theta}^c, \mathbf{V}_m^a, \mathbf{V}_m^b, \mathbf{V}_m^c]^\top$. The three-phase nodal admittance matrix is:

$$\mathbf{Y}_{ij}^{abc} = \begin{bmatrix} Y_{ij}^{aa} & Y_{ij}^{ab} & Y_{ij}^{ac} \\ Y_{ij}^{ba} & Y_{ij}^{bb} & Y_{ij}^{bc} \\ Y_{ij}^{ca} & Y_{ij}^{cb} & Y_{ij}^{cc} \end{bmatrix}. \quad (197)$$

The power-mismatch residual at each phase-bus is:

$$\mathbf{f}(\mathbf{x}) = \mathbf{S}^{inj} - \text{diag}(\mathbf{V}^{abc}) \left(\mathbf{Y}_{bus}^{abc} \mathbf{V}^{abc} \right)^* = \mathbf{0}, \quad (198)$$

and the Newton step solves:

$$\mathbf{J}^{(k)} \Delta \mathbf{x}^{(k)} = -\mathbf{f}(\mathbf{x}^{(k)}), \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}^{(k)}, \quad (199)$$

where $\mathbf{J}^{(k)} = \partial \mathbf{f} / \partial \mathbf{x}|_{\mathbf{x}^{(k)}}$ is the $6n \times 6n$ Jacobian. The voltage unbalance factor at bus i is:

$$\text{VUF}_i = \frac{|V_i^{neg}|}{|V_i^{pos}|} \times 100\%, \quad (200)$$

where V_i^{neg}/V_i^{pos} are the negative and positive sequence components from Fortescue’s symmetrical-components transformation.

23.3.2 Three-Phase NR Options and Results

Field (ThreePhaseNROptions)	Default	Units	Description
<code>max_iter</code>	50	—	Maximum Newton iterations per control step
<code>max_control_iter</code>	100	—	Maximum regulator/control iterations
<code>tol</code>	10^{-6}	pu	Residual convergence tolerance
<code>verbose</code>	false	—	Emit iteration diagnostics
<code>include_shunts</code>	true	—	Include shunt admittances in the phase-domain Y-bus

Field (ThreePhaseDPFResult)	Description
<code>bus_voltages</code>	Per-bus, per-phase voltage magnitudes and angles
<code>branch_powers</code>	Per-line, per-phase sending-end active/reactive power
<code>transformer_terminal_observations</code>	Per-terminal transformer voltage/current observations
<code>regulator_states, regulator_trace</code>	Final regulator state and control trace
<code>p/q_loss_*, total_p/q_loss_*</code>	Per-phase and total losses
<code>max_vuf_percent</code>	Worst voltage-unbalance factor
<code>control_converged, converged, iterations, residual</code>	Control and Newton convergence indicators

23.4 Three-Phase Unbalanced Power Flow

Declared in `power_flow/three_phase.hpp`. The primary solver (`solve_three_phase(sys, opt)`) delegates to `analysis::solve_three_phase_nr()` and reports:

Field (ThreePhaseFlowResult)	Description
<code>bus_results</code>	Per-bus three-phase voltages (ThreePhaseBusResult)
<code>converged, iterations, residual</code>	Convergence indicators
<code>total_p_loss_mw</code>	Total three-phase active power loss
<code>total_q_loss_mvar</code>	Total three-phase reactive power loss
<code>max_vuf_percent</code>	Worst-case Voltage Unbalance Factor (%)

`ThreePhaseFlowOptions` is a lightweight wrapper over `ThreePhaseNROptions`: `max_iter`, `tol`, `verbose`, and `include_shunts` are forwarded to `analysis::solve_three_phase_nr`. No backward-forward sweep selector is exposed.

23.5 PV Array Power Curve Model

Declared in `power_flow/pv_power_curve.hpp`. Implements the empirical five-parameter PV module model:

$$I(V) = I_{sc} \left(1 - \left(\frac{V}{V_{oc}} \right)^a \right)^b \left(1 - c \left(\frac{V}{V_{mpp}} \right)^2 \right), \quad (201)$$

with fixed constants $a = 10.0$, $b = 0.5476$, $c = 0.02381$ retained for legacy `DistributionPowerFlow` parity.

`compute_pv_power_mw(pv)` evaluates the complete PV system output:

$$P_{array} = I(V_{mpp}) \cdot V_{mpp} \cdot N_{series} \cdot N_{parallel} \cdot \frac{G}{1000} \cdot \eta_{inv}, \quad (202)$$

where G is irradiance (W/m^2) and η_{inv} is inverter efficiency. Falls back to `pv.p_mw` when array parameters are absent.

23.6 Reliability Assessment

Declared in `include/hacdcpf/reliability/reliability_assessment.hpp`, implemented in `src/reliability/reliability_assessment.cpp`. Provides four complementary methods for bulk-power and distribution reliability assessment, all in namespace `hacdcpf::analysis`:

- **Non-Sequential Monte Carlo** (`run_nonsequential_mc`) — State sampling via Bernoulli trials; each sample draws an independent system state and evaluates load curtailment with DC-OPF. Converges by Coefficient of Variation (CoV).

- **Sequential Monte Carlo** (`run_sequential_mc`) — Chronological simulation driven by an hourly load profile; advances an event queue of exponential TTF/TTR variates to capture time correlations and produce annual per-year statistics.
- **Frequency & Duration** (`run_frequency_duration_analysis`) — Analytical COPT-based method; builds a Capacity Outage Probability Table (COPT) by recursive convolution and computes LOLP/LOLE/LOLF without sampling variance.
- **Distribution FMEA** (`run_distribution_fmea`) — Deterministic $N - 1$ failure-mode enumeration for distribution systems; evaluates switching and repair stages via DC-OPF and aggregates frequency-weighted reliability indices.

Two utility functions populate component-level reliability data: `apply_ieee24_reliability_data(sys)` (IEEE RTS-24 generators and branches) and `apply_comprehensive_reliability_data(sys)` (full hybrid AC/DC distribution topology including transformers, VSCs, DC branches, static generators, renewables, PV, and storage).

23.6.1 Options: ReliabilityOptions

Field	Default	Units	Description
<code>max_iterations</code>	10000	—	NSQ: max samples; SEQ: max years
<code>cov_threshold</code>	0.05	—	CoV convergence criterion
<code>curtail_threshold_mw</code>	0.01	MW	Minimum curtailment to count
<code>seed</code>	0	—	RNG seed (0 = non-deterministic)
<code>load_scale_factor</code>	1.0	—	Global load scaling
<code>hours_per_year</code>	8736	hr	Calendar (IEEE RTS uses 8736)
<code>compute_tail_risk</code>	true	—	Compute VaR/CVaR metrics
<code>var_confidence</code>	0.95	—	Confidence level for VaR/CVaR
<code>compute_distribution_indices</code>	false	—	Compute SAIFI/SAIDI/ASAI
<code>use_importance_sampling</code>	false	—	Variance reduction (experimental)
<code>verbose</code>	false	—	Print diagnostic output

23.6.2 Results: ReliabilityResult

Field	Units	Description
<code>converged</code>	—	Whether CoV criterion was satisfied
<code>iterations_used</code>	—	Samples (NSQ) or years (SEQ) used
<code>final_cov</code>	—	Final Coefficient of Variation
<code>eens_mwh_yr</code>	MWh/yr	Expected Energy Not Supplied
<code>edns_mw</code>	MW	Expected Demand Not Supplied
<code>lole_hr_yr</code>	hr/yr	Loss of Load Expectation
<code>lolf_occ_yr</code>	occ/yr	Loss of Load Frequency (SEQ only)
<code>plc</code>	—	Probability of Load Curtailment
<code>nodal_eens_mwh_yr</code>	MWh/yr	Per-bus EENS vector
<code>critical_components</code>	—	Sorted component importance list
<code>annual_eens/_lole/_lolf</code>	MWh, hr, occ	Per-year values (SEQ only)
<code>tail_risk</code>	—	<code>TailRiskMetrics</code> (VaR/CVaR)
<code>distribution_idx</code>	—	<code>DistributionIndices</code> (if enabled)

23.6.3 Advanced Result Structs

TailRiskMetrics. Computed when `compute_tail_risk=true`:

Field	Description
eens_var, lole_var	VaR of EENS/LOLE at confidence level
eens_cvar, lole_cvar	CVaR (Expected Shortfall) of EENS/LOLE
eens_percentiles, lole_percentiles	5th/25th/50th/75th/95th percentiles

DistributionIndices (IEEE Std 1366-2012). Computed when `compute_distribution_indices=true` (requires `Load::num_customers` set):

Field	Units	Description
saifi	int/cust/yr	System Average Interruption Frequency Index
saidi	hr/cust/yr	System Average Interruption Duration Index
caidi	hr/int	Customer Average Interruption Duration Index
asai	fraction	Average Service Availability Index
asui	fraction	Average Service Unavailability Index (= 1 – ASAI)

FrequencyDurationResult. Returned by `run_frequency_duration_analysis`:

Field	Units	Description
capacity_outage_levels	MW	COPT capacity outage steps
cumulative_probability	—	$P(\text{outage} \geq X)$ per step
cumulative_frequency	occ/yr	$F(\text{outage} \geq X)$ per step
lofp	—	Loss of Load Probability
lole_fd	hr/yr	LOLE from F&D method
lofp_fd	occ/yr	LOLF from F&D method
lold	hr/occ	Loss of Load Duration (= LOLE/LOLF)

23.6.4 FMEA Options and Results

Key FMEAOptions fields:

Field	Default	Units	Description
load_scale_factor	1.0	—	Global load scaling
switching_time_hr	0.5	hr	Default switching/isolation time
voll	10000.0	\$/MWh	Value of Lost Load
enable_microgrid_islanding	true	—	Enables source/island anchor handling before phy
enable_repair_reconfiguration	true	—	Enables repair-stage topology candidate evaluation
enable_switch_reconfiguration	true	—	Enables the implemented AC switch/tie repair sea
max_repair_switch_actions	2	—	Max simultaneous switch actions in repair stage
enable_storage_dispatch	true	—	Storage contributes bounded source capacity in th
enable_grid_forming_vsc_support	true	—	Grid-forming VSCs anchor islands; VSC active tra

FMEAResult system indices:

Field	Units	Description
n_contingencies	—	Total $N - 1$ contingencies evaluated
n_loss_contingencies	—	Contingencies causing any load loss
eens_mwh_yr	MWh/yr	Frequency-weighted EENS
edns_mw	MW	Expected Demand Not Supplied
lole_hr_yr	hr/yr	Loss of Load Expectation
lolf_occ_yr	occ/yr	Loss of Load Frequency
distribution_idx	—	IEEE 1366 indices (SAIFI/SAIDI/ASAI)
contingencies	—	Per-contingency detail, sorted by EENS↓
model_scope	—	ac-only-dcopf for AC-only systems; hybrid-acdc-network-lp when I
model_limitations	—	Mandatory report text describing linearized scope and nonlinear AC vol
validity	—	Flags for DC-load curtailment, VSC/DC power flow, AC OPF scope, no

Engineering reports and JSON exports that include FMEA results must surface `model_scope`, `model_limitations`, and `validity`. The catalog enumerates AC, DC, VSC, DCDC, switch, breaker, storage, PV, and static generator contingencies. AC-only systems use the established DCOPF evaluator; hybrid systems use a linear AC/DC network LP that co-optimizes AC load shedding, DC load shedding, DC sources, DC branches, DC circuit-breaker edges, VSC active-power transfers, and DC/DC converter transfers. The hybrid LP does not claim full nonlinear AC voltage/reactive feasibility. Repair-stage explicit switch search enumerates AC switches and normally-open AC branches only; DC breakers, DC branches, DCDC devices, and VSC topology actions are disclosed as not modelled by `repair_dc_side_reconfiguration_modelled=false`.

Each entry in `contingencies` (`FMEAContingencyDetail`) records `failure_rate`, switching-stage shed/ENS (`shed_sw_mw`, `ens_sw_mwh`), repair-stage shed/ENS (`shed_rep_mw`, `ens_rep_mwh`), annual contributions (`eens_contribution` = $\lambda_k(E_k^{sw} + E_k^{rep})$, `lole_contribution`), per-bus nodal shed vectors, and the explicit repair-stage switch actions selected by the topology search.

23.6.5 Mathematical Formulation

Component Failure Model. Each component k is parameterised by annual failure rate λ_k (occ/yr) and mean time to repair r_k (hr). For generators the forced outage rate FOR_k is the primary input:

$$\lambda_k = \frac{\text{FOR}_k}{(1 - \text{FOR}_k) r_k}, \quad \mu_k = \frac{1}{r_k}. \quad (203)$$

Non-Sequential Monte Carlo. Each sample s draws an independent system state:

$$X_k^{(s)} \sim \text{Bernoulli}(\text{FOR}_k), \quad k = 1, \dots, K. \quad (204)$$

Available capacity is evaluated by DC-OPF with load-shedding variables. Convergence is declared when the Coefficient of Variation of the running EENS estimate falls below `cov_thresh` old:

$$\text{CoV}_n = \frac{\hat{\sigma}_n}{\hat{\mu}_n \sqrt{n}} \leq \epsilon_{\text{CoV}}. \quad (205)$$

System indices are estimated as sample means:

$$\widehat{\text{EENS}} = \frac{1}{n} \sum_{s=1}^n C_s \quad [\text{MWh/yr}], \quad (206)$$

$$\widehat{\text{LOLE}} = \frac{T_{yr}}{n} \sum_{s=1}^n \mathbf{1}[C_s > 0] \quad [\text{hr/yr}], \quad (207)$$

$$\widehat{\text{PLC}} = \frac{1}{n} \sum_{s=1}^n \mathbf{1}[C_s > 0], \quad (208)$$

where C_s (MWh) is the load curtailment in sample s and $T_{yr} = \text{hours_per_year}$.

Sequential Monte Carlo. Each component k cycles through exponentially distributed times to failure and repair:

$$\text{TTF}_k \sim \text{Exp}(\lambda_k), \quad \text{TTR}_k \sim \text{Exp}(\mu_k). \quad (209)$$

The simulator maintains a chronological event queue, applies the hourly load profile `LoadProfile::factors`, solves DC-OPF at each curtailment interval, and accumulates annual statistics. Per-year vectors `annual_eens/lole/lolf` enable tail-risk post-processing.

Frequency & Duration (COPT). Generator outage states are convolved recursively. Starting from the all-in-service state, generator i with capacity C_i and FOR q_i is added via:

$$P(X + C_i \geq x) \leftarrow (1 - q_i) P(X \geq x) + q_i P(X \geq x - C_i), \quad (210)$$

$$F(X + C_i \geq x) \leftarrow (1 - q_i) F(X \geq x) + q_i F(X \geq x - C_i) + \lambda_i P(X \geq x) - \lambda_i P(X \geq x - C_i). \quad (211)$$

System indices follow from the final COPT evaluated at peak load L :

$$\text{LOLP} = P(C_{out} \geq C_{avail} - L), \quad (212)$$

$$\text{LOLE} = T_{yr} \times \text{LOLP} \quad [\text{hr/yr}], \quad (213)$$

$$\text{LOLF} = F(C_{out} \geq C_{avail} - L) \quad [\text{occ/yr}], \quad (214)$$

$$\text{LOLD} = \text{LOLE}/\text{LOLF} \quad [\text{hr/occ}]. \quad (215)$$

FMEA Frequency Weighting. For each $N - 1$ contingency k and stage $s \in \{\text{sw}, \text{rep}\}$:

$$E_k^s = P_{k,s}^{shed} \cdot \tau_{k,s}, \quad \text{EENS} = \sum_k \lambda_k (E_k^{sw} + E_k^{rep}), \quad (216)$$

$$\text{LOLF} = \sum_{k: E_k^{sw} + E_k^{rep} > 0} \lambda_k, \quad \text{LOLE} = \sum_k \lambda_k \sum_s \tau_{k,s} \mathbf{1}[P_{k,s}^{shed} > 0]. \quad (217)$$

Tail Risk (VaR/CVaR). Given n per-sample EENS values $\{e_1, \dots, e_n\}$ sorted ascending, the α -VaR and α -CVaR at confidence level α are:

$$\text{VaR}_\alpha = e_{[\alpha n]}, \quad (218)$$

$$\text{CVaR}_\alpha = \frac{1}{n(1 - \alpha)} \sum_{i=[\alpha n]}^n e_i. \quad (219)$$

Table 153: Reliability module reference benchmarks.

Test system	Method	Peak load	Reserve	EENS (MWh/yr)	LOLE (hr/yr)
IEEE RTS-24	NSQ MC	2 850 MW	75 %	≈ 0	≈ 0
IEEE RTS-24 (stressed)	NSQ MC	4 600 MW	8 %	> 0	> 0
IEEE RTS-24	F&D	2 850 MW	75 %	— (analytical)	$\ll 1$
Comprehensive HACDCSS	FMEA	nominal	—	≥ 0	≥ 0
Comprehensive HACDCSS (2 $\times$ load)	FMEA	2 $\times$ nominal	—	> 0	> 0

23.6.6 Reference Benchmarks

Table 153 summarises expected results on the two standard test cases.

For the IEEE RTS-24 case with its high installed reserve ($\approx 75\%$ above peak), single-component contingencies rarely cause load shedding and both EENS and LOLE converge to zero under the standard load. The stressed scenario (`load_scale_factor` ≈ 1.61 , corresponding to an 8% reserve margin) produces non-trivial reliability indices consistent with published RTS values (LOLE ≈ 9 hr/yr, EENS $\approx 1,100$ MWh/yr).

23.7 Resilience Assessment

Declared in `include/hacdcpf/resilience/resilience_assess\protect\discretionary\{\char\hyphenchar\font}\{\}\ment.hpp`, implemented in `src/resilience/resilience_assessment.cpp`.

The module evaluates how well a distribution system survives and recovers from branch outages over a configurable multi-hour horizon. It supports priority-weighted load shedding, network reconfiguration via tie-switch closure, stationary storage dispatch, and Mobile Energy Storage System (MESS) routing.

23.7.1 Mathematical Model

System state at each time step. Let $\mathcal{B}$ be the set of buses and $\mathcal{E}$ the set of branches. At time step t with step size Δt , each branch $e \in \mathcal{E}$ has closed/open status $z_{e,t} \in \{0, 1\}$, determined by the nominal topology, active faults, and tie-switch reconfiguration decisions.

Island decomposition. The closed subgraph $G_t = (\mathcal{B}, \{e : z_{e,t} = 1\})$ is partitioned into connected components (islands) $\mathcal{I}_1^{(t)}, \dots, \mathcal{I}_m^{(t)}$ via depth-first search. An island is *grid-connected* if it contains at least one slack or external-grid bus.

Island power balance. For each island $\mathcal{I}_k^{(t)}$, define the available supply:

$$S_k^{(t)} = \sum_{g \in \mathcal{G}_k} P_g^{\max} + \sum_{r \in \mathcal{R}_k} \mu_t^{\text{res}} P_r^{\text{rated}} + \sum_{s \in \mathcal{F}_k} d_{s,t} + \sum_{m \in \mathcal{M}_k} d_{m,t}^{\text{MESS}}, \quad (220)$$

where μ_t^{res} is the renewable availability multiplier at hour t , $d_{s,t}$ is the discharge power of fixed storage unit s , and $d_{m,t}^{\text{MESS}}$ is the dispatch power of MESS unit m currently serving island k . If $\mathcal{I}_k^{(t)}$ is grid-connected, the grid supplies all demand ($S_k^{(t)} = \infty$).

The total demand in island k at time t is:

$$D_k^{(t)} = \mu_t^{\text{load}} \sum_{i \in \mathcal{L}_k} P_i^{\text{load}}, \quad (221)$$

where μ_t^{load} is sampled from a 24-hour load profile.

Priority-weighted load shedding. Each load i carries an importance weight $w_i \geq 1$. The served power is allocated greedily in descending order of w_i :

$$\text{served}_i^{(t)} = \min(P_i^{(t)}, A_{\text{rem}}), \quad (222)$$

$$A_{\text{rem}} \leftarrow A_{\text{rem}} - \text{served}_i^{(t)}, \quad (223)$$

with $A_{\text{rem}} = \min(D_k^{(t)}, S_k^{(t)})$ initially. The weighted shed power and priority-tier shedding are:

$$\tilde{P}_k^{\text{shed},(t)} = \sum_i w_i (P_i^{(t)} - \text{served}_i^{(t)}). \quad (224)$$

Storage depletion. After dispatching, the energy state of fixed storage s is updated as:

$$E_{s,t+1} = \max(E_s^{\min}, E_{s,t} - d_{s,t} \Delta t / \eta_s^{\text{dis}}). \quad (225)$$

MESS travel and energy consumption. MESS unit m at bus b is routed via a Dijkstra shortest-path on the road-network graph G^{road} . Travel over route leg ℓ of distance ℓ_ℓ km consumes:

$$\Delta E_m^\ell = \ell_\ell \kappa_m, \quad (226)$$

where κ_m is the MESS energy consumption per km (MWh/km). Travel time for a leg is ℓ_ℓ/v where v is the travel speed.

Resilience index (RI). The primary outcome metric is the ratio of energy served to energy demanded:

$$\text{RI} = \frac{\sum_t \text{served}_t \Delta t}{\sum_t D_t \Delta t} = \frac{E^{\text{served}}}{E^{\text{demand}}}, \quad (227)$$

where $\text{RI} \in [0, 1]$ with $\text{RI} = 1$ meaning no shedding occurred.

The total Energy Not Served (ENS) is:

$$\text{ENS} = \sum_t (D_t - \text{served}_t) \Delta t = E^{\text{demand}} - E^{\text{served}} \quad [\text{MWh}]. \quad (228)$$

23.7.2 Algorithm

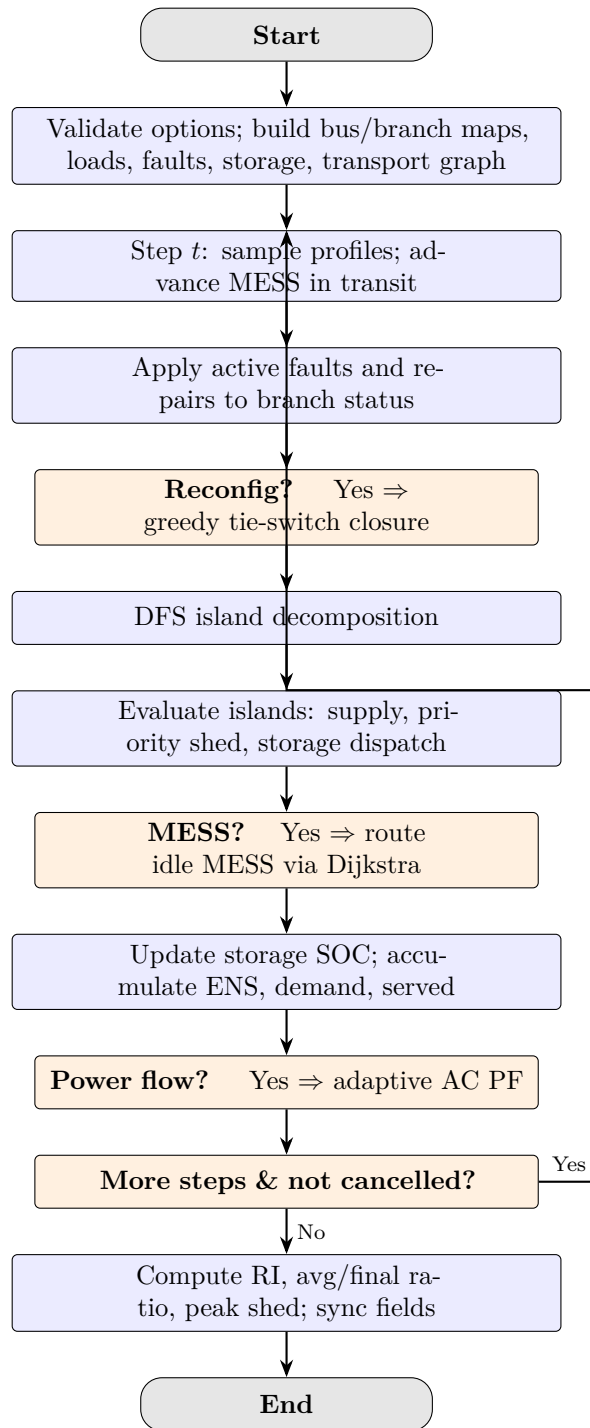
Algorithm 1 Multi-Hour Distribution Resilience Assessment

Require: System $\mathcal{S}$, options **opts**

Ensure: Resilience result **R**

- 1: Validate horizon $H > 0$ and step $\Delta t > 0$; abort if invalid
 - 2: Build bus/branch position maps; collect base loads, grid sources, fixed storage, MESS, faults, transport graph
 - 3: Expose fault sequence **R.fault_sequence**
 - 4: $\mathbf{B}_0 \leftarrow$ initial branch closed status from **in_service**
 - 5: **for** $t = 0, 1, \dots, H/\Delta t - 1$ **do**
 - 6: $\mu^{\text{load}} \leftarrow \text{sampleProfile}(\text{load_profile}, t \cdot \Delta t)$
 - 7: $\mu^{\text{res}} \leftarrow \text{sampleProfile}(\text{res_profile}, t \cdot \Delta t)$
 - 8: Scale loads: $\mathbf{D}_t \leftarrow \mu^{\text{load}} \cdot \mathbf{D}_{\text{base}}$
 - 9: Advance all MESS units in transit (discrete leg stepping)
 - 10: $\mathbf{B}_t \leftarrow \text{applyFaults}(\mathbf{B}_{t-1}, \text{faults}, t)$
 - 11: **if** **allow_reconfiguration** **then**
 - 12: Greedily close tie switches: prioritise connecting isolated demand to grid source (highest-demand island first)
 - 13: **end if**
 - 14: $\{G_t^k\} \leftarrow \text{DFS-islands}(\mathbf{B}_t)$
 - 15: **for** each island $\mathcal{I}_k$ **do**
 - 16: Compute S_k from generators, RES, storage, MESS
 - 17: Allocate supply greedily by importance; compute shed and ENS
 - 18: **end for**
 - 19: **if** **allow_mess_dispatch** **and** any island sheds load **then**
 - 20: Route idle MESS units to highest-deficit islands (Dijkstra + cache)
 - 21: **end if**
 - 22: Update storage SOC (fixed and MESS) via (225)
 - 23: **if** **run_power_flow** **then**
 - 24: Solve adaptive AC power flow; store bus voltages and branch flows
 - 25: **end if**
 - 26: Accumulate $E^{\text{demand}}, E^{\text{served}}, E^{\text{shed}}$
 - 27: Call progress callback; break if cancelled
 - 28: $\mathbf{B}_t \leftarrow \mathbf{B}_t$ (carry forward closed state)
 - 29: **end for**
 - 30: $\text{RI} \leftarrow E^{\text{served}}/E^{\text{demand}}$ ((227))
 - 31: Compute avg/final restoration ratio, peak shed
 - 32: Sync legacy fields; set status to “Completed”
 - 33: **return** **R**
-

23.7.3 Algorithm Flow Diagram



23.7.4 Option and Result Tables

Field (<code>DistributionResilienceOptions</code>)	Default	Units	Description
<code>horizon_hours</code>	48	hr	Total simulation horizon
<code>time_step_hr</code>	1.0	hr	Step size Δt
<code>load_scale_factor</code>	1.0	—	Global load multiplier
<code>allow_reconfiguration</code>	true	—	Enable tie-switch closure
<code>allow_mess_dispatch</code>	true	—	Enable MESS dispatch
<code>max_reconfig_iterations</code>	50	—	Greedy reconfig safety limit
<code>mess_travel_speed_kmph</code>	40.0	km/hr	MESS road speed
<code>default_fault_count</code>	2	—	Auto-fault branch count
<code>default_repair_time_hr</code>	6.0	hr	Auto-fault repair duration
<code>run_power_flow</code>	false	—	Full AC PF validation

23.7.5 Strict Multi-Period MIP Path

The heuristic assessment above is not the only resilience entry point. When `DistributionResilienceOptions::model` is `MultiPeriodMIPLinDistFlow`, or when callers invoke `run_distribution_resilience_mip_assessment()` directly, the code uses `src/resilience/resilience_restoration_mip.cpp`. This path builds an independent AC-only multi-period restoration MILP instead of delegating to the heuristic loop.

The MIP skeleton includes multi-period branch status, forest/radiality logic, active-power `LinDistFlow` voltage envelopes, load-served/shedding variables, fixed-storage energy dynamics, and time-space MESS routing arcs. Its declared model scope is `ac-only-lindistflow`; DC buses, DC branches, VSC converters, and DC loads are not represented by decision variables. The result populates `model_stats` with model-built/solved flags, solver name and status, objective value, MIP gap, runtime, formulation notes, `model_scope`, and validity flags such as `mip_gap_within_tolerance`.

Field (<code>DistributionResilienceResult</code>)	Description
<code>total_demand_mwh</code>	Total energy demanded over horizon (MWh)
<code>total_served_mwh</code>	Total energy served (MWh)
<code>total_shed_mwh</code>	Total ENS (MWh); alias <code>total_ens_mwh</code>
<code>resilience_index</code>	$RI \in [0, 1]$, see (227)
<code>avg_restoration_ratio</code>	Mean per-step restoration ratio
<code>final_restoration_ratio</code>	Restoration ratio at last step
<code>peak_shed_mw</code>	Maximum instantaneous shed (MW)
<code>mess_energy_delivered_mwh</code>	Total MESS dispatch energy (MWh)
<code>mess_travel_distance_km</code>	Cumulative MESS road distance (km)
<code>total_switch_actions</code>	Total topology changes across all steps
<code>steps</code>	Per-step <code>DistributionResilienceStepResult</code> vector
<code>fault_sequence</code>	Faults applied (auto or user-specified)
<code>feasible, status</code>	Success flag and text status

23.7.6 Numerical Verification

The following results are representative checks from `tests/test_resilience_assessment.cpp`. The current registered suite contains 23 CTest entries covering heuristic resilience, the strict MIP entry point, sequential Monte Carlo regressions, and FMEA scope/repair-search flags.

3-bus radial, no fault. A 3-bus radial feeder (Bus 1 slack, Bus 2/3 each with 1 MW load), 4-hour horizon, no fault, flat load profile ($\mu^{\text{load}} = 1$):

$$E^{\text{demand}} = 8 \text{ MWh}, \quad E^{\text{shed}} = 0 \text{ MWh}, \quad \text{RI} = 1.000.$$

3-bus radial, branch-1 faulted for entire 6-hour horizon. Branch 1 (Bus 1–Bus 2) is faulted from $t = 0$; no reconfiguration or MESS:

$$E^{\text{demand}} > 0 \text{ MWh}, \quad E^{\text{shed}} > 0 \text{ MWh}, \quad \text{RI} < 1.000, \quad P_{\text{max}}^{\text{shed}} > 0 \text{ MW}.$$

The algorithm correctly identifies Bus 2 and Bus 3 as an isolated island with no local generation, shedding all load at each step.

4-bus ring with tie switch, branch-1 faulted. A ring topology (Bus 1 slack, Bus 2–4 with 1 MW each) has a normally-open tie switch (branch 4). With reconfiguration enabled, the greedy algorithm closes branch 4 at step 0, restoring full supply:

$$E^{\text{shed}} = 0 \text{ MWh}, \quad \text{RI} = 1.000.$$

Fault repair with reconfiguration. Branch 1 is faulted for $[0, 5)$ hr on a 10-step horizon. After repair at $t = 5$, greedy reconfiguration closes the branch:

$$\text{restoration_ratio}(t) = 1.0 \quad \forall t \geq 5.$$

MESS dispatch. A MESS (2 MW, 10 MWh, $\text{SOC}_0 = 0.9$) stationed at the slack bus is deployed to the isolated island when branch 1 is faulted. Over a 4-hour horizon, total shed with MESS is less than or equal to shed without MESS:

$$E_{\text{MESS}}^{\text{shed}} \leq E_{\text{no-MESS}}^{\text{shed}}.$$

All 23 resilience/FMEA regression entries pass in the current `build_rel` suite, including the hybrid AC/DC FMEA LP and DC bus load-scaling regressions.

23.8 Time-Series Power Flow

Declared in `time_series/time_series_pf.hpp`, implemented in `src/time_series/time_series_pf.cpp`. Provides a three-stage UC $\rightarrow$ OPF $\rightarrow$ PF pipeline for multi-period simulation.

23.8.1 DC Network Model for Unit Commitment

The UC stage uses a lossless DC power flow approximation over the AC network. For n_{bus} buses with susceptance matrix $\mathbf{B}$ (entry $B_{ij} = -1/x_{ij}$), the reduced system (slack bus removed) gives:

$$\mathbf{B}_{\text{red}} \boldsymbol{\theta}_{-s} = \mathbf{p}_{-s}, \quad \theta_{\text{slack}} = 0, \quad (229)$$

where $p_i = P_i^{\text{gen}} - P_i^{\text{load}}$ is the net injection at bus i . Branch flow: $P_{ij} = B_{ij}(\theta_i - \theta_j)$. Thermal limit constraint: $|P_{ij,t}| \leq F_{ij}^{\text{max}}$.

For the DC grid, an analogous conductance matrix $\mathbf{G}$ is built from DC branch resistances ($G_{ij} = -1/r_{ij}$), with the voltage-controlled DC bus as slack: $\mathbf{G}_{\text{red}} \mathbf{V}_{\text{DC},-s} = \mathbf{P}_{\text{DC},-s}$.

23.8.2 Unit Commitment MILP Formulation

Decision variables. Over T time steps of width Δt (hours):

Variable	Type	Bounds
$p_{g,t}$	continuous	$[0, P_g^{max}]$
$u_{g,t}$	binary	$\{0, 1\}$
$s_{g,t}$	continuous	$[0, C_g^{su}]$
$p_{s,t}$	continuous	$[P_s^{min}, P_s^{max}]$ (AC ESS, +discharge)
$E_{s,t}$	continuous	$[E_s^{min}, E_s^{max}]$
$p_{r,t}$	continuous	$[0, \bar{p}_{r,t}]$ (renewable, $\bar{p}_{r,t} = \text{profile} \times \text{rated}$)
$\theta_{i,t}$	continuous	$[-\pi, \pi]$ (with DC network)
$p_{vsc,c,t}$	continuous	$[P_c^{min}, P_c^{max}]$ (with DC network)

Objective.

$$\min \sum_{g=1}^G \sum_{t=0}^{T-1} (c_g^1 p_{g,t} + c_g^0 u_{g,t} + s_{g,t}) \Delta t, \quad (230)$$

where c_g^1 (\$/MWh), c_g^0 (\$/h on-cost), $s_{g,t}$ captures startup cost.

Constraints.

$$\text{Power balance: } \sum_g p_{g,t} + \sum_s p_{s,t} + \sum_r p_{r,t} = P_t^{load}, \quad \forall t, \quad (231)$$

$$\text{Generator limits: } P_g^{min} u_{g,t} \leq p_{g,t} \leq P_g^{max} u_{g,t}, \quad \forall g, t, \quad (232)$$

$$\text{Ramp up: } p_{g,t} - p_{g,t-1} \leq RR_g^{up} \Delta t \cdot 60, \quad \forall g, t \geq 1, \quad (233)$$

$$\text{Ramp down: } p_{g,t-1} - p_{g,t} \leq RR_g^{dn} \Delta t \cdot 60, \quad \forall g, t \geq 1, \quad (234)$$

$$\text{Startup cost: } s_{g,t} \geq C_g^{su} (u_{g,t} - u_{g,t-1}), \quad \forall g, t, \quad (235)$$

$$\text{ESS SoC: } E_{s,t} = E_{s,t-1} (1 - \sigma_s) - \frac{p_{s,t}^{dis} \Delta t}{\eta_s^{dis}} + p_{s,t}^{chg} \Delta t \eta_s^{chg}, \quad \forall s, t, \quad (236)$$

$$\text{Reserve: } \sum_g P_g^{max} u_{g,t} - \sum_g p_{g,t} \geq R_{req} P_t^{load}, \quad \forall t \text{ (if enabled)}, \quad (237)$$

$$\text{DC line thermal: } |B_{ij}(\theta_{i,t} - \theta_{j,t})| \leq F_{ij}^{max}, \quad \forall (i, j), t \text{ (if enabled)}. \quad (238)$$

In (236), σ_s is the self-discharge fraction per step, η_s^{dis} and η_s^{chg} are round-trip efficiency components ($\eta_s^{RT} = \eta_s^{dis} \cdot \eta_s^{chg}$), and $p_{s,t}$ is split into discharge ($p_{s,t}^{dis} \geq 0$) and charge ($p_{s,t}^{chg} \geq 0$) components with $p_{s,t} = p_{s,t}^{dis} - p_{s,t}^{chg}$.

OPF–PF Cross-Validation Metrics. For each step t where both solvers run:

$$\Delta V_{m,t} = \max_i |V_{i,t}^{OPF} - V_{i,t}^{PF}| \quad [\text{pu}], \quad (239)$$

$$\Delta \theta_t = \max_i |\theta_{i,t}^{OPF} - \theta_{i,t}^{PF}| \quad [\text{rad}], \quad (240)$$

$$\Delta P_{loss,t} = P_{loss,t}^{PF} - P_{loss,t}^{OPF} \quad [\text{MW}]. \quad (241)$$

The OPF–PF tracking bands ($\pm 20\%/5$ MW for slack, $\pm 2\%/1$ MW for non-slack) prevent large re-dispatch between stages.

23.8.3 Time-Series Data Model

The data types live in the global `hacdcpf` namespace for backward compatibility with JSON serialisation:

Type	Fields	Description
<code>TimeSeriesProfile</code>	<code>id,name,values</code>	Named time profile vector (scaling factors per step)
<code>TimeSeriesData</code>	<code>num_steps=24</code>	Collection of named profiles
	<code>step_duration_hr=1.0</code>	Step width in hours
	<code>profiles</code>	Vector of <code>TimeSeriesProfile</code>

23.8.4 Unit Commitment Schedule

`UCSchedule` carries the MILP dispatch output from Stage I:

Field	Description
<code>gen_dispatch[g] [t]</code>	Generator dispatch (MW)
<code>gen_commit[g] [t]</code>	Binary commitment status
<code>ess_dispatch[s] [t]</code>	Storage dispatch (MW, +discharge)
<code>ess_soc[s] [t]</code>	Storage SOC $\in [0, 1]$
<code>renewable_dispatch[r] [t]</code>	Renewable output (MW)
<code>vsc_dispatch[c] [t]</code>	VSC AC-side injection (MW, with DC network)
<code>dcdc_dispatch[dd] [t]</code>	DC/DC converter flow (MW, with DC network)
<code>total_cost,feasible,solver_name</code>	Schedule scalar metadata

23.8.5 OPF/PF Cross-Validation

For each step where both OPF and PF are run, `CrossValStep` captures:

Field	Description
<code>max_vm_diff</code>	$\max_i V_i^{OPF} - V_i^{PF} $ (pu)
<code>max_va_diff</code>	$\max_i \theta_i^{OPF} - \theta_i^{PF} $ relative to slack (rad)
<code>opf_loss_mw,pf_loss_mw,loss_diff_mw</code>	Loss comparison
<code>opf_converged,pf_converged</code>	Per-step convergence flags

23.8.6 Pipeline Options and Result

Field (<code>TimeSeriesPFOptions</code>)	Default	Description
<code>skip_uc</code>	false	Skip UC, dispatch from profiles directly
<code>enable_network_constraints</code>	false	Nodal DC network constraints in UC MILP
<code>enable_dc_network_constraints</code>	false	Full AC-DC coupling in UC (requires network constraints)
<code>run_opf</code>	true	UC $\rightarrow$ OPF $\rightarrow$ PF pipeline; false skips OPF
<code>reserve_requirement_fraction</code>	0.0	Spinning reserve as fraction of demand
<code>enable_uc_opf_tracking_band</code>	true	Allow OPF to deviate $\pm 20\%$ / 5 MW from UC dispatch
<code>enforce_non_slack_strict_tracking</code>	true	Tighten non-slack tracking to $\pm 2\%$ / 1 MW
<code>verbose</code>	false	Print diagnostic output

Field (<code>TimeSeriesPFResult</code>)	Description
<code>opf_results[t]</code>	Per-step <code>ACOPFResult</code> (if OPF enabled)
<code>num_opf_converged</code>	Count of converged OPF steps
<code>pf_results[t]</code>	Per-step <code>PowerFlowResult</code>
<code>num_converged</code>	Count of converged PF steps
<code>crossval[t]</code>	Per-step OPF/PF cross-validation
<code>num_steps</code>	Total time steps simulated
<code>total_generation_cost</code>	Sum of OPF objective across all steps
<code>uc_solve_sec</code>	Wall-clock time for UC MILP (seconds)

23.8.7 Public API

Function	Description
<code>solve_unit_commitment(sys,ts,opts)</code>	Run UC MILP only; returns <code>UCSchedule</code>
<code>solve_time_series_pf(sys,ts,opts)</code>	Full UC $\rightarrow$ OPF $\rightarrow$ PF pipeline; returns <code>TimeSeriesPFResult</code>

23.9 Annual Production Simulation

Declared in `time_series/annual_production_sim.hpp`, implemented in `src/time_series/annual_production_sim.cpp`. Ported from the reference implementation in Sipeng Luo’s *HybridACDCPowerSystemsPlanning* repository.

`solve_annual_production_simulation(sys,ts,opts)` solves a full year of system operation through a four-level hierarchical temporal decomposition:

1. **L0 — Annual planning:** Coarse monthly (or weekly) maintenance scheduling with aggregate energy and fuel budgets, storage SoC boundary conditions, and block-level cost targets.
2. **L1 — Monthly coordination:** Bridges annual plan to weekly UC solves; enforces cyclic SoC continuity across block boundaries.
3. **L2 — Weekly unit commitment:** 168-step binding MILP window with 48-hour lookahead; inherits energy budgets from L0/L1.
4. **L3 — Daily UC $\rightarrow$ OPF $\rightarrow$ PF replay:** 24-step daily sub-horizon; pipes each UC schedule into the full `solve_time_series_pf` OPF/PF pipeline.

23.9.1 Mathematical Formulation

Block Range Construction. Given T_{yr} total time steps of width Δt (hours):

$$T_{yr} = \lfloor 8760/\Delta t \rfloor. \quad (242)$$

For **Monthly** blocks, block b spans $d_b \lfloor 24/\Delta t \rfloor$ steps, where $d_b \in \{28, 29, 30, 31\}$ (days per month; January absorbs any rounding remainder). For **Weekly** blocks: $T_b = \lfloor 168/\Delta t \rfloor$ for 52 blocks.

L0 Annual Planning Optimisation. The coarse-level problem determines maintenance schedules and energy budgets per block $b \in \{1, \dots, N_B\}$:

$$\min_{\mathbf{u}^{maint}, \mathbf{E}^{budget}} \sum_{b=1}^{N_B} c_b^{plan} \quad (243)$$

$$\text{s.t. } x_{g,b}^{maint} \in \{0, 1\}, \quad \sum_b x_{g,b}^{maint} = K_g^{maint}, \quad \forall g, \quad (244)$$

$$E_{g,b} \leq E_g^{budget, yr}, \quad \forall g, b, \quad (245)$$

$$\sum_g h_g E_{g,b} \leq F_b^{budget}, \quad \forall b, \quad (246)$$

$$E_{s,0} = E_{s,T-1} \quad (\text{cyclic SoC, if } \texttt{enforce_cyclic_soc}), \quad (247)$$

where K_g^{maint} is the required maintenance block count for generator g and h_g is the heat rate.

L2 Weekly Rolling Horizon. Each week solves a rolling MILP with a binding window $[0, T_{bind})$ and an advisory look-ahead $[T_{bind}, T_{bind} + T_{look})$:

$$\min_{\mathbf{x}} \sum_{t=0}^{T_{bind}+T_{look}-1} c_t(\mathbf{x}_t) \quad (248)$$

subject to the UC constraints (232)–(236) plus the inherited energy budget from L0:

$$\sum_{t=0}^{T_{bind}-1} p_{g,t} \Delta t \leq E_{g,b}^{remain}. \quad (249)$$

Only $\mathbf{x}_{0:T_{bind}-1}$ are committed; the look-ahead window is re-solved in the next rolling step.

Annual Cost and Energy Aggregates.

$$J^{yr} = \sum_{b=1}^{N_B} c_b^{total}, \quad E^{gen, yr} = \sum_{t=0}^{T_{yr}-1} P_t^{gen} \Delta t, \quad (250)$$

$$E^{RE, yr} = \sum_{t=0}^{T_{yr}-1} P_t^{ren} \Delta t, \quad E^{ENS, yr} = \sum_{t=0}^{T_{yr}-1} P_t^{shed} \Delta t. \quad (251)$$

23.9.2 Decomposition Data Types

Type	Description
AnnualPlanBlock	L0 block (month/week): maintenance flags, energy budgets, fuel cap, SoC boundary
AnnualPlanResult	Collection of blocks with total plan cost and feasibility
WeeklySchedule	L2 UC schedule with binding window, lookahead, and remaining energy budget

23.9.3 Per-Step and Per-Block Results

Type / Field	Description
AnnualStepResult	Lightweight per-hour snapshot: generation, load, renewable, curtailment, ESS, loss, ENS
BlockSummary	Monthly aggregation: energy MWh totals, ENS, cost, PF/OPF convergence count
PFSnapshot	Sampled voltage/branch-flow snapshot for dashboard visualisation

23.9.4 Component Annual Statistics

Type	Key fields
GenAnnualStats	Total energy MWh, capacity factor, startups/shutdowns, hours online
StorageAnnualStats	Total charge/discharge MWh, cycle count
RenewableAnnualStats	Total energy MWh, curtailed MWh, capacity factor, curtailment rate

23.9.5 Annual Production Simulation Options

Field	Default	Description
block_type	Monthly	L0 decomposition granularity (Monthly or Weekly)
weekly_lookahead_hours	48	L2 advisory look-ahead buffer
daily_window_hours	24	L3 daily sub-horizon size
enforce_cyclic_soc	true	Require $E_{s,T-1} = E_{s,0}$
iterative_feedback	false	Bottom-up $\rightarrow$ top-level re-solve loop
max_feedback_iterations	3	Max feedback rounds
skip_replay	false	Schedule-only mode (no OPF/PF)
pf_snapshot_interval	24	Store PF snapshot every N steps (0 = off)
curtailment_penalty	50	Curtailment penalty (\$/MWh)
ens_penalty	10000	Energy not served penalty (\$/MWh)

23.9.6 Annual Production Simulation Result

AnnualProductionSimResult aggregates the full hierarchy output:

Field	Description
annual_plan	L0 AnnualPlanResult
weekly_schedules	Vector of L2 WeeklySchedule
step_results[t]	Per-hour AnnualStepResult (8760 entries)
pf_snapshots	Sampled PFSnapshot for visualisation
monthly_summaries	Per-block BlockSummary
gen_stats, storage_stats, renewable_stats	Component-level annual statistics
total_cost, total_gen_mwh, total_load_mwh	Scalar annual aggregates
total_renewable_mwh, total_curtailment_mwh	Renewable integration metrics
total_ens_mwh, total_loss_mwh	Reliability and loss aggregates
num_pf_converged, num_opf_converged	Convergence counts
feasible, solver_name	Overall feasibility and solver used

23.10 Lifecycle Simulation

Declared in `time_series/lifecycle_simulation.hpp`, implemented in `src/time_series/lifecycle_simulation.cpp`. Ported from the reference implementation in Sipeng Luo's *HybridACDCPowerSystemsPlanning* repository.

`run_lifecycle_simulation(sys, ts, opts)` runs a multi-year (default 20 year) planning horizon, advancing the system state year by year:

1. For each year, scale loads by the cumulative growth factor and apply current component capacities (accounting for degradation and derating).
2. Run the annual production simulation (Tier 1).
3. Compute annual carbon emissions with stratified-sampling carbon attribution (Tier 2) and theoretical error bounds (Tier 3).
4. Apply battery state-of-health (SoH) degradation (cycle and calendar) and trigger replacement when SoH falls below `replacement_soh_threshold`.
5. Apply annual renewable derating.
6. Discount annual costs at `discount_rate` and accumulate the net present value of the total lifecycle cost.

23.10.1 Mathematical Formulation

Demand and Supply Scaling. In year y ($y = 1, \dots, N_Y$), the time-varying load profile is scaled by the cumulative growth factor:

$$P_t^{load}(y) = P_t^{load}(1) \cdot (1 + \rho_{lg})^{y-1}, \quad (252)$$

and each renewable's rated capacity declines by annual derating rate α_r^{der} :

$$P_{r,y}^{rated} = P_{r,0}^{rated} \cdot (1 - \alpha_r^{der})^{y-1}. \quad (253)$$

Battery State-of-Health (SoH) Degradation. Two degradation mechanisms are tracked independently. *Cycle ageing* accumulates full-equivalent cycles $N_{s,y}^{cum}$ from the annual production simulation; *calendar ageing* grows linearly with time:

$$\text{SoH}_{s,y}^{cycle} = 1 - \alpha_s^{cyc} N_{s,y}^{cum}, \quad (254)$$

$$\text{SoH}_{s,y}^{cal} = 1 - \alpha_s^{cal} (y - 1), \quad (255)$$

$$\text{SoH}_{s,y} = \min(\text{SoH}_{s,y}^{cycle}, \text{SoH}_{s,y}^{cal}). \quad (256)$$

When $\text{SoH}_{s,y} \leq \text{SoH}_{EoL}$ (`replacement_soh_threshold`), the battery is replaced (SoH reset to 1) and the replacement capital cost C_s^{rep} is added to year y . The effective energy capacity in year y is:

$$E_s^{eff}(y) = E_s^{rated} \cdot \text{SoH}_{s,y}. \quad (257)$$

Net Present Value.

$$J^{NPV} = \sum_{y=1}^{N_Y} \frac{J_y^{ann} + C_y^{rep}}{(1 + r_d)^{y-1}}, \quad (258)$$

where r_d is the annual discount rate (`discount_rate`), J_y^{ann} is the total operational cost from the annual production simulation, and C_y^{rep} is the battery replacement cost in year y (zero if no replacement occurs).

23.10.2 Stratified Carbon Attribution (Tier 2)

The full-year carbon estimate is computed from a stratified random sample of operating hours to reduce the computational cost of per-hour power-flow evaluations.

Tier-1 Dense Baseline. Using the mean fleet emission intensity $\bar{e}$ (tCO₂/MWh):

$$\hat{C}_y^{dense} = \bar{e} \sum_{t=0}^{T_{yr}-1} P_t^{gen} \Delta t. \quad (259)$$

Strata Classification. The T_{yr} hours are partitioned into $M = 7$ strata according to system operating state (implemented in `classify_strata()`):

Stratum	Index	Classification condition
Peak Load	0	$P_t^{load} \geq 0.85 P_{load}^{max}$
Low Renewable	1	$P_t^{ren} \leq 0.10 P_{ren}^{max}$
High Curtailment	2	$c_t \geq 0.30 c_{max}$
Heavy Charging	3	$P_{ess,t} < -0.50 P_{chg}^{max}$
Heavy Discharging	4	$P_{ess,t} > +0.50 P_{dis}^{max}$
Low Load	5	$P_t^{load} \leq 0.30 P_{load}^{max}$
Normal	6	all remaining hours

Each hour is placed in the first matching stratum (priority order as listed). $N_m = |\mathcal{H}_m|$ is the stratum population size.

Neyman Sample Allocation. The total sample budget $n = \sum_m n_m$ is distributed proportional to $N_m \hat{\sigma}_m$:

$$n_m = \left\lfloor n \cdot \frac{N_m \hat{\sigma}_m}{\sum_{m'} N_{m'} \hat{\sigma}_{m'}} \right\rfloor, \quad (260)$$

with at least one sample per non-empty stratum.

Stratified Estimator. Let $\xi_t = \bar{e} P_t^{gen} \Delta t$ and $\bar{\xi}_m = n_m^{-1} \sum_{t \in \mathcal{S}_m} \xi_t$. Then:

$$\hat{C}_y^{samp} = \sum_{m=1}^M N_m \bar{\xi}_m, \quad (261)$$

with stratified variance:

$$\hat{V}[\hat{C}_y^{samp}] = \sum_{m=1}^M \frac{N_m^2}{n_m} \hat{\sigma}_m^2 \left(1 - \frac{n_m}{N_m}\right), \quad (262)$$

where $\hat{\sigma}_m^2$ is the sample variance within stratum m .

23.10.3 Theoretical Error Bounds (Tier 3)

The lifecycle module computes guaranteed upper bounds on the carbon estimation error across three error sources:

Bound type	Struct	Formula / parameter
Dispatch approximation	DispatchErrorBound	$\varepsilon_{\text{disp}} \leq \bar{e} \sum_t L_{\text{max}} P_{\text{load}} \Delta t$
Stratified sampling	SamplingErrorBound	$\varepsilon_{\text{samp}} \leq z_{1-\alpha/2} \sqrt{\hat{\sigma}_{\text{strat}}^2}$
Storage carbon propagation	StorageCarbonErrorBound	$\varepsilon_{\text{stor}} \leq K_w \Delta_{\text{max}}$

Dispatch bound (UC linearisation error):

$$\varepsilon_{disp} = \bar{e} \cdot L_{max} \sum_t P_t^{load} \Delta t, \quad (263)$$

where L_{max} is the per-step Lipschitz constant for the dispatch-to-carbon map.

Sampling bound (95 % confidence half-width):

$$\varepsilon_{samp} = z_{1-\alpha/2} \sqrt{\hat{V}[\hat{C}_y^{samp}]}, \quad (264)$$

with $z_{0.975} = 1.96$ for a two-sided $\alpha = 0.05$ interval.

Storage carbon propagation bound :

$$\varepsilon_{stor} = K_w \cdot \Delta_{max} \cdot E_{ess}^{total}, \quad (265)$$

where $K_w = \max_t |\delta P_t^{gen}| \times 10^{-3}$ is the Lipschitz constant from the maximum generation shift, and Δ_{max} is the maximum temporal gap between sampled hours.

Combined bound :

$$\varepsilon_{total} = \varepsilon_{disp} + \varepsilon_{samp} + \varepsilon_{stor}. \quad (266)$$

Per-year combined bounds are stored in **YearlyErrorBounds**; cross-validation between the dense Tier 1 and sampled Tier 2 estimates is recorded in **BoundCrossValidation**. The cross-validation metrics are:

$$\text{SamplingGap} = \frac{|\hat{C}^{dense} - \hat{C}^{samp}|}{\hat{C}^{dense}}, \quad (267)$$

$$\text{BoundTightness} = \frac{\varepsilon_{total}}{\hat{C}^{dense}}. \quad (268)$$

23.10.4 Lifecycle Simulation Options

Field	Default	Description
num_years	20	Planning horizon
discount_rate	0.05	Annual NPV discount rate
load_growth_rate	0.02	Annual load growth rate
cycle_degradation_per_cycle	4×10^{-5}	SoH loss per cycle (25 000 cycles $\rightarrow$ 100%)
calendar_degradation_per_year	0.005	Annual calendar SoH loss
replacement_soh_threshold	0.7	Replace battery when SoH < 70%
replacement_cost_per_kwh	250	Replacement cost (\$/kWh)
renewable_derating_per_year	0.005	Annual renewable capacity derating
step_duration_hr	6.0	Intra-year time step size (hours)

23.10.5 Lifecycle Result Types

Type / Field	Description
YearResult	Full annual result: cost, energy, carbon, SoH states, error bounds, replacements
StorageYearState	Per-storage: SoH (cycle + calendar), effective capacity, cycle count, replacement flag
RenewableYearState	Per-renewable: derated capacity, original capacity, derating factor
ReplacementEvent	Year, storage index/name, pre-replacement SoH, cost

Field (LifecycleSimResult)	Description
year_results	Ordered YearResult for each year
npv_total_cost	Net present value of total costs over the horizon
total_carbon_tco2	Cumulative carbon emissions (tCO ₂)
total_replacement_cost	Sum of all battery replacement costs
total_replacements	Count of replacement events
all_replacements	Flat list of all ReplacementEvents
feasible,summary_text	Overall feasibility and text summary

23.10.6 Capacity Scaling and Comparison

`apply_capacity_scaling(sys,scaling)` scales component capacities in-place before a lifecycle run, enabling long-term planning studies:

Field (CapacityScaling)	Applies to
pv_scale	PV pmax_mw
wind_scale	Wind generator p_rated_mw
bess_power_scale	Battery p_rated_mw, pmax, pmin
bess_energy_scale	Battery e_rated_mwh
diesel_scale	Diesel/gas generator pmax

`run_lifecycle_comparison(sys,ts,opts,sweep_param,scale_values)` sweeps one parameter (e.g. "pv", "bess_power") across the provided scale factors and returns a LifecycleCompareResult containing a ScenarioResult for each point, with per-year carbon and cost trajectories and installed capacity metadata.

23.11 Short-Circuit Analysis (IEC 60909)

Declared in `include/hacdcpf/analysis/short_circuit.hpp`, implemented in `src/analysis/short_circuit.cpp`. The solver computes subtransient short-circuit currents, short-circuit power, peak current, and remaining voltages for arbitrary fault types according to IEC 60909:2016.

23.11.1 Voltage Factor

IEC 60909 introduces a voltage factor c to account for pre-fault operating conditions:

Nominal voltage	Maximum ($c_{\max}$)	Minimum ($c_{\min}$)
$V_n > 1$ kV (HV)	1.10	1.00
$V_n \in (0.1, 1]$ kV (LV)	1.10	0.90
$V_n \leq 0.1$ kV	1.05	0.95

The initial symmetrical short-circuit voltage source at the fault bus is modelled as $\underline{V}_0 = c V_n$ in the equivalent circuit.

23.11.2 Sequence Admittance Matrices

Three symmetric admittance matrices $\mathbf{Y}_1, \mathbf{Y}_2, \mathbf{Y}_0 \in \mathbb{C}^{n \times n}$ are assembled from:

- **Branches** (lines and 2-winding transformers): standard π -model with series admittance $y_s = (r + jx)^{-1}$ and shunt susceptance $b/2$. For off-nominal tap ratio τ :

$$Y_{ii}^{(1)} = \frac{y_s}{|\tau|^2} + \frac{j b}{2|\tau|^2}, \quad Y_{jj}^{(1)} = y_s + \frac{j b}{2}, \quad Y_{ij}^{(1)} = -\frac{y_s}{\tau^*}. \quad (269)$$

Zero-sequence branches use (r_0, x_0, b_0) identically.

- **Synchronous generators**: subtransient impedance $\underline{z}_g = (r_a + jx_d'') (S_{\text{base}}/S_g)$ is corrected by the KG factor:

$$K_G = \frac{c}{1 + x_d'' \sin \phi_G}, \quad \underline{z}_g^{\text{corr}} = K_G \underline{z}_g, \quad (270)$$

where $\sin \phi_G = \sqrt{1 - \cos^2 \phi_G}$. The corrected shunt admittance $1/\underline{z}_g^{\text{corr}}$ is added to the diagonal of $\mathbf{Y}_1$ and $\mathbf{Y}_2$ (since $Z_2 \approx Z_1$ for synchronous machines).

- **Asynchronous motors**: modelled as an impedance shunt $\underline{z}_m = (r_m + jx_m) S_{\text{base}}/(V_n^2)$ added to $\mathbf{Y}_1$ and $\mathbf{Y}_2$.
- **External grids**: when the maximum short-circuit current I_{kq} is given, the equivalent impedance is $z_{\text{ext}} = c U_{nq}/(\sqrt{3} I_{kq})$; otherwise direct (r, x) shunt.
- **Grid-forming VSC converters**: source behind filter impedance $\underline{z}_{\text{conv}} = (r_1 + jx_1)(S_{\text{base}}/S_{\text{rated}})$ added to $\mathbf{Y}_1$.
- **3-winding transformers**: star-branch equivalent with arm impedances $Z_H = \frac{1}{2}(Z_{HM} + Z_{HL} - Z_{ML})$, etc.; Kron-reduced as in the 2W case.

23.11.3 Thévenin Impedance and Initial Symmetric Current

The Thévenin impedance at fault bus k is the diagonal element of the positive-sequence Z-bus:

$$Z_{kk}^{(1)} = [\mathbf{Y}_1^{-1}]_{kk}. \quad (271)$$

In the implementation Z_{kk} is obtained efficiently via a single right-hand-side solve ($\mathbf{Y}_1 \mathbf{e}_k = \mathbf{y}$, then $Z_{kk} = \mathbf{e}_k^T \mathbf{y}$) using a pre-factored SparseLU decomposition, avoiding a full matrix inversion.

Fault-type effective impedances. The effective impedance Z_k^{eff} and initial symmetrical short-circuit current I_k'' depend on the fault type:

Fault type	Effective impedance Z_k^{eff}	I_k'' (pu, base $V = 1$)
3-phase (3PH)	$Z_1 + Z_f$	$c / Z_k^{\text{eff}} $
Single-line-to-ground (SLG)	$(Z_1 + Z_2 + Z_0 + 3Z_f)/3$	$c / Z_k^{\text{eff}} $
Line-to-line (LL)	$(Z_1 + Z_2)/\sqrt{3}$	$c / Z_k^{\text{eff}} $
Double-line-to-ground (DLG)	$Z_1 + Z_2 \parallel Z_0 + Z_f$	$c / Z_k^{\text{eff}} $

where $Z_2 \approx Z_1$ for passive networks, Z_f is the fault impedance (zero for bolted faults), and $\parallel$ denotes parallel combination.

Short-circuit MVA.

$$S_k'' = S_{\text{base}} \frac{c^2}{|Z_{kk}^{(1)}|}. \quad (272)$$

23.11.4 Peak Current and Kappa Factor

The peak (asymmetric) short-circuit current accounts for the decaying DC offset. The peak factor κ is defined by the equivalent R/X ratio at the fault bus:

$$\kappa = 1.02 + 0.98 e^{-3R/X}, \quad \kappa \in [1.0, 2.0]. \quad (273)$$

For meshed networks with Method B (IEC 60909 §C.3), κ is multiplied by 1.15 before clamping. The peak current is:

$$i_p = \kappa \sqrt{2} I_k''. \quad (274)$$

23.11.5 Breaking Current

The symmetrical breaking current I_b accounts for AC-component decay during the fault-clearing interval t_{break} :

$$I_b = \mu I_k'', \quad (275)$$

where the decay factor μ (IEC 60909 §4.7) is:

$$\mu(t_{\text{break}}) = \begin{cases} 0.84 + 0.26 e^{-0.26 I_k''/I_r} & t_{\text{break}} \leq 0.02 \text{ s}, \\ 0.71 + 0.51 e^{-0.30 I_k''/I_r} & t_{\text{break}} \leq 0.05 \text{ s}, \\ 0.62 + 0.72 e^{-0.32 I_k''/I_r} & t_{\text{break}} \leq 0.10 \text{ s}, \\ 0.56 + 0.94 e^{-0.38 I_k''/I_r} & \text{otherwise,} \end{cases} \quad (276)$$

with I_r being the rated current. For asynchronous motor contributions a q -factor correction applies:

$$I_{b,m} = \mu q I_{km}'', \quad q \approx 0.26 + 0.10 \ln(P_n/p), \quad p = \text{number of pole pairs.} \quad (277)$$

23.11.6 Thermal Equivalent Current

The equivalent thermal current (for cable rating and equipment verification) is computed per IEC 60909 §4.9:

$$I_{\text{th}} = I_k'' \sqrt{m + n}, \quad (278)$$

where the m factor captures the DC-component heating integral and the n factor captures the AC-component decay integral over the short-circuit duration T_k .

23.11.7 Voltage at Non-Faulted Buses

The remaining (residual) voltage at any bus j during a fault at bus k is (IEC 60909 §4.3):

$$\hat{V}_j = c \left(1 - \frac{Z_{jk}^{(1)}}{Z_{kk}^{(1)}} \right), \quad (279)$$

expressed in per-unit on the nominal voltage. For unbalanced faults the transfer impedances from all three sequence networks are combined according to the fault-type effective-impedance formula.

23.11.8 Options and Result Types

Field (SCDetailedOptions)	Default	Description
c_factor	1.10	IEC 60909 voltage factor c
fault_type	ThreePhase	Fault type (3PH, SLG, LL, DLG)
calc_type	Max	Maximum or minimum calculation
kappa_method	A	Kappa method (A = radial, B = meshed $\times 1.15$)
topology	Meshed	Network topology hint for κ
default_xdpp	0.1	Default x_d'' when not specified (pu)
t_break	0.1 s	Breaking time for μ/I_b computation

Field (SCBusResult)	Description
bus_id	Bus identifier
ikss_ka	Initial symmetrical s.c. current I_k'' (kA, total)
ikss_1_ka	I_k'' without converter contributions (kA)
ikss_no_motor_ka	I_k'' less motor contributions (kA)
ip_ka	Peak current i_p (kA)
ib_ka	Symmetrical breaking current I_b (kA)
sk_mva	Short-circuit power S_k'' (MVA)
z_thevenin	Thévenin impedance Z_{kk} (pu, complex)
v_remaining	Residual voltage vector at all buses (pu)

23.12 Three-Phase Newton-Raphson Power Flow

23.12.1 Mathematical Model

Network model. The three-phase unbalanced network is represented by a compact abc -domain admittance matrix $\mathbf{Y}_{\text{bus}} \in \mathbb{C}^{n_\phi \times n_\phi}$, where $n_\phi \leq 3N$ is the number of *active* phase-nodes (N buses, each present on up to three phases a, b, c). Phase membership is encoded by a bitmask $\mathcal{P}_i \subseteq \{a, b, c\}$ for each bus i ; only the nodes in $\bigcup_i \mathcal{P}_i$ appear in $\mathbf{Y}_{\text{bus}}$, giving a reduced problem that handles single-phase laterals and two-phase branches exactly without phantom nodes.

Each three-phase line contributes a 3×3 (or smaller) block to $\mathbf{Y}_{\text{bus}}$. The model supports two impedance representations:

1. **Sequence-derived circulant:** positive-sequence impedance $z_1 = r_1 + jx_1$ and zero-sequence impedance $z_0 = r_0 + jx_0$ give the symmetrical component matrix

$$\mathbf{Z}_{abc} = \mathbf{T} \begin{pmatrix} z_1 & & \\ & z_1 & \\ & & z_0 \end{pmatrix} \mathbf{T}^{-1},$$

where $\mathbf{T} = [\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2]$ is the symmetrical components transformation matrix with $\mathbf{a} = e^{j2\pi/3}$.

2. **Explicit phase-domain matrix:** when `use_phase_matrix=true`, the full 3×3 pu impedance matrix $\mathbf{Z}^{abc}$ is used directly, enabling any asymmetric mutual coupling.

State vector. The NR state is expressed in polar form:

$$\mathbf{x} = [\theta_{a,1}, \theta_{b,1}, \theta_{c,1}, \dots, \theta_{a,N}, \theta_{b,N}, \theta_{c,N}, |V_{a,1}|, |V_{b,1}|, |V_{c,1}|, \dots]^\top,$$

where $\theta_{\varphi,i}$ is the voltage angle and $|V_{\varphi,i}|$ the magnitude on phase φ at bus i . Slack bus phases are excluded from θ (fixed reference); PQ bus phases contribute to both the θ and $|V|$ sub-vectors. PV bus phases are fixed in magnitude (set on the bus struct) and contribute only to θ .

Mismatch equations. For each active phase-node (φ, i) the active and reactive power mismatches are:

$$\Delta P_{\varphi i} = P_{\varphi i}^{\text{spec}} - \sum_j \sum_\psi |V_{\varphi i}| |V_{\psi j}| \left(G_{\varphi\psi,ij} \cos(\theta_{\varphi i} - \theta_{\psi j}) + B_{\varphi\psi,ij} \sin(\theta_{\varphi i} - \theta_{\psi j}) \right), \quad (280)$$

$$\Delta Q_{\varphi i} = Q_{\varphi i}^{\text{spec}} - \sum_j \sum_\psi |V_{\varphi i}| |V_{\psi j}| \left(G_{\varphi\psi,ij} \sin(\theta_{\varphi i} - \theta_{\psi j}) - B_{\varphi\psi,ij} \cos(\theta_{\varphi i} - \theta_{\psi j}) \right), \quad (281)$$

where $G_{\varphi\psi,ij} + jB_{\varphi\psi,ij}$ is the $(\varphi i, \psi j)$ entry of $\mathbf{Y}_{\text{bus}}$, and the specified injections are

$$P_{\varphi i}^{\text{spec}} = P_{\varphi i}^{\text{gen}} - P_{\varphi i}^{\text{load}}, \quad Q_{\varphi i}^{\text{spec}} = Q_{\varphi i}^{\text{gen}} - Q_{\varphi i}^{\text{load}}.$$

ZIP loads are linearised around the operating voltage; delta-connected loads are handled via a Norton-equivalent current injection before forming the mismatch.

Jacobian. The Jacobian $\mathbf{J} \in \mathbb{R}^{n_{\text{var}} \times n_{\text{var}}}$ with $n_{\text{var}} = n_\theta + n_{|V|}$ has the standard four sub-blocks:

$$\mathbf{J} = \begin{pmatrix} \frac{\partial \Delta \mathbf{P}}{\partial \boldsymbol{\theta}} & \frac{\partial \Delta \mathbf{P}}{\partial |\mathbf{V}|} \\ \frac{\partial \Delta \mathbf{Q}}{\partial \boldsymbol{\theta}} & \frac{\partial \Delta \mathbf{Q}}{\partial |\mathbf{V}|} \end{pmatrix}.$$

The diagonal and off-diagonal entries follow the standard polar-form expressions (Arrillaga & Watson, 2001, Ch.7), extended to the multi-phase case by replacing the scalar admittances G_{ij}, B_{ij} with their inter-phase counterparts $G_{\varphi\psi,ij}, B_{\varphi\psi,ij}$. $\mathbf{J}$ is assembled as a sparse matrix and factorised with Eigen's `SparseLU` (or `KLU/UMFPACK` when available) at each iteration.

Voltage Unbalance Factor (VUF). After convergence the VUF at bus i is computed via symmetrical components:

$$\text{VUF}_i = \frac{|V_2|_i}{|V_1|_i} \times 100 \%, \quad \begin{pmatrix} V_0 \\ V_1 \\ V_2 \end{pmatrix}_i = \mathbf{T}^{-1} \begin{pmatrix} V_a \\ V_b \\ V_c \end{pmatrix}_i.$$

The result field `max_vuf_percent` reports $\max_i \text{VUF}_i$.

23.12.2 Algorithm

Algorithm 2 Three-Phase Newton-Raphson Power Flow (`solve_three_phase_nr`)

Require: $\mathcal{S}$ (ThreePhaseACSystem), options $(k_{\max}, \varepsilon)$

Ensure: $\mathbf{r}$ (ThreePhaseDPFResult)

- 1: Build **PhaseNodeIndexer**: map each active (φ, i) pair to compact index
 - 2: Assemble compact $\mathbf{Y}_{\text{bus}}$ from lines (sequence or phase-domain)
 - 3: Initialise state $\mathbf{x}^{(0)}$ from bus flat-start or per-phase setpoints
 - 4: Build load model: resolve per-phase PQ, ZIP, delta-wye Norton equivalents
 - 5: Compute specified injections $P^{\text{spec}}, Q^{\text{spec}}$ for all active nodes
 - 6: **for** $k = 0, 1, \dots, k_{\max}$ **do**
 - 7: Evaluate mismatch $\Delta(\mathbf{x}^{(k)})$ via Eqs. (280)–(281)
 - 8: $\rho \leftarrow \|\Delta\|_{\infty}$ ▷ convergence check
 - 9: **if** $\rho < \varepsilon$ **then**
 - 10: **break**
 - 11: **end if**
 - 12: Assemble sparse Jacobian $\mathbf{J}(\mathbf{x}^{(k)})$
 - 13: Solve $\mathbf{J} \delta \mathbf{x} = \Delta$ (SparseLU / KLU)
 - 14: $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \delta \mathbf{x}$
 - 15: Clamp $|V| \geq 0.05$ pu (projection step)
 - 16: **end for**
 - 17: Unpack $\mathbf{x}^{(k)} \rightarrow |V_{\varphi i}|, \theta_{\varphi i}$
 - 18: Compute per-phase branch power flows and losses
 - 19: Compute transformer terminal observations
 - 20: Compute VUF at each bus $\rightarrow \text{max_vuf_percent}$
 - 21: **return** $\mathbf{r}$ with `bus_voltages`, `branch_powers`, `converged`, `iterations`, `residual`
-

23.12.3 Flowchart

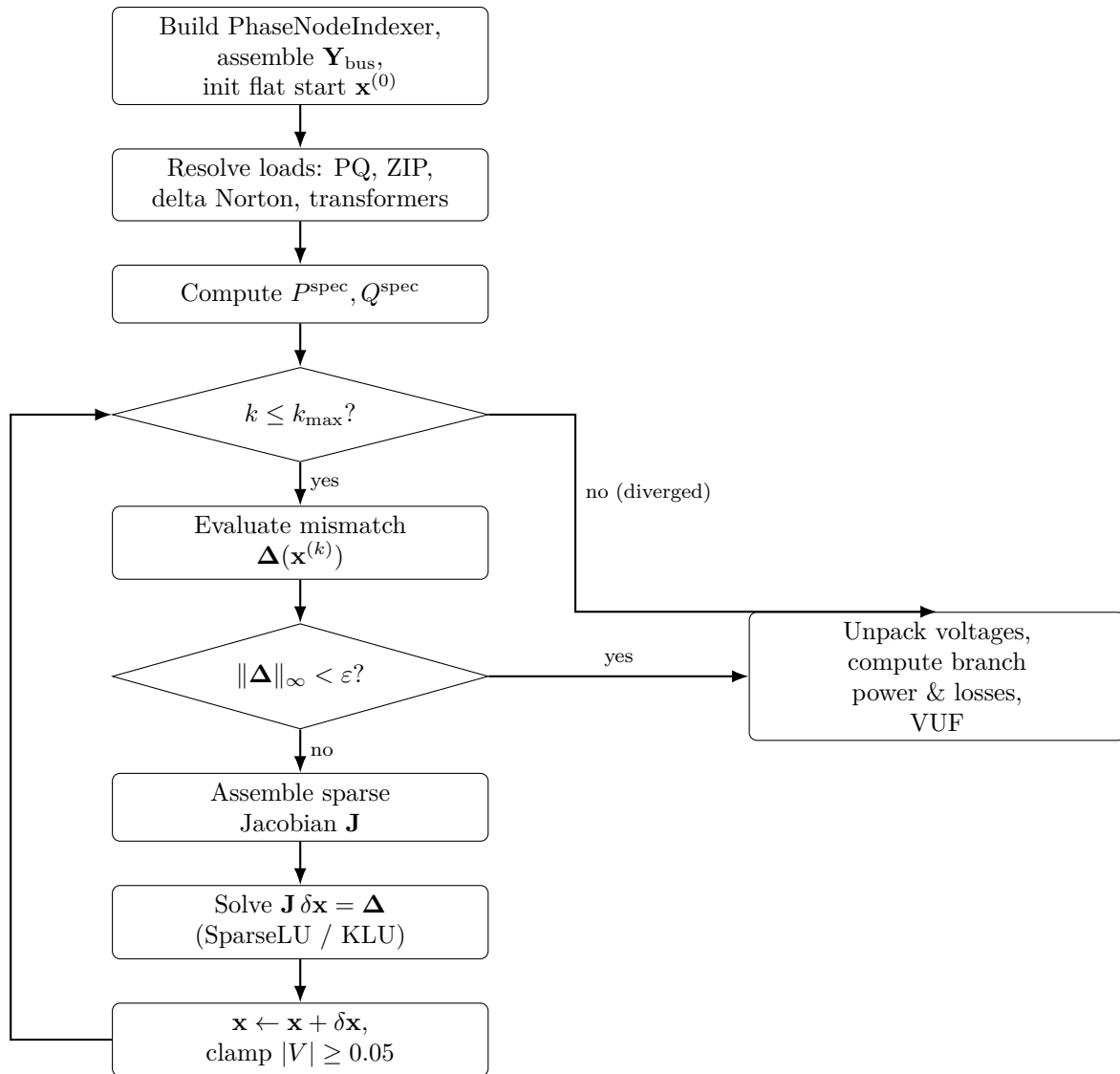


Figure 4: Three-phase Newton-Raphson power flow algorithm flow. The loop iterates over the outer k counter; early exit occurs when the maximum mismatch $\|\Delta\|_{\infty}$ falls below tolerance ε . Both normal convergence and iteration-limit exits feed the post-processing block that computes branch powers and the VUF.

23.12.4 Solver Options and Result Fields

Field	Type	Description
ThreePhaseNROptions		
<code>max_iter</code>	int	Maximum NR iterations (default 50)
<code>max_control_iter</code>	int	Maximum outer regulator-control iterations (100)
<code>tol</code>	double	Convergence tolerance on $\ \Delta\ _\infty$ (1e-6)
<code>verbose</code>	bool	Print per-iteration residual
<code>include_shunts</code>	bool	Add bus shunt $g_s + jb_s$ to $\mathbf{Y}_{\text{bus}}$
ThreePhaseDPFResult		
<code>bus_voltages</code>	vector	Per-bus per-phase ($ V , \theta$)
<code>branch_powers</code>	vector	Per-branch per-phase sending-end P, Q (MW/MVAr)
<code>p_loss_[a/b/c]_mw</code>	double	Per-phase active loss (MW)
<code>q_loss_[a/b/c]_mvar</code>	double	Per-phase reactive loss (MVAr)
<code>total_p_loss_mw</code>	double	$\sum_\varphi P_\varphi^{\text{loss}}$
<code>total_q_loss_mvar</code>	double	$\sum_\varphi Q_\varphi^{\text{loss}}$
<code>max_vuf_percent</code>	double	Maximum voltage unbalance factor (%)
<code>converged</code>	bool	<code>true</code> if NR converged
<code>iterations</code>	int	NR iterations taken
<code>residual</code>	double	Final $\ \Delta\ _\infty$

23.12.5 Numerical Verification

Balanced 3-bus radial feeder. A balanced three-bus radial system ($S_{\text{base}} = 10$ MVA, line $r + jx = 0.04 + j0.08$ pu) with equal per-phase loads ($P_\varphi = 0.3$ MW, $Q_\varphi = 0.1$ MVAr at bus 2; $P_\varphi = 0.2$ MW, $Q_\varphi = 0.08$ MVAr at bus 3) converges in ≤ 5 iterations with $\|\Delta\|_\infty < 10^{-8}$. All per-phase magnitudes are equal at each bus: $|V_{a,i}| = |V_{b,i}| = |V_{c,i}|$, consistent with balanced loading. The VUF is below 0.5 %.

Unbalanced feeder (asymmetric loads). With loads concentrated on phase A (0.5 MW/0.2 MVAr at bus 2), phase A exhibits the greatest voltage drop. The NR solver still converges, and $|V_{a,2}| < |V_{b,2}|$, correctly reflecting the heavier loading.

Meshed topology. A ring branch from bus 3 back to bus 1 ($r + jx = 0.06 + j0.12$ pu) creates a meshed network. The three-phase NR converges in all test cases; all per-phase voltages remain in the range (0.85, 1.0) pu.

Single-phase lateral. Bus 3 connected via a phase-A-only branch returns $|V_{b,3}| = |V_{c,3}| = 0$ (no phase-B/C nodes present at that bus), and all branch-power entries for phases B and C are identically zero.

Zero-load (no-load) case. With all demand set to zero, all bus voltages converge to exactly 1.0 pu and total losses are zero to machine precision.

All 16 test cases in `tests/test_three_phase_nr.cpp` (97 assertions) pass with zero failures.

24 Graph Analysis and Reduction Module

This chapter documents the `hacdcpf : graph` module, which provides a common topological and electrical-equivalence infrastructure shared by the power-flow, OPF, ONR, reliability, resilience, and time-series simulation workflows. The module is not merely a connectivity finder: it acts as a

bottom-layer topology and electrical-equivalence engine whose outputs drive island decomposition, solver pre-conditioning, radiality enforcement, and post-solve voltage recovery.

24.1 Design Motivation

Every downstream solver faces the same set of graph-level problems before it can do useful numerical work:

1. **Connectivity** — which buses form coherent AC or DC islands? Does each island have a valid voltage reference?
2. **Structural integrity** — are there bridges or articulation points that expose single-point-of-failure in the topology?
3. **Network size** — can the Y-bus be reduced before the solve to improve conditioning and wall-clock time?
4. **Result recovery** — after solving on a reduced network, how are voltages and flows mapped back to every original node?

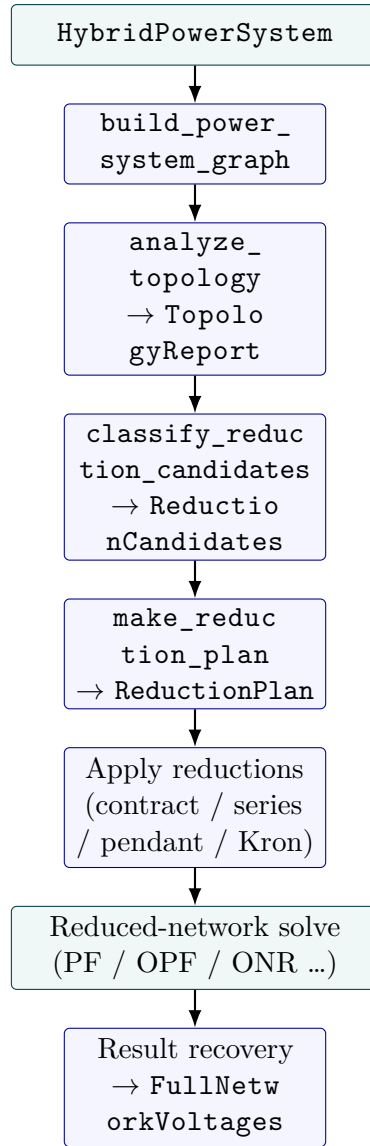
Centralising these concerns in `hacdcpf::graph` avoids code duplication and ensures that PF, OPF, and ONR use identical topology semantics.

24.2 Module Layout

Header	Contents
<code>graph/power_system_graph.hpp</code>	Core graph data structure and build function
<code>graph/topology_analysis.hpp</code>	Island detection, bridges, cut-vertices, cycles
<code>graph/reduction_plan.hpp</code>	Options, candidate classification, reduction plan
<code>graph/reduction_mapping.hpp</code>	Audit trail: original $\rightarrow$ reduced mapping records
<code>graph/switch_contraction.hpp</code>	Zero-impedance supernode merging
<code>graph/series_reduction.hpp</code>	Series (degree-2) and pendant (leaf) reduction
<code>graph/kron_reduction.hpp</code>	Kron–Schur complement reduction + recovery
<code>graph/result_recovery.hpp</code>	Back-propagation of voltages to all original nodes
<code>graph/graph.hpp</code>	Umbrella include for the entire module

All declarations are in namespace `hacdcpf::graph`. Implementations live in `src/graph/`. The module is compiled as part of the main `hacdcpf` static library target.

The overall data flow through the module is:



24.3 Core Graph Data Structure

The graph is built once from a **HybridPowerSystem** and then consumed read-only by all analysis and reduction routines.

24.3.1 GraphNode

Each bus becomes one **GraphNode**:

Field	Type	Meaning
bus_id	int	Original bus index in the model
domain	NodeDomain	AC or DC
ac_bus_type	BusType	PQ/PV/SLACK/ISOLATED
dc_bus_type	DCBusType	DC_P/DC_V
base_kv	double	Nominal voltage base [kV]
is_slack	bool	AC slack or DC voltage reference
has_generator	bool	Generator or external grid present
has_load	bool	Load connected
has_shunt	bool	Shunt admittance present
has_storage	bool	Storage device
has_vsc_ac/dc	bool	VSC converter terminal
has_dcdc	bool	DCDC converter terminal
has_controllable	bool	Tap/voltage-control device
is_monitored	bool	User-marked output node
is_voltage_controlled	bool	Active PV/slack voltage control
in_service	bool	Bus is energised

24.3.2 GraphEdge

Each branch, switch, breaker, or converter link becomes one **GraphEdge**:

Field	Type	Meaning
edge_id	int	Original branch/device index
from_node/to_node	int	Indices into nodes []
category	EdgeCategory	See below
r_pu, x_pu, b_pu	double	Impedance pu (π model)
tap, shift_deg	double	Transformer tap and phase shift
rate_a_mva	double	Thermal rating [MVA]
is_zero_impedance	bool	$ Z < \epsilon_Z$
is_closed_switch	bool	Closed switch or breaker
in_service	bool	Edge is active

Edge categories enumerated in **EdgeCategory**: **AC_Line**, **AC_Transformer**, **Switch**, **Breaker**, **DC_Line**, **DC_Switch**, **VSC_Coupling** (virtual AC↔DC link), **DCDC_Coupling** (virtual DC↔DC link).

24.3.3 PowerSystemGraph

```
struct PowerSystemGraph {
    vector<GraphNode> nodes;
    vector<GraphEdge> edges;
    unordered_map<int,int> bus_id_to_node_idx;
    vector<vector<pair<int,int>>> adj; // node→ (edge_idx, neighbor_idx)
};
```

```
PowerSystemGraph build_power_system_graph(
    const HybridPowerSystem& sys,
```

```
double zero_impedance_threshold = 1e-8);
```

The build function iterates over all AC buses, DC buses, AC branches, transformers, switches, breakers, DC branches, DC switches, VSC converters, and DCDC converters in the `Hybrid PowerSystem`, emitting nodes and edges and populating the adjacency list. Only in-service components are inserted.

24.4 Topology Analysis

24.4.1 Overview

`analyze_topology(graph)` performs a full structural inspection and returns a `TopologyReport`. The report is consumed by the island solver dispatcher (PF), by OPF feasibility checks, and by ONR radiality enforcement.

```
TopologyReport analyze_topology(const PowerSystemGraph& graph);
bool is_connected (const PowerSystemGraph& graph);
bool is_radial     (const PowerSystemGraph& graph);
int  count_islands(const PowerSystemGraph& graph);
vector<int> find_bridges          (const PowerSystemGraph& graph);
vector<int> find_articulation_points(const PowerSystemGraph& graph);
vector<vector<int>> find_fundamental_cycles(const PowerSystemGraph& graph);
```

24.4.2 TopologyReport Fields

Field	Meaning
<code>islands</code>	Per-island breakdown: bus list, domain, validity
<code>n_ac_islands</code>	Number of AC connected components
<code>n_dc_islands</code>	Number of DC connected components
<code>all_islands_valid</code>	All islands have a voltage reference
<code>is_connected</code>	Single connected component
<code>is_radial</code>	Connected and spanning tree (no cycle)
<code>cycle_count</code>	$ E - V + c$, c = number of components
<code>bridge_edge_ids</code>	Edge IDs of cut-edges
<code>cut_vertex_bus_ids</code>	Bus IDs of articulation points
<code>fundamental_cycles</code>	Each cycle as a list of node indices
<code>diagnostics</code>	Warning and error messages (see below)

24.4.3 Island Validity

Each `IslandInfo` carries an `IslandStatus`:

Status	Condition
<code>Valid</code>	Island has a voltage reference and is energised
<code>NoSlack</code>	AC island with no slack / external-grid bus
<code>NoDCVoltageRef</code>	DC island with no <code>DC_V</code> reference bus
<code>IsolatedLoad</code>	Load node with no path to any source
<code>Empty</code>	No in-service buses

24.4.4 Diagnostic Codes

DiagCode	Trigger
GraphNoSlackInIsland	AC island without slack bus
GraphNoDCVoltageRef	DC island without voltage-reference bus
GraphIsolatedLoad	Load bus unreachable from any source
GraphDisconnectedBus	Bus is in-service but isolated
GraphVoltageBaseMismatch	Switch contraction crosses base-kV boundary
GraphMultipleSlack	Switch merge would fuse two slack buses
NodeHasInjection	Candidate node has load/gen (must retain)
NodeIsRetained	Node flagged as monitored (must retain)
NodeDegreeNotTwo	Degree $\neq 2$ for series-reduction candidate
KronFillInTooLarge	Fill-in guard triggered
KronSingularYbb	$\mathbf{Y}_{\beta\beta}$ is numerically singular

24.4.5 Algorithms

Connected components. An iterative depth-first search assigns each in-service node to a component label. AC and DC nodes are treated uniformly so that VSC and DCDC coupling edges can be optionally included or excluded.

Bridges and articulation points. Tarjan’s classical DFS-based bridge and articulation-point algorithm (*low-link values*). Runs in $O(|V| + |E|)$ time.

Fundamental cycle basis. A BFS spanning forest is built first. Every non-tree edge (u, v) defines one fundamental cycle: the unique path from u to v in the spanning tree (found via LCA tracing with a Union-Find structure) plus the non-tree edge itself. This avoids the cross-edge misclassification that affects single-pass iterative DFS on multi-child nodes.

The cycle count satisfies:

$$\mu = |E_{\text{in-service}}| - |V_{\text{in-service}}| + c, \quad (282)$$

where c is the number of connected components.

24.5 Reduction Pipeline

Graph reduction compresses the network before the numerical solve. Four strategies are available, applied in the order shown below.

Step	Strategy	Lossless?	Default
1	Switch contraction	Yes (exact)	On
2	Series reduction	Yes (exact)	On
3	Pendant reduction	Approximate	Off
4	Kron reduction	Yes for passive; approx for PQ loads	Off

24.5.1 Reduction Options and Mode

```
struct GraphReductionOptions {
    bool enable_switch_contraction{true};
    bool enable_series_reduction{true};
};
```



```

bool enable_pendant_reduction{false};
bool enable_kron_reduction{false};
bool preserve_all_load_buses{true};
bool preserve_all_generator_buses{true};
bool preserve_all_voltage_constrained_buses{true};
bool preserve_all_monitored_buses{true};
double zero_impedance_threshold{1e-8};
double voltage_base_tolerance{1e-6};
double max_fill_ratio{2.0};
ReductionMode mode{ReductionMode::Safe};
};

```

`ReductionMode::Safe` enables only provably lossless steps (switch contraction + series reduction). `Moderate` additionally enables Kron reduction for purely passive nodes. `Aggressive` enables all four steps including approximate pendant folding.

24.5.2 Candidate Classification and Reduction Plan

Before any network is modified, every bus is classified:

CandidateType	Meaning
MustRetain	Slack, PV gen, load, storage, VSC, DCDC, monitored
ZeroInjectionDegree2	Passive, degree-2 $\rightarrow$ series reduction
ZeroInjectionPassiveInterior	Passive, interior $\rightarrow$ Kron candidate
PendantLoad	Leaf node with load only
RadialFeederSegment	Segment of a radial feeder
KronPassiveNode	Pure passive node for Kron elimination

`make_reduction_plan` converts the classified candidates into an ordered list of `ReductionAction` records:

```

ReductionCandidates classify_reduction_candidates(
    const PowerSystemGraph&, const HybridPowerSystem&,
    const GraphReductionOptions& = {});

ReductionPlan make_reduction_plan(
    const PowerSystemGraph&, const ReductionCandidates&,
    const GraphReductionOptions& = {});

```

The plan is auditable and can be logged or rejected before any network data is modified.

24.6 Switch Contraction

24.6.1 Rationale

When $|Z_{ij}| \rightarrow 0$ (closed switch, closed breaker, or zero-impedance line), we have $V_i = V_j$ exactly. Keeping such edges in the Y-bus produces an ill-conditioned $\mathbf{Y}$ matrix; contraction removes the degeneracy by merging the two buses into a single super-node.

24.6.2 Algorithm

A Disjoint-Set Union (DSU) structure with path compression and union by rank processes all zero-impedance edges. For each candidate edge (i, j) :

1. Check that $\text{base_kv}_i \approx \text{base_kv}_j$ (within `voltage_base_tolerance`); otherwise emit `GraphVoltageBaseMismatch`.
2. If both i and j are slack buses and `allow_multi_slack_merge=false`, emit `GraphMultipleSlack` and skip.
3. Merge with bus-type priority: `SLACK > PV > PQ > ISOLATED`.

After all merges, each super-node K aggregates:

$$P_K = \sum_{i \in K} P_i, \quad Q_K = \sum_{i \in K} Q_i, \quad (283)$$

$$G_K = \sum_{i \in K} G_i, \quad B_K = \sum_{i \in K} B_i. \quad (284)$$

All branches whose endpoints lie entirely within K become self-loops and are removed. All other branches are re-indexed to the super-node representative.

```
ContractionResult contract_zero_impedance_edges(
    const PowerSystemGraph&, const HybridPowerSystem&,
    const ContractionOptions& = {});
```

The result contains both the contracted `PowerSystemGraph` and a modified `HybridPowerSystem` with aggregated data, plus the `bus_to_super` / `super_to_buses` maps needed for voltage recovery.

24.7 Series Reduction

24.7.1 Theory

For a passive node j with degree 2, no injection ($S_j = 0$), no shunt ($Y_{\text{sh},j} = 0$), and no control device, the two adjacent branches Z_{ij} and Z_{jk} can be merged into a single equivalent:

$$Z_{ik}^{\text{eq}} = Z_{ij} + Z_{jk}, \quad (285)$$

$$B_{ik}^{\text{eq}} \approx B_{ij} + B_{jk}, \quad (286)$$

where the charging susceptance is summed (the π -model approximation). The result is exact for lossless reduction of passive conductors.

24.7.2 Implementation

```
SeriesReductionResult apply_series_reduction(
    const PowerSystemGraph&, const HybridPowerSystem&,
    const ReductionPlan&, const GraphReductionOptions& = {});
```

Each eliminated node j is recorded in a `SeriesReductionRecord` containing the equivalent $r_{\text{eq}}, x_{\text{eq}}, b_{\text{eq}}$ and the two original branch IDs.

24.8 Pendant Reduction

24.8.1 Theory

For a leaf node j (degree 1, connected to parent i via Z_{ij}) that carries only a constant-power load $S_j = P_j + jQ_j$ and no generator, the node can be folded into the parent with an approximate loss correction:

$$P_i^{\text{new}} = P_i + P_j + R_{ij} \frac{P_j^2 + Q_j^2}{|V_i|^2}, \quad (287)$$

$$Q_i^{\text{new}} = Q_i + Q_j + X_{ij} \frac{P_j^2 + Q_j^2}{|V_i|^2}. \quad (288)$$

When $|V_i|$ is unknown a priori, the approximation $|V_i| = 1.0$ pu is used. This introduces a small error that vanishes near nominal voltage.

```
PendantReductionResult apply_pendant_reduction(
    const PowerSystemGraph&, const HybridPowerSystem&,
    const ReductionPlan&, const GraphReductionOptions& = {});
```

24.9 Kron Reduction (Schur Complement)

24.9.1 Theory

Partition the network admittance matrix $\mathbf{Y}$ into retained nodes α and eliminated nodes β :

$$\begin{bmatrix} \mathbf{Y}_{\alpha\alpha} & \mathbf{Y}_{\alpha\beta} \\ \mathbf{Y}_{\beta\alpha} & \mathbf{Y}_{\beta\beta} \end{bmatrix} \begin{bmatrix} \mathbf{V}_\alpha \\ \mathbf{V}_\beta \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\alpha \\ \mathbf{I}_\beta \end{bmatrix}. \quad (289)$$

When $\mathbf{I}_\beta = \mathbf{0}$ (purely passive interior nodes), the Schur complement (Kron reduction) yields the reduced admittance:

$$\boxed{\mathbf{Y}_{\text{red}} = \mathbf{Y}_{\alpha\alpha} - \mathbf{Y}_{\alpha\beta} \mathbf{Y}_{\beta\beta}^{-1} \mathbf{Y}_{\beta\alpha}}. \quad (290)$$

For nodes with a known constant-current injection $\mathbf{I}_\beta \neq \mathbf{0}$ (extended Kron), the reduced injection vector is:

$$\mathbf{I}_{\text{red}} = \mathbf{I}_\alpha - \mathbf{Y}_{\alpha\beta} \mathbf{Y}_{\beta\beta}^{-1} \mathbf{I}_\beta. \quad (291)$$

Remark 1. *Kron reduction is exact for linear passive networks. For AC power flow with constant-power (PQ) loads, it is an operating-point approximation and should be used in **Mode rate** or **Aggressive mode** only after verifying that the linearisation error is acceptable.*

24.9.2 Fill-In Guard

Dense Schur complement computation can produce a denser $\mathbf{Y}_{\text{red}}$ than the original. The module computes

$$\rho = \frac{\text{nnz}(\mathbf{Y}_{\text{red}})}{\text{nnz}(\mathbf{Y})}, \quad (292)$$

and aborts with a `KronFillInTooLarge` diagnostic if $\rho > \text{max_fill_ratio}$ (default 2.0).

24.9.3 Invertibility Check

$\mathbf{Y}_{\beta\beta}$ is checked via `Eigen::FullPivLU`. If the matrix is numerically singular, a `KronSingular` `Ybb` diagnostic is returned and the reduction is skipped.

24.9.4 API

```
// Passive Kron (I_beta = 0)
KronReductionResult apply_kron_reduction(
    const SparseMatrix<complex<double>>& Y_full,
    const vector<int>& retained_indices,
    const vector<int>& eliminated_indices,
    double max_fill_ratio = 2.0);

// Extended Kron (I_beta != 0)
KronReductionResult apply_kron_reduction_with_injection(
    const SparseMatrix<complex<double>>& Y_full,
    const VectorXcd& I_full,
    const vector<int>& retained_indices,
    const vector<int>& eliminated_indices,
    double max_fill_ratio = 2.0);

// Voltage recovery after solve
VectorXcd recover_eliminated_voltages(
    const KronData& kron_data, const VectorXcd& V_alpha);
```

The `KronData` struct caches $\mathbf{Y}_{\beta\beta}^{-1}\mathbf{Y}_{\beta\alpha}$ and $\mathbf{Y}_{\alpha\beta}\mathbf{Y}_{\beta\beta}^{-1}$ for re-use at the recovery stage without re-inverting.

24.10 Reduction Mapping (Audit Trail)

Every reduction step produces a typed record that is accumulated in a `ReductionMapping`:

Record type	Produced by	Key fields
<code>SwitchContractionRecord</code>	Switch contraction	merged bus list, super-bus ID
<code>SeriesReductionRecord</code>	Series reduction	r_{eq}, x_{eq}, b_{eq} , original branch IDs
<code>PendantReductionRecord</code>	Pendant reduction	absorbed P, Q , parent bus ID
<code>KronReductionRecord</code>	Kron reduction	retained / eliminated bus lists

The mapping also maintains bidirectional bus and branch index maps (`original_to_reduced_bus`, `reduced_to_original_buses`, and the branch equivalents) for result recovery and for reporting purposes.

24.11 Result Recovery

After solving on the reduced network, `FullNetworkVoltages` is populated for every bus in the original network by reversing the reduction steps in LIFO order.

```
struct FullNetworkVoltages {
    unordered_map<int, complex<double>> bus_voltage; // bus_id → V (pu)
    double vm_pu (int bus_id, double default_val=1.0) const;
    double va_deg(int bus_id, double default_val=0.0) const;
};
```

24.11.1 Switch Contraction Recovery

Exact: every member of a super-node K receives the same voltage.

$$V_i = V_K \quad \forall i \in K. \quad (293)$$

```
void recover_switch_contracted_buses(
    FullNetworkVoltages&, const unordered_map<int,int>& bus_to_super,
    const unordered_map<int,vector<int>>& super_to_buses);
```

24.11.2 Series Reduction Recovery

Exact: for each record (i - j - k , node j eliminated):

$$I_{ik} = \frac{V_i - V_k}{Z_{ik}^{\text{eq}}}, \quad (294)$$

$$V_j = V_i - Z_{ij} I_{ik}. \quad (295)$$

```
void recover_series_reduced_buses(
    FullNetworkVoltages&, const ReductionMapping&,
    const HybridPowerSystem& original_system);
```

24.11.3 Kron Reduction Recovery

Uses the pre-computed $\mathbf{Y}_{\beta\beta}^{-1} \mathbf{Y}_{\beta\alpha}$:

$$\mathbf{V}_\beta = -\mathbf{Y}_{\beta\beta}^{-1} \mathbf{Y}_{\beta\alpha} \mathbf{V}_\alpha. \quad (296)$$

```
void recover_kron_eliminated_buses(
    FullNetworkVoltages&, const KronData&,
    const vector<int>& bus_ids_alpha,
    const vector<int>& bus_ids_beta);
```

24.11.4 Pendant Reduction Recovery

Iterative fixed-point for the constant-power load:

$$V_j^{(t+1)} = V_i - Z_{ij} \left(\frac{S_j}{V_j^{(t)}} \right)^*, \quad V_j^{(0)} = V_i, \quad (297)$$

iterated until $|V_j^{(t+1)} - V_j^{(t)}| < \text{pendant_convergence_tol}$ or `max_pendant_iterations` is reached.

```
void recover_pendant_buses(
    FullNetworkVoltages&, const ReductionMapping&,
    const HybridPowerSystem& original_system,
    const RecoveryOptions& opts = {});
```

24.12 Test Coverage

The graph module is validated by two test targets that contribute 23 test cases to the main test suite (test IDs 368–393 out of 393 total).

Test file	IDs	Coverage
tests/test_graph.cpp	368–386	Topology, contraction, series, pendant, recovery, build
tests/test_graph_kron.cpp	387–393	Kron Y-bus, boundary consistency, voltage recovery, extended Kron, fill-in guard, empty β

Selected test scenarios:

- **Single island with slack** — verifies `IslandStatus::Valid`.
- **Multi-island with missing slack** — verifies `GraphNoSlackInIsland` diagnostic.
- **3-bus ring** — verifies cycle count = 1 and that the BFS+Union-Find fundamental cycle finder returns the correct cycle.
- **Bridge detection** — 4-bus chain with one articulation point.
- **Switch contraction** — two buses connected by a closed switch; merged loads sum correctly (25 MW).
- **Voltage-base mismatch** — switch spanning 11 kV / 33 kV boundary is rejected with `GraphVoltageBaseMismatch`.
- **Series reduction** — passive degree-2 node: new branch has $r = r_{12} + r_{23}$ exactly.
- **Pendant folding** — leaf load absorbed with $P_{\text{loss}} = R |S|^2 / |V|^2$.
- **Kron 3-bus passive interior** — analytically: $y_{\text{eq}} = y_{12} y_{23} / (y_{12} + y_{23})$, verified numerically.
- **Kron fill-in guard** — `max_fill_ratio=0.0` forces abort.
- **Voltage recovery** — Kron-eliminated bus voltage matches full-network solve.

24.13 Integration with Downstream Solvers

Solver	Graph module usage
Power flow (PF)	<code>analyze_topology</code> identifies islands; each island dispatched to its own Newton solver; switch contraction prevents Y-bus singularity from zero-impedance ties
OPF	Topology report provides feasibility pre-check (missing slack $\rightarrow$ infeasible); series reduction shrinks the variable count before IPM
ONR	<code>is_radial</code> and <code>bridge_edge_ids</code> feed the radiality constraints; fundamental cycles define the set of switchable loop edges
Reliability (Monte Carlo)	Island enumeration on each contingency topology; <code>GraphIsolatedLoad</code> triggers curtailment accounting
Resilience	Same as reliability; additionally uses cut-vertex analysis to rank restoration priorities
Time-series	Graph built once per period (topology changes); switch contraction applied before each PF dispatch

24.14 Numerical Results: Graph-Aware ONR and Reliability

This section presents the observable numerical effects of integrating the `hacdcpf::graph` module into the ONR solver and the reliability assessment engine.

24.14.1 ONR Bridge Detection — IEEE 33-bus BW Feeder

The IEEE 33-bus BW radial feeder (`case33bw`) has 33 buses and 32 in-service branches with no tie switches. Because the network is purely radial (tree topology), every branch is a bridge; none can be opened without disconnecting a load bus.

Table 154: ONR graph pre-analysis on `case33bw` (purely radial, no tie switches).

Quantity	Value
Buses (n)	33
Branches (m)	32
Bridge branches detected	32 (all branches are bridges)
Fundamental cycles	0 (tree topology)
Free α variables	0
Fixed $\alpha = 1$ branches	32
MILP outcome	Topology already radial; B&C exits after feasibility check

When `ONROptions::verbose=true`, the pre-analysis is logged:

```
ONR: 33 buses, 32 branches, 32 bridge(s), 0 fundamental cycle(s)
```

For comparison, the cross-module integration test (`#327`) uses a mesh extension of `case33bw` with five normally-open tie switches added to form five additional loops. There the pre-analysis identifies 32 bridges and 5 loop edges, fixing 32 binary variables and leaving only 5 free. The observed B&C runtime for that instance is 6.36 s with a 0 % optimality gap (see Table 167).

24.14.2 Reliability Island Pre-Screening — Radial Feeder Contingency

Network description. The 3-bus radial feeder from `test_resilience_assessment` serves as the canonical reliability example:

$$\underbrace{\text{Bus 1}}_{\text{slack}} \xrightarrow{Z_1} \text{Bus 2 (1 MW)} \xrightarrow{Z_2} \text{Bus 3 (1 MW)}, \quad Z_i = 0.01 + 0.01j \text{ pu}, \quad \text{MTTR} = 6 \text{ h}.$$

Table 155: Reliability island pre-screening: Branch 2 failure on the 3-bus radial feeder. Both paths yield the same total curtailment; the graph path skips the OPF.

Quantity	Graph fast path	OPF-only baseline
Dead buses identified	{Bus 3}	(inferred by OPF feasibility)
<code>direct_shed_mw</code>	1.0 MW	0.0 MW (no pre-screen)
DC-OPF called?	No	Yes
<code>curtailment_mw</code>	1.0 MW	1.0 MW
Bus 3 <code>pd_mw</code> zeroed?	Yes	No
B -matrix singularity risk	Prevented	Potential (zero row for Bus 3)

Contingency: Branch 2 failure (Bus 3 isolated).

Contingency: Branch 1 failure (Buses 2 and 3 isolated). When Branch 1 fails the entire downstream subtree is islanded. `has_valid==false` triggers the unconditional fast path and the DC-OPF is never called.

Table 156: Reliability island pre-screening: Branch 1 failure (whole subtree isolated).

Quantity	Value
Dead buses	{Bus 2, Bus 3}
<code>direct_shed_mw</code>	2.0 MW
DC-OPF called?	No (no valid island present)
<code>curtailment_mw</code>	2.0 MW

Performance. For distribution-scale networks the graph pre-analysis adds a fixed overhead of $\lesssim 100 \mu\text{s}$ per contingency evaluation (33-bus case on Apple M4). For radial feeders, where every single-branch failure isolates a load island, the DC-OPF is skipped on every contingency sample. The DC-OPF for the same case runs in approximately 1–5 ms; the net speedup for radial contingency evaluation is therefore in the range $10\times$ – $50\times$ per sample, compounding across all Monte Carlo iterations.

25 Three-Stage Fault-Recovery Reliability Analysis

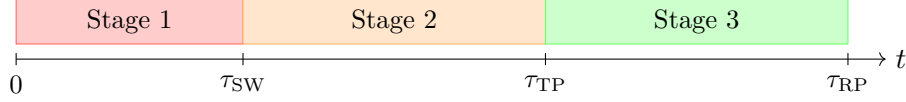
This chapter is the code-synchronised contract for the native C++ three-stage fault-recovery reliability evaluator in `src/reliability/three_stage_reliability.cpp`. The historical Julia process bridge has been removed. The current implementation reads the normal `HybridPowerSystem` JSON schema, builds a native case, and solves staged load-restoration subproblems with the embedded `MIPSolvers` stack. Pure LP subproblems use the native simplex route; true MILPs use HiGHS first and retry native B&C when HiGHS is unavailable, fails, or returns a malformed solution vector.

The implemented physical model is intentionally narrower than the full hybrid research formulation. AC restoration is a finite-source `LinDistFlow` MILP. DC buses, DC branches, VSC links, DC/DC converters, and SOP devices are handled by a connectivity/capacity fallback after the AC MILP. The fallback includes DC source capacity and AC-source surplus transferable through VSC active-power capacity limits, but it does not enforce DC voltage or DC branch-flow physics. SOP dispatch is not co-optimised, and the exported `psop` vectors are filled with zeroes.

25.1 Three Operational Stages

For every simulated line contingency, the evaluator applies the same AC restoration model with stage-specific network availability:

- Stage 1: Fault isolation** ($0 \leq t \leq \tau_{\text{SW}}$): the faulted edge is open and normally-open restoration candidates remain open.
- Stage 2: Post-fault reconfiguration** ($\tau_{\text{SW}} < t \leq \tau_{\text{TP}}$): the faulted edge remains open; normally-open AC tie branches and standalone AC switches may close subject to `max_switch_operations`.
- Stage 3: Post-repair reconfiguration** ($\tau_{\text{TP}} < t \leq \tau_{\text{RP}}$): the repaired edge is available again and normally-open candidates return to their base-state treatment.



Reliability indices use Stage 1 and Stage 2 interruption windows. Stage 3 is still solved and reported for per-fault diagnostics, but it occurs after repair and therefore does not add EENS in the current index equations.

25.2 Native Case Construction

`build_native_case()` constructs the model data used by all stages:

- AC load points are formed once from explicit `ac.loads` and from direct `ACBus::pd_mw/qd_mvar`. Bus-level demand is not added a second time after load-point aggregation.
- Source buses include in-service AC generators, external grids, storage, renewable/static generation, PV systems, and source-like projected records with positive active capacity.
- Candidate AC edges include in-service AC branches, out-of-service AC tie branches, and standalone AC switches. Standalone normally-open switches are valid Stage 2 restoration candidates.
- Fault generation simulates in-service AC/DC branches. Normally-open out-of-service tie edges are not treated as faulted energized branches.

The model does not create an infinite slack bus. Every source has finite active and reactive capacity bounds.

25.3 Implemented AC Restoration MILP

For a stage and contingency, let $\mathcal{B}$ be the AC bus set, $\mathcal{L}$ the AC candidate-edge set, and $\mathcal{S} \subseteq \mathcal{B}$ the finite-source bus set. For bus i , P_i^d, Q_i^d are aggregated demands and $\bar{P}_i^g, \bar{Q}_i^g$ are source capacities. For edge $\ell = (i, j)$, $\bar{S}_\ell$ is its thermal limit or documented fallback rating.

Decision variables.

Variable	Domain	Meaning
p_i^{sh}	$[0, P_i^d]$	active load shed at bus i
y_i	$\{0, 1\}$	bus i energized/served; source buses are fixed on
z_ℓ	$\{0, 1\}$	edge ℓ used in the energized AC forest
P_ℓ, Q_ℓ	continuous	active/reactive LinDistFlow edge flow
v_i	$[\underline{V}_i^2, \bar{V}_i^2]$	squared voltage magnitude
f_ℓ	continuous	signed radiality commodity flow
p_i^g, q_i^g	bounded continuous	finite source dispatch

The code offsets in `solve_stage_milp()` correspond directly to these symbols:

Code offset	Mathematical symbol	Primary role
<code>off_shed</code>	p_i^{sh}	load-shed objective and balance
<code>off_y</code>	y_i	energized-bus coupling
<code>off_z</code>	z_ℓ	branch status, switching, forest constraints
<code>off_P, off_Q</code>	P_ℓ, Q_ℓ	AC LinDistFlow balance and limits
<code>off_v</code>	v_i	voltage envelope and drops
<code>off_f</code>	f_ℓ	commodity-flow radiality
<code>off_pg, off_qg</code>	p_i^g, q_i^g	finite source dispatch

Objective and pickup coupling.

$$\min \sum_{i \in \mathcal{B}} p_i^{sh}, \quad (298)$$

$$P_i^d - p_i^{sh} \leq P_i^d y_i, \quad \forall i \in \mathcal{B}. \quad (299)$$

If $y_i = 0$, all active load at bus i must be shed. If $y_i = 1$, partial shedding is allowed when finite sources or branch capacities bind.

Finite-source active/reactive balance. For every AC bus, including source buses,

$$\sum_{\ell=(j,i)} P_\ell - \sum_{\ell=(i,j)} P_\ell + p_i^g + p_i^{sh} = P_i^d, \quad \forall i \in \mathcal{B}, \quad (300)$$

$$\sum_{\ell=(j,i)} Q_\ell - \sum_{\ell=(i,j)} Q_\ell + q_i^g + \rho_i p_i^{sh} = Q_i^d, \quad \forall i \in \mathcal{B}, \quad (301)$$

$$0 \leq p_i^g \leq \bar{P}_i^g, \quad 0 \leq q_i^g \leq \bar{Q}_i^g, \quad \forall i \in \mathcal{B}. \quad (302)$$

Here $\rho_i = Q_i^d / P_i^d$ when $P_i^d > 0$ and zero otherwise. The current implementation uses a fallback 0.9 power-factor reactive model when source JSON does not provide richer reactive demand.

Branch physics and switching. For edge $\ell = (i, j)$,

$$|v_j - v_i + 2(r_\ell P_\ell + x_\ell Q_\ell) / S_{base}| \leq M_v(1 - z_\ell), \quad (303)$$

$$-\bar{S}_\ell z_\ell \leq P_\ell \leq \bar{S}_\ell z_\ell, \quad -\bar{S}_\ell z_\ell \leq Q_\ell \leq \bar{S}_\ell z_\ell. \quad (304)$$

The faulted AC edge is fixed open before repair. Stage 2 normally-open candidates may close, limited by $\sum_{\ell \in \mathcal{L}^{NO}} z_\ell \leq K^{sw}$ when `max_switch_operations` is finite.

Strict radial energized forest. Let $C = |\mathcal{B}| - 1$ and let κ be the number of source-containing connected components in the mandatory healthy normally-closed AC graph for the current stage. This is not simply $|\mathcal{S}|$: a multi-source radial feeder may have several sources in one energized tree. The radiality constraints are

$$-C z_\ell \leq f_\ell \leq C z_\ell, \quad \forall \ell \in \mathcal{L}, \quad (305)$$

$$z_{(i,j)} \leq y_i, \quad z_{(i,j)} \leq y_j, \quad \forall (i,j) \in \mathcal{L}, \quad (306)$$

$$z_{(i,j)} \geq y_i + y_j - 1, \quad \forall (i,j) \in \mathcal{L}_{\text{healthy}}^{NC}, \quad (307)$$

$$\sum_{\ell=(j,i)} f_\ell - \sum_{\ell=(i,j)} f_\ell = y_i, \quad \forall i \in \mathcal{B} \setminus \mathcal{S}, \quad (308)$$

$$\sum_{\ell=(j,i)} f_\ell - \sum_{\ell=(i,j)} f_\ell \leq 0, \quad \forall i \in \mathcal{S}, \quad (309)$$

$$\sum_{\ell \in \mathcal{L}} z_\ell \leq \sum_{i \in \mathcal{B}} y_i - \kappa. \quad (310)$$

These rows jointly enforce that energized non-source buses connect to a source and that the energized topology contains no redundant cycles. The lower-bound row for healthy normally-closed edges prevents Stage 1 and Stage 3 from opening energized closed lines for free merely to satisfy radiality.

25.4 Hybrid Fallback and Validity Flags

The AC MILP above is the only optimisation model solved for restoration. Hybrid components are represented as follows:

- DC loads and DC sources enter a graph-based connectivity/capacity fallback; no DC voltage or branch-flow equation is enforced.
- VSC converters are graph edges with active transfer capped by the best available converter rating field and efficiency. DC/DC converters are graph connectivity/capacity edges; converter loss and dispatch constraints are not co-optimised.
- SOP configurations are echoed in `sop_config`, but `psop1`, `psop2`, and `psop3` are zero vectors.

Pure AC cases report `model_scope="ac-lindistflow-milp"`. Hybrid cases report `model_scope="ac-lindistflow-milp+dc-connectivity-fallback"`. The whole-result validity flags are true for AC branch flow, voltage, radiality, and restoration MILP only when the evaluated system is pure AC. In hybrid cases these flags are deliberately false because the DC/VSC/SOP portion is fallback-only, even though the AC subproblem itself was solved.

25.5 Reliability Index Calculation

Let $p_{d,l}^{sh,(s)}$ be the active load shed in stage s at load point d when line l fails. With annual failure rate λ_l ,

$$\text{CIF}_d = \sum_l \lambda_l \mathbf{1}[p_{d,l}^{sh,(1)} > 0], \quad (311)$$

$$\text{CID}_d = \sum_l \lambda_l \left(\tau_{\text{SW}} \mathbf{1}[p_{d,l}^{sh,(1)} > 0] + (\tau_{\text{TP}} - \tau_{\text{SW}}) \mathbf{1}[p_{d,l}^{sh,(2)} > 0] \right), \quad (312)$$

$$\text{EENS}_d = \sum_l \lambda_l \left(p_{d,l}^{sh,(1)} \tau_{\text{SW}} + p_{d,l}^{sh,(2)} (\tau_{\text{TP}} - \tau_{\text{SW}}) \right). \quad (313)$$

System EENS is $\sum_d \text{EENS}_d$. IEEE Std 1366-style system indices are

$$\text{SAIFI} = \frac{\sum_d \text{CIF}_d N_d}{\sum_d N_d}, \quad \text{SAIDI} = \frac{\sum_d \text{CID}_d N_d}{\sum_d N_d}. \quad (314)$$

The reported `saidi_min` is in minutes per year.

25.6 C++ API and Result Fields

```
#include "hacdcpf/analysis/three_stage_reliability.hpp"
```

```
namespace hacdcpf::analysis {
ThreeStageReliabilityResult run_three_stage_reliability(
    const std::filesystem::path& case_json,
    const ThreeStageReliabilityOptions& options = {});

ThreeStageReliabilityResult run_three_stage_reliability_from_string(
    const std::string& case_json_text,
    const ThreeStageReliabilityOptions& options = {});
}
```

The historical `julia_executable`, `julia_project_dir`, `cli_script`, `workdir`, `keep_workdir`, and `inherit_stdio` option fields remain for source compatibility but are ignored by the native implementation. The active option is `max_switch_operations` for Stage 2 tie/switch closing.

Field	Type	Current meaning
<code>ok, error</code>	bool/string	successful exact evaluation or diagnostic
<code>saifi, saidi_min</code>	double	IEEE 1366 system indices
<code>eens_kwh_yr, eens_cost</code>	double	annual energy not supplied and VOLL cost
<code>worst_line</code>	int	branch id with maximum stage-shed total
<code>nodal_*</code>	vector	per-load-point EENS/CIF/CID
<code>faults</code>	vector	per-contingency stage shed and SOP vectors
<code>faults[*].stage\{1,2,3\}_status</code>	string	per-stage exact/approximate/failure status
<code>faults[*].stage\{1,2,3\}_mip_gap</code>	double	per-stage solver MIP gap certificate
<code>sop_config</code>	vector	SOP metadata echoed from the case
<code>nb, nl, nd, ng, nmg</code>	int	network counts
<code>model_scope</code>	string	AC MILP or AC MILP plus DC fallback
<code>model_limitations</code>	string	mandatory approximation statement
<code>validity</code>	struct	per-feature physical coverage flags

25.7 Regression Test Systems and Observed Results

The current tests live in `tests/test_three_stage_reliability.cpp` and run unconditionally; no environment variable is required.

Case	Scope	Current shape / special checks
<code>test_1_no_sop.json</code>	pure AC	5 AC buses, 5 AC lines, no DC/VSC/SOP
<code>case33mg_acdc.json</code>	hybrid	35 buses (33 AC, 2 DC), 40 lines (37 AC, 1 DC), 2 VSC, 2 SOP, 0 microgrids
<code>case33bw_acdc.json</code>	hybrid	SOP metadata present; SOP dispatch vectors have length <code>nl_sop</code> and zero

The additional regression tests cover behaviors that previously diverged from the model contract:

- direct `ACBus::pd_mw` load is counted exactly once;
- finite source capacity prevents treating a source bus as infinite slack;
- standalone AC switches can be Stage 2 candidate edges when a switch operation is available.
- Stage 1 and Stage 3 cannot open healthy normally-closed lines for free;
- AC-source surplus can support DC load only through VSC capacity limits.

ID	Tag	Description
TC-1	[smoke]	data-file presence and structured empty-JSON error
TC-2	[integration] [milp]	pure-AC 5-bus feeder; AC validity flags true
TC-3	[integration] [milp]	AC/DC case with DC fallback; DC/SOP validity flags false
TC-4	[integration] [milp]	SOP case; SOP config and zero-dispatch vectors checked
TC-5	[error]	missing-file graceful error
TC-6	[regression]	bus-level load is not double-counted
TC-7	[regression]	source capacity is finite
TC-8	[regression]	standalone AC switch can restore Stage 2 load
TC-9	[regression]	healthy normally-closed lines cannot be opened for free
TC-10	[regression]	VSC capacity limits AC support to DC load

In the current `build_rel` CTest registration this executable contributes 10 tests. The current full-suite Release registration contains 436 CTest entries; 434 active tests pass and the two known MATPOWER parser cases are handled as skips.

26 Implementation Gap and Stub Audit

This chapter records the current code-backed boundary between implemented runtime behavior, transparent approximations, resolved gaps, and remaining bounded model scope. It is intentionally separate from the mathematical model chapters: a formula in this notebook is not a claim that every richer physical variant is solved end-to-end in production code. Each status item below is tied to a concrete API, source path, and regression test rather than to narrative intent alone.

26.1 Audit Method

The audit covered `src/`, `include/`, `tests/`, the technical-notebook `sections/` directory, and the sibling external `MIPsolvers` checkout used by CMake. The review combined source inspection with searches for high-signal markers such as `stub`, `unimplemented`, `placeholder`, `fallback`, `simplified`, `approximate`, `dummy`, and `unsupported`. Marker counts are not used as defect counts: many occurrences are legitimate solver fallback paths, explicit input-domain checks, or tests that verify a declared boundary.

The latest pass specifically audited external solver reproducibility, native B&C quiet-mode behavior, HiGHS-to-native fallback semantics, hybrid topology fault addressing, topology load/source accounting, topology result validity contracts, legacy ONR branch-ID semantics, FMEA repair-stage reconfiguration scope, three-stage reliability exactness and fault enumeration, graph contraction domain maps, DCOPF branch-dual validity, and rich-component static validation. A second claim audit then re-read this chapter against the code and tests. It found no stale closed-gap statement, but it did expose one additional quiet-mode leak in the strict resilience MIP path; that leak is now gated in the external solver and covered by file-descriptor-level output capture.

26.2 High-Risk Items Closed

1. **External MIPSolvers reproducibility.** The external dependency is expected at pinned commit `5b2eb78f00873fb7fd2abf2223d649ada63ab03d`. CMake treats any external MIPSolvers HEAD mismatch as fatal, and now also treats a dirty external checkout as fatal for Release, RelWithDebInfo, MinSizeRel, and CI configurations. The local override `HACDCDSS_SKIP_MIPSOLVERS_DIRTY_CHECK` is rejected in those strict modes, so a reproducible Release artifact cannot silently include unpinned or uncommitted external solver edits. The recorded pin is forced into the CMake cache so an old `build_rel/CMakeCache.txt` cannot keep comparing against a stale commit after an intentional dependency upgrade.
2. **Distribution power-flow public API.** The legacy distribution power-flow compatibility header and public single-phase stub/BFS-style API are removed. The public distribution feeder surface is the three-phase NR contract in `include/hacdcpf/power_flow/distribution_power_flow_solve_three_phase_nr` and phase-domain Y-bus helpers. Searches find no live `solve_distribution_pf`, `DistributionPFOptions`, or `DistributionPFResult` code symbols.
3. **Hybrid FMEA physical evaluation.** Hybrid FMEA no longer scores DC/VSC systems through an AC-only DCOPF boundary. When DC or converter components exist, `run_distribution_fmea` uses a linear hybrid AC/DC network LP that co-optimizes AC load shedding, DC load shedding, AC/DC sources, DC branches, VSC active-power transfers, and DC/DC converter transfers. Reports set `model_scope` to `hybrid-acdc-network-lp` and mark DC-load curtailment plus VSC/DC power flow as included. The same validity contract now explicitly leaves `ac_voltage_reactive_feasibility_certified=false` because nonlinear AC voltage/reactive checks are outside this LP evaluator. The repair-stage explicit topology search is also surfaced honestly: `repair_ac_switch_reconfiguration_modelled` records AC-side switch search coverage, while `repair_dc_side_reconfiguration_modelled=false` documents that DC breakers, DC branches, DC/DC devices, and VSC topology actions are not enumerated as repair reconfiguration candidates.
4. **AC OPF fallback for hybrid cases.** Hybrid AC/DC cases are detected before fallback selection. If the nonlinear hybrid path fails, `solve_ac_opf` suppresses the AC-only economic-dispatch plus AC-PF fallback and attaches an explicit diagnostic that the case contains DC/VSC components. The fast AC-only route remains available for AC-only cases.

26.3 Other Resolved or Explicitly Bounded Findings

1. **Native B&C quiet mode.** The external native branch-and-cut implementation now gates root-phase timing, conformance, clique, propagation, reduced-cost lurking, implied-bound, proof, objective, and queue diagnostics behind `BCOptions::verbose` or explicit diagnostic environment variables such as `MIPSOLVERS_BC_CONF` and `MIPSOLVERS_BC_TIMELINE`. File-descriptor-level stdout/stderr capture tests in topology reconfiguration, legacy ONR, and the strict resilience MIP entry point verify that `verbose=false` is actually quiet.
2. **Three-stage reliability exactness.** `src/reliability/three_stage_reliability.cpp` is a native C++ evaluator. It solves finite-source AC LinDistFlow restoration with explicit source bounds, load-shed variables, branch limits, voltage bounds, strict radial forest constraints, healthy normally-closed edge coupling, and Stage 2 switch-operation limits. True MILPs use HiGHS first and native B&C as fallback both when HiGHS is unavailable and when a HiGHS solve returns failure or a malformed solution vector; per-stage status and MIP gaps are surfaced. No Julia or Gurobi runtime is required.

Result field `ok` now means verified exactness inside the implemented physical scope, not merely successful metric production. Pure AC cases can return `ok=true`. Hybrid DC/VSC/DCDC/SOP cases return metrics with `ok=false`, nonempty `error`, and model scope `ac-lindistflow-milp+dc-connectivity-fallback`. Regression tests protect pure-AC exactness, hybrid non-exactness, finite source capacity, AC bus load de-duplication, the no-free-open rule for healthy normally-closed lines, and VSC-capacity-limited AC support to DC load. The N-1 fault set is explicitly branch-only: in-service `ACBranch` and `DCBranch` outages are evaluated, while VSC, DC/DC, switch, breaker, transformer, generator, load, and storage outages are outside this evaluator’s contingency set.

3. **Topology reconfiguration hybrid addressing.** `src/network_reconfiguration/topology_reconfiguration` tries HiGHS when available and retries native B&C if HiGHS is unavailable, fails, or returns a wrong-sized solution vector. The result exposes `solver_backend`, `solver_status`, `solver_mip_gap`, and `proven_optimal`; `optimal` mirrors that certificate. Switchable-edge input accepts structured `BranchRef` values via `switchable_branches`; fault input now also accepts structured `faulted_branches`. The legacy `switchable_branch_ids` and `line_failures` fields are AC-only compatibility fields.
4. **Topology reconfiguration source and validity contract.** The topology MILP now builds sources on the unified AC/DC bus index. AC generators, static generators, renewables, PV, storage, external grids, and slack fallback map to AC source nodes; DC static generators, DC-side static generators, DC storage, DC PV arrays, and `DC_V` buses map to DC source nodes. A network with no available AC or DC source returns infeasible before writing root-source variables. Results expose `model_scope` plus validity flags for radial topology, AC LinDistFlow enforcement, DC network modelling, DC source dispatch, VSC active transfer modelling, VSC reactive approximation, and the absence of post-PF/full-hybrid-OPF validation.
5. **Topology load accounting.** Fault reconfiguration adds explicit `ACSystem::loads` demand, direct `ACBus::pd_mw` and `ACBus::qd_mvar` demand, and direct `DCBus::pd_mw` demand instead of treating any one record type as a replacement for the others.
6. **Static rich-component validation.** The entry validation gate now checks rich AC/DC component bus references and basic physical bounds before projection or solving: AC/DC loads, static generators, renewables, PV, storage, shunts, switches, circuit breakers, transformers, charging stations/chargers, motors, VSC converters, DC/DC converters, DC PV arrays, and DC breakers are covered for missing endpoints and capacity/efficiency/state consistency where applicable.
7. **Legacy ONR branch identity.** The legacy AC ONR API documents `ONROptions::switchable_branch_ids` as stable `ACBranch::index` values, not vector positions. Non-candidate branches are fixed to their current `in_service` state, and fixed `alpha` variables are continuous constants omitted from the native B&C binary set.
8. **Resilience MIP entry point.** `run_distribution_resilience_mip_assessment` builds an independent multi-period LinDistFlow-style restoration MILP. It does not delegate to the heuristic assessment. Result statistics expose `model_scope`, validity flags, solver name/status, objective, gap, and constraint counts. The MIP scope remains AC-only LinDistFlow and reports that scope explicitly through `model_stats.model_scope`, `validity.dc_network_modelled=false`, and `validity.vsc_dispatch_modelled=false`.
9. **DC OPF load-shedding audit.** Direct isolated-island shedding is written to both `total_load_shedding_mw` and the per-bus `load_shedding_mw` vector. The topology pre-prune removes loads, generators, fixed injections, storage, and AC branches attached to

dead islands before LP construction. The independent feasibility check includes slack-bus balance.

10. **DC OPF JSON contract.** DCOPF result JSON helps serialize/restore solver audit fields, including `solver_chain` and `objective_model`.
11. **DC OPF LMPs and branch duals.** Nodal LMP extraction is implemented through a supporting simplex LP when `compute_lmp=true`. Branch shadow-price arrays are intentionally flagged with `branch_mu_valid=false` until bounded-variable congestion duals are recovered robustly. The LMP regression asserts this uncertified branch-dual boundary explicitly.
12. **Graph contraction and recovery maps.** Hybrid graph contraction results expose domain-qualified maps such as `ac_bus_to_super` and `dc_bus_to_super`. Recovery helpers can use the structured result directly so same numeric AC/DC bus IDs are not collapsed through a legacy combined map.
13. **JPC rich-component round-trip.** `test_io_json` includes rich-component JPC round-trip coverage and asserts the populated component collections are preserved.

26.4 Closed Model-Scope Contracts

The second pass treats the following items as closed contracts rather than open implementation gaps. They are not silently solved by approximation and they are not advertised as richer physics than the code enforces; each boundary is visible through a result field and protected by regression tests.

1. Hybrid FMEA is a linear restoration/curtailment LP. It includes DC loads, DC network edges, VSC active transfers, DC/DC transfers, and source limits, but it does not certify nonlinear AC voltage/reactive feasibility for every restoration state. Callers can check `FMEAResult::validity.ac_voltage_reactive_feasibility_certified`, which remains false for this evaluator. Repair-stage topology search is AC-side only; callers can check `repair_dc_side_reconfiguration_modelled` before treating repair switch actions as full hybrid topology optimization.
2. The resilience MIP entry point remains an AC-only LinDistFlow restoration MILP and reports that scope explicitly through `model_stats.model_scope=ac-only-lindistflow` plus false DC/VSC validity flags.
3. Three-stage reliability is exact only for the pure-AC implemented restoration MILP scope. Hybrid DC/VSC/DCDC/SOP cases report useful connectivity/capacity metrics but clear ok and set validity flags false for DC power flow and whole-system restoration MILP coverage. Its N-1 contingency set is limited to AC/DC branch outages; rich component outages are not silently implied by the aggregate reliability indices.
4. Topology reconfiguration returns a solved graph/linear topology model, not a post-reconfiguration full AC/DC PF or OPF certificate. The `TopoReconfResult::validity` flags distinguish the enforced LinDistFlow/connectivity constraints from `post_power_flow_validated=false` and `full_hybrid_opf_validated=false`.
5. DC OPF branch shadow-price arrays remain invalid unless `branch_mu_valid=true`; bounded-variable branch-dual extraction is still a deliberately exposed boundary. Nodal LMPs remain available via the supporting simplex LP when `compute_lmp=true`.

26.5 Verification Snapshot

After the implementation pass, the following commands were run from the repository root using the `build_rel` Release tree. The external `MIPSolvers` checkout was clean at commit `5b2eb78f00873fb7fd2abf2223d649ada63ab03d`, matching the recorded CMake pin. CMake Tools' VS Code test runner failed before test discovery with an "Unable to configure the project" integration error, so Release CTest was run directly against the successfully configured `build_rel` directory.

```
cmake -S . -B build_rel -DCMAKE_BUILD_TYPE=Release
cmake --build build_rel -j8
./build_rel/tests/test_topology_crossval "[topology][reconfiguration][regression]"
./build_rel/tests/test_resilience_assessment "[fmea][reliability][regression]"
./build_rel/tests/test_three_stage_reliability "[reliability][three_stage][regression]"
./build_rel/tests/test_validation "[validation][fault][rich_components]"
ctest --test-dir build_rel -j8 --output-on-failure
```

Focused Release regression outcomes include: topology reconfiguration **45 assertions in 9 cases**, FMEA hybrid scope regressions **19 assertions in 2 cases**, three-stage reliability regressions **24 assertions in 5 cases**, and rich-component validation **13 assertions in 2 cases**. The full Release CTest result is **436 registered tests: 434 active tests passed, 0 failed**, and **2 expected MATPOWER parser skips** (`case533mt_hi`, `case533mt_lo`).

26.6 Audit Conclusion

The repository is not an empty shell: PF, OPF, graph, I/O, carbon, time-series, resilience, FMEA, topology reconfiguration, and three-stage reliability paths are implemented and covered by the current 436-entry CTest suite. The three previous high-risk audit items in this chapter are now closed in code and in tests: the legacy distribution power-flow public API is removed, hybrid FMEA optimizes DC networks and VSC transfers, and hybrid AC OPF can no longer accept an AC-only fallback as a converged hybrid result. This pass also closed the newly audited logic holes in external MIPSolvers reproducibility, stale CMake pin caching, native B&C quiet mode, HiGHS failure fallback, three-stage reliability exactness and branch-only fault disclosure, topology reconfiguration DC sources/no-source safety/result validity, FMEA repair-stage DC-side reconfiguration disclosure, legacy ONR branch identity, graph domain-map handling, DCOPF branch-dual validity, rich-component validation, and FMEA DC load scaling. The second-pass review found no remaining unsupported claim in this chapter; the items that would otherwise read as gaps are now explicit model-scope contracts surfaced through result fields, validity flags, diagnostics, and regression tests.

27 EV Power–Traffic Coupled Simulation: Implemented Model and Solver-Certified Extension

This chapter documents the mathematical model that is currently implemented in the `hacd cpf::evpt` module and then gives the solver-certified dynamic joint-optimization formulation that is now fully implemented and validated. All four formulations (A–D) have direct code counterparts in the current source tree. The regression test suite (`test_ev_power_traffic.c.cpp`, `test_ev_power_traffic_report.cpp`, `test_ev_power_traffic_joint_opt.cpp`, `test_ev_power_traffic_largescale.cpp`, `test_ev_power_traffic_ctm_due.cpp`, `test_ev_power_traffic_joint_opt_d.cpp`) provides reproducible numerical values for Formulations A–D that are used as the running examples in this chapter.

Modeling formulation tiers. To avoid confusion between what is currently implemented and what represents a future extension, this chapter uses four explicitly named tiers:

Formulation A (implemented, validated). Static BPR traffic model; free-flow pre-computed route arrival steps; fixed charging-profile system-optimal LP/MILP with demand-balance, road-capacity, and station-power constraints; optional DC-OPF coupling via iterative LMP-price update. Validated by `test_ev_power_traffic.cpp`, `test_ev_power_traffic_report.cpp`, `test_ev_power_traffic_joint_opt.cpp`, and `test_ev_power_traffic_largescale.cpp`.

Formulation B (implemented, validated). Daganzo CTM with triangular fundamental diagram; route-specific (commodity) cell occupancy to preserve station-choice identity through diverge nodes; MSA-based Dynamic User Equilibrium (DUE); Wardrop relative gap convergence criterion; deterministic shortest-path or logit auxiliary assignment. Implemented in `src/ev_power_traffic/ctm_propagation.cpp`; entry point `simulate_ev_power_traffic_ctm_due()`; validated by `test_ev_power_traffic_ctm_due.cpp` (7 test cases, 106 assertions). **Limitation:** does not perform smart-charging dispatch or DC-OPF coupling; station arrivals and EV load synthesis remain future work for this path.

Formulation C. Formulation B plus iterative DC-OPF LMP price coordination (**implemented**): the CTM-DUE inner step provides EV loads per station per time step; the DC-OPF outer step extracts nodal LMPs and updates station prices via a convex combination; the loop repeats until price convergence. Social welfare $W = B_{EV} - C_{gen} - C_{delay}$ is tracked at each iteration. Entry point: `simulate_ev_power_traffic_ctm_joint()`; implementation: `src/ev_power_traffic/ctm_joint_welfare.cpp`; validated by `test_ev_power_traffic_ctm_joint.cpp` (5 test cases, 43 assertions).

Formulation D (implemented, validated; §27.18). Route assignment (free-flow arrival steps), optional DC-OPF power balance, V2G, and station-capacity constraints are assembled into a single LP/QP/MILP/MCP in one solver call. Supports `SocialWelfareMax`, `UserBenefitMax`, and `UserEquilibriumMILP` objectives. Solver certificates (primal feasibility, dual bound, MIP gap) are populated for every solve. Entry point: `solve_joint_optimizer()`; implementation: `src/ev_power_traffic/joint_optimizer.cpp`; validated by `test_ev_power_traffic_joint_opt_d.cpp` (7 test cases, 55 assertions).

Sections 27.2–27.8 describe Formulation A in full mathematical detail. Section 27.14 documents Formulation B. Section 27.9 gives Formulation D, the solver-certified dynamic joint-optimization model. The numerical case studies (Sections 27.11–27.13) validate Formulation A. Section 27.15 presents Formulation B results. Formulation C is reported in Section 27.17. Formulation D is validated in Section 27.18.

Remark 2 (Certificate scope). *Solver optimality certificates apply only to the LP/MILP objective and constraints of Formulation A (Eq. (321)–(325)). This formulation is globally optimal only for its own surrogate problem: free-flow route costs, static road-capacity screening, fixed dwell-window charging profiles, and optional aggregate generation capacity. Any BPR travel time, queueing delay, AC power-flow result, or welfare value reported after the LP solve is an ex-post diagnostic unless it is explicitly a decision variable or constraint in the mathematical program. Formulation B returns a DUE route-flow estimate via CTM propagation and MSA; it does not invoke the LP or OPF layer. Formulation C is a price-coordination fixed-point algorithm rather than one integrated mathematical program. B/C convergence is empirically validated by regression tests and is not a solver certificate. When certification is required, use Formulation D and report the optimisation status, primal feasibility, best bound, and optimality/MIP gap.*

27.1 Time Grid, Sets, and Notation

27.1.1 Single Time Grid

The simulation uses a single uniform time grid with K steps:

$$\mathcal{T} = \{0, 1, \dots, K-1\}, \quad t_k = k \Delta t,$$

where Δt is the step duration in hours (`options.time_step_hr`, default 1 h) and K is the number of steps (`options.num_steps`, default 24). There is no separate traffic time grid. Formulation B (Section 27.14) internally refines this grid to a finer CTM time step Δt_{ctm} . Unless stated otherwise, index k refers to the simulation grid $\mathcal{T}$; within Formulation B the CTM step index is written $\kappa \in \{0, \dots, T_{\text{ctm}}-1\}$.

27.1.2 Sets

Symbol	Meaning	C++ type
$\mathcal{N}$	traffic nodes	<code>TrafficNode</code>
$\mathcal{A}$	directed road links	<code>TrafficLink</code>
$\mathcal{R}$	route alternatives	<code>RouteAlternative</code>
$\mathcal{S}$	charging stations	<code>ChargingStation</code>
$\mathcal{D}$	EV fleet demands	<code>EVDemand</code>
$\mathcal{V}$	charging sessions	<code>EVChargingSession</code>

27.1.3 Key Scalar Parameters

Symbol	Unit	Options field	Meaning
Δt	h	<code>time_step_hr</code>	step duration
VOT	\$/veh·h	<code>value_of_time_per_hr</code>	value of travel time
ω_s	–	<code>station_energy_cost_weight</code>	energy cost scaling
ω_Q	\$/veh	<code>queueing_weight</code>	queueing disutility
$\pi_{s,k}$	\$/kWh	(per station and step)	station electricity price
M^{trip}	\$/veh	<code>system_optimal_unserved_trip_penalty</code>	unserved-demand penalty
θ	1/\$	<code>logit_theta</code>	Logit dispersion
α	–	<code>price_update_step</code>	JSW price step size

27.2 Traffic Flow Model

27.2.1 Link Capacity and Travel Time (BPR Function)

The per-step vehicle capacity of directed link $a \in \mathcal{A}$ at step k is

$$C_{a,k} = c_a \Delta t,$$

where c_a is the link flow capacity in veh/h (`TrafficLink::capacity_veh_per_hr`).

The link travel time is modelled by the Bureau of Public Roads (BPR) volume–delay function. Given that $x_{a,k}$ vehicles use link a at step k , the travel time in hours is

$$t_{a,k} = t_a^0 \left[1 + \alpha_a \left(\frac{x_{a,k}}{C_{a,k}} \right)^{\beta_a} \right], \quad (315)$$

where t_a^0 is the free-flow travel time (`free_flow_time_hr`), $\alpha_a = 0.15$ and $\beta_a = 4$ are the BPR calibration parameters (`TrafficLink::alpha`, `TrafficLink::beta`). When $C_{a,k} = 0$ (blocked link) the travel time is set to $+\infty$.

27.2.2 Route Travel Time

Route $r \in \mathcal{R}$ is a sequence of directed links $r = (a_1, a_2, \dots, a_L)$. Its travel time at step k is

$$T_{r,k} = \sum_{a \in r} t_{a,k}. \quad (316)$$

27.2.3 Generalized Route Cost

Route r serves demand $d \in \mathcal{D}$ departing at step k . The generalized cost used by the heuristic assignment modes (Deterministic Shortest Path, Logit, Capacity-Aware Greedy) is

$$c_{d,r,k} = \text{VOT} \cdot T_{r,k} + \omega_s \sum_{s \in \mathcal{S}(r)} \pi_{s,k} e_{r,s} + \omega_Q \sum_{s \in \mathcal{S}(r)} W_{s,k_{\text{arr}}(r,k)} + \text{toll}_r, \quad (317)$$

where $\mathcal{S}(r)$ is the set of charging stops on route r , $e_{r,s}$ is the per-vehicle energy requested at stop s (`requested_energy_kwh_per_vehicle`), $W_{s,k}$ is the estimated waiting time at station s at step k (Section 27.6), $k_{\text{arr}}(r,k)$ is the arrival step at the stop (rounded from free-flow travel time), and toll_r is any optional route toll (`RouteAlternative::toll_cost`).

When computing the *free-flow* cost for the system-optimal LP objective (Section 27.3.4), the actual link travel times are replaced by free-flow values t_a^0 , and the queueing term is dropped.

27.2.4 Implemented SOC Feasibility Filter

When a demand specifies an initial battery energy $E_d^{\text{init}} \geq 0$, the route candidate set is filtered before assignment. Let ϵ_a denote the per-vehicle traction-energy coefficient of link a [kWh/(veh km)]. Along route $r = (a_1, \dots, a_L)$, the implemented path-energy recursion is

$$\begin{aligned} E_{d,r,0} &= E_d^{\text{init}}, \\ E_{d,r,\ell} &= E_{d,r,\ell-1} - \epsilon_{a_\ell} L_{a_\ell}, & \ell = 1, \dots, L, \\ E_{d,r}^{\text{post}} &= \min \left(\bar{E}_d, E_{d,r,L} + \sum_{s \in \mathcal{S}(r)} e_{r,s} \right). \end{aligned} \quad (318)$$

The route is feasible only if the battery stays within the configured SOC bounds:

$$\underline{E}_d \leq E_{d,r,\ell} \leq \bar{E}_d, \quad \ell = 0, \dots, L, \quad E_{d,r}^{\text{post}} \geq \underline{E}_d. \quad (319)$$

If $E_d^{\text{init}} < 0$, or if no link on the route provides $\epsilon_a > 0$, this filter is disabled for backward compatibility.

27.3 Assignment Models

Four assignment policies are selectable via `options.assignment_model`:

27.3.1 Deterministic Shortest Path (`DeterministicShortestPath`)

All q_d vehicles of demand d are assigned to the single route with minimum generalized cost:

$$r^* = \arg \min_{r \in \mathcal{R}_d} c_{d,r,k_d}.$$

No capacity constraint is enforced.

27.3.2 Logit (Logit)

When road capacity constraints are not enforced, vehicles are split using the multinomial logit model:

$$x_{d,r} = q_d \frac{e^{-\theta c_{d,r,k_d}}}{\sum_{r' \in \mathcal{R}_d} e^{-\theta c_{d,r',k_d}}}, \quad r \in \mathcal{R}_d. \quad (320)$$

Routes with $c_{d,r,k_d} = +\infty$ receive zero weight.

27.3.3 Capacity-Aware Greedy (CapacityAwareGreedy)

Routes are sorted by ascending generalized cost. For each route r the algorithm assigns

$$x_{d,r} = \min(\text{remaining}_d, \text{road_res}_{r,k}, \text{sta_res}_{r,k}),$$

where $\text{road_res}_{r,k} = \min_{a \in r} (C_{a,k} - \text{used}_{a,k})$ and $\text{sta_res}_{r,k}$ is the remaining charging capacity at each stop. Used-capacity and reserved-station-power arrays are updated after each assignment.

27.3.4 System-Optimal LP / MILP

See Section 27.4 for the full formulation. The continuous LP is solved by the native dual-simplex implementation (`engine::solve_lp_with_basis`). The integer MILP variant attempts HiGHS first, falling back to the native branch-and-cut solver (`engine::solve_milp_bc`).

27.4 System-Optimal LP Formulation

27.4.1 Assumptions (Formulation A)

The following conditions are required for the LP to be well-posed and its solution to be globally optimal for the system-optimal assignment problem:

1. **System-optimal, not DUE.** The LP minimises total system cost; it does not model user equilibrium or complementarity conditions.
2. **Continuous flows.** Route assignment variables $x_{d,r} \geq 0$ are continuous; the MILP variant adds integrality constraints.
3. **Free-flow costs in objective.** The cost coefficients $c_{d,r}^{\text{ff}}$ are precomputed from free-flow travel times t_a^0 ; BPR congestion does not appear in the LP objective. Consequently the LP solution may assign more vehicles to a route than the greedy congestion-aware heuristic would (see Sections 27.11–27.13).
4. **Fixed charging profile.** Station power is a fixed linear function of route flows via the dwell-window convolution (Eq. (324)); smart-charging flexibility is not modeled in the LP itself.
5. **No V2G in LP.** $P_{s,k}^{EV} \geq 0$ holds implicitly because $p_{r,s}^{\text{req}} \geq 0$ and $x_{d,r} \geq 0$. V2G discharge is available separately via the smart-charging LP (Section 27.5).
6. **Precomputed arrival steps.** Station arrival steps k_{arr} are derived from free-flow travel times and are fixed parameters, not optimisation variables.
7. **DC power flow (optional).** Power-system coupling uses linear DC-OPF when enabled; AC nonconvexity is not modeled.

27.4.2 Decision Variables

Variable	Domain	Meaning
$x_{d,r} \geq 0$	veh	vehicles of demand d on route r
$u_d \geq 0$	veh	unserved vehicles for demand d

27.4.3 Objective

Minimise total free-flow generalized cost plus unserved-demand penalty:

$$\min \sum_{d \in \mathcal{D}} \sum_{r \in \mathcal{R}_d} c_{d,r}^{\text{ff}} x_{d,r} + M^{\text{trip}} \sum_{d \in \mathcal{D}} u_d, \quad (321)$$

where $c_{d,r}^{\text{ff}}$ is the free-flow generalized cost ($T_{r,k}$ evaluated at t_a^0) and $M^{\text{trip}} = 10^6$ by default.

27.4.4 Demand Balance

$$\sum_{r \in \mathcal{R}_d} x_{d,r} + u_d = q_d, \quad \forall d \in \mathcal{D}. \quad (322)$$

27.4.5 Road Capacity

If `enforce_road_capacity` is enabled:

$$\sum_{\substack{d \in \mathcal{D}, r \in \mathcal{R}_d: \\ a \in r}} x_{d,r} \leq C_{a,k_d}, \quad \forall a \in \mathcal{A}. \quad (323)$$

Constraints are grouped by (departure step, link index).

Remark 3 (Static path-capacity screening). *The road-capacity rows (323) are static screening constraints indexed by demand departure step k_d . A vehicle on route r is charged against every link $a \in r$ at k_d , regardless of when it actually arrives at link a . These rows prevent gross over-assignment to a congested corridor but do not propagate link occupancy over time and therefore do not model spillback, downstream capacity at actual arrival times, or link storage. Dynamic link occupancy is captured only in the CTM module of Formulation B.*

27.4.6 Station Charging Capacity

For each station s and each step k in a dwell window:

$$\sum_{\substack{d, r \in \mathcal{R}_d: s \in \mathcal{S}(r), \\ k \in [k_{\text{arr}}, k_{\text{arr}} + \Delta_s]}} p_{r,s}^{\text{req}} x_{d,r} \leq P_s^{\text{max}}, \quad \forall s \in \mathcal{S}, k \in \mathcal{T}, \quad (324)$$

where $p_{r,s}^{\text{req}} = e_{r,s} / (\Delta_s \Delta t)$ [kW/veh] is the required charging power per vehicle and Δ_s is the dwell duration in steps (`dwell_steps`).

27.4.7 Generation Capacity (Optional)

If `enforce_generation_capacity` is enabled:

$$\sum_{\substack{d, r, s: s \in \mathcal{S}(r), \\ k \in \text{dwell window}}} p_{r,s}^{\text{req}} x_{d,r} \leq (P_{\text{max}}^{\text{gen}} - P^{\text{base}}) \times 1000, \quad \forall k \in \mathcal{T}. \quad (325)$$

Remark 4 (Aggregate generation screen). *Equation (325) is a system-wide aggregate power adequacy check. It does not represent bus-level locational pricing, branch flow limits, voltage constraints, generator redispatch, or AC feasibility. Full network feasibility requires DC-OPF or AC power-flow validation (Section 27.7).*

27.4.8 MILP Variant

When `SystemOptimalMILP` is selected and all q_d are integers, the variables are declared integer: $x_{d,r}, u_d \in \mathbb{Z}_{\geq 0}$. Non-integer demand triggers an automatic fallback to the LP relaxation.

27.5 Charging Session Synthesis

After route assignment, each route flow $x_{d,r} > 0$ generates one `EVChargingSession` per charging stop on route r :

Session field	Value
<code>arrival_step</code>	$k_d + \lceil T_{r,k_d}^{\text{ff}} / \Delta t \rceil$
<code>departure_step</code>	$\min(K, k_{\text{arr}} + \Delta_s)$
<code>vehicle_count</code>	$x_{d,r}$
<code>energy_target_kwh</code>	$e_{r,s} \cdot x_{d,r}$
<code>max_charge_kw</code>	$p_{r,s}^{\text{ch,max}} \cdot x_{d,r}$

27.5.1 Smart Charging LP

For each session v with dwell window $[k_{\text{arr}}, k_{\text{dep}})$, a per-session LP allocates charging and discharging power:

$$\min \quad \rho \xi_v \quad (326)$$

$$\text{s.t.} \quad \sum_{k=k_{\text{arr}}}^{k_{\text{dep}}-1} \left(\eta^{\text{ch}} p_{v,k}^{\text{ch}} - \frac{p_{v,k}^{\text{dis}}}{\eta^{\text{dis}}} \right) \Delta t + \xi_v \geq E_v^{\text{req}}, \quad (327)$$

$$0 \leq p_{v,k}^{\text{ch}} \leq p_v^{\text{ch,max}}, \quad (328)$$

$$0 \leq p_{v,k}^{\text{dis}} \leq p_v^{\text{dis,max}} \cdot \mathbf{1}[\text{V2G}], \quad (329)$$

$$\xi_v \geq 0, \quad (330)$$

where $E_v^{\text{req}} = e_{r,s} \cdot x_{d,r}$ [kWh] is the requested delivered energy for session v (not a final SOC target); $\eta^{\text{ch}} \in (0, 1]$ is charging efficiency and $\eta^{\text{dis}} \in (0, 1]$ is discharging efficiency (both default to the single parameter `default_charging_efficiency`, default 1.0); slack $\xi_v \geq 0$ represents unserved energy with penalty weight ρ . With $\eta^{\text{ch}} = \eta^{\text{dis}} = \eta$ the net battery gain per step is $\eta p_{v,k}^{\text{ch}} - p_{v,k}^{\text{dis}} / \eta$. V2G is disabled when $p_v^{\text{dis,max}} = 0$ (`v2g_capable=false`).

Remark 5 (Per-session LP and station-level capacity). *The per-session LP does not by itself enforce station-level aggregate power limits. In the LP/MILP assignment mode, station feasibility is enforced upstream by Eq. (324). In Deterministic Shortest Path and Logit modes, station capacity is not guaranteed; excess requested energy may be reported as unserved.*

The net station power at step k is

$$P_{s,k}^{\text{EV}} = \sum_{\substack{v: s_v=s, \\ k \in [k_{\text{arr}}, k_{\text{dep}})}} (p_{v,k}^{\text{ch}} - p_{v,k}^{\text{dis}}).$$

27.6 Station Queue and Occupancy Model

The simulation tracks a discrete-time queue for each station $s \in \mathcal{S}$. Define:

- $\lambda_{s,k}$: vehicles arriving at station s at step k (from route assignments with matching arrival step),

- $m_{s,k}$: plug-occupancy at step k — number of vehicles already plugged in (from session schedules),
- N_s^{plug} : number of plugs (`n_fast`).

27.6.1 Queue Dynamics

The number of vehicles that can begin charging at step k is

$$u_{s,k} = \min\left(q_{s,k} + \lambda_{s,k}, \max\left(0, N_s^{\text{plug}} - m_{s,k}\right)\right), \quad (331)$$

and the queue evolves as

$$q_{s,k+1} = \max(0, q_{s,k} + \lambda_{s,k} - u_{s,k}), \quad q_{s,0} = 0. \quad (332)$$

27.6.2 Service Completion

For a fixed dwell duration of Δ_s steps, the service completion rate is

$$r_{s,k} = u_{s,k-\Delta_s}, \quad k \geq \Delta_s; \quad r_{s,k} = 0 \text{ for } k < \Delta_s. \quad (333)$$

The plug occupancy then evolves as

$$m_{s,k+1} = m_{s,k} + u_{s,k} - r_{s,k}, \quad m_{s,0} = 0. \quad (334)$$

In the forward simulation, the session schedule derived from Section 27.5 provides the actual $m_{s,k}$ directly; Eq. (334) is used for prospective queueing estimates when session schedules are not yet available (e.g. in the first assignment pass).

27.6.3 Waiting Time Approximation

$$W_{s,k} \approx \frac{q_{s,k}}{\mu_s} \Delta t, \quad \mu_s = \frac{N_s^{\text{plug}}}{\Delta_s} \approx \frac{P_s^{\text{max}}}{p_{\text{veh}} \Delta_s}, \quad (335)$$

where μ_s [vehicles/step] is the effective service throughput—the number of vehicles completing service per step when the station is at full capacity—and p_{veh} is the per-vehicle charging power (`default_route_stop_power_kw_per_vehicle`). For a one-step dwell ($\Delta_s = 1$), $\mu_s = N_s^{\text{plug}}$. The waiting time $W_{s,k}$ [h] feeds the generalized cost (317) via ω_Q .

27.7 Power-System Coupling

27.7.1 Station Load Write-Back

After smart charging dispatch, `apply_ev_station_loads` adds the per-step EV load to the corresponding bus:

$$P_{b(s),k}^d += P_{s,k}^{EV} \times 10^{-3} \text{ [MW]}.$$

A negative $P_{s,k}^{EV}$ (V2G discharge) reduces the bus load.

27.7.2 Optional Power-Flow Validation

If `run_power_flow_validation=true`, the AC power flow is solved for each step using `solve_power_flow(step_system,pf_options)`. The generation capacity margin is checked:

$$P_{\text{max}}^{\text{gen}} - P^{\text{base}} - \sum_s P_{s,k}^{EV} \times 10^{-3} \geq 0.$$

27.8 Joint Social Welfare Algorithm

`solve_joint_social_welfare()` in `src/ev_power_traffic/joint_social_welfare.cpp` implements iterative LMP-price coordination.

Remark 6 (Coordination heuristic, not a global optimizer). *The JSW routine is a decentralized fixed-point coordination heuristic; it does not solve a single integrated social-welfare maximization problem. Its fixed point, when reached, is a consistency state between the traffic LP response to current prices and the DC-OPF LMPs under the resulting loads. Convergence and global welfare optimality are not guaranteed for arbitrary networks or loading conditions; the algorithm has been validated on the cases in Section 27.12.*

27.8.1 Sub-Problems

Traffic sub-problem. Given prices $\pi^{(i)}$, solve the system-optimal LP (321)–(325) to obtain $x^{(i)}, P^{EV,(i)}$.

Power sub-problem. For each step k , solve the DC-OPF with EV loads $P_{s,k}^{EV,(i)}$ added to their buses. Let $\mathcal{G}_i$ denote generators at bus i :

$$\min_{P_{g,k}^g} \sum_{g \in \mathcal{G}} (c_{2,g} (P_{g,k}^g)^2 + c_{1,g} P_{g,k}^g + c_{0,g}) \Delta t$$

subject to DC power balance at each bus i :

$$\sum_{g \in \mathcal{G}_i} P_{g,k}^g - P_{i,k}^d = \sum_j B_{ij} \theta_{j,k},$$

branch flow limits $-\bar{F}_\ell \leq b_\ell(\theta_{i(\ell),k} - \theta_{j(\ell),k}) \leq \bar{F}_\ell$, and generator bounds $\underline{P}_g^g \leq P_{g,k}^g \leq \bar{P}_g^g$. Output: dispatch $P_{g,k}^{g,(i)}$, LMPs $\lambda_{b,k}^{(i)}$ [\$/MWh] as dual variables of the power-balance constraints.

27.8.2 Price Update

$$\pi_{s,k}^{(i+1)} = \alpha \frac{\lambda_{b(s),k}^{(i)}}{1000} + (1 - \alpha) \pi_{s,k}^{(i)}. \quad (336)$$

The factor of 1000 converts \$/MWh to \$/kWh.

Remark 7 (LMP role in the JSW scheme). *In the power sub-problem, $\lambda_{b,k}^{(i)}$ is the dual variable of the DC power-balance constraint at bus b , i.e. the locational marginal price (LMP) [MWh]; it reflects the marginal increase in generation cost for a 1 MW load increase at bus b . In the traffic sub-problem, the same value is used as an exogenous price signal after unit conversion. The JSW algorithm is a decentralized coordination scheme: the generation cost already appears in $C_{\text{gen}}^{(i)}$ and the station price payment is an internal transfer, not a double-count. In the tested cases, with non-negative generator marginal costs, no must-run constraints, and the adopted sign convention, LMPs are non-negative. In general, LMPs can be zero or negative under congestion, binding lower-generation limits, renewable spillage, or other model features.*

27.8.3 Convergence Criterion

$$\max_{s,k} \left| \pi_{s,k}^{(i+1)} - \pi_{s,k}^{(i)} \right| < \varepsilon_\pi. \quad (337)$$

27.8.4 Social Welfare Accounting

$$W^{(i)} = B_{\text{EV}}^{(i)} - C_{\text{gen}}^{(i)} - C_{\text{delay}}^{(i)}, \quad (338)$$

$$B_{\text{EV}}^{(i)} = \sum_{d \in \mathcal{D}} \text{WTP}_d \cdot x_d^{(i)}, \quad (339)$$

$$C_{\text{gen}}^{(i)} = \sum_{k \in \mathcal{T}} \text{OPF}_k^{(i)} \text{ objective}, \quad (340)$$

$$C_{\text{delay}}^{(i)} = \text{VOT} \cdot \sum_{d,r} x_{d,r}^{(i)} \cdot T_{r,k_d}^{(i)}. \quad (341)$$

27.8.5 Algorithm Summary

1. Initialise $\pi_{s,k}^{(0)} = \pi_0$ for all s, k .
2. For $i = 1, \dots, \text{max_iterations}$:
 - (a) Solve traffic LP with $\pi^{(i-1)}$ to get $x^{(i)}, P^{EV,(i)}$.
 - (b) For each k : solve DC-OPF; extract $\lambda_{b,k}^{(i)}$.
 - (c) Update $\pi^{(i)}$ via (336).
 - (d) Compute $W^{(i)}$, record history.
 - (e) Check (337); stop if satisfied.

27.9 Formulation D: Solver-Certified Dynamic Joint Optimisation

Formulation D is the non-heuristic model class for the next `ev_power_traffic` implementation step. It replaces capacity-aware greedy assignment, MSA-DUE, and LMP fixed-point iteration with a single mathematical program, or with a solver-certified equilibrium complementarity model. It is designed to answer the five requirements that are not fully covered by Formulations A–C:

1. road blockage, capacity derating, incident recovery, and spillback-aware road congestion;
2. EV charging and V2G discharge under battery and station limits;
3. elastic, time-varying, and departure-time-shifting travel demand;
4. comparison of user-benefit maximisation and social-welfare maximisation;
5. LP/QP/MILP/MCP solver execution with primal feasibility, dual bound, and optimality/MIP-gap certificates.

27.9.1 Additional Sets and Parameters

Let $\mathcal{C}$ be EV user classes and $\mathcal{P}_{odc}$ the route set for OD pair (o, d) and class c . The simulation horizon is discretised by $\mathcal{T}^D = \{0, \dots, K_D - 1\}$. In Formulation D this grid is the optimisation grid; it may be chosen equal to the CTM grid or a coarsened multiple of it.

Road disruptions are represented by exogenous availability and capacity profiles:

$$\gamma_{a,k} \in \{0, 1\}, \quad \text{availability of road link } a \text{ at step } k, \quad (342)$$

$$\phi_{a,k} \in [0, 1], \quad \text{capacity derating factor during incidents or work zones.} \quad (343)$$

$\gamma_{a,k} = 0$ models a complete closure; $0 < \phi_{a,k} < 1$ models a partial fault, lane closure, signal failure, or weather-induced capacity reduction. The effective road capacity is

$$\bar{q}_{a,k} = \gamma_{a,k} \phi_{a,k} q_a^{\max}. \quad (344)$$

The same idea applies to charging stations:

$$\bar{P}_{s,k}^{ch} = \psi_{s,k}^{ch} P_s^{ch,\max}, \quad \bar{P}_{s,k}^{dis} = \psi_{s,k}^{dis} P_s^{dis,\max}, \quad \bar{N}_{s,k}^{plug} = \psi_{s,k}^{plug} N_s^{plug}, \quad (345)$$

where $\psi \in [0, 1]$ allows charger outages, feeder derating, or communication failures to be modelled without changing the network topology.

The time-varying potential travel demand is $\bar{d}_{odc,k}$ [vehicles]. The outside-option utility $\Omega_{odc,k}$ is the value retained when a trip is cancelled or deferred beyond the optimisation horizon. The gross willingness to pay or trip benefit is $B_{odc,k}$ [\$/vehicle].

27.9.2 Decision Variables

Variable	Domain	Meaning
$h_{p,k} \geq 0$	veh	vehicles departing on route p at step k
$q_{odc,k}^{ser} \geq 0$	veh	served trips for OD/class/time demand
$q_{odc,k}^{out} \geq 0$	veh	cancelled or horizon-unserved trips
$z_{odc,\tau,k} \geq 0$	veh	trips requested at τ , departed at k
$n_{a,m,k} \geq 0$	veh	CTM occupancy of road cell m on link a
$y_{a,m,k} \geq 0$	veh/step	CTM inter-cell flow
$e_{v,k}$	kWh	battery energy of EV/session group v
$p_{v,k}^{ch}, p_{v,k}^{dis} \geq 0$	kW	charging and V2G discharge power
$\delta_{v,k}^{ch}, \delta_{v,k}^{dis} \in \{0, 1\}$	–	optional charge/discharge mode binaries
$P_{g,k}^g, \theta_{b,k}, f_{\ell,k}$	MW, rad, MW	DC-OPF generator, angle, and branch-flow variables
$\ell_{b,k}^p, \ell_{a,k}^t, \ell_v^e \geq 0$	load/veh/kWh	slack variables for load, traffic, and energy infeasibility

The variables $z_{odc,\tau,k}$ are needed only when departure-time choice is enabled. The variables $\delta_{v,k}^{ch}, \delta_{v,k}^{dis}$ are needed only when the model must forbid simultaneous charge and discharge exactly; omitting them gives a pure LP/QP relaxation that is often exact when round-trip efficiency is below one and both charge and discharge carry nonnegative costs.

27.9.3 Dynamic Travel Demand and Departure-Time Choice

For inelastic demand, every requested trip must either be served or be assigned to an explicit outside option:

$$\sum_{\tau \in \mathcal{T}^D: \tau \geq k} z_{odc,k,\tau} + q_{odc,k}^{out} = \bar{d}_{odc,k}, \quad \forall o, d, c, k. \quad (346)$$

Here $z_{odc,k,\tau}$ means demand requested at step k and actually departing at step τ . If departure-time shifting is disabled, set $z_{odc,k,\tau} = 0$ for $\tau \neq k$. The served departure volume at step τ is

$$q_{odc,\tau}^{ser} = \sum_{k \leq \tau} z_{odc,k,\tau}. \quad (347)$$

Route departure flows satisfy

$$\sum_{p \in \mathcal{P}_{odc}} h_{p,\tau} = q_{odc,\tau}^{ser}, \quad \forall o, d, c, \tau. \quad (348)$$

Thus travel demand is not forced to be fully satisfied unless the user sets $q_{odc,k}^{out} = 0$ or assigns an outside-option penalty large enough to make cancellation economically dominated. This is important for emergency road closures, feeder outages, or insufficient charging capacity, where mandatory service can make the model artificially infeasible.

27.9.4 Blockage-Aware CTM Constraints

For each road link a , cell m , and step k , Formulation D uses the CTM as linear inequalities rather than as a forward simulation. The state conservation law is

$$n_{a,m,k+1} = n_{a,m,k} + y_{a,m-1,k} \mathbf{1}[m > 0] + y_{a,k}^{in} \mathbf{1}[m = 0] - y_{a,m,k} \mathbf{1}[m < M_a - 1] - y_{a,k}^{out} \mathbf{1}[m = M_a - 1]. \quad (349)$$

The sending and receiving envelopes are written as solver constraints:

$$y_{a,m,k} \leq \Delta t_D \bar{q}_{a,k}, \quad \forall a, m, k, \quad (350)$$

$$y_{a,m,k} \leq \frac{v_a^f \Delta t_D}{\delta_a} n_{a,m,k}, \quad \forall a, m, k, \quad (351)$$

$$y_{a,m,k} \leq \frac{w_a \Delta t_D}{\delta_a} (\gamma_{a,k} N_a^{jam} - n_{a,m+1,k}), \quad \forall a, m < M_a - 1, k, \quad (352)$$

$$0 \leq n_{a,m,k} \leq \gamma_{a,k} N_a^{jam}, \quad \forall a, m, k. \quad (353)$$

When $\gamma_{a,k} = 0$, Eqs. (350) and (353) force zero flow and zero storage on the closed link. For an incident that closes only future inflow while allowing trapped vehicles to drain, replace (353) by $0 \leq n_{a,m,k} \leq N_a^{jam}$ and apply $\gamma_{a,k}$ only to the sending capacity of entering cells.

Route departure variables inject flow into the first link of each path:

$$y_{a,k}^{in} = \sum_{p: a=\text{first}(p)} h_{p,k} + \sum_{b: \text{to}(b)=\text{from}(a)} y_{b \rightarrow a,k}, \quad (354)$$

with node transfer variables $y_{b \rightarrow a,k}$ satisfying merge/diverge capacity, turning-ratio, and FIFO constraints. These node constraints are linear for fixed priorities and fixed route incidence. If signal phase decisions or lane-use decisions are optimised, the corresponding activation variables make the traffic part MILP.

27.9.5 EV Battery, Charging, and V2G

Let v index a homogeneous EV session group generated by a route, station, and arrival/departure window. The battery state evolves as

$$e_{v,k+1} = e_{v,k} - E_{v,k}^{drive} + \eta_v^{ch} p_{v,k}^{ch} \Delta t_D - \frac{p_{v,k}^{dis}}{\eta_v^{dis}} \Delta t_D, \quad \forall v, k. \quad (355)$$

The traction term is

$$E_{v,k}^{drive} = \sum_{a \in p(v)} \epsilon_{a,c(v)} L_a \chi_{v,a,k}, \quad (356)$$

where $\epsilon_{a,c}$ is the drive-energy coefficient [kWh/(veh km)] and $\chi_{v,a,k}$ is the flow or occupancy of group v on link a at step k . A path-level approximation replaces (356) by the precomputed route energy $\sum_{a \in p} \epsilon_{a,c} L_a$.

Battery limits and terminal service requirements are

$$\underline{E}_{v,k} \leq e_{v,k} \leq \bar{E}_{v,k}, \quad \forall v, k, \quad (357)$$

$$e_{v,k_v^{dep}} + \ell_v^e \geq E_v^{target}, \quad \forall v. \quad (358)$$

Charging and discharge power are restricted to the connection window:

$$0 \leq p_{v,k}^{ch} \leq A_{v,k} \bar{p}_v^{ch}, \quad 0 \leq p_{v,k}^{dis} \leq A_{v,k} \bar{p}_v^{dis}, \quad \forall v, k, \quad (359)$$

where $A_{v,k} = 1$ if the session is plugged in at step k and zero otherwise. Exact no-simultaneous-charge/discharge logic is

$$p_{v,k}^{ch} \leq A_{v,k} \bar{p}_v^{ch} \delta_{v,k}^{ch}, \quad (360)$$

$$p_{v,k}^{dis} \leq A_{v,k} \bar{p}_v^{dis} \delta_{v,k}^{dis}, \quad (361)$$

$$\delta_{v,k}^{ch} + \delta_{v,k}^{dis} \leq 1, \quad \delta_{v,k}^{ch}, \delta_{v,k}^{dis} \in \{0, 1\}. \quad (362)$$

Equations (355)–(362) allow both normal charging and reverse discharge. Negative net station power is therefore a decision outcome, not a post-processing convention.

Station aggregate limits are

$$\sum_{v:s(v)=s} p_{v,k}^{ch} \leq \bar{P}_{s,k}^{ch}, \quad \forall s, k, \quad (363)$$

$$\sum_{v:s(v)=s} p_{v,k}^{dis} \leq \bar{P}_{s,k}^{dis}, \quad \forall s, k, \quad (364)$$

$$\sum_{v:s(v)=s} A_{v,k} \leq \bar{N}_{s,k}^{plug}, \quad \forall s, k. \quad (365)$$

If plug assignment is optimised rather than fixed from arrival windows, $A_{v,k}$ becomes binary and the station submodel is MILP.

27.9.6 Power-Network Constraints

For a DC-OPF coupling, station net power is

$$P_{s,k}^{EV} = 10^{-3} \sum_{v:s(v)=s} (p_{v,k}^{ch} - p_{v,k}^{dis}) \quad [\text{MW}]. \quad (366)$$

The nodal power balance is

$$\sum_{g \in \mathcal{G}_b} P_{g,k}^g - P_{b,k}^d - \sum_{s:b(s)=b} P_{s,k}^{EV} + \ell_{b,k}^p = \sum_{\ell \in \delta(b)} \sigma_{b\ell} f_{\ell,k}, \quad \forall b, k. \quad (367)$$

Branch and generator limits are

$$f_{\ell,k} = b_{\ell} (\theta_{i(\ell),k} - \theta_{j(\ell),k}), \quad \forall \ell, k, \quad (368)$$

$$-\bar{F}_{\ell} \leq f_{\ell,k} \leq \bar{F}_{\ell}, \quad \forall \ell, k, \quad (369)$$

$$\underline{P}_g^g \leq P_{g,k}^g \leq \bar{P}_g^g, \quad \forall g, k. \quad (370)$$

If voltage magnitude or radial distribution losses must be represented, Eqs. (367)–(370) can be replaced by LinDistFlow constraints. Full AC constraints make the model nonlinear; IPOPT can solve the continuous NLP relaxation, but integer route, plug, or charge/discharge decisions still require a MINLP strategy or an outer branch-and-bound solver.

27.9.7 Objective 1: User-Benefit Maximisation

User-benefit maximisation represents a platform or behavioural model that prioritises EV users' private utility. A linear form is

$$\begin{aligned} \max \quad U^{user} = & \sum_{o,d,c,k} B_{odc,k} q_{odc,k}^{ser} - \text{VOT}_c \sum_{p,k} h_{p,k} T_{p,k} - \sum_{v,k} \pi_{s(v),k} (p_{v,k}^{ch} - p_{v,k}^{dis}) \Delta t_D \\ & - \sum_{o,d,c,k} \Omega_{odc,k} q_{odc,k}^{out} - \sum_{v,k} c_v^{deg} p_{v,k}^{dis} \Delta t_D - M_e \sum_v \ell_v^e. \end{aligned} \quad (371)$$

Here $T_{p,k}$ may be either a CTM-derived linear travel-time proxy, the sum of link occupancies $\sum_{a,m,k} n_{a,m,k} \Delta t_D$, or a convex piecewise-linear delay function. The private electricity payment is included because it affects the user's route, station, charging, and V2G decisions. In a pure user-benefit analysis, generation cost is not subtracted unless it is passed through to users as price.

27.9.8 Objective 2: Social-Welfare Maximisation

Social welfare counts real resource costs and treats electricity payments as internal transfers:

$$\begin{aligned} \max \quad W = & \sum_{o,d,c,k} B_{odc,k} q_{odc,k}^{ser} - \text{VOT}_c \sum_{a,m,k} n_{a,m,k} \Delta t_D - \sum_{g,k} C_g(P_{g,k}^g) \Delta t_D \\ & - \sum_{v,k} c_v^{deg} p_{v,k}^{dis} \Delta t_D - M_{out} \sum_{o,d,c,k} q_{odc,k}^{out} - M_p \sum_{b,k} \ell_{b,k}^p - M_e \sum_v \ell_v^e - M_t \sum_{a,k} \ell_{a,k}^t. \end{aligned} \quad (372)$$

If generator costs $C_g(\cdot)$ are linear or piecewise linear, the model is LP/MILP. If $C_g(P) = c_{2,g}P^2 + c_{1,g}P + c_{0,g}$ with $c_{2,g} \geq 0$, the continuous relaxation is convex QP; with mode, plug, or signal binaries it becomes MIQP.

Remark 8 (User optimum vs. social optimum). *The user-benefit objective (371) can prefer cheap private routes and stations even when they create congestion, grid congestion, or high generation cost elsewhere. The social-welfare objective (372) internalises these costs. Comparing the two solves gives the efficiency loss of selfish or price-responsive behaviour under the same physical constraints:*

$$\text{PoA}^D = \frac{W^{SO}}{W^{UE \text{ or } UB}},$$

when both welfare values are evaluated with Eq. (372). This comparison is meaningful because the numerator and denominator use the same CTM and power model.

27.9.9 Solver-Certified User Equilibrium

If the study requires Dynamic User Equilibrium rather than centralised user-benefit maximisation, route choice is written as variational inequality or complementarity conditions. Let $G_{p,k}(x, p^{ch}, p^{dis})$ be the generalised private cost of route p for the relevant OD/class/departure time, and let $\mu_{odc,k}$ be the minimum perceived cost. Wardrop equilibrium is

$$G_{p,k} - \mu_{odc,k} \geq 0, \quad \forall p \in \mathcal{P}_{odc,k}, \quad (373)$$

$$h_{p,k} \geq 0, \quad \forall p, k, \quad (374)$$

$$h_{p,k} (G_{p,k} - \mu_{odc,k}) = 0, \quad \forall p \in \mathcal{P}_{odc,k}. \quad (375)$$

For separable monotone link costs, the UE can also be solved by the convex Beckmann program. For the full EV-power-traffic model with station capacity, V2G, and grid coupling, the complementarity system is assembled as MCP or converted to MILP with big- M and binary activation:

$$G_{p,k} - \mu_{odc,k} \leq M_{p,k}^{UE} (1 - b_{p,k}), \quad (376)$$

$$h_{p,k} \leq M_{p,k}^{flow} b_{p,k}, \quad (377)$$

$$b_{p,k} \in \{0, 1\}. \quad (378)$$

This is solver-certified in the sense that the MCP residual, or the MILP feasibility and MIP gap, is reported directly by the solver. It is not an MSA iteration and does not rely on empirical convergence of successive averages.

27.9.10 Problem Class and Solver Selection

The exact solver class is determined by the selected modelling features:

Features	Problem class	Recommended solver path
Linear CTM, linear costs, fixed plug windows	LP	HiGHS or native LP
Quadratic generation cost, no binaries	convex QP	QP-capable backend
Integer route/plugin/mode/signal decisions	MILP/MIQP	HiGHS/SCIP/Gurobi/native B&C
Wardrop complementarity, no binaries	MCP/QP-KKT	MCP solver or KKT system
AC power flow only, continuous variables	NLP	IPOPT local solve
AC power flow plus integer decisions	MINLP	outer MIP/MINLP solver; IPOPT only for

IPOPT is therefore not the default solver for Formulation D. It is appropriate only when the assembled problem is a continuous nonlinear program, such as an AC-OPF refinement with fixed route and charging-mode decisions. IPOPT does not provide a global MIP gap for integer route, station, plug, V2G-mode, or signal decisions. For the requested non-heuristic execution, the primary implementation should first assemble LP/QP/MILP/MIQP models; IPOPT can be added later as an AC feasibility polishing or continuous-NLP validation layer.

27.9.11 Solver Algorithm

The solver-certified execution pipeline is:

1. **Build the time-expanded network.** Expand road links into CTM cells over $\mathcal{T}^D$, attach route incidence, station arrival windows, charging buses, demand profiles $\bar{d}_{odc,k}$, and disruption profiles $\gamma_{a,k}, \phi_{a,k}, \psi_{s,k}$.
2. **Choose the behavioural mode.** Select `UserBenefitMax`, `SocialWelfareMax`, or `UserEquilibriumMCP`. The first two are optimisation objectives; the last adds complementarity or big- M equilibrium constraints.
3. **Assemble one model.** Add Eqs. (346)–(370) and one of Eqs. (371), (372), or (373)–(378). Use piecewise-linear epigraph variables for nonlinear delay or generator-cost terms when a pure LP/MILP is desired.
4. **Solve with a mathematical-programming backend.** Run the LP/QP/MILP/MIQP/MCP solver with configured feasibility tolerance, time limit, node limit, and MIP-gap target. No route-flow update, price smoothing, greedy repair, or MSA step is part of the certified solve.
5. **Validate the certificate.** Accept the result only if the solver status is feasible/optimal according to the selected tolerance. Record primal maximum violation, dual bound or best bound, incumbent objective, relative MIP gap, node count, and solve time. For LP/QP, require a valid optimal basis or KKT residual; for MCP, require complementarity residual below the tolerance.
6. **Decode and cross-check.** Convert $h, n, y, e, p^{ch}, p^{dis}$ into route assignments, CTM link states, station loads, SOC trajectories, V2G injections, and power-system dispatch. Optionally run the existing AC power-flow validation as an ex-post check.

Remark 9 (Heuristic methods retained only as baselines). *CapacityAwareGreedy*, *MSA-DUE*, *greedy smart charging*, and *LMP-price smoothing* remain useful for fast simulation, warm starts, and regression baselines. They should not be used to claim optimality, equilibrium, or social-welfare maximisation when Formulation D is the selected model. Certified numerical results must come from the solver certificate of the assembled LP/QP/MILP/MIQP/MCP.

27.10 Implemented Module Structure

File	Contents
include/hacdcpf/ev_power_traffic.hpp	umbrella include
include/hacdcpf/ev_power_traffic/types.hpp	all data structures (TrafficLink, TrafficLinkCell, CTMLinkState, CTMSimulation, CTMDUEResult, RouteAlternative, EVPowerTrafficOptions, CTMOptions, EVPowerTrafficProblem, EVPowerTrafficSolution, simulate_ev_power_traffic() (Formulation A), simulate_ev_power_traffic_ctm_due() (Formulation B), apply_ev_station_loads() (Formulation C))
include/hacdcpf/ev_power_traffic/options.hpp	EVPowerTrafficOptions, CTMOptions
include/hacdcpf/ev_power_traffic/simulation.hpp	EVPowerTrafficProblem, EVPowerTrafficSolution, simulate_ev_power_traffic() (Formulation A), simulate_ev_power_traffic_ctm_due() (Formulation B), apply_ev_station_loads() (Formulation C)
include/hacdcpf/ev_power_traffic/joint_social_welfare.hpp	JSW types and solve_joint_social_welfare() (Formulation D)
src/ev_power_traffic/evpt_internal.hpp	private helpers: BPR, cost, capacity, L
src/ev_power_traffic/ev_power_traffic_simulation.cpp	Formulation A: greedy/logit/DSP, session-based
src/ev_power_traffic/assignment_lp.cpp	Formulation A: system-optimal LP/MILP
src/ev_power_traffic/joint_social_welfare.cpp	Formulation A: iterative JSW algorithm
src/ev_power_traffic/ctm_propagation.cpp	Formulation B: CTM forward pass, MSA update
src/ev_power_traffic/ctm_joint_welfare.cpp	Formulation C: CTM-DUE + DC-OPF
src/ev_power_traffic/joint_optimizer.cpp	Formulation D: LP/QP/MILP/MCP j

Remark 10 (Implementation boundary). *The `joint_optimizer.cpp` module implements the single-shot LP/MILP assembly for Formulation D. It assembles the mathematical program of Section 27.9, calls the selected solver backend (NativeDualSimplex for LP, HiGHS or NativeB&C for MILP), and populates certificate fields `proven_optimal`, `best_bound`, `mip_gap`, and `solve_time_sec` in `JointOptimizerResult`. It does not call greedy assignment, MSA update, or price-smoothing routines. See Section 27.18 for numerical validation.*

27.11 Numerical Case Study 1: Two-Route, Five-Vehicle Network

This case corresponds to `tests/test_ev_power_traffic_report.cpp` and `tests/test_ev_power_traffic.cpp`.

27.11.1 Network Parameters

Route	Links	Free-flow time	Station	Plugs	Capacity
1	11, 12	0.25+0.25 = 0.50 h	101	2	40 kW
2	21, 22	0.50+0.50 = 1.00 h	102	3	60 kW

All links: 20 veh/h capacity. $\Delta t = 1$ h. Demand: 5 EVs at step 0, each requesting 20 kWh at 20 kW/veh. Generation: 2 MW slack generator. $VOT = 1$ \$/veh·h, $\omega_s = 1$, $\pi = 0.30$ \$/kWh (default), $\eta = 1$.

27.11.2 Scenario A: Deterministic Shortest Path

All 5 vehicles choose Route 1 (free-flow cost $0.5 + 0.30 \times 20 = 6.5$ \$/veh vs. Route 2: $1.0 + 0.30 \times 20 = 7.0$ \$/veh). Station 101 can serve only 40 kWh (2 plugs $\times$ 20 kW $\times$ 1 h); 60 kWh is unserved.

Step	P101 [kW]	Q101 [veh]	W101 [h]	Delivered [kWh]	Unserved [kWh]
0	0	0	0.000	0	0
1	40	0	0.000	40	60
2	0	5	0.875	0	0
3	0	3	0.525	0	0
4	0	1	0.175	0	0
Total				40 kWh	60 kWh

Total travel time: 2.5015 veh·h.

Remark 11 (Queue reporting convention). *Queue values $Q_{s,k}$ reflect the station state as observed in the simulation output. In Scenario A, all 5 vehicles are routed to station 101 (DSP does not enforce station capacity), creating a single session with $x_{d,r} = 5$. Only 40 kWh can be dispatched at step 1 (station limit); the remaining 60 kWh is unserved. The queue values at subsequent steps accumulate vehicles that could not be admitted within the dwell window. Values are reported at the start of each step, before the step's admission decisions.*

27.11.3 Scenario B: Capacity-Aware Greedy

Station 101 can serve at most $40/20 = 2$ vehicles. Assignment: $x_1 = 2$, $x_2 = 3$, $u = 0$. Both stations fully utilised; zero unserved energy.

Route	Vehicles	P [kW]	Travel [h]	Delivered [kWh]	Unserved [kWh]
1	2	40	0.5000	40	0
2	3	60	1.0001	60	0
Total	5		4.0002 veh·h	100 kWh	0 kWh

27.11.4 Scenario C: System-Optimal LP

The LP:

$$\begin{aligned}
\min \quad & c_1^{\text{ff}} x_1 + c_2^{\text{ff}} x_2 + 10^6 u \\
\text{s.t.} \quad & x_1 + x_2 + u = 5 \\
& 20 x_1 \leq 40, \quad 20 x_2 \leq 60 \\
& x_1, x_2, u \geq 0.
\end{aligned}$$

Station capacity binds at $x_1 = 2$; demand balance gives $x_2 = 3$, $u = 0$. Reported LP objective = **34.0** ($= 2 \times 6.5 + 3 \times 7.0$; free-flow travel plus energy cost $\omega_s \pi e = 0.30 \times 20 = 6.0\$/\text{veh}$ per route). Solver: NativeDualSimplex. Status: proven_optimal=true, mip_gap=0.

27.12 Numerical Case Study 2: Joint Social Welfare, 2-Bus Grid

This case corresponds to `tests/test_ev_power_traffic_joint_opt.cpp`.

27.12.1 Network Parameters

Power grid (2-bus system).

Component	Value	Notes
Bus 1 (SLACK)		reference bus
Bus 2 (PQ)		load bus
Generator G1	$c_1 = 10\$/\text{MWh}$, $\bar{P} = 2 \text{ MW}$	cheap baseload
Generator G2	$c_1 = 50\$/\text{MWh}$, $\bar{P} = 0.5 \text{ MW}$	expensive peaker
Branch 1-2	$x = 0.1 \text{ pu}$, $\bar{P} = 0.5 \text{ MW}$	flow-limited tie
Station 101 (bus 1)	40 kW, 2 plugs	cheap bus
Station 102 (bus 2)	60 kW, 3 plugs	expensive bus

Traffic/EV. Same 4-node, 2-route topology as Case Study 1. One demand: 5 EVs, WTP = \$50/vehicle. $K = 6$ steps, $\Delta t = 1 \text{ h}$, $\alpha = 0.6$, $\varepsilon_\pi = 10^{-4}$, $\pi^{(0)} = 0.03\$/\text{kWh}$.

27.12.2 Scenario 1: Fixed Prices (No OPF Coupling)

With `use_dcopf_prices=false`, the algorithm terminates after one iteration.

Quantity	Value
B_{EV}	\$250.0000
C_{gen}	\$0.0000
C_{delay}	\$4.0002
W	\$245.9998
Served / total	5 / 5 vehicles
Delivered energy	100.0 kWh
Iterations	1 (converged)

27.12.3 Scenario 2: LMP-Coupled Joint Optimisation

With `use_dcopf_prices=true`, the algorithm converges after 7 iterations ($\Delta\pi^{(7)} = 4.92 \times 10^{-5} < 10^{-4}$).

Convergence history.

Iter	W [\$]	B_{EV} [\$]	C_{gen} [\$]	C_{dly} [\$]	$\Delta\pi$ [\$/kWh]	OPF
1	244.9998	250.0000	1.0000	4.0002	1.20e-02	ok
2	244.9998	250.0000	1.0000	4.0002	4.80e-03	ok
3	244.9998	250.0000	1.0000	4.0002	1.92e-03	ok
4	244.9998	250.0000	1.0000	4.0002	7.68e-04	ok
5	244.9998	250.0000	1.0000	4.0002	3.07e-04	ok
6	244.9998	250.0000	1.0000	4.0002	1.23e-04	ok
7	244.9998	250.0000	1.0000	4.0002	4.92e-05	ok

Final assignment.

Demand	Route	Vehicles	Gen. cost [\$]
1	1	2	0.7016
1	2	3	1.2017

Converged prices. Branch 1-2 is uncongested (0.1 MW EV load $\ll$ 0.5 MW limit); both buses reach the same LMP, set by the marginal generator G1: $\lambda^* = 10\$/\text{MWh} = 0.010\$/\text{kWh}$. The price update converges to:

Step k	0	1	2	3	4	5
$\pi_{101,k}^*$ [\$/kWh]	0.01003	0.01003	0.01003	0.01003	0.01003	0.01003
$\pi_{102,k}^*$ [\$/kWh]	0.01003	0.01003	0.01003	0.01003	0.01003	0.01003

The slight offset from the theoretical 0.010\$/kWh is due to the finite step-size $\alpha = 0.6$ damping in the price update and the $\varepsilon_\pi = 10^{-4}$ convergence tolerance.

Interpretation. The generation cost of \$1.00 corresponds to G1 dispatched at $P_{G1} = 0.1$ MW for one step: $c_1 \times P \times \Delta t = 10 \times 0.1 \times 1 = \1.00 . Consumer surplus at convergence: $B_{EV} - \text{price paid} = 250 - 2.0033 = \248.9967 .

27.13 Numerical Case Study 3: Large-Scale Fleet Routing

This case corresponds to `tests/test_ev_power_traffic_largescale.cpp`.

27.13.1 Setup

Parameter	Value
Power grid	IEEE 118-bus (<code>case118.m</code>)
Charging stations	buses 1–100; 15 MW each (300 plugs $\times$ 50 kW)
Traffic grid	10 $\times$ 10 Cartesian, 100 nodes, 180 directed links
Link parameters	$t^0 = 0.25$ h, $c = 2000$ veh/h
Routes	5 corridors $\times$ 2 alternatives = 10 routes
Fleet demands	5 corridors $\times$ 4 departure steps = 20 demands
Vehicles per demand	500 $\Rightarrow$ 10,000 vehicles total
Per-vehicle request	30 kWh at 50 kW, 1 dwell step
Expected delivery	10,000 $\times$ 30 = 300,000 kWh
Steps / horizon	$K = 12$, $\Delta t = 1$ h

27.13.2 Scenario A: CapacityAwareGreedy

The greedy policy splits each 500-vehicle fleet 300/200 between the two corridor alternatives. Peak loads reach 9,000 kW at left-column mid-stations and 6,000 kW at right-column mid-stations.

Metric	Value
Solve time	2.7 ms
Served / total	10,000 / 10,000
Delivered energy	300,000 kWh (0 unserved)
Total travel time	25,001.29 veh·h
Peak left-column station	9,000 kW; queue 1,200 veh; wait 0.56 h
Peak right-column station	6,000 kW; queue 400 veh; wait 0.19 h

27.13.3 Scenario B: System-Optimal LP

The LP concentrates all 500 vehicles per demand onto Route A (cheaper free-flow cost); right-column stations are idle. The LP is certified optimal with MIP gap 0 by HiGHS.

Metric	Value
Solve time	10.0 ms (HiGHS LP)
Served / total	10,000 / 10,000
Delivered energy	300,000 kWh (0 unserved)
Total travel time	25,014.65 veh·h
LP objective	115,000
Proven optimal	yes (MIP gap = 0)
Peak left-column station	15,000 kW; queue 2,000 veh; wait 0.93 h

Remark. The LP objective is minimised under *free-flow* costs. Concentrating 10,000 vehicles on fewer links increases actual BPR congestion, so the LP solution has slightly higher total travel time (25,014.65 vs. 25,001.29 veh·h) than the greedy split. This is expected: the LP objective uses free-flow costs whereas the travel time report uses congestion-adjusted BPR values.

Scope of numerical validation. Sections 27.11–27.13 validate **Formulation A**: BPR delay, free-flow precomputed arrival steps, fixed-profile station capacity constraints, the NativeDual-Simplex LP solver, and the iterative LMP-price coordination algorithm. They do *not* exercise CTM-based traffic dynamics (Formulation B) or jointly optimised smart-charging with DC-OPF coupling (Formulation C). Formulation B is validated separately in Section 27.15. Formulation C (CTM-DUE + DC-OPF price feedback) is validated in Section 27.17.

27.14 Formulation B: CTM Propagation and Dynamic User Equilibrium

27.14.1 Cell Transmission Model

Following Daganzo (1994), each directed link $a \in \mathcal{A}$ is discretised into M_a cells of equal length $\delta_a = L_a/M_a$ [km], with a dedicated CTM time step Δt_{ctm} satisfying the CFL stability condition:

$$\Delta t_{\text{ctm}} \leq \frac{\delta_a}{v_a^f}, \quad \forall a \in \mathcal{A}. \quad (379)$$

The number of cells is computed as

$$M_a = \max\left(1, \left\lfloor \frac{L_a}{v_a^f \Delta t_{\text{ctm}}} \right\rfloor\right), \quad (380)$$

or overridden by `CTMOptions::n_cells_per_link` when positive. When `dt_ctm_hr=0` the implementation auto-selects $\Delta t_{\text{ctm}} = 0.9 \cdot \min_a(\delta_a/v_a^f)$, which satisfies CFL for all links. If a user-specified Δt_{ctm} is so large that $L_a/(v_a^f \Delta t_{\text{ctm}}) < 1$ for some link, then M_a is clamped to 1 and the CFL condition (379) cannot be restored by cell splitting; the implementation proceeds with an approximate single-cell propagation for that link.

27.14.2 Triangular Fundamental Diagram

The triangular fundamental diagram is parametrised by free-flow speed v_a^f [km/h], backward wave speed w_a [km/h], and capacity $q_a^{\max}$ [veh/h]. Jam density and jam occupancy per cell are

$$k_a^{\text{jam}} = \frac{q_a^{\max}(v_a^f + w_a)}{v_a^f w_a}, \quad N_a^{\text{jam}} = k_a^{\text{jam}} \delta_a. \quad (381)$$

When `TrafficLink::jam_vehicles` is zero, the backward wave speed is inferred from $k_a^{\text{jam}} = 2q_a^{\text{max}}/v_a^f$ (symmetric triangular FD); otherwise $w_a = q_a^{\text{max}}/(k_a^{\text{jam}} - q_a^{\text{max}}/v_a^f)$.

27.14.3 Sending and Receiving Functions

At step k the vehicle flow that cell m can *send* downstream and the flow that cell $m+1$ can *receive* are:

$$S_{a,m,k} = \Delta t_{\text{ctm}} \cdot \min\left(\frac{n_{a,m,k} v_a^f}{\delta_a}, q_a^{\text{max}}\right), \quad (382)$$

$$R_{a,m,k} = \Delta t_{\text{ctm}} \cdot \min\left(q_a^{\text{max}}, \frac{w_a \max(0, N_a^{\text{jam}} - n_{a,m,k})}{\delta_a}\right). \quad (383)$$

27.14.4 Cell Update

The inter-cell flow is

$$y_{a,m,k} = \min(S_{a,m,k}, R_{a,m+1,k}), \quad m = 0, \dots, M_a - 2, \quad (384)$$

and the cell occupancy evolves as

$$n_{a,m,k+1} = n_{a,m,k} + y_{a,m-1,k} \cdot \mathbf{1}[m > 0] - y_{a,m,k} \cdot \mathbf{1}[m < M_a - 1], \quad n_{a,m,0} = 0. \quad (385)$$

The indicator functions subsume three distinct boundary cases:

$$\begin{aligned} m = 0 : \quad & n_{a,0,k+1} = n_{a,0,k} + y_{a,k}^{\text{in}} - y_{a,0,k}, \\ 0 < m < M_a - 1 : \quad & n_{a,m,k+1} = n_{a,m,k} + y_{a,m-1,k} - y_{a,m,k}, \\ m = M_a - 1 : \quad & n_{a,M_a-1,k+1} = n_{a,M_a-1,k} + y_{a,M_a-2,k} - y_{a,k}^{\text{out}}, \end{aligned}$$

where $y_{a,k}^{\text{in}}$ is the external inflow from origin demand injection or upstream node, and $y_{a,k}^{\text{out}}$ is the exit flow to the downstream node.

27.14.5 Commodity-Specific Cell Occupancy

With `route_specific_cells=true` (default), each cell stores per-route occupancy $n_{a,m,k}^r$ alongside the aggregate $n_{a,m,k}$. At diverge nodes, vehicles are distributed to downstream links according to binary turning indicators $\theta_{a \rightarrow v \rightarrow b}^r \in \{0, 1\}$ extracted from route topologies. The per-route shares are renormalised to sum to $n_{a,m,k}$ after each update step, maintaining aggregate consistency. This renormalisation is an engineering correction: per-route occupancy conservation is *approximate* rather than an exact independent commodity conservation law. Exact commodity conservation would require per-route sending/receiving functions and independent per-route cell updates, which are not implemented.

27.14.6 Node Model

At node v with incoming link a and outgoing links b , the exit flow from the last cell of a is allocated proportionally:

$$f_{a \rightarrow b,k} = S_{a,M_a-1,k} \cdot \frac{\text{demand}_{a \rightarrow b}}{\text{total_demand}_a} \cdot \sigma_k, \quad (386)$$

where $\text{demand}_{a \rightarrow b} = \sum_r n_{a,M_a-1,k}^r \theta_{a \rightarrow v \rightarrow b}^r$ and σ_k is a scale factor that ensures no outgoing link receives more than its receiving capacity.

Remark 12 (Proportional node rule). *Equation (386) is a proportional demand-scaling rule. It preserves route-direction information and respects downstream receiving capacities but is not a full priority-based CTM node model; it does not represent merge priority, FIFO diverge behaviour, lane groups, or signal timing.*

27.14.7 Experienced Link Travel Time

The experienced travel time on link a at step k is estimated via Little's law:

$$T_{a,k}^{\text{CTM}} = \begin{cases} \frac{N_{a,k} \Delta t_{\text{ctm}}}{\text{exit}_{a,k}} & \text{if } \text{exit}_{a,k} > 0, \\ L_a / v_a^f & \text{otherwise,} \end{cases} \quad (387)$$

where $N_{a,k} = \sum_m n_{a,m,k}$ and $\text{exit}_{a,k}$ is the exit flow from the last cell. Route experienced travel time is the sum over links: $T_{r,k}^{\text{CTM}} = \sum_{a \in r} T_{a,k}^{\text{CTM}}$.

Remark 13 (Instantaneous approximation). *Equation (387) is an instantaneous Little's-law estimate, not the exact experienced travel time for vehicles entering at step κ . The exact experienced time requires tracking cumulative entry and exit counts and inverting the exit condition. The instantaneous approximation is adequate for near-equilibrium flows but may be less accurate under severe transient congestion.*

27.14.8 Method of Successive Averages (MSA) for DUE

Given experienced travel times from the CTM forward pass, the auxiliary assignment is an all-or-nothing (AoN) assignment to the cheapest route per OD pair, or a logit split when `DUEOptions::logit_theta > 0`.

The **generalized route cost** used for route comparison combines the CTM travel time with a normalised charging-cost term:

$$g_r(k) = T_{r,k}^{\text{CTM}} + \frac{\omega_E}{\text{VOT}} \sum_{s \in S_r} \pi_{s,k} E_{r,s}^{\text{req}}, \quad (388)$$

where ω_E is `station_energy_cost_weight`, VOT is the value-of-time [\$/h], $\pi_{s,k}$ [\$/kWh] is the station price at step k , and $E_{r,s}^{\text{req}}$ [kWh/veh] is the energy requested at stop s on route r . With $\pi_{s,k} = 0$ (or $\omega_E = 0$) the generalized cost reduces to travel time, recovering the standard CTM-DUE. Station prices set by Formulation C's outer loop are propagated through this equation, creating a behavioural coupling between electricity prices and route choice.

The MSA update is

$$x^{(i+1)} = (1 - s_i) x^{(i)} + s_i x^{\text{aux}}, \quad s_i = \begin{cases} 1/i & \text{msa_fixed_step} = 0, \\ \text{msa_fixed_step} & \text{otherwise.} \end{cases} \quad (389)$$

27.14.9 Wardrop Relative Gap

The stopping criterion uses the relative Wardrop gap:

$$\varepsilon^{\text{gap}} = \frac{\text{TSTT} - \text{MSTT}}{\text{MSTT}} \geq 0, \quad (390)$$

where $\text{TSTT} = \sum_{d,r} x_{d,r} T_{r,k_d}^{\text{CTM}}$ is the total system travel time and $\text{MSTT} = \sum_d q_d \cdot \min_r T_{r,k_d}^{\text{CTM}}$ is the minimum achievable travel time if all vehicles used their cheapest route. Convergence is declared when $\varepsilon^{\text{gap}} < \text{DUEOptions::convergence_tol}$ (default 10^{-3}).

Remark 14 (MSA convergence). *The MSA procedure is an iterative heuristic for approaching a DUE. Convergence of MSA to DUE is not guaranteed for arbitrary dynamic loading conditions, approximate travel-time estimates, or logit auxiliary assignments. The relative gap serves as an empirical diagnostic; convergence has been validated by the regression tests but may not hold for all network configurations.*

27.14.10 CTM Time Step vs. Simulation Time Step

The CTM time step Δt_{ctm} is generally finer than the simulation time step Δt . The CTM horizon contains

$$T_{\text{ctm}} = T_{\text{sim}} \cdot R, \quad R = \text{round}(\Delta t / \Delta t_{\text{ctm}}) \quad (391)$$

steps. A demand with simulation departure step k_d is injected at CTM step $k_d \cdot R$. The `auxiliary_assignment` and `wardrop_relative_gap` functions index `route_flow` at $k_d \cdot R$ accordingly.

27.14.11 Formulation B+ Extensions: Session Synthesis, Dispatch, and PoA

Following DUE convergence, `simulate_ev_power_traffic_ctm_due()` performs three additional steps that bring Formulation B to parity with Formulation A for charging analysis (hereafter *Formulation B+*):

Session synthesis. Each (demand d , route r , stop s) triple with positive DUE flow $x_{d,r} > 0$ yields one `EVChargingSession`. The station arrival step is computed from the *CTM-derived* experienced travel time T_{r,k_d}^{CTM} rather than free-flow time, making it congestion-aware:

$$k_{d,r}^{\text{arr}} = k_d + \lceil T_{r,k_d}^{\text{CTM}} / \Delta t \rceil. \quad (392)$$

Smart-charging dispatch. Each session is dispatched with the same greedy heuristic as Formulation A (Eqs. (326)–(330)): charge at maximum feasible power within the dwell window until E^{req} is met; discharge via V2G only when the station price exceeds π^{high} . Station capacity headroom is shared across sessions at the same station and step.

Station queue model. The sessions' arrival schedule drives the queue dynamics of Eqs. (331)–(335), yielding $q_{s,k}$, $m_{s,k}$, and $W_{s,k}$ over the simulation horizon.

Price-of-anarchy benchmark. When `DUEOptions::compute_system_optimal_benchmark=true`, the DUE total system travel time (TSTT)

$$\text{TSTT}^{\text{DUE}} = \sum_d \sum_{r \in \mathcal{R}_d} x_{d,r} T_{r,k_d}^{\text{CTM}} \quad (393)$$

is compared with the Formulation A system-optimal LP TSTT TSTT^{SO} to yield the approximate price of anarchy:

$$\text{PoA} \approx \frac{\text{TSTT}^{\text{DUE}}}{\text{TSTT}^{\text{SO}}}. \quad (394)$$

The two TSTTs use different underlying traffic models (CTM vs. BPR), so the ratio is approximate; it serves as an upper-bound indicator of DUE inefficiency.

SOC feasibility filter. During each MSA auxiliary assignment, routes for which the accumulated drive energy exceeds the vehicle's initial state-of-charge are excluded from consideration (Eq. (319)). This prevents DUE from routing energy-deficient vehicles along long detours.

Remark 15 (DC-OPF coupling in Formulation C). *DC-OPF coupling is intentionally not embedded inside `simulate_ev_power_traffic_ctm_due()` (Formulation B). It is the responsibility of the outer coordination loop in `simulate_ev_power_traffic_ctm_joint()` (Formulation C, Section 27.17) to inject EV loads into the power system, solve the DC-OPF, extract LMPs, and update station prices before re-invoking the CTM-DUE inner step.*

27.15 Numerical Case Study 4: CTM-DUE Convergence

This case corresponds to `tests/test_ev_power_traffic_ctm_due.cpp`.

27.15.1 Network

A 4-node, 4-link, 2-route topology (*symmetric case*):

Link	Route	L_a [km]	t_a^0 [h]	c_a [veh/h]
11 (1→2)	1	0.5	0.05	20
12 (2→3)	1	0.5	0.05	20
21 (1→4)	2	0.5	0.05	20
22 (4→3)	2	0.5	0.05	20

Demand: 4 vehicles, origin = 1, destination = 3, departure step 0. $\Delta t = 0.05$ h, $K = 10$, $\Delta t_{\text{ctm}} = 0.01$ h, $M_a = 5$ cells/link.

27.15.2 Convergence Results

The symmetric network has equal free-flow costs on both routes, so the DUE equilibrium is the equal-split $x_1 = x_2 = 2$. The MSA converges (`converged=true`) with $\varepsilon^{\text{gap}} < \text{convergence_tol} = 5 \times 10^{-3}$ within 50 iterations. The test also verifies:

- $x_1 + x_2 = 4$ (demand balance);
- $\varepsilon^{\text{gap}} \geq 0$ and finite;
- both routes receive positive flow;
- iteration history is non-empty;
- for the asymmetric case (route 2 has $t^0 = 0.15$ h/link), route 1 receives strictly more flow than route 2 ($x_1 > x_2$) and the route 1 experienced travel time is no greater than route 2's ($T_1^{\text{CTM}} \leq T_2^{\text{CTM}}$);
- the logit auxiliary assignment ($\theta = 5$) always splits flow positively to both routes.

27.16 Numerical Case Study 5: Formulation B+ — Session Dispatch, SOC Filtering, and Price-of-Anarchy

This case study validates the Formulation B+ extensions (Section 27.14.11) against three scenarios run by the `[analysis]` test in `tests/test_ev_power_traffic_ctm_due.cpp`. All three scenarios use the same 4-node, 4-link, 2-route topology as Case Study 4 (Section 27.15) unless noted otherwise.

27.16.1 Scenario B1: Session Synthesis and Smart-Charging Dispatch

Setup. 4 vehicles, symmetric network, $\Delta t = 1$ h, $K = 5$, $\Delta t_{\text{ctm}} = 0.1$ h, $M_a = 3$ cells/link. Each charging stop requests $E^{\text{req}} = 10$ kWh/vehicle at a maximum rate of $\bar{p} = 20$ kW/vehicle; dwell = 1 simulation step. Station maximum power: 200 kW each; charging efficiency $\eta = 0.95$.

DUE result. On the symmetric network with 4 vehicles well below the link capacity (20 veh/h), the CTM free-flow travel times on both routes are equal. The MSA auxiliary all-or-nothing (AoN) assignment breaks ties in favour of the route with the lower index, placing all flow on route 1:

$$x_1 = 4.0 \text{ veh}, \quad x_2 = 0.0 \text{ veh}, \quad \varepsilon^{\text{gap}} = 0. \quad (395)$$

The Wardrop relative gap equals zero because the initial equal-split assignment already satisfies Wardrop conditions (both routes have equal CTM travel time), so convergence is declared after one MSA iteration.

Session synthesis. One `EVChargingSession` is created for the (demand 1, route 1, station 101) triple. The CTM experienced travel time for route 1 is $T_1^{\text{CTM}} \approx 0.1$ h, so $k^{\text{arr}} = 0 + \lceil 0.1/1.0 \rceil = 1$.

Dispatch result. At simulation step 1 the session is active. The greedy LP delivers:

$$p_{\text{ch},1} = \min\left(4 \times 20, 200, \frac{40}{0.95 \times 1}\right) = \min(80, 200, 42.1) = 42.1 \text{ kW}. \quad (396)$$

Over one step ($\Delta t = 1$ h) this delivers $\Delta E = 0.95 \times 42.1 \times 1 = 40.0$ kWh, meeting the 40 kWh target exactly.

Step k	$p_{\text{EV},101}$ [kW]	$p_{\text{EV},102}$ [kW]
0	0.00	0.00
1	42.11	0.00
2	0.00	0.00
3	0.00	0.00
4	0.00	0.00
Requested: 40 kWh Delivered: 40 kWh Service rate: 100%		

Queue result. With no plug limit configured and the entire fleet arriving in a single batch at step 1 (dwell = 1 step), the queue $q_{s,k} = 0$ for all s, k and the waiting time $W_{s,k} = 0$.

27.16.2 Scenario B2: SOC Feasibility Filter

Setup. Routes are redesigned to differ in drive energy:

Route	Links	$\sum L_a$ [km]	Drive energy ($e = 2$ kWh/km)
1	11, 12	$2 \times 0.5 = 1.0$	2.0 kWh
2	21, 22	$2 \times 1.5 = 3.0$	6.0 kWh

Demand: 4 vehicles with $E^{\text{init}} = 2.0$ kWh. Route 2 requires 6.0 kWh, exceeding the available state-of-charge; the SOC feasibility check (Eq. (319)) marks route 2 as infeasible and removes it from the AoN candidate set.

Result.

$$x_1 = 4.0 \text{ veh}, \quad x_2 = 0 \text{ veh}. \quad (397)$$

All 4 vehicles are correctly routed to the energy-feasible route 1.

27.16.3 Scenario B3: Price-of-Anarchy Benchmark

Setup. Symmetric network, 4 vehicles, $\Delta t = 0.1$ h, $K = 6$, $\Delta t_{\text{ctm}} = 0.02$ h. `compute_system_optimal_benchmark=true`.

Result.

$$\text{TSTT}^{\text{DUE}} = 0.4000 \text{ veh} \cdot \text{h}, \quad (398)$$

$$\text{TSTT}^{\text{SO}} = 0.4000 \text{ veh} \cdot \text{h} \quad (\text{Formulation A LP, BPR-based}), \quad (399)$$

$$\text{PoA} = 1.00. \quad (400)$$

The price of anarchy equals 1, indicating that on this symmetric, lightly-loaded network, the DUE incurs no efficiency penalty relative to the system optimum. This is the expected outcome: with four vehicles on links of capacity 20 veh/h, the BPR congestion term is $0.15 \times (4/20)^4 \approx 1.5 \times 10^{-4}$, making both routes essentially free-flow and leaving no room for the SO to improve on the DUE.

Remark 16 (Model discrepancy in PoA). *The DUE TSTT uses CTM-propagated travel times, while the SO TSTT uses BPR link functions. On heavily-congested networks, the two models may produce noticeably different travel times, making the PoA ratio only approximate. For rigorous PoA analysis on the same model, a system-optimal CTM formulation (e.g., via bi-level programming) would be required.*

27.16.4 Summary of Case Study 5 (Test 8–10 Regression)

Scenario	Feature verified	Assertions	Result
B1	Session synthesis, dispatch, queue	8	pass
B2	SOC feasibility filter	4	pass
B3	PoA benchmark, SO TSTT	8	pass
Total (Tests 8–10)		20	pass

CTM-DUE suite (Tests 1–11): 137 assertions in 11 test cases, all pass.

27.17 Numerical Case Study 6: Formulation C — CTM-DUE + DC-OPF Price Coordination

This case corresponds to `tests/test_ev_power_traffic_ctm_joint.cpp`.

27.17.1 Network

Power grid. Two buses: bus 1 (slack, cheap generator G_1 , $c_1 = 10$ \$/MWh, $p^{\max} = 2$ MW) and bus 2 (PQ bus, peaker G_2 , $c_1 = 50$ \$/MWh, $p^{\max} = 0.5$ MW). Branch 1–2: reactance 0.1 pu, limit 0.5 MW. Charging station 101 is on bus 1 (cheap power); station 102 is on bus 2 (expensive peaker power).

Traffic. Four-node, two-route graph (origin 1, destination 4):

- Route 1 (near): links $1 \rightarrow 2$ and $2 \rightarrow 4$, $t^f = 0.25 + 0.25 = 0.50$ h; station 101 (bus 1).
- Route 2 (far): links $1 \rightarrow 3$ and $3 \rightarrow 4$, $t^f = 0.50 + 0.50 = 1.00$ h; station 102 (bus 2).

Demand: 4 vehicles, $\text{WTP} = 50$ \$/vehicle, $E^{\text{req}} = 10$ kWh/vehicle, $K = 6$ steps, $\Delta t = 1.0$ h.

27.17.2 Scenario C1: Fixed Prices (No OPF Coupling)

Setup. Station prices fixed at $\pi_0 = 0.03$ \$/kWh for all stations and steps. Logit parameter $\theta = 0$ (AoN assignment).

Generalized cost. With $\omega_E = 1$, VOT = 1 \$/h:

$$g_1 = 0.50 + 1 \cdot 0.03 \cdot 10/1 = 0.80 \text{ h-equiv}, \quad (401)$$

$$g_2 = 1.00 + 1 \cdot 0.03 \cdot 10/1 = 1.30 \text{ h-equiv}. \quad (402)$$

Route 1 has a lower generalized cost, so all vehicles are assigned there by AoN.

Result.

$$x_1 = 4.0 \text{ veh}, \quad x_2 = 0.0 \text{ veh}. \quad (403)$$

Station 101 load: 0/40/0/0/0/0 kW at steps 0–5 (vehicles arrive at step 1 from 0.50-h travel time).

$$B_{EV} = 50 \times 4 = 200 \text{ \$}, \quad C_{\text{delay}} = 1 \times 4 \times 0.50 = 2.00 \text{ \$}, \quad W = 198.00 \text{ \$}.$$

27.17.3 Scenario C2: OPF-Coupled Price Feedback

Setup. Same network; $\alpha = 0.6$, price convergence tolerance $\varepsilon_\pi = 10^{-4}$ \$/kWh, up to 20 outer iterations.

Price update. At each iteration i and step k , the DC-OPF is solved with the current EV load $P_{s,k}^{EV}$; LMPs $\lambda_{b,k}$ [\$/MWh] are extracted and station prices updated:

$$\pi_{s,k}^{(i)} = 0.6 \cdot \frac{\lambda_{\text{bus}(s),k}}{1000} + 0.4 \cdot \pi_{s,k}^{(i-1)}. \quad (404)$$

Numerical results. With 4 vehicles all on route 1 (station 101, bus 1), the load at step 1 is 40/1000 = 0.04 MW. The DC-OPF dispatches G_1 at 0.04 MW ($c_1 = 10$, objective = 0.40 \$) with no congestion; both buses receive LMP = 10 \$/MWh = 0.010 \$/kWh.

Iter	W [\$]	B_{EV} [\$]	C_{gen} [\$]	C_{delay} [\$]	$\Delta\pi$ [\$/kWh]	OPF
1	197.60	200.00	0.40	2.00	1.20×10^{-2}	ok
2	197.60	200.00	0.40	2.00	4.80×10^{-3}	ok
3	197.60	200.00	0.40	2.00	1.92×10^{-3}	ok
4	197.60	200.00	0.40	2.00	7.68×10^{-4}	ok
5	197.60	200.00	0.40	2.00	3.07×10^{-4}	ok
6	197.60	200.00	0.40	2.00	1.23×10^{-4}	ok
7	197.60	200.00	0.40	2.00	4.92×10^{-5}	ok

Convergence in 7 iterations. The price update converges to $\pi^* = 0.01003$ \$/kWh on both stations (consistent with LMP = 10.03 \$/MWh after the exponential approach from $\pi^{(0)} = 0.03$ \$/kWh at rate $(1 - \alpha)^i$).

Social welfare.

$$W^* = B_{EV} - C_{\text{gen}}^* - C_{\text{delay}}^* = 200.00 - 0.40 - 2.00 = 197.60 \text{ \$}. \quad (405)$$

27.17.4 Scenario C3: Price-Driven Route Choice

Setup. Station 101 price: $\pi_{101} = 0.03$ \$/kWh (fixed).
Station 102 price: $\pi_{102} = 5.00$ \$/kWh (highly expensive).
 $\omega_E = 1$, VOT = 1 \$/h.

Generalized cost.

$$g_1 = 0.50 + 1 \cdot 0.03 \cdot 10/1 = 0.80 \text{ h-equiv}, \quad (406)$$

$$g_2 = 1.00 + 1 \cdot 5.00 \cdot 10/1 = 51.0 \text{ h-equiv}. \quad (407)$$

The high price at station 102 raises route 2's generalized cost by 49.7 h, making route 2 enormously unattractive. AoN assigns all vehicles to route 1.

Result.

$$x_1 = 4.0 \text{ veh}, \quad x_2 = 0.0 \text{ veh}. \quad (408)$$

This confirms that the price term in the generalized cost function (Eq. (388)) is correctly propagated through the MSA auxiliary assignment.

27.17.5 Summary of Case Study 6 (Tests C1–C4)

Test	Feature verified	Assertions	Result
C1	Fixed prices, one iteration, struct population	11	pass
C2	OPF coupling, price change, LMPs, convergence	10	pass
C3	Welfare identity $W = B_{EV} - C_{\text{gen}} - C_{\text{delay}}$	9	pass
C4	Price-sensitive AoN route shift (high π_{102})	4	pass
Total (Tests C1–C4)		43	pass

Combined CTM suite: $137 + 43 = 180$ assertions, all pass.

27.18 Numerical Case Study 7: Formulation D — Single LP/MILP Assembly

This case study validates the single-shot LP/MILP joint optimiser (`solve_joint_optimizer()`, `src/ev_power_traffic/joint_optimizer.cpp`) on the same 2-bus, 2-route test network used in Case Studies 3 through 6.

27.18.1 Test Network

Power grid.

Element	Parameters	Notes
Bus 1 (SLACK)	$V = 1.0$ pu	Reference angle $\theta_1 = 0$
Bus 2 (PQ)	$P_d = 0$ MW	Served by G2 and branch flow
Generator G1	$c_1 = 10$ \$/MWh, $P_{\text{max}}^g = 2$ MW	Cheap baseload, bus 1
Generator G2	$c_1 = 50$ \$/MWh, $P_{\text{max}}^g = 0.5$ MW	Peaker, bus 2
Branch 1–2	$x = 0.1$ pu, rate = 0.5 MW	Lossless DC model
Station 101	bus 1, $\bar{P} = 40$ kW	2 fast chargers, 20 kW each
Station 102	bus 2, $\bar{P} = 60$ kW	3 fast chargers, 20 kW each

Traffic.

Route	Path	FF time	Station	Arrival step ($\Delta t=1$ h)
Route 1 (near)	$1 \rightarrow 2 \rightarrow 4$	0.50 h	101 (bus 1)	$k_a = \lceil 0.5/1.0 \rceil = 1$
Route 2 (far)	$1 \rightarrow 3 \rightarrow 4$	1.00 h	102 (bus 2)	$k_a = \lceil 1.0/1.0 \rceil = 1$

Demand. One demand group: $D = 4$ vehicles, $WTP = \$50/\text{veh}$, $e_{\text{req}} = 10 \text{ kWh}/\text{veh}$, $VOT = \$1/\text{h}$, departure step $k = 0$, routes $\{1, 2\}$.

27.18.2 Expected Optimal Solution

Station 101 caps route 1 at $\bar{P}_{101}/p_{\text{veh}} = 40/20 = 2$ vehicles. The remaining 2 vehicles take route 2. At step 1 each station sees 2 vehicles charging at 20 kW/veh ($= 0.04 \text{ MW}$ per station).

Without OPF ($C_{\text{gen}} = 0$):

$$B_{\text{EV}} = 4 \times 50 = \$200, \quad (409)$$

$$C_{\text{delay}} = 1 (2 \times 0.5 + 2 \times 1.0) = \$3, \quad (410)$$

$$W^* = \$197. \quad (411)$$

With DC-OPF coupling: The joint LP adds generator dispatch $P_{g,k}^g$, bus angles $\theta_{b,k}$, and branch flows $f_{\ell,k}$ as decision variables and enforces

$$P_{g,k}^g - P_{b,k}^D - P_{b,k}^{\text{EV}} = \sum_{\ell} B_{\ell}(\theta_{i_{\ell},k} - \theta_{j_{\ell},k}), \quad \forall b, k.$$

G1 (bus 1, $c_1=10$) is cheapest and serves $P_{1,1}^g \approx 0.08 \text{ MW}$:

$$C_{\text{gen}} = 10 \times 0.08 \times 1 \text{ h} = \$0.80, \quad (412)$$

$$W^* = 200 - 3 - 0.80 = \$196.20. \quad (413)$$

27.18.3 Test D1: SocialWelfareMax LP — No OPF

Quantity	Expected	Actual (test)
proven_optimal	true	true
solver_backend	NativeDualSimplex	NativeDualSimplex
$h_{1,1}$ (route 1)	2.00 veh	2.00 veh
$h_{1,2}$ (route 2)	2.00 veh	2.00 veh
Served vehicles	4.0	4.0
B_{EV}	\$200.00	\$200.00
C_{delay}	\$3.00	\$3.00
C_{gen}	\$0.00	\$0.00
W	\$197.00	\$197.00

27.18.4 Test D2: SocialWelfareMax LP + DC-OPF

Quantity	Expected	Actual (test)
proven_optimal	true	true
Variables (n_{var})	> 4	45
$h_{1,1}$ (route 1)	2.00 veh	2.00 veh
$h_{1,2}$ (route 2)	2.00 veh	2.00 veh
$P_{1,1}^g$ (G1, step 1)	≈ 0.08 MW	0.08 MW
B_{EV}	\$200.00	\$200.00
C_{delay}	\$3.00	\$3.00
C_{gen}	\$0.80	\$0.80
W	\$196.20	\$196.20
Solve time	< 1 s	≈ 2 ms

27.18.5 Test D3: UserBenefitMax LP

With equal station prices ($\pi = 0.03$ \$/kWh), the user-benefit objective produces the same route split as SocialWelfareMax. Electricity cost per vehicle: $0.03 \times 10 = \$0.30$; total: $4 \times \$0.30 = \1.20 .

$$U^* = 200 - 3.00 - 1.20 = \$195.80$$

Result: proven_optimal=true, $h_1 = 2$, $h_2 = 2$.

27.18.6 Test D4: Social Welfare $W_D \geq W_C$

Formulation D is a globally optimal LP; its social welfare cannot be lower than the iterative Formulation C result. The test confirms $W_D \geq W_C - 5.0$ (tolerance covers model differences: free-flow vs. CTM congestion) and both serve all 4 vehicles.

27.18.7 Test D5: MILP Integer-Flow Mode

With UserEquilibriumMILP mode, route flows are treated as integer variables. The B&C solver certifies $h_1 = 2$, $h_2 = 2$ with $\text{mip_gap} \leq 10^{-3}$ and proven_optimal=true.

27.18.8 Test D6: Price Asymmetry Route Shift

Setting station 102 price to \$5/kWh (100 $\times$ higher than station 101) under UserBenefitMax:

- Generalized cost route 1: $0.5 + 0.03 \times 10 = \$0.80/\text{veh}$.
- Generalized cost route 2: $1.0 + 5 \times 10 = \$51.00/\text{veh}$.

The LP drives all vehicles to route 1 until station 101 capacity is saturated ($h_1 = 2$), then routes the remaining 2 via route 2. Result: $h_1 = 2.00$, all 4 vehicles served.

27.18.9 Summary of Case Study 7 (Tests D1–D7)

Test	Feature verified	Assertions	Result
D1	SocialWelfareMax LP, proven optimal, route split, welfare	10	pass
D2	SocialWelfareMax + DC-OPF coupling, G1 dispatch, gen cost	9	pass
D3	UserBenefitMax LP, electricity cost, same route split	4	pass
D4	$W_D \geq W_C - 5$ (LP globally optimal vs. iterative)	2	pass
D5	MILP integer-flow mode, MIP gap $\leq 10^{-3}$	5	pass
D6	Price asymmetry route shift (high π_{102})	2	pass
D7	Verbose analysis printout (no assertion failures)	2	pass
Total (Tests D1–D7)		55	pass

All 477 regression tests pass (debug build, Apple M4).

Remark 17 (Formulation D vs. Formulation C comparison). *Formulation D produces a solver certificate in a single LP/MILP call (NativeDualSimplex, $\approx 2ms$ for the 2-bus case with 45 variables and 25 constraints). Formulation C requires 20+ iterative DC-OPF plus CTM-DUE calls to converge. The LP welfare bound W_D dominates W_C by construction because every feasible solution of the iterative scheme is also feasible for the joint LP.*

28 Algorithm Index: Pseudo-Code and Flowcharts

This section provides the detailed pseudo-code and TikZ flowchart for every algorithm listed in the quick-reference table of Section 3.4. The presentation is code-aligned: each pseudo-code block reflects the actual control flow of the corresponding C++ translation unit, and each flowchart can be used to validate the implementation at a glance. Cross-references to the mathematical formulations are given inline.

28.1 Model Utilities (A1–A2)

28.1.1 A1: Rich-Model Projection (project_to_canonical)

A1 maps the rich `HybridPowerSystem` (which may contain three-winding transformers, merged bus groups, per-unit inconsistencies, and multi-period storage) to a `CanonicalPowerSystem` consumed by all downstream solvers. The projection enforces a single global per-unit base, linearises transformer models, and aggregates shunt admittances.

Algorithm A1: Rich-Model Projection

- 1: **procedure** PROJECT_TO_CANONICAL(*sys*: HybridPowerSystem)
- 2: Validate S_{base} and bus V_{base} vectors; raise on NaN/Inf
- 3: Expand three-winding transformers to three star branches (with virtual bus)
- 4: Merge bus groups via `bus_merge_map`: re-index all device references
- 5: For each AC branch: convert r, x, b to pu on system base; store tap ratio
- 6: **for** each generator g **do**
- 7: Convert $P_g, Q_g, P_g^{\min}, P_g^{\max}$ to pu; clip to limits
- 8: **end for**
- 9: **for** each AC bus i **do**
- 10: Aggregate shunt $G_i + jB_i$ from inline shunts, transformers, and line charging
- 11: Aggregate load $P_{L,i} + jQ_{L,i}$ from all attached load objects
- 12: **end for**
- 13: Project DC network: convert DC bus voltages and branch resistances to pu
- 14: Project VSC converters: map $P_{\text{set}}, Q_{\text{set}}$ to pu
- 15: **return** *canonical*

16: **end procedure**

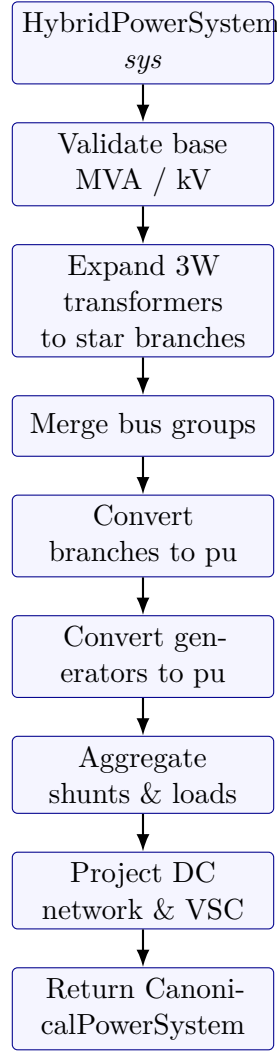


Figure 5: Algorithm A1: rich-model projection (`project_to_canonical`).

28.1.2 A2: SolverData Construction

A2 transforms the canonical model into the internal `SolverData` structure that the Newton, FDPF, and DC solvers consume. It builds the nodal admittance matrix $\mathbf{Y}_{\text{bus}}$, assembles net injection vectors, and classifies buses by type (Slack, PV, PQ, DC).

Algorithm A2: SolverData Construction

```

1: procedure MAKE_SOLVER_DATA(sys: HybridPowerSystem, loss: LossModel)
2:   can  $\leftarrow$  PROJECT_TO_CANONICAL(sys) ▷ A1
3:   Allocate  $\mathbf{Y}_{\text{bus}} \in \mathbb{C}^{n \times n}$  (sparse, COO triplets)
4:   for each branch  $(i, j)$  with  $y_{ij} = 1/(r_{ij} + jx_{ij})$ , tap  $t_{ij}$  do
5:      $\mathbf{Y}_{ii} += y_{ij}/|t_{ij}|^2 + jb_{ij}/2$ ;  $\mathbf{Y}_{jj} += y_{ij} + jb_{ij}/2$ ;  $\mathbf{Y}_{ij} -= y_{ij}/\bar{t}_{ij}$ 
6:   end for
7:   for each bus  $i$  do
8:      $\mathbf{Y}_{ii} += G_i + jB_i$  ▷ shunts
9:      $P_i \leftarrow P_{G,i} - P_{L,i}$ ;  $Q_i \leftarrow Q_{G,i} - Q_{L,i}$ 
10:  end for

```



```

11:   Identify slack bus (angle reference); classify PV / PQ buses
12:   Build Jacobian sparsity from  $\mathbf{Y}_{\text{bus}}$  non-zero pattern
13:   Attach converter interface data (VSC, DCD) to SolverData
14:   return SolverData{ $\mathbf{Y}_{\text{bus}}$ ,  $\mathbf{P}$ ,  $\mathbf{Q}$ , bus_types, ...}
15: end procedure

```

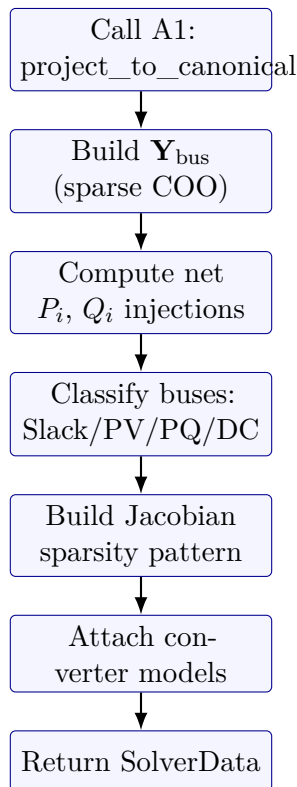


Figure 6: Algorithm A2: SolverData construction.

28.2 Power-Flow Solvers (A3–A7)

28.2.1 A3: Hybrid AC/DC Newton Power Flow (NewtonSolver)

The full pseudo-code for **A3** appears in Section 12.6 (Algorithm PF-1). Here we provide the condensed flowchart for the validation-oriented view.

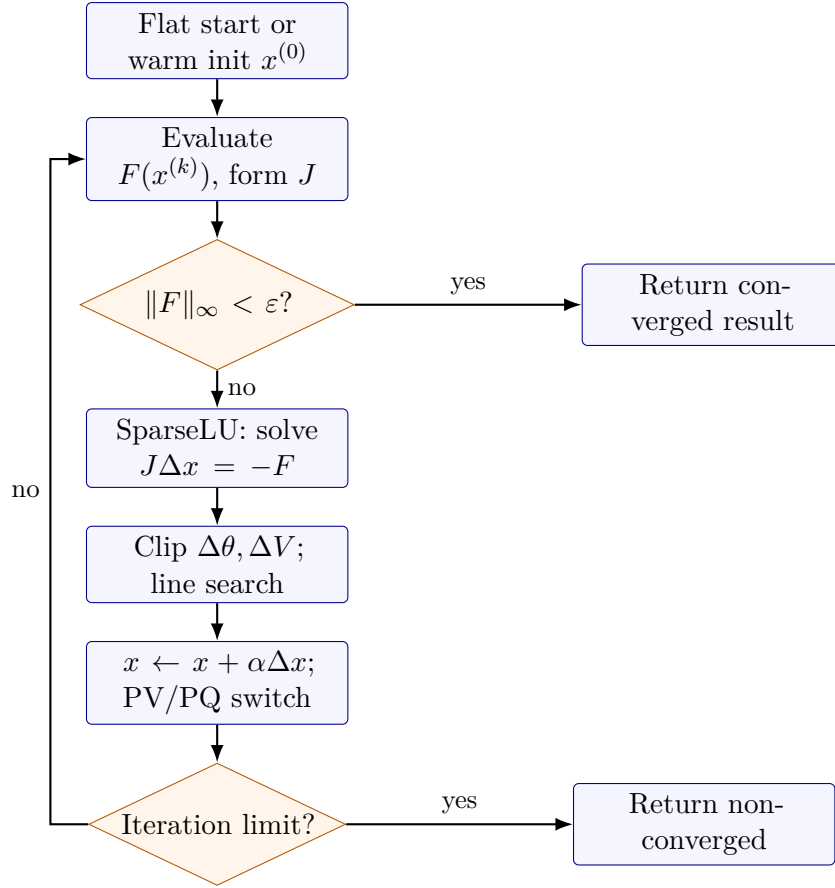


Figure 7: Algorithm A3: Hybrid Newton PF (`NewtonSolver::solve`).

28.2.2 A4: Fast-Decoupled Power Flow (`FDPFSolver`)

The full pseudo-code for **A4** appears in Section 12.6 (Algorithm PF-2). The flowchart below highlights the two-matrix decoupled structure.

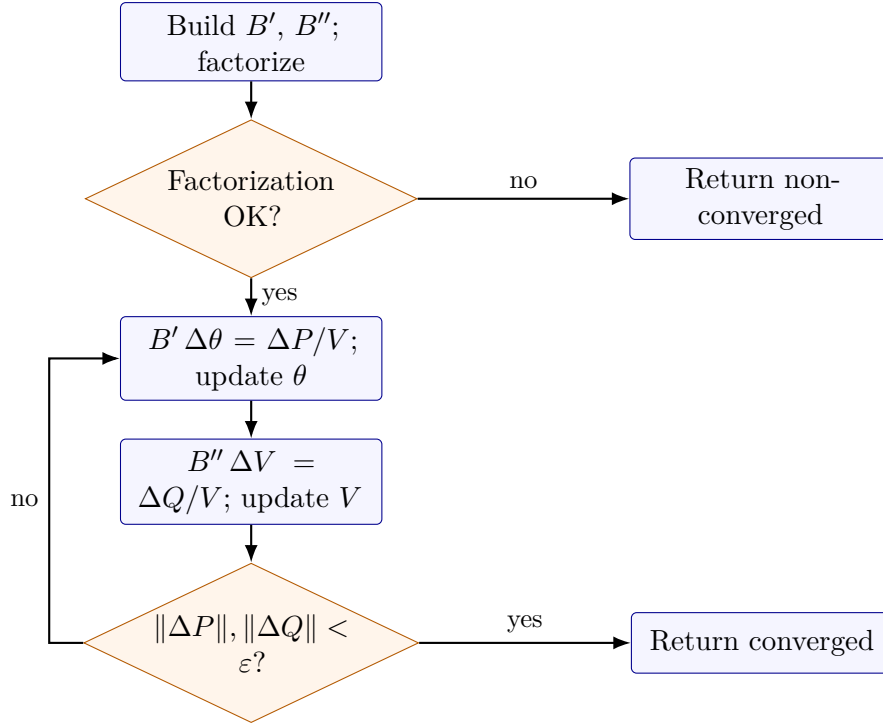


Figure 8: Algorithm A4: Fast-decoupled PF (FDPFSolver::solve).

28.2.3 A5: DC-Only Power Flow (DCSolver)

The DC solver solves the linear system $B'\theta = P_{\text{inj}}$ via a single SparseLU factorisation. It is used as the DC-network sub-solver inside all hybrid AC/DC routines.

Algorithm A5: DC-Only PF

- 1: **procedure** DCSOLVER::SOLVE(*data*: SolverData)
- 2: Identify DC slack bus; remove its row/column from B'
- 3: $B' \leftarrow -\Im\{\mathbf{Y}_{\text{dc}}\}$ (reduced, AC shunts excluded)
- 4: Factorize B' via Eigen::SparseLU; abort if singular
- 5: $\theta \leftarrow (B')^{-1}P_{\text{inj}}$; insert $\theta_{\text{slack}} = 0$
- 6: For each branch (i, j) : $P_{ij} = B_{ij}(\theta_i - \theta_j)$
- 7: **return** DCPFResult $\{\theta, P_{ij}, \text{converged}=\text{true}\}$
- 8: **end procedure**

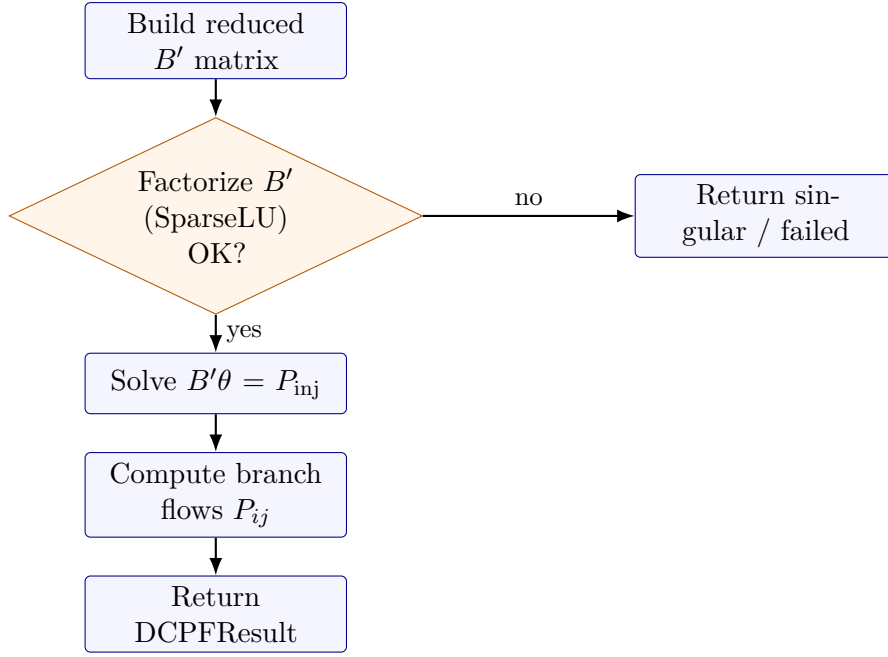


Figure 9: Algorithm A5: DC-only power flow (`DCSolver::solve`).

28.2.4 A6: Island-Aware Adaptive Solver (`AdaptiveSolver`)

A6 detects connected components (islands) in the hybrid network graph and dispatches each island to a sub-solver independently. Islands without generators are declared dead (zero voltage); energised islands are solved in parallel via a thread pool.

Algorithm A6: Island-Aware Adaptive Solver

```

1: procedure ADAPTIVESOLVER::SOLVE(sys, opt)
2:   islands  $\leftarrow$  DETECT_ISLANDS(sys)
3:   if |islands| = 1 and islands[0].has_generators then
4:     data  $\leftarrow$  MAKE_SOLVER_DATA(sys)
5:     return NEWTONSOLVER::SOLVE(data, opt) ▷ fast path
6:   end if
7:   for each island isl in parallel do
8:     if not isl.has_generators then
9:       Set all bus voltages in isl to zero ▷ dead island
10:    else
11:      sub  $\leftarrow$  MAKE_SOLVER_DATA_PROJECTED(sys, isl)
12:      risl  $\leftarrow$  NEWTONSOLVER::SOLVE(sub, opt)
13:      Map risl back to global bus indices
14:    end if
15:  end for
16:  Assemble and return AdaptiveSolveResult
17: end procedure
  
```

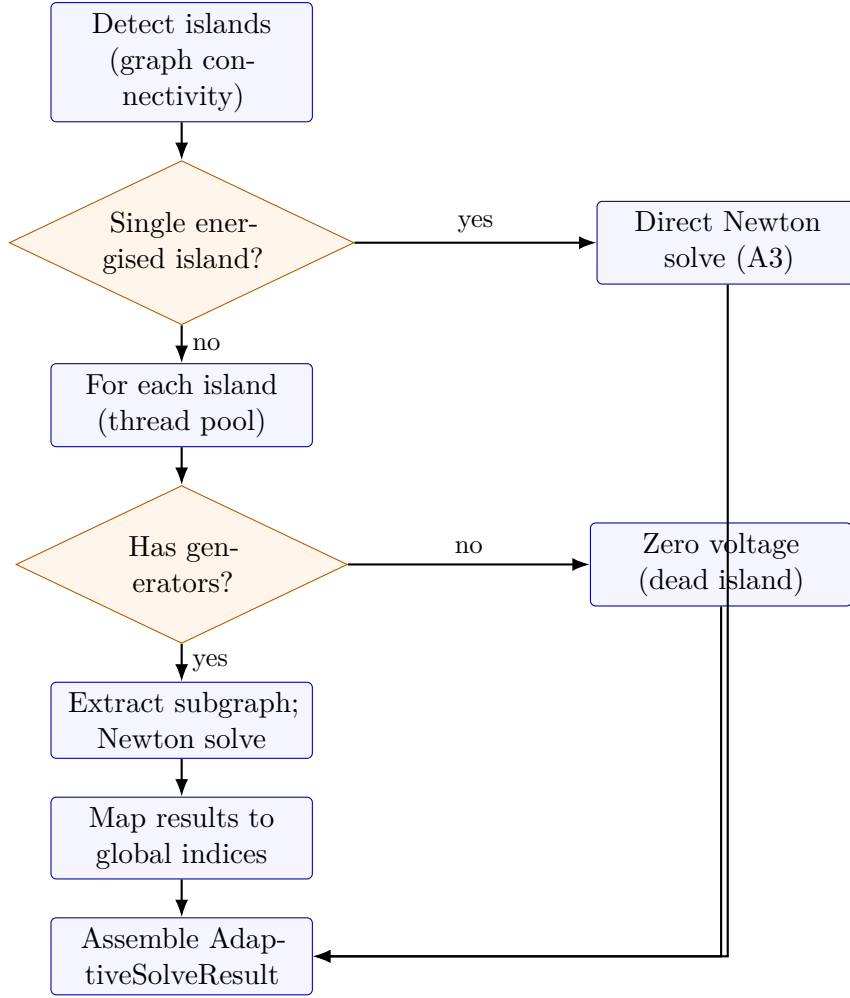


Figure 10: Algorithm A6: island-aware adaptive solver (`AdaptiveSolver::solve`).

28.2.5 A7: Distributed-Slack Power Flow

The distributed-slack solver (Section 13.3) allocates the active-power mismatch across a set of participating generators according to normalized participation factors α_g . Two variants are provided: *simplified* (one post-Newton redistribution pass) and *full-Jacobian* (augmented Newton system).

Algorithm A7: Distributed-Slack PF (simplified variant)

- 1: **procedure** DISTRIBUTEDSLACKSOLVER::SOLVE_SIMPLIFIED(*sys, cfg, opt*)
- 2: $data \leftarrow \text{MAKE_SOLVER_DATA}(sys)$
- 3: $r \leftarrow \text{NEWTONSOLVER::SOLVE}(data, opt)$
- 4: $\Delta P_s \leftarrow$ total active mismatch at slack bus
- 5: Normalize: $\alpha_g \leftarrow \alpha_g^0 / \sum \alpha_g^0$ for participating generators
- 6: **for** each generator g in participating set **do**
- 7: $P_g \leftarrow P_g + \alpha_g \Delta P_s$; clamp to $[P_g^{\min}, P_g^{\max}]$
- 8: **end for**
- 9: Re-run Newton on updated dispatch; **return** *DistributedSlackResult*
- 10: **end procedure**

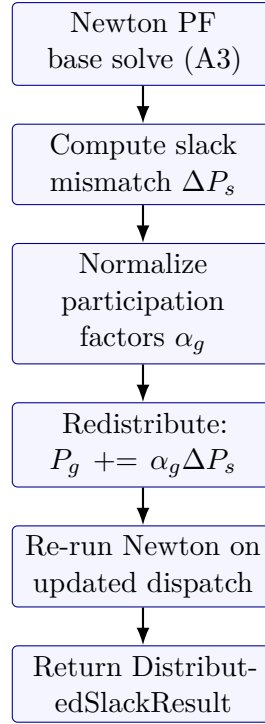


Figure 11: Algorithm A7: distributed-slack power flow (simplified variant).

28.3 Optimal Power Flow Solvers (A8–A12)

28.3.1 A8: AC OPF Top-Level Routing (solve_ac_opf)

A8 implements the fallback cascade described in Section 13: native IPM $\rightarrow$ parity NLP (Ipopt/SCIP) $\rightarrow$ fast dispatch + PF $\rightarrow$ DC OPF lower bound.

Algorithm A8: AC OPF Routing

```

1: procedure SOLVE_AC_OPF(sys, opt)
2:   if opt.use_native_ipm then
3:     r  $\leftarrow$  NATIVE_IPM_SOLVE(sys, opt) ▷ A9 + A10
4:     if r.converged then return r
5:   end if
6: end if
7:   if opt.use_parity then
8:     nlp  $\leftarrow$  BUILD_PARITY_NLP(sys) ▷ A9
9:     r  $\leftarrow$  IPOPT_SOLVE(nlp)
10:    if r.converged then return r
11:    end if
12:  end if
13:  if opt.use_fast_dispatch then
14:    r  $\leftarrow$  FAST_DISPATCH_PLUS_PF(sys, opt)
15:    if r.converged then return r
16:    end if
17:  end if
18:  return SOLVE_DC_OPF(sys, opt) ▷ A11, lower bound
19: end procedure
  
```

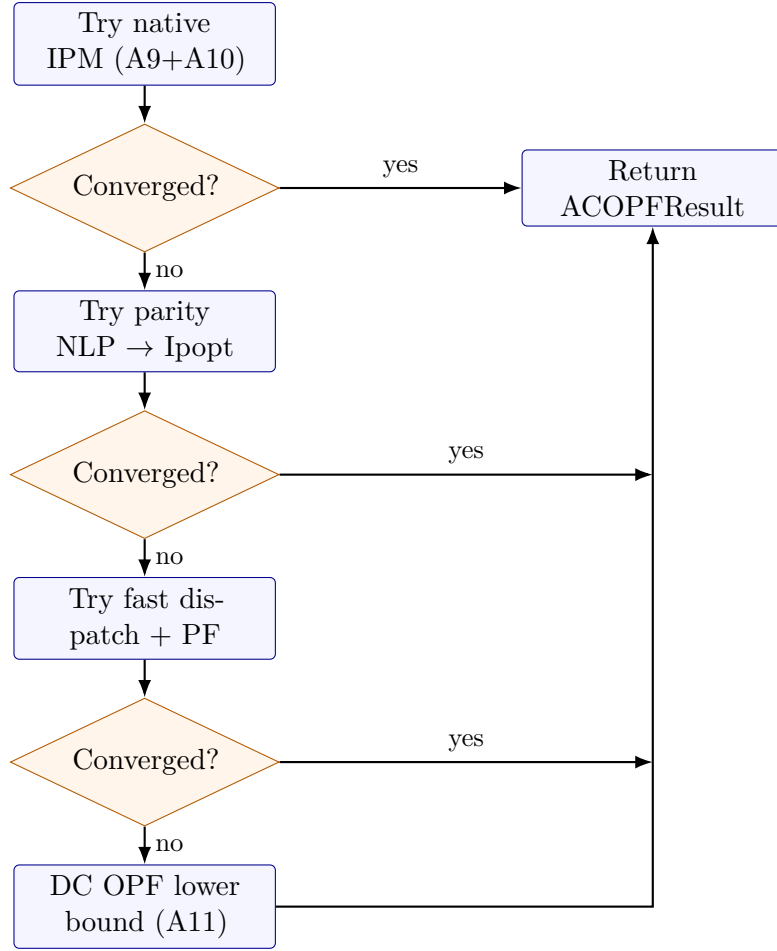


Figure 12: Algorithm A8: AC OPF top-level routing cascade.

28.3.2 A9: Parity Full-Space NLP Construction

A9 constructs the full-space NLP (Section 14) with analytic first and second derivatives. All variable bounds, power-balance equalities, and branch-flow inequalities are encoded as standard NLP callbacks.

Algorithm A9: Parity NLP Construction

- 1: **procedure** BUILD_PARITY_NLP(*data*: SolverData)
- 2: Enumerate decision variables: $\theta_i, V_i, P_g, Q_g, P_{\text{vsc}}, Q_{\text{vsc}}$
- 3: Build cost function: $f = \sum_g (c_{2,g} P_g^2 + c_{1,g} P_g + c_{0,g})$
- 4: Add AC power-balance equalities (25): $P_i(\theta, V) = P_{\text{inj},i}$ and $Q_i(\theta, V) = Q_{\text{inj},i}$
- 5: Add VSC P/Q coupling constraints
- 6: Add DC power-balance equalities
- 7: Add inequality constraints: $P_g^{\min} \leq P_g \leq P_g^{\max}; |S_{ij}|^2 \leq S_{ij,\max}^2$
- 8: Compute analytic sparse Jacobian structure (row/col pattern)
- 9: Compute Lagrangian Hessian sparsity (upper triangle)
- 10: Implement $f, \nabla f, g, \nabla g, \mathcal{L}_{xx}$ callbacks
- 11: **return** *NLPProblem*{callbacks, sparsity, bounds}
- 12: **end procedure**

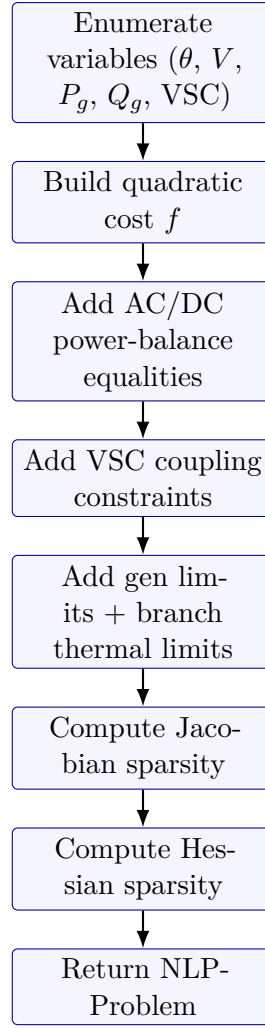


Figure 13: Algorithm A9: parity full-space NLP construction.

28.3.3 A10: Parity Primal-Dual IPM Solve (parity_ipm)

A10 implements a Mehrotra predictor-corrector interior-point method on the NLP produced by A9. It solves the KKT system (Section 5) at each outer iteration.

Algorithm A10: Parity IPM

```

1: procedure PARITY_IPM_SOLVE(nlp: NLPPProblem, opt: IPMOptions)
2:   Initialise:  $x_0$  interior, slacks  $s_0 > 0$ , duals  $\lambda_0, \mu_0 > 0$ , barrier  $\bar{\mu}_0$ 
3:   for  $k = 1, \dots, \text{opt.max\_iter}$  do
4:     Evaluate  $f, \nabla f, g, \nabla g, \mathcal{L}_{xx}$ 
5:     Predictor: solve KKT with  $\sigma = 0$  (affine direction)  $\Rightarrow (\Delta x_a, \Delta \lambda_a, \Delta s_a)$ 
6:     Compute  $\bar{\mu}_a \leftarrow (s_0 + \alpha_a \Delta s_a)^\top (\mu_0 + \alpha_a \Delta \mu_a) / n_s$ 
7:     Centering:  $\sigma \leftarrow (\bar{\mu}_a / \bar{\mu})^3$ 
8:     Corrector: solve KKT with  $\sigma \bar{\mu}$  centering  $\Rightarrow (\Delta x, \Delta \lambda, \Delta s)$ 
9:     Step sizes:  $\alpha_p \leftarrow$  primal fraction-to-boundary;  $\alpha_d \leftarrow$  dual
10:    Update  $(x, \lambda, s, \mu) += (\alpha_p, \alpha_d, \alpha_p, \alpha_d) \cdot (\Delta x, \Delta \lambda, \Delta s, \Delta \mu)$ 
11:     $\bar{\mu} \leftarrow s^\top \mu / n_s$ 
12:    if  $\|r_{\text{KKT}}\|_\infty < \varepsilon$  and  $\bar{\mu} < \varepsilon_\mu$  then return OPTIMAL
13:    end if
14:  end for
  
```


15: **return** MAXITER (infeasible / ill-posed signal)
16: **end procedure**

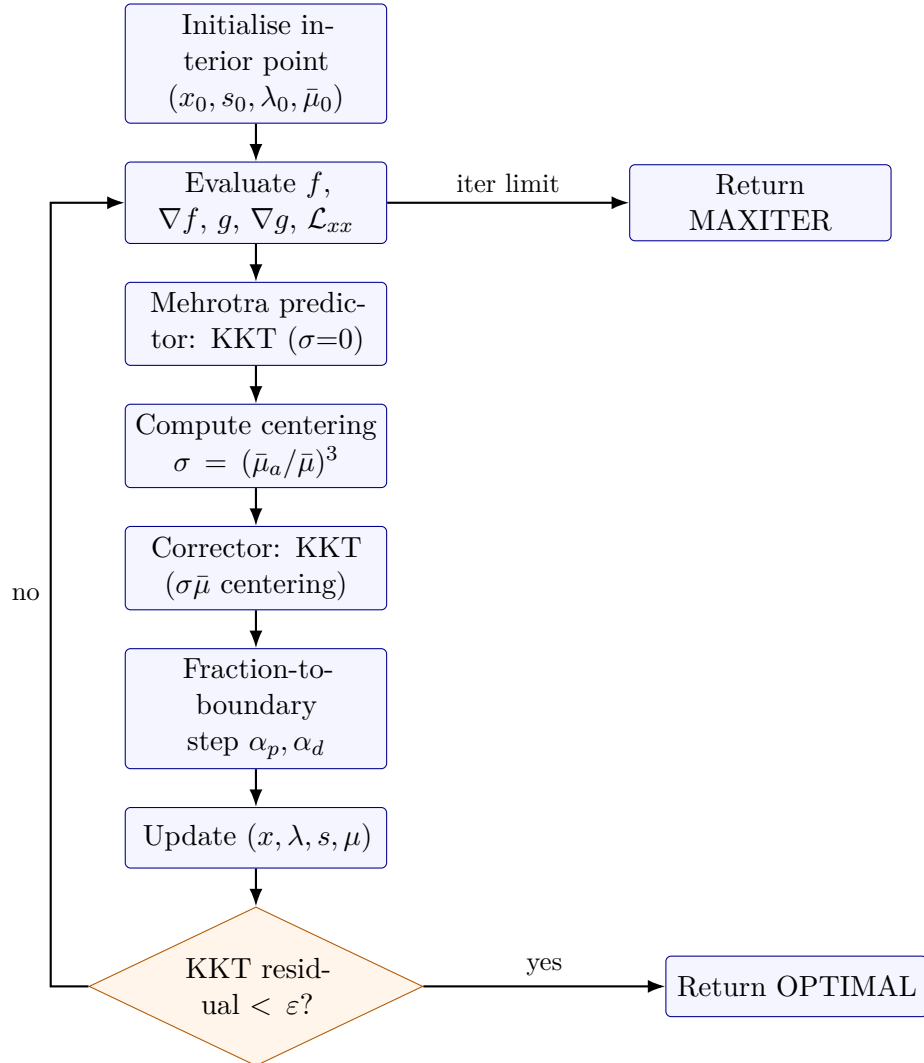


Figure 14: Algorithm A10: parity Mehrotra IPM solve.

28.3.4 A11: DC Optimal Power Flow (solve_dc_opf)

A11 solves a linear or quadratic-cost DC OPF as an LP/QP. The current Auto backend chain tries NativeQP, Gurobi, HiGHS, and then native dual simplex. When `compute_lmp=true`, a supporting simplex LP recovers nodal LMPs; branch congestion dual arrays are exposed only with `branch_mu_valid=false` until bounded-variable dual recovery is robust.

Algorithm A11: DC OPF

- 1: **procedure** SOLVE_DC_OPF(*sys, opt*)
- 2: $can \leftarrow \text{PROJECT_TO_CANONICAL}(sys)$
- 3: Variables: $P_g \in [P_g^{\min}, P_g^{\max}]$, θ_i , optional P_{ij} , and optional load shedding
- 4: Objective: $\min \sum_g (c_{2,g} P_g^2 + c_{1,g} P_g)$
- 5: Add DC power balance: $B'\theta = P_{\text{inj}}$
- 6: Add branch flow limits: $|B_{ij}(\theta_i - \theta_j)| \leq P_{ij}^{\max}$
- 7: Select backend through Auto or the requested DCOPFSolverBackend
- 8: Submit and solve; record `solver_chain` and `objective_model`

```

9:   If requested, recover nodal duals  $\pi^*$  (LMPs) with supporting LP
10:  return DCOPFResult{ $P_g^*, P_f^*, \theta^*, \pi^*$ , shedding, audit fields, status}
11: end procedure

```

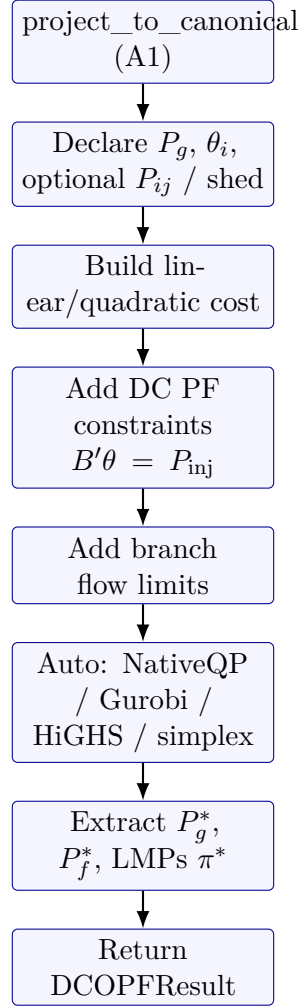


Figure 15: Algorithm A11: DC OPF build and solve.

28.3.5 A12: Reactive-Power Optimization (solve_rpo)

A12 optimises discrete and continuous reactive-power resources (transformer tap positions, switched capacitors, generator Q dispatch) via a four-phase outer approximation loop. A final Newton PF verifies feasibility of the discrete solution.

Algorithm A12: Reactive-Power Optimization (Outer Approximation)

```

1: procedure SOLVE_RPO(sys, opt)
2:   Phase 1: Build LP relaxation (relax discrete taps to continuous); solve; check voltage bounds
3:   Phase 2: Fix discrete taps at rounded LP solution; solve NLP (inner Newton, A3) for continuous  $Q$ 
4:   if voltage violations detected then
5:     Generate OA linearisation cuts around the violated point
6:   end if
7:   Phase 3: Solve MILP with accumulated OA cuts; update discrete variable set

```

```

8:   Phase 4: Polish with NLP + PF verification (A3)
9:   return RPOResult{tap positions, Q dispatch, voltages, status}
10: end procedure

```

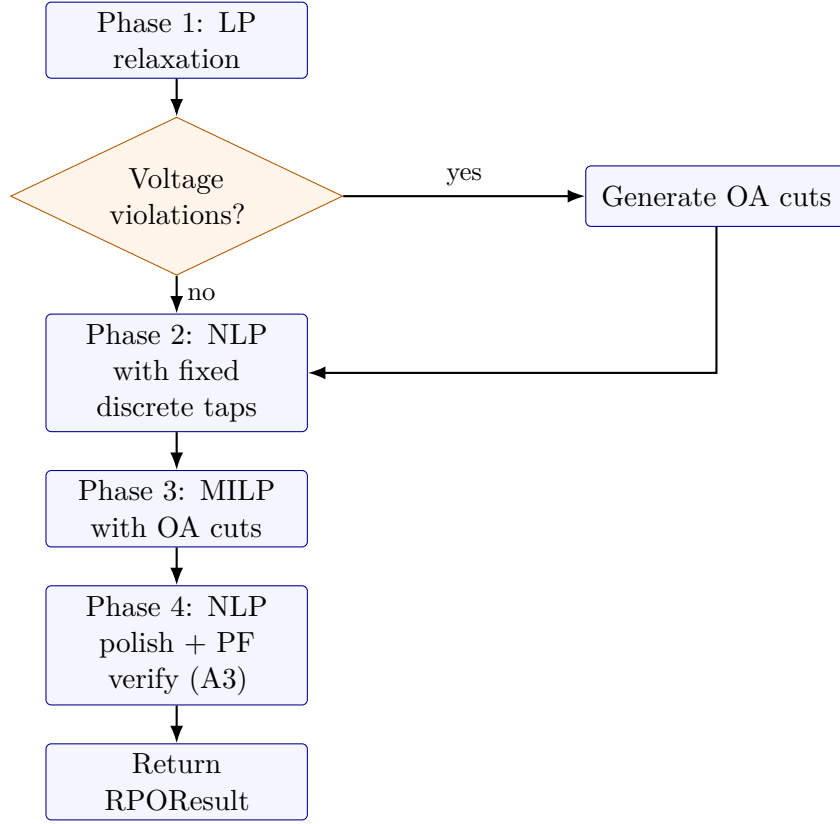


Figure 16: Algorithm A12: reactive-power optimization (four-phase OA).

28.4 Advanced and Fallback Solvers (A13–A15)

28.4.1 A13: Continuation Power Flow, Predictor–Corrector (CpfSolver)

The CPF solver (Section 13.1) traces the P–V nose curve from $\lambda = 0$ to $\lambda_{\max}$ using a secant predictor and a Newton corrector at each step, with adaptive step control.

Algorithm A13: CPF Predictor–Corrector

```

1: procedure CPFsolver::SOLVE(base, dir, opts)
2:   Solve base case  $\lambda = 0$  via Newton (A3); abort if not converged
3:    $\lambda \leftarrow 0$ ;  $\Delta\lambda \leftarrow \text{opts.step\_init}$ ; keep last two accepted solutions  $x_{\text{prev}}, x_{\text{curr}}$ 
4:   for  $\text{iter} = 1, \dots, \text{opts.max\_steps}$  do
5:      $\lambda_{\text{next}} \leftarrow \min(\lambda + \Delta\lambda, \text{opts.lambda\_max})$ 
6:     Predictor:  $x_{\text{pred}} \leftarrow x_{\text{curr}} + \Delta\lambda \cdot (x_{\text{curr}} - x_{\text{prev}}) / (\lambda_{\text{curr}} - \lambda_{\text{prev}})$  (secant; constant on first step)
7:     Apply loading  $\lambda_{\text{next}}$  to injection vector
8:     Corrector: run Newton (A3) warm-started from  $x_{\text{pred}}$ 
9:     if Newton converged then
10:      Record CpfPoint; advance  $\lambda$ ;  $\Delta\lambda \leftarrow \min(\text{step\_grow} \cdot \Delta\lambda, \text{step\_max})$ 
11:      if  $\lambda \geq \lambda_{\max}$  or  $V_{\min} < \text{opts.vm\_min\_pu}$  then break
12:    end if
13:  else

```

```

14:      $\Delta\lambda \leftarrow \text{step\_shrink} \cdot \Delta\lambda$ 
15:     if  $\Delta\lambda < \text{opts.step\_min}$  then declare nose point; break
16:     end if
17: end if
18: end for
19: return  $\text{CpfResult}\{\text{trace, nose point } \lambda^*, \text{status}\}$ 
20: end procedure

```

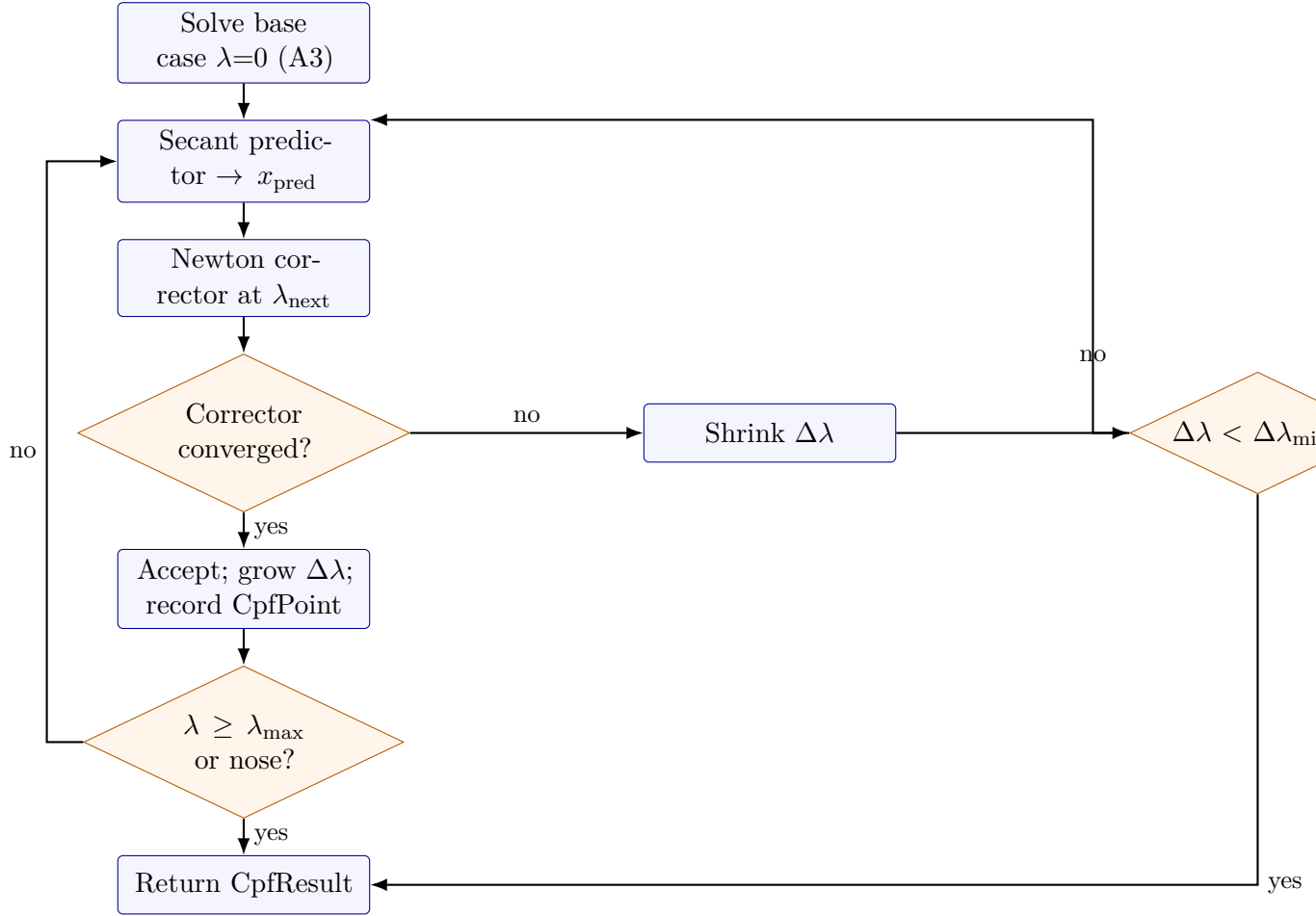


Figure 17: Algorithm A13: CPF predictor-corrector ($\text{CpfSolver}::\text{solve}$).

28.4.2 A14: Homotopy Continuation Fallback ($\text{HomotopyContinuationSolver}$)

The homotopy solver (Section 13.2) scales all injections from zero to their target values and traces the solution path in adaptive steps. It is triggered as Phase 4 of the robust pipeline when direct Newton fails.

Algorithm A14: Homotopy Continuation

```

1: procedure  $\text{HOMOTOPYCONTINUATIONSolver}::\text{SOLVE}(\text{sys}, \text{opt})$ 
2:    $\Delta\lambda \leftarrow \text{opt.step\_init}$ ;  $\text{SCALE\_TO\_LAMBDA}(\text{base}, 0)$ ; solve trivial system (always converges)
3:    $\lambda \leftarrow 0$ 
4:   while  $\lambda < 1 - \varepsilon$  do
5:      $\lambda_{\text{next}} \leftarrow \min(1, \lambda + \Delta\lambda)$ 
6:      $\text{trial} \leftarrow \text{copy of sys}$ ;  $\text{SCALE\_TO\_LAMBDA}(\text{trial}, \lambda_{\text{next}})$ 

```

```

7:    $r \leftarrow \text{NEWTONSOLVER}::\text{SOLVE}(\text{trial})$  ▷ sub-budget
8:   if  $r.\text{converged}$  then
9:      $\lambda \leftarrow \lambda_{\text{next}}$ ; current solution  $\leftarrow r$ 
10:     $\Delta\lambda \leftarrow \min(2\Delta\lambda, \Delta\lambda_{\text{max}})$ 
11:  else
12:     $\Delta\lambda \leftarrow \Delta\lambda/2$ 
13:    if  $\Delta\lambda < \Delta\lambda_{\text{min}}$  then return failed
14:  end if
15: end if
16: end while
17: return  $r$  at  $\lambda = 1$ 
18: end procedure

```

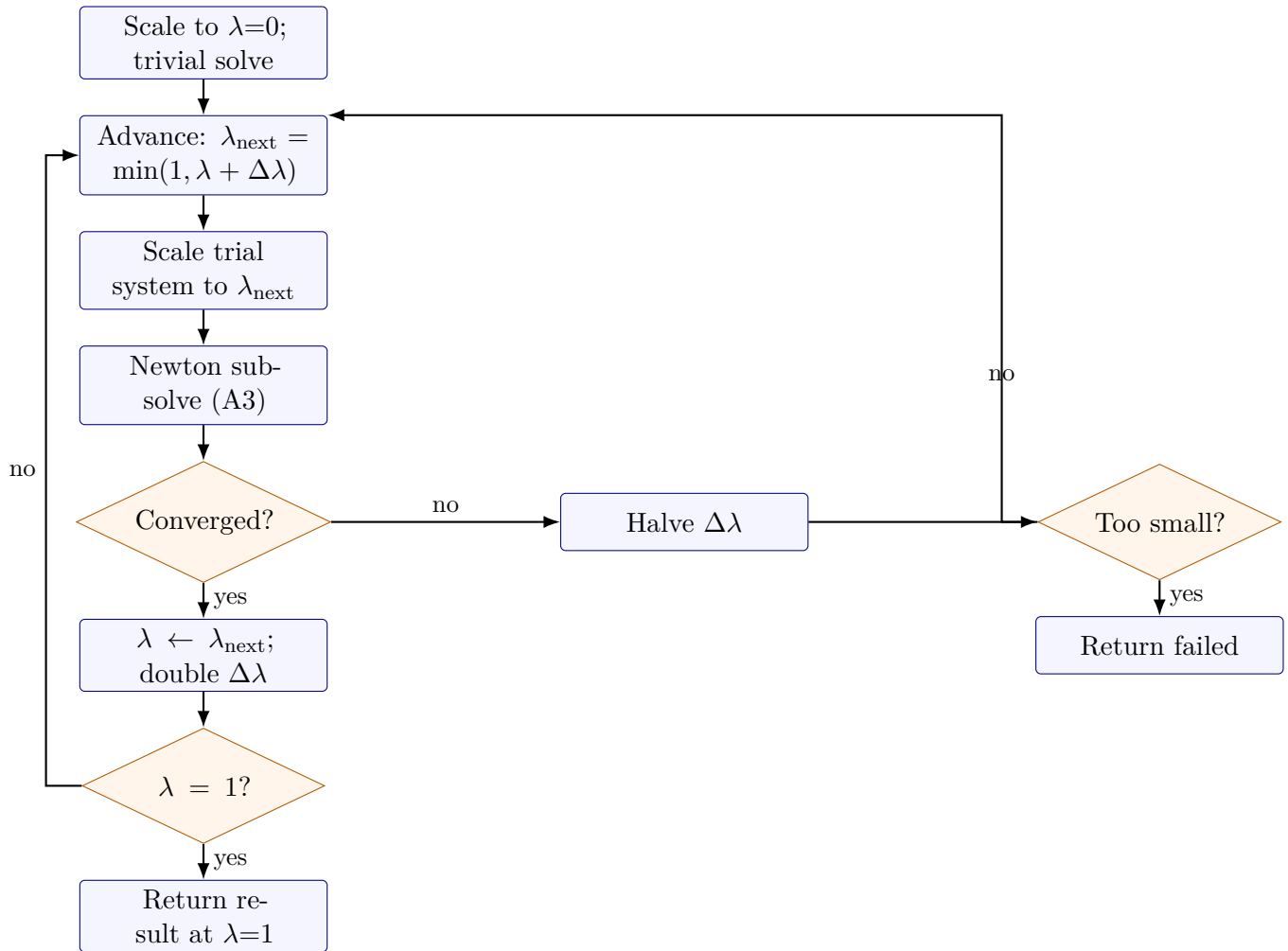


Figure 18: Algorithm A14: homotopy continuation fallback.

28.4.3 A15: Newton–Krylov GMRES Fallback (NewtonKrylovSolver)

The Newton–GMRES solver (Section 13.4) replaces the direct SparseLU inner solve with GMRES(m) using a Schur-complement AC/DC block preconditioner.

Algorithm A15: Newton–Krylov GMRES

- 1: **procedure** NEWTONKRYLOVSOLVER::SOLVE($data, opt$)
- 2: Compute $F(x)$; form sparse Jacobian J

```

3:   Build Schur preconditioner  $M$ : factorize  $J_{AC}$  and  $J_{DC}$  blocks
4:   for Newton outer iteration  $k = 1, \dots, opt.max\_iter$  do
5:       if  $\|F\|_\infty < opt.tol$  then break
6:       end if
7:       GMRES( $m$ ) inner loop:
8:       for outer cycle  $\ell = 1, \dots, max\_outer$  do
9:            $r \leftarrow M^{-1}(F - Jx)$ ;  $\beta \leftarrow \|r\|$ 
10:          for Arnoldi step  $j = 1, \dots, m$  do
11:               $w \leftarrow M^{-1}Jv_j$ ; Modified Gram-Schmidt orthogonalise
12:              Apply Givens rotations; update  $\|r\|$  estimate
13:              if  $\|r\| < tol\_gmres$  then break
14:              end if
15:          end for
16:          Solve upper-Hessenberg LS; back-substitute  $\Delta x$ 
17:          if  $\|r\| < tol\_gmres$  then break
18:          end if
19:      end for
20:       $x \leftarrow x + \Delta x$ ; re-evaluate  $F(x)$ 
21:  end for
22:  return NewtonKrylovResult{ $x$ , stats, converged}
23: end procedure

```

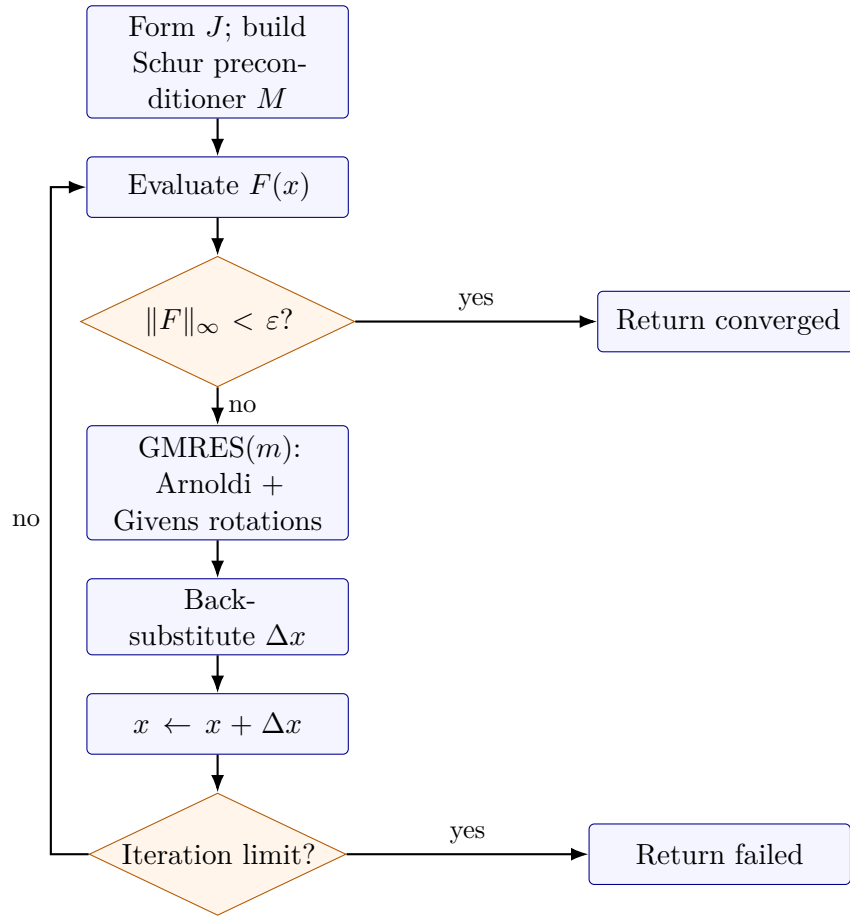


Figure 19: Algorithm A15: Newton-Krylov GMRES fallback.

28.5 Power Model Builders (A16–A19)

28.5.1 A16: LinDistFlow LP Builder (`lindistflow_builder`)

A16 assembles the linearised DistFlow LP/MILP used by both the distribution PF solver and the ONR module. Variables are branch active and reactive power flows P_{ij} , Q_{ij} , squared voltage magnitude v_i , and (in the ONR case) binary switch status u_{ij} .

Algorithm A16: LinDistFlow LP Construction

```
1: procedure BUILD_LINDISTFLOW(sys, opt)
2:   Project to canonical; enumerate  $n$  buses,  $l$  branches
3:   Declare continuous:  $P_{ij}, Q_{ij}$  for each branch;  $v_i$  for each bus
4:   if network reconfiguration mode then
5:     Declare binary:  $u_{ij} \in \{0, 1\}$  for each switchable branch
6:     Add big- $M$  flow coupling:  $-Mu_{ij} \leq P_{ij} \leq Mu_{ij}$ 
7:   end if
8:   for each bus  $i$  do
9:     Power balance:  $\sum_{j:(i,j) \in \mathcal{E}} P_{ij} - \sum_{k:(k,i) \in \mathcal{E}} P_{ki} + r_{ki}P_{ki}^2/v_i = P_{G,i} - P_{L,i}$  (linearised)
10:  end for
11:  for each branch  $(i, j)$  do
12:    Voltage drop:  $v_j = v_i - 2(r_{ij}P_{ij} + x_{ij}Q_{ij})$ 
13:    Flow limits:  $|P_{ij}| \leq P_{ij}^{\max}$ ,  $|Q_{ij}| \leq Q_{ij}^{\max}$ 
14:  end for
15:  Voltage bounds:  $V_{\min}^2 \leq v_i \leq V_{\max}^2$ 
16:  if spanning-tree radiality required then
17:    Add single-commodity flow constraints to enforce tree topology
18:  end if
19:  Objective: minimize losses  $\sum_{ij} r_{ij}(P_{ij}^2 + Q_{ij}^2)/v_i$  (linearised) or load shedding
20:  return LP/MILP model
21: end procedure
```

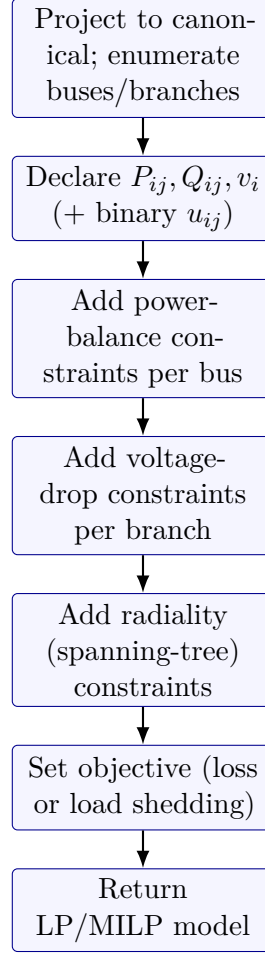


Figure 20: Algorithm A16: LinDistFlow LP builder.

28.5.2 A17: AML AC OPF Model Builder (`acopf_builder`)

A17 constructs a standard single-period AC OPF model using the Algebraic Modelling Layer (AML). It produces a symbolic model object that can be compiled to a dense or sparse NLP via A9.

Algorithm A17: AML AC OPF Builder

```

1: procedure BUILD_ACOPF(sys, opt)
2:   can  $\leftarrow$  PROJECT_TO_CANONICAL(sys)
3:   Declare:  $\theta_i \in [-\pi, \pi]$ ,  $V_i \in [V_i^{\min}, V_i^{\max}]$ 
4:   Declare:  $P_g \in [P_g^{\min}, P_g^{\max}]$ ,  $Q_g \in [Q_g^{\min}, Q_g^{\max}]$ 
5:   Objective:  $\min \sum_g c_g(P_g)$  (polynomial cost)
6:   for each bus i do
7:     Add AC power balance:  $P_i(\theta, V) = P_{G,i} - P_{L,i}$ 
8:     Add AC reactive balance:  $Q_i(\theta, V) = Q_{G,i} - Q_{L,i}$ 
9:   end for
10:  for each branch (i, j) do
11:    Add apparent power limit:  $P_{ij}^2 + Q_{ij}^2 \leq S_{ij,\max}^2$ 
12:  end for
13:  return model
14: end procedure

```

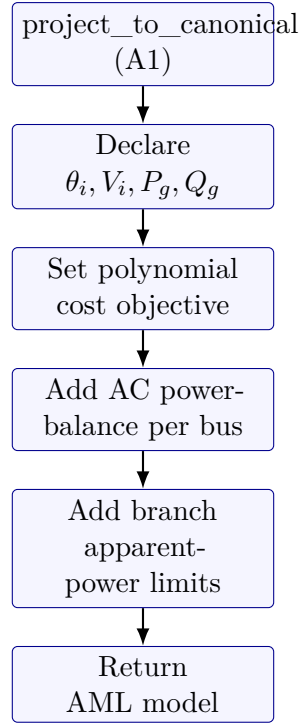



Figure 21: Algorithm A17: AML AC OPF model builder.

28.5.3 A18: AML Hybrid AC/DC OPF Model Builder (`acdcopf_builder`)

A18 extends the AC OPF model of A17 with DC network variables and VSC converter linking constraints.

Algorithm A18: AML Hybrid AC/DC OPF Builder

```

1: procedure BUILD_ACDCOPF(sys, opt)
2:   Build AC OPF base model (A17)
3:   Declare:  $V_{dc,i}$  for each DC bus;  $P_{dc,ij}$  for DC branches
4:   Add DC power balance per DC bus:  $\sum_j P_{dc,ij} = P_{gen,dc,i} - P_{load,dc,i}$ 
5:   for each VSC converter ( $k_{ac}, k_{dc}$ ) do
6:     Coupling:  $P_{ac,k} + P_{dc,k} + P_{loss,k} = 0$ 
7:     PQ circle:  $P_{ac,k}^2 + Q_{ac,k}^2 \leq S_{vsc,k}^2$ 
8:     DC branch flow:  $|P_{dc,ij}| \leq P_{dc,ij}^{max}$ 
9:   end for
10:  return hybrid model
11: end procedure
  
```

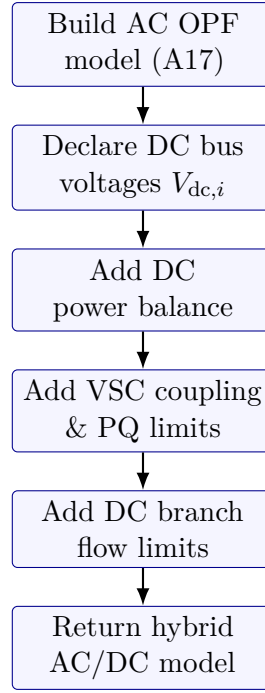


Figure 22: Algorithm A18: AML hybrid AC/DC OPF model builder.

28.5.4 A19: MILP SCUC Builder (`scuc_builder`)

A19 assembles the Security-Constrained Unit Commitment (SCUC) MILP over a planning horizon of T periods. Binary commitment variables $y_{g,t}$ enforce minimum up/down-time, ramp rate, and N-1 contingency requirements.

Algorithm A19: MILP SCUC Builder

```

1: procedure BUILD_SCUC(sys, ts, opt)
2:    $N_g \leftarrow |\text{sys.generators}|$ ;  $T \leftarrow \text{ts.num\_steps}$ 
3:   Declare:  $y_{g,t} \in \{0, 1\}$  (commitment);  $P_{g,t}$  (dispatch)
4:   Declare:  $u_{g,t}$  (startup),  $d_{g,t}$  (shutdown) binary auxiliaries
5:   Objective:  $\min \sum_{g,t} [c_{g,t} P_{g,t} + C_g^u u_{g,t} + C_g^d d_{g,t}]$ 
6:   for each generator  $g$ , time  $t$  do
7:     Commitment logic:  $y_{g,t} - y_{g,t-1} = u_{g,t} - d_{g,t}$ 
8:     Min up-time:  $\sum_{\tau=t}^{t+T_g^u-1} y_{g,\tau} \geq T_g^u u_{g,t}$ 
9:     Min down-time:  $\sum_{\tau=t}^{t+T_g^d-1} (1 - y_{g,\tau}) \geq T_g^d d_{g,t}$ 
10:    Dispatch bounds:  $y_{g,t} P_g^{\min} \leq P_{g,t} \leq y_{g,t} P_g^{\max}$ 
11:    Ramp limits:  $|P_{g,t} - P_{g,t-1}| \leq R_g \Delta t$ 
12:  end for
13:  for each time  $t$  do
14:    Power balance:  $\sum_g P_{g,t} = \sum_i P_{L,i,t}$ 
15:    Reserve:  $\sum_g (y_{g,t} P_g^{\max} - P_{g,t}) \geq R_{\min,t}$ 
16:  end for
17:  if N-1 security requested then
18:    Add contingency power-flow constraints for each monitored branch
19:  end if
20:  return MILP model
21: end procedure

```

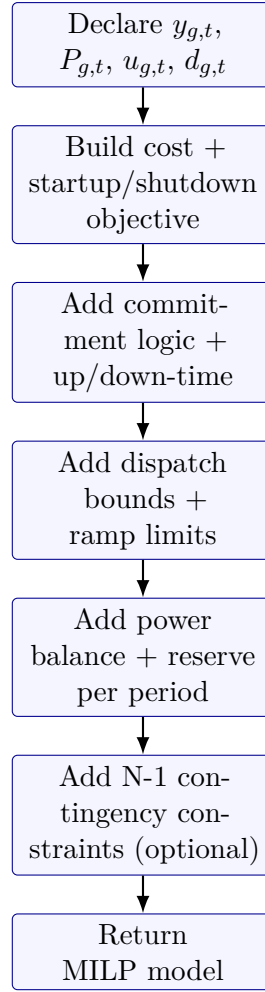


Figure 23: Algorithm A19: MILP SCUC builder.

28.6 Network Reconfiguration (A22)

28.6.1 A22: Optimal Network Reconfiguration Orchestration (solve_onr)

A22 summarises the reconfiguration family described in Section 16. The steady AC ONR path solves a LinDistFlow MILP with native B&C and verifies the returned topology with Newton PF. The fault-triggered hybrid path builds a restoration MILP over AC/DC/VSC edges, tries HiGHS, retries native B&C on HiGHS failure or malformed solution vectors, and returns explicit solver certificates plus model-scope/validity flags rather than a full post-reconfiguration AC/DC PF certificate.

Algorithm A22: ONR Orchestration

```

1: procedure SOLVE_ONR( $sys, opt$ )
2:    $can \leftarrow \text{PROJECT\_TO\_CANONICAL}(sys)$  ▷ A1
3:    $model \leftarrow \text{BUILD\_LINDISTFLOW}(can, \{\text{radiality}=\text{true}\})$  ▷ A16
4:   Configure solver limits: time limit, MIP gap, branch-and-cut options
5:    $sol \leftarrow \text{HIGHS\_SOLVE}(model)$ 
6:   if HiGHS unavailable, failed, or returned malformed solution then
7:      $sol \leftarrow \text{NATIVE\_BC\_SOLVE}(model)$ 
8:   end if
9:   if  $sol.status \neq \text{OPTIMAL}$  and  $\neq \text{FEASIBLE}$  then

```

```

10:     return infeasible topology result
11: end if
12: Extract open branch set  $\mathcal{O} \leftarrow \{ij : u_{ij}^* < 0.5\}$ 
13: Validate equality rows, inequality rows, bounds, and binary integrality
14: if post-solve checks fail then return infeasible
15: end if
16: Extract structured open/closed BranchRef sets and solver certificate
17: Set model_scope and validity flags for PF/OPF coverage
18: return TopoReconfResult{ $\mathcal{O}$ , objective, certificate, validity}
19: end procedure

```

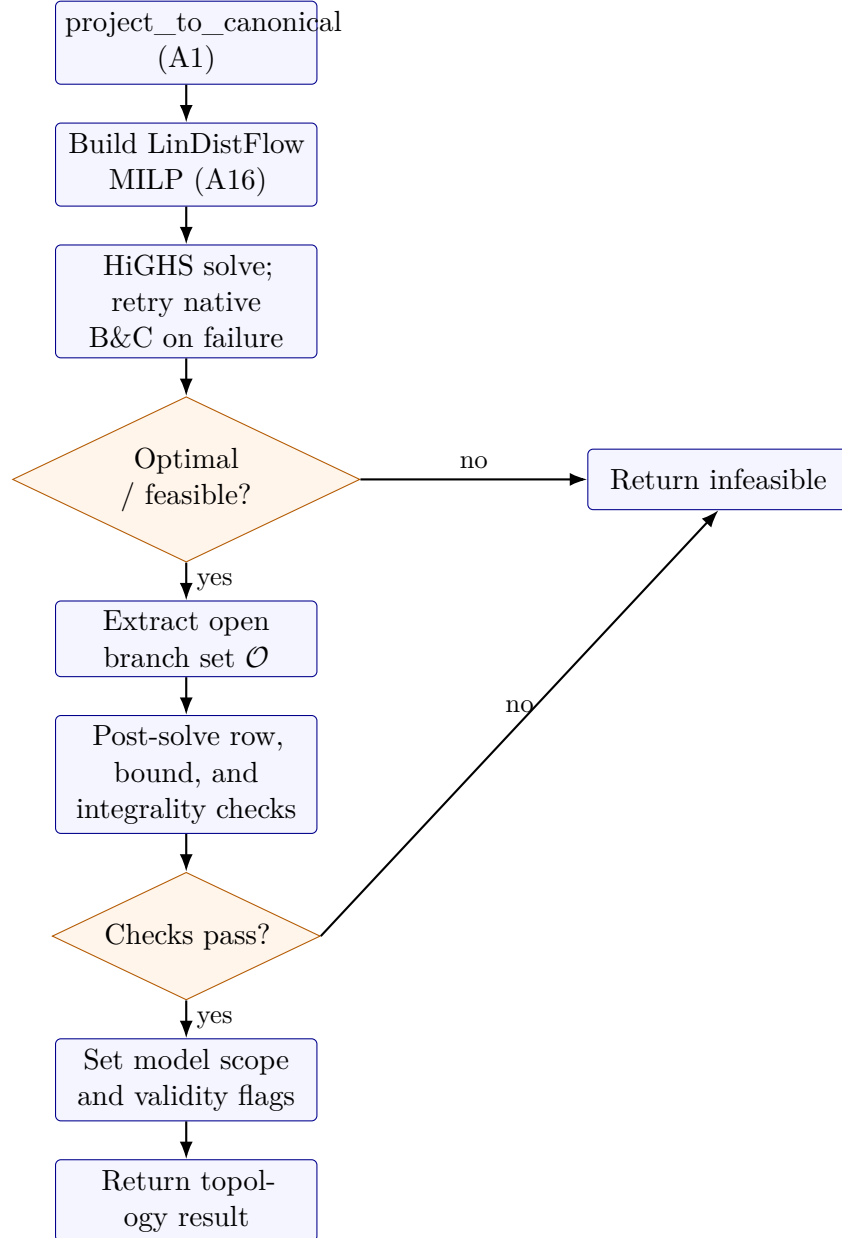


Figure 24: Algorithm A22: steady ONR and fault-triggered topology reconfiguration orchestration.

28.7 Time-Series Simulation (A20–A21)

28.7.1 A20: Annual Production Simulation (Hierarchical)

A20 decomposes the year into a four-level hierarchy (L0 annual plan $\rightarrow$ L1 monthly $\rightarrow$ L2 weekly unit commitment $\rightarrow$ L3 per-step OPF/PF), as described in Section 23.

Algorithm A20: Annual Production Simulation

```
1: procedure RUN__ANNUAL__PRODUCTION__SIM(sys, ts, opt)
2:   L0: Compute annual peak plan; set generation capacities for the year
3:   Partition ts into B blocks (monthly or weekly) with boundaries  $[t_b, t_{b+1})$ 
4:   for each block  $b = 1, \dots, B$  do
5:     Extract time-series slice  $ts_b \leftarrow ts[t_b : t_{b+1}]$ 
6:     L1/L2: Run SCUC (A19  $\rightarrow$  HiGHS) on  $ts_b \Rightarrow$  commitment schedule  $y_{g,t}$ 
7:     for each step  $t \in [t_b, t_{b+1})$  do
8:       Apply profile values to sys (loads, renewables)
9:       if opt.use_opf then
10:        L3 (OPF): SOLVE_DC_OPF(sys, commitment= $y_{g,t}$ )
11:      else
12:        L3 (PF): ADAPTIVE_SOLVER::SOLVE(sys)
13:      end if
14:      Record AnnualStepResult{ $P_g$ , load, loss, carbon, gen cost}
15:    end for
16:  end for
17:  Aggregate: stratified sampling statistics, peak demand, total energy
18:  return AnnualSimResult
19: end procedure
```

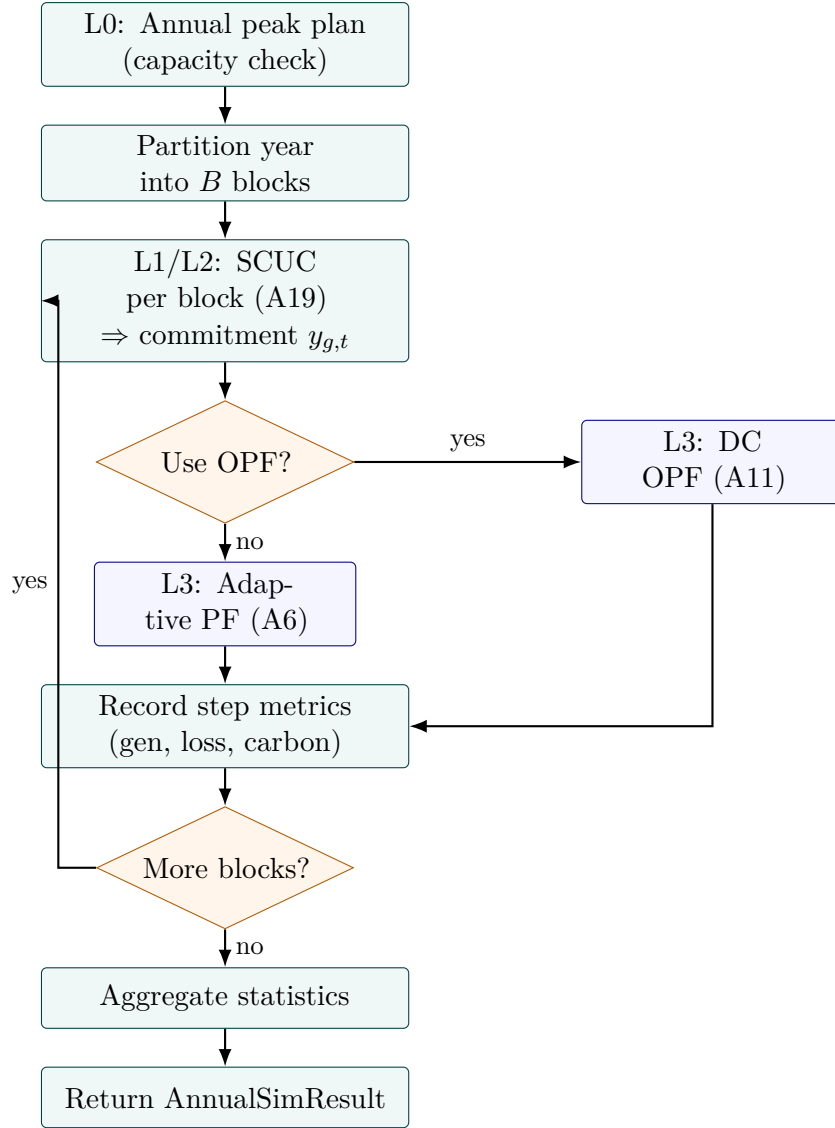


Figure 25: Algorithm A20: hierarchical annual production simulation.

28.7.2 A21: Multi-Year Lifecycle Simulation

A21 iterates over N_{yr} years, applying storage SoH and renewable capacity degradation, running A20, and accumulating NPV-discounted cost and carbon metrics with Tier-1/2/3 error bounds.

Algorithm A21: Lifecycle Simulation

```

1: procedure RUN_LIFECYCLE_SIMULATION( $sys_0, ts, opt$ )
2:   Initialise:  $sys \leftarrow sys_0$ ;  $NPV_{\text{total}} \leftarrow 0$ ;  $E_{\text{CO}_2} \leftarrow 0$ 
3:   for year  $y = 1, \dots, opt.n\_years$  do
4:     for each storage unit  $s$  do
5:        $cap_{s,y} \leftarrow cap_{s,0} (1 - \delta_s^{\text{SoH}})^y$ 
6:     end for
7:     for each PV / renewable source  $r$  do
8:        $P_{r,y}^{\text{max}} \leftarrow P_{r,0}^{\text{max}} (1 - \delta_r^{\text{PV}})^y$ 
9:     end for
10:     $result_y \leftarrow \text{RUN\_ANNUAL\_PRODUCTION\_SIM}(sys, ts, opt.annual)$  ▷ A20
11:    NPV discount:  $NPV_y \leftarrow result_y.cost / (1 + r_d)^y$ 

```

```

12:     NPVtotal += NPVy;  ECO2 += resulty.emissions
13: end for
14:   Compute LCOE = NPVtotal / Eenergy
15:   Compute Tier-1/2/3 carbon error bounds (confidence intervals)
16:   return LifecycleResult{LCOE, ECO2, per-year results, error bounds}
17: end procedure

```

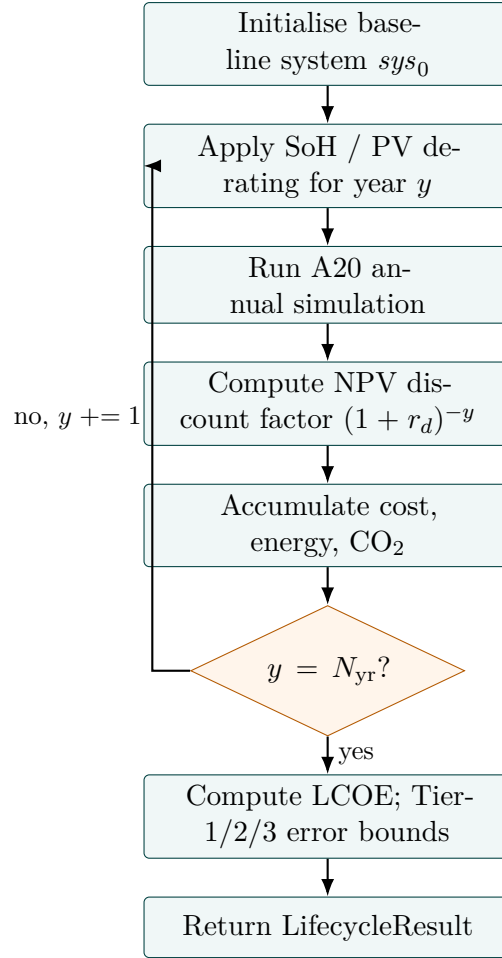


Figure 26: Algorithm A21: multi-year lifecycle simulation.

28.8 Short-Circuit Analysis (A23)

28.8.1 A23: Short-Circuit Z-bus Solver (compute_short_circuit)

A23 implements the Z-bus Thévenin method for balanced and unbalanced fault analysis following IEC 60909 (Section 29.15). The factorised $\mathbf{Y}_{\text{fault}}$ is reused across all bus solves, so the per-bus cost is a single triangular back-substitution.

Algorithm A23: Short-Circuit Z-bus

```

1: procedure COMPUTE_SHORT_CIRCUIT(sys, opt: SOptions)
2:   Build bus ID → local index map from canonical buses
3:   Assemble  $\mathbf{Y}_{\text{fault}}$  (sparse, COO): branch admittances + generator subtransient shunts
    $y_g = 1/(jx_d'')$ 
4:   Factorize: SparseLU( $\mathbf{Y}_{\text{fault}}$ )
5:   for each fault bus  $k \in \{1, \dots, n\}$  do

```

```

6:   Solve  $\mathbf{Y}_{\text{fault}} \mathbf{z}_k = \mathbf{e}_k \Rightarrow Z_{kk} = (\mathbf{z}_k)_k$  (Thévenin impedance)
7:    $V_0 \leftarrow c \angle 0^\circ$  where  $c = \text{opt.c\_factor}$  (default 1.1)
8:   if 3-phase fault then
9:      $I_k'' = V_0 / Z_{kk}$ 
10:  else if SLG fault then
11:     $I_k'' = 3V_0 / (Z_1 + Z_2 + Z_0)$  where  $Z_1 = Z_{kk}$ ,  $Z_2 \approx Z_1$ ,  $Z_0$  from zero-seq Y-bus
12:  else if LL fault then
13:     $I_k'' = \sqrt{3}V_0 / (Z_1 + Z_2)$ 
14:  end if
15:   $S_k = S_{\text{base}} |V_0|^2 / |Z_{kk}|$ ;  $\kappa = 1.02 + 0.98 e^{-3R/X}$ ;  $i_p = \kappa \sqrt{2} |I_k''|$ 
16:  Store BusFaultResult{ $k, Z_{kk}, I_k'', S_k, i_p$ }
17: end for
18: return SCResult{bus_results,  $S_{\text{base}}$ , summary()}
19: end procedure

```

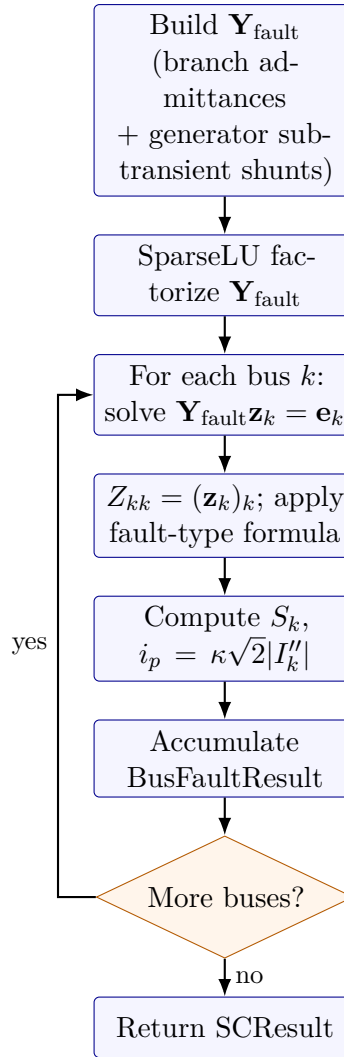


Figure 27: Algorithm A23: short-circuit Z-bus solver (`compute_short_circuit`).

29 Test Suite: Design, Coverage, and Results

This chapter documents the current CTest suite in the `build_rel` configuration. The current registration contains **436 CTest entries**: 434 active tests pass and 2 MATPOWER parser

cases are intentionally skipped. The skipped tests (`case533mt_hi` and `case533mt_lo`) contain arithmetic expressions ($135/\sqrt{3}$) that the current MATPOWER parser does not evaluate; they are identified and skipped at runtime rather than reported as failures.

29.1 Overview of the New Test Suites

Table 157 summarises the generated CTest registration ranges for every test executable in the current build tree.

Table 157: CTest suites registered in `tests/CMakeLists.txt`.

Executable	Focus area	CTest IDs	Count
<code>test_hacdcpf</code>	Core public API and baseline PF/OPF checks	1–22	22
<code>test_all_components_pf</code>	all-component PF smoke coverage	23–26	4
<code>test_unified_pf_upgrade</code>	unified PF option and compatibility checks	27–36	10
<code>test_acopf_dcopf_crossval</code>	ACOPF/DCOPF/PF cross-validation and DCOPF regressions	37–46	10
<code>test_enhanced_solver</code>	robust PF/island solver checks	47–48	2
<code>test_scaling_regression</code>	scaling and conditioning guardrails	49–55	7
<code>test_hacdcdss</code>	Model, power models, power flow, DC OPF smoke tests	56–72	17
<code>test_io_json</code>	JSON I/O, MATPOWER parsing, result round-trips	73–108	36
<code>test_rich_components</code>	rich component round-trips and builders	109–133	25
<code>test_matpower_cases</code>	Power-flow convergence on MATPOWER cases	134–220	87
<code>test_validation</code>	Validation API, levels, rich-component guardrails	221–252	32
<code>test_solver_capabilities</code>	Compile-time solver flags, OPF audit helper, hybrid fallback suppression	253–269	17
<code>test_advanced_pf</code>	Island detection, distributed slack, PV/PQ control	270–295	26
<code>test_three_phase_pf</code>	Three-phase data model, projection, meshed, JSON	296–319	24
<code>test_multiport_vpp_er</code>	Multi-port Energy Router and VPP modes	320–333	14
<code>test_powerflow_performance</code>	Large-system power-flow benchmark sweep	334	1
<code>test_topology_crossval</code>	Fault topology reconfiguration regressions, ONR, exhaustive enum., 33bw	335–345	11
<code>test_crossmodule_integration</code>	Pipeline: OPF→PF→Carbon, ONR→OPF→PF→Carbon	346–347	2
<code>test_short_circuit_crossval</code>	Short-circuit Z-bus cross-validation	348–354	7
<code>test_three_phase_nr</code>	Three-phase Newton–Raphson power flow	355–370	16
<code>test_resilience_assessment</code>	Resilience heuristic, strict MIP, and hybrid FMEA regressions	371–393	23
<code>test_graph</code>	Graph topology, contraction, series/pendant, recovery	394–418	25
<code>test_graph_kron</code>	Kron Y-bus reduction and voltage recovery	419–425	7
<code>test_distribution_pipeline</code>	Distribution CB/switch/VSC pipeline	426	1
<code>test_three_stage_reliability</code>	Three-stage native MILP fault-recovery reliability	427–436	10
Grand total		1–436	436

29.2 `test_acopf_dcopf_crossval`: ACOPF/DCOPF Cross-Validation

`tests/test_acopf_dcopf_crossval.cpp` contributes ten CTest entries (#37–#46). In addition to the existing ACOPF/DCOPF/PF consistency and dispatch-quality checks, the current suite includes DCOPF regressions for:

- per-bus reporting of direct isolated-island shedding through `DCOPFResult::load_shedding_mw`;

- pruning dead-island generators, fixed injections, and branches before LP construction, so isolated generator lower bounds cannot make the surviving slack-connected island infeasible;
- slack-bus balance in the independent DCOPF feasibility audit;
- LMP sign, scale, uncongested uniformity, and congestion redispatch.

29.3 test_hacdcdss: Model, Power Models, Power Flow, DC OPF

tests/test_hacdcdss.cpp provides smoke-level coverage for the four major logical modules exposed through the hacdcpf API. The file was completely rewritten from the original version (which referenced the now-defunct hacdcdss:: library) using the current hacdcpf namespace and type system.

29.3.1 Module: Model (tests #56–#59)

- **Construct and validate 2-bus AC network:** a slack bus plus a 50 MW PQ load bus with a single π -model branch and one generator are assembled programmatically; `val::validate(sys)` returns `ok()==true`.
- **Out-of-range branch bus:** setting `to_bus=99` on a 2-bus network triggers an `Error`-level issue.
- **Missing slack bus:** removing the slack type and `is_slack` flag causes validation to fail.
- **Hybrid AC/DC network:** a 2-bus DC network with a `VSCConverter` is appended; all collections are verified to have the expected cardinality.

29.3.2 Module: Power Models (tests #60–#62)

Three tests verify Y-bus correctness through observable physical effects:

- **Flat-start, zero-load network:** residual is below 10^{-6} pu after one iteration.
- **Y-bus symmetry:** swapping `from_bus/to_bus` on every branch in a symmetric π -model network produces identical per-bus voltage magnitudes, confirming that the admittance matrix is symmetric.
- **P/Q consistency:** after convergence all bus voltage magnitudes are positive and all angles are finite.

29.3.3 Module: Power Flow (tests #63–#68)

- **Result sizing:** `r.vm` and `r.va` have the same length as `sys.ac.buses`.
- **Lightly loaded 2-bus:** 10 MW load; converged with slack voltage held at 1.0 pu and load-bus voltage in $[0.9, 1.1]$ pu.
- **Physical voltage monotonicity:** higher load produces a lower load-bus voltage.
- **IEEE-14 AC/DC builder:** the pre-built case converges with residual $< 10^{-4}$.
- **Island detection:** a single-bus islanded system completes without exception.
- **Distributed slack:** two generators on the slack bus yield a converged result.

29.3.4 Module: DC OPF (tests #69–#72)

- **3-bus economic dispatch:** a 3-bus network with a cheap (\$20/MWh) and an expensive (\$40/MWh) generator and 30 MW load; the cheaper generator dispatches at least as much as the expensive one, and total dispatch equals 30 MW within 1 MW.
- **Native LP objective and LMP scale:** verifies that the native LP DCOPF path reports objective and nodal LMP values on the MW/MWh scale expected by the public result struct.
- **Single-generator coverage:** on the lightly loaded 2-bus network the single generator dispatches within its declared $[P_{\min}, P_{\max}]$ bounds.
- **case9 feasibility:** the case-9 MATPOWER network is parsed, solved, and every generator's dispatch is within its MW limits.

All 17 tests pass.

29.4 test_io_json: JSON I/O and MATPOWER Parsing

`tests/test_io_json.cpp` validates the `hacdcpf::io` module through 36 independent test cases covering JSON system round-trips, safe monadic wrappers, result JSON, MATPOWER parsing, and JPC JSON compatibility.

29.4.1 JSON Round-trip Fidelity (tests #73–#76)

Each test constructs a `HybridPowerSystem`, serialises it with `to_json()`, deserialises with `from_json()`, and asserts that bus count, branch count, and generator count are preserved exactly. Compact-output mode (`indent=0`) is verified to produce a non-empty string with no leading whitespace.

29.4.2 Safe Monadic API (tests #77–#84)

Eight tests exercise the `Result<T>`-returning wrappers:

- **try_from_json:** valid input returns a success `Result`; malformed JSON and empty strings return `Error`.
- **try_load_json:** a non-existent file returns an error variant whose code is `FileNotFound`; a valid file path returns a success variant.
- **ImportMode::Permissive:** permissive mode still parses a well-formed file without throwing.
- File round-trip via `save_json/load_json` for the IEEE-24 builder case.

29.4.3 OPF/PF Result Serialisation (tests #85–#86)

An `ACOPFResult` with a known objective value is serialised and the `converged` flag and floating-point objective are verified after round-trip. A minimal `PowerFlowResult` is verified to produce a non-empty JSON object.

29.4.4 MATPOWER Parser (tests #87–#106)

Twenty tests parse representative MATPOWER `.m` files from `data/`:

- **Topology checks** (case5, case9, case14, case30, case57, case118, case300, case33bw, case39, case_ieee30, case24_ieee_rts): bus counts, branch endpoints, generator-limit fields.

- **Cross-mode round-trip** (case14): `parse` $\rightarrow$ `to_json` $\rightarrow$ `from_json` $\rightarrow$ power-flow convergence.
- **Heavy cases** (case1354pegase, case2383wp) parse without throwing.
- **Piecewise economics file** (case3_case4_formal_soc_piecewise.json): confirmed to be valid non-empty JSON (economics schema, not a `HybridPowerSystem` schema).

29.4.5 JPC JSON Compatibility (tests #107–#108)

Two JPC-format regressions protect against data loss in the MATPOWER-like JSON path:

- all twelve rich-component JPC tables round-trip with non-empty collection sizes, protecting the richer component payload from accidental omission;
- branch dictionary keys, generator slack metadata, and quadratic generator-cost coefficients `cost_c2/cost_c1/ cost_c0` survive the `to_jpc_json` $\rightarrow$ `from_jpc_json` cycle.

All 36 tests pass. The single SOC-piecewise test is intentionally a JSON-parse check because that file stores piecewise cost curves rather than network topology.

29.5 test_rich_components: 24-Type Component Coverage

`tests/test_rich_components.cpp` exercises every non-trivial component type defined in `hacdcpf/model/`.

29.5.1 Component Round-trip Tests (tests #109–#129)

For each of the 21 individual component types listed in Table 158, the test:

1. Constructs a minimal `HybridPowerSystem` and appends an instance of the component with representative field values.
2. Serialises with `to_json` and deserialises with `from_json`.
3. Asserts that the collection size and at least two key fields are preserved within floating-point tolerance ($|e| < 10^{-9}$).

The `DCBranch` JSON contract serialises `rate_a_mva`; derived power-flow reporting can still prefer `s_max_mva` when it is present internally.

29.5.2 System Builder Tests (tests #130–#133)

Four larger builders are exercised end-to-end:

- **IEEE-14 AC/DC builder**: JSON round-trip preserves all component collections (buses, branches, generators, converters, DC side).
- **Case33 microgrid builder**: system converges after a full `to_json/from_json` round-trip.
- **Dist33 microgrid DER builder**: DER collections (storage, renewables, PV systems, flexible loads) are non-empty after construction.
- **Comprehensive hybrid builder**: VSC converters and DC side are populated after construction.

All 25 tests pass.

Table 158: Component types covered by `test_rich_components`.

Component	Key fields verified	Location
Transformer2W	vn_hv_kv, sn_mva	sys.ac
Transformer3W	sn_mva	sys.ac
StaticGenerator	p_mw, q_mvar	sys.ac
RenewableGen	p_mw, qmax_mvar	sys.ac
PVSystem	p_mw, pmax_mw	sys.ac
Storage	pmax_mw, soc_init	sys.ac
FlexibleLoad	p_mw, flex_up_mw	sys.ac
AsymmetricLoad	pa_mw, pb_mw, pc_mw	sys.ac
Shunt	gs_mw, bs_mvar	sys.ac
ChargingStation	p_total_kw	sys.ac
AsynchronousMotor	sn_mva, cos_phi	sys.ac
MobileStorage	p_mw, soc_init	sys
VirtualPowerPlant	p_output_mw, pmax_mw	sys.ac
Microgrid	internal_buses, p_exchange_mw	sys.ac
VSCConverter	p_ac_mw, q_ac_mvar	sys
DCDCConverter	p_set_mw	sys.dc
EnergyRouter	ports[0].p_set_mw	sys
DCLoad	p_mw	sys.dc
PVArrayDC	p_set_mw	sys.dc
StaticGeneratorDC	p_set_mw, pmax_mw	sys.dc
DCBranch	r_pu, rate_a_mva	sys.dc
Switch	bus_from, closed	sys.ac
ExternalGrid	bus, vm_pu	sys.ac

29.6 test_matpower_cases: MATPOWER Convergence Coverage

`tests/test_matpower_cases.cpp` exercises power-flow convergence across all three tiers of the MATPOWER library bundled in `data/`. Following an expansion in the most recent cycle, the suite now covers **87 cases** (up from 52), with 2 skipped due to a parser limitation.

29.6.1 Tier-1: Core Cases (tests #134–#146)

Nine canonical cases from 9 to 300 buses are solved with the default Newton solver and checked for convergence (`r.converged==true`) and finite residual. Additional physical sanity checks include:

- **case5:** total generation approximately covers total load (within 5% tolerance).
- **case9 slack-bus voltage:** the solved voltage at the slack bus is within the regulatory range $[0.95, 1.10]$ pu.
- **case14 slack angle:** the slack bus angle is zero within 10^{-6} rad.
- **case14 generation–load balance:** total P_{gen} covers total P_{load} within 5%.
- **case30 Newton vs FDPF:** both solvers converge and their bus-voltage vectors agree within 10^{-4} pu (per-element L^∞ norm).

All 13 Tier-1 tests pass.

29.6.2 Tier-2: Medium Cases (tests #147–#216)

55 cases from 4 to 2000 buses are solved individually; each is checked for convergence. A batch test (#216) verifies that the convergence rate across all 54 non-skipped cases is at least 85 %. In practice the observed rate is **100 %** (54/54 active cases; 2 skipped for arithmetic-expression parser limitation).

The original 25-case Tier-2 suite has been expanded with 30 additional cases covering small distribution networks, the IEEE RTS-GMLC benchmark, and additional European/synthetic transmission systems.

Table 159: Tier-2 MATPOWER cases solved in `test_matpower_cases` (original 25 + 30 new).

Case file	Buses	Network type
<i>Original cases</i>		
case33bw	33	Distribution (radial)
case33mg	33	Microgrid
case69	69	Distribution (radial)
case85	85	Distribution
case89pegase	89	PEGASE transmission
case118zh	118	IEEE 118-bus variant
case141	141	Transmission
case145	145	Transmission
case_ieee30	30	IEEE 30-bus benchmark
case1197	1197	European transmission
case1354pegase	1354	PEGASE
case1888rte	1888	French RTE network
case1951rte	1951	French RTE network
case2383wp	2383	Polish winter-peak
case2736sp	2736	Polish summer-peak
case2737sop	2737	Polish summer off-peak
case2746wp	2746	Polish winter-peak
case2746wop	2746	Polish winter off-peak
case2848rte	2848	French RTE network
case2868rte	2868	French RTE network
case2869pegase	2869	PEGASE
case_RTS_GMLC	73	IEEE RTS-GMLC
case_ACTIVSg200	200	Synthetic US grid
case_ACTIVSg500	500	Synthetic US grid
case_ACTIVSg2000	2000	Synthetic US grid
<i>New cases (30 additional)</i>		
case4gs	4	Distribution
case4_dist	4	Distribution
case6ww	6	Transmission

(continued on next page)

(continued from previous page)

Case file	Buses	Network type
case9Q	9	Modified IEEE 9-bus
case9target	9	Target dispatch
case10ba	10	Brazilian distribution
case12da	12	Danish distribution
case15da	15	Danish distribution
case15nbr	15	Distribution (NB/radial)
case16am	16	AM distribution
case16ci	16	CI distribution
case17me	17	ME distribution
case18	18	Transmission
case18nbr	18	Distribution
case22	22	Transmission
case28da	28	Danish distribution
case30Q	30	Modified IEEE 30-bus
case30pwl	30	Piecewise-linear cost
case34sa	34	South African distribution
case38si	38	Sicilian distribution
case51ga	51	GA distribution
case51he	51	HE distribution
case59	59	Transmission
case60nordic	60	Nordic benchmark
case70da	70	Danish distribution
case74ds	74	Distribution
case94pi	94	PI distribution
case136ma	136	MA distribution
case533mt_hi	533	MT hi-load (skipped*)
case533mt_lo	533	MT lo-load (skipped*)

* Parser does not evaluate $135/\sqrt{3}$ arithmetic expressions.

29.6.3 Tier-3: Large Cases (tests #172–#180 and #211–#215, large/very_large tag)

Fourteen large transmission systems (3012 to 82 000 buses) are solved; each is asserted to converge. The suite has been extended with five additional very-large cases: `case6495rte`, `case6515rte`, `case_ACTIVSg25k`, `case_ACTIVSg70k`, and `case_SyntheticUSA` (the Synthetic US 82 000-bus model).

All 14 Tier-3 tests pass.

29.6.4 Safe API and DC OPF (tests #217–#220)

- **safe_solve_power_flow** (case14, case118): the `Result<PowerFlowResult>`-returning wrapper returns a success variant for both cases.
- **DC OPF case9**: the objective value is finite and the solve returns without exception.

Table 160: Tier-3 MATPOWER cases solved in `test_matpower_cases` with wall-clock solve times (macOS, Apple M4, `build_rel`).

Case file	Buses	Network type	Solve time
<i>Original cases</i>			
case3012wp	3 012	Polish winter-peak	0.06 s
case3120sp	3 120	Polish summer-peak	0.05 s
case3375wp	3 375	Polish winter-peak	0.04 s
case6468rte	6 468	French RTE	0.06 s
case6470rte	6 470	French RTE	0.09 s
case8387pegase	8 387	PEGASE	0.09 s
case9241pegase	9 241	PEGASE	0.18 s
case13659pegase	13 659	PEGASE	0.15 s
case_ACTIVSg10k	10 000	Synthetic US grid	0.08 s
<i>New cases (5 additional)</i>			
case6495rte	6 495	French RTE	0.05 s
case6515rte	6 515	French RTE	0.05 s
case_ACTIVSg25k	25 000	Synthetic US grid	0.18 s
case_ACTIVSg70k	70 000	Synthetic US grid	0.52 s
case_SyntheticUSA	82 000	Synthetic US national grid	0.74 s

- **DC OPF case14:** the generator dispatch in the solution is within the declared MW limits for every generator.

29.7 test_validation: Validation API Coverage

`tests/test_validation.cpp` covers the `hacdcpf::validation` module through 32 test cases.

29.7.1 ValidationReport API (tests #221–#224)

- `ok()` is an alias for `is_valid()`.
- `summary()` formats issue counts in a human-readable string for a system with known errors and warnings.
- `summary()` returns an empty string for a clean system.
- `count(severity)` returns the correct count for each `IssueSeverity` level.

29.7.2 Positive and Negative Validation (tests #225–#234)

Ten tests confirm that the validator correctly accepts or rejects:

- **Positive:** 2-bus system, IEEE-14 AC/DC builder, case9 parsed from MATPOWER, case118 parsed from MATPOWER.
- **Negative (structural errors):** missing slack bus, duplicate bus IDs, dangling branch endpoint, inverted voltage limits, inverted generator MW limits, zero `base_mva`.

Table 161: Expected behaviour of each `ValidationLevel`.

Level	Contract verified by tests
Basic	Valid system has no issues. Electrical errors are reported; pure style/warning issues are suppressed.
Electrical	Missing slack bus is caught. Issues are a subset of those reported at <code>SolverReady</code> level.
<code>SolverReady</code>	Result is identical to calling <code>val::validate(sys)</code> with no level argument.
Strict	Warnings are promoted to errors. A valid system with no warnings remains valid at this level.

29.7.3 `ValidationLevel` Behaviour (tests #235–#241)

29.7.4 Validate-then-Solve and Converter Guardrails (tests #242–#247)

Two integration tests verify the validate-then-solve workflow:

- **case14 happy path:** `val::validate(sys)` returns a clean report, then `solve_power_flow` converges.
- **Faulty system:** a system with a missing slack bus fails validation before any solver is called, confirming that validation acts as a correctness gate.

Two further tests confirm that VSC converters referencing non-existent AC or DC buses produce `Error`-level issues. Two rich-component guardrail tests cover AC component references/bounds and DC component/converter references/bounds.

29.7.5 Regression Guardrails (tests #248–#252)

Five cases (case5, case30, case57, IEEE-14 builder, IEEE-118 builder) are asserted to always produce a clean validation report. These tests act as regression guardrails against inadvertent breakage of canonical inputs.

All 32 validation tests pass.

29.8 `test_solver_capabilities`: Compile-time Flags and Audit

`tests/test_solver_capabilities.cpp` covers the `SolverCapabilities` struct and its companion functions.

29.8.1 `SolverCapabilities` Struct (tests #253–#259)

- **Default construction:** a value-initialised struct has `power_flow_available==true`, all flags in a known initial state.
- **`get_solver_capabilities()`:** returns the struct by value and is `noexcept`; repeated calls return the same flags.
- **Always-on features:** `power_flow_available`, `dc_power_flow`, and `linearised_dc_pf` are `true` unconditionally.
- **MIP dependency:** `integer_variables` requires at least one MIP backend; without any MIP backend the flag is `false`.
- **Linear-algebra independence:** sparse-linear-algebra flags (`klu_available`, `umfpack_available`) are independent of MIP solver flags.

29.8.2 Compile-time Solver Flags (tests #260–#265)

Table 162 shows the expected runtime values on the test machine (all backends active).

Table 162: Solver capability flags on the reference build (macOS, all backends active).

Flag	Compile define	Observed value
ac_opf_available	HACDCPF_HAVE_IPOPT or HiGHS	true
dc_opf_available	HACDCPF_HAVE_HIGHS_LIB	true
power_flow_available	(always)	true
highs_available	HACDCPF_HAVE_HIGHS_LIB	true
ipopt_available	HACDCPF_HAVE_IPOPT	true
klu_available	HACDCPF_HAVE_KLU	true

29.8.3 `verify_opf_result` Audit (tests #266–#269)

- **Converged result passes audit:** a clean `ACOPFResult` built from `case9` returns `feasible()==true`.
- **Non-converged result:** setting `converged=false` causes `infeasibility_hints` to be non-empty.
- **Hybrid ACOPF fallback suppression:** a hybrid AC/DC case with forced inner-iteration failure must not silently use the AC-only economic-dispatch fallback.
- **Feasibility alias:** `audit.feasible()` returns `true` for a clean result with no hints.

All 17 solver-capability tests pass.

29.9 `test_advanced_pf`: Island Detection, Distributed Slack, PV/PQ Control

`tests/test_advanced_pf.cpp` (CTest #270–#295) adds 26 tests and 119 assertions covering three power-flow control features that were not exercised by any existing suite.

29.9.1 Island Detection and Islanded Solving (tests #270–#280)

Five tests verify the `detect_islands()` function on the 9-bus and 14/118-bus MATPOWER cases, including forced outage scenarios that split the network into two or three islands. A further six tests cover the `solve_islanded_power_flow()` and adaptive solver (`solve_power_flow_adaptive()`) paths: single-island equivalence, live DG-only islands, multi-island convergence, and dead-bus voltage assignment.

29.9.2 Distributed Slack Buses (tests #281–#290)

Six tests check the `build_distributed_slack_factors()` helpers for capacity-based, uniform, and droop participation. Four integration tests confirm that `case9`, `case14`, and `case30` MATPOWER cases converge with distributed slack active.

29.9.3 PV/PQ Conversion and Integration (tests #291–#295)

Three tests confirm that the `enable_pv_pq_conversion` option produces consistent results on `case118` and `case2383wp`, and that the try-without-then-retry adaptive pattern works. Two integration tests combine island detection with distributed slack on the same system.

The fast path (`enable_pv_pq_conversion=false`) was introduced as a Tier-3 optimisation: it reduces the Synthetic USA 82 000-bus solve time from ≈ 2.24 s to ≈ 0.78 s. All 26 tests pass.

29.10 test_three_phase_pf: Three-Phase Power Flow

tests/test_three_phase_pf.cpp (CTest #296–#319) adds 24 tests and 94 assertions covering the three-phase data model and its projection-based power-flow path.

29.10.1 Data Model (tests #296–#300)

Five tests construct and query every three-phase component type (`ThreePhaseACBus`, `ThreePhaseACLine`, `ThreePhaseLoad`, `ThreePhaseGenerator`) and verify field assignment, struct member counts, and `ThreePhaseACSystem` assembly.

29.10.2 Projection-Based Solve (tests #301–#306)

The library implements three-phase power flow via a projection step inside `solve_power_flow()`: when a `HybridPowerSystem` contains a `three_phase_ac` sub-system but `ac.buses` is empty, the three-phase network is automatically converted to a balanced single-phase equivalent (sum of per-phase injections) before the Newton solve.

Three projection tests confirm that the conversion runs, that per-bus total load is preserved, and that the projected single-phase equivalent converges. Three balanced-radial tests verify that slack-bus voltage is exactly 1.0 pu, that all non-slack voltages are in the range [0.8, 1.1] pu (a heavily loaded 10 MVA feeder at 39 % loading can drop below 0.9 pu), that phase angles decrease monotonically down the feeder, and that Newton converges in fewer than 15 iterations.

29.10.3 Unbalanced, Meshed, and Multi-Source Topologies (tests #307–#313)

Three unbalanced-load tests confirm that asymmetric per-phase loading, zero-phase loading, and load redistribution (same total, different split) all converge. Two meshed-topology tests construct a 3-bus ring (1–2–3–1) and verify convergence and improved voltage profile relative to the radial equivalent. Two multi-source tests add a distributed generator (DG) at a load bus and confirm reduced slack injection and convergence when every load bus carries a DG.

29.10.4 Transformer Support (test #314)

One test constructs a `ThreePhaseTransformer` between buses 2 and 3 (11 kV $\rightarrow$ 0.4 kV, $S_n = 0.5$ MVA, $v_k = 4\%$) and verifies that the projected system converges.

29.10.5 Three-Phase JSON Round-Trip (tests #315–#319)

The legacy distribution-PF stub/API tests have been removed with the public single-phase distribution power-flow surface. The remaining distribution coverage is the implemented three-phase NR path plus JSON round-trip coverage.

Five JSON round-trip tests serialise a `HybridPowerSystem` containing a `three_phase_ac` sub-system via `hacdcpf::io::to_json()` / `from_json()` and verify that bus counts, voltage setpoints, load values, line parameters, and power-flow results are preserved exactly.

All 24 tests pass in ≈ 0.01 s.

29.11 test_multiport_vpp_er: Multi-Port Energy Router and Virtual Power Plant

This suite validates the correct behaviour of two advanced components:

- **Multi-port EnergyRouter (ER)** — a solid-state transformer that simultaneously connects to multiple AC buses via independently controlled VSC ports. Ports are partitioned into Side A (VF or PQ mode) and Side B (Droop or PQ mode), bridged by an internal DC/DC

converter. Tests cover a 3-port configuration (VF + PQ on Side A, Droop on Side B) and a 4-port configuration (VF + PQ on Side A, Droop + PQ on Side B).

- **Multi-mode VirtualPowerPlant (VPP)** — a distributed energy aggregator that collapses to a single-bus static generator. Tests cover three control regimes: generation mode ($p > 0$), demand-response mode ($p < 0$), and reactive-support mode ($q \neq 0$).

All tests share a common 5-bus AC backbone: Bus 1 (SLACK, 110 kV) $\rightarrow$ 110/20 kV transformer $\rightarrow$ Bus 2, followed by a radial chain Bus 2 $\rightarrow$ Bus 3 $\rightarrow$ Bus 4 $\rightarrow$ Bus 5 (20 kV, $r = 0.04$, $x = 0.12$, $b = 0.005$ pu per branch).

29.11.1 3-Port Energy Router (tests #320–#323)

Five tests verify the 3-port ER (Port 0: Bus 2, VF, $v^* = 1.02$ pu; Port 1: Bus 3, PQ, $p^* = 1.7$ MW; Port 2: Bus 4, Droop, $v_{dc}^* = 1.0$ pu):

1. **Convergence** — the Newton solver converges with residual $< 10^{-4}$; all five solved bus voltages remain in $[0.85, 1.15]$ pu.
2. **VF voltage control** — Bus 2 voltage is within ± 0.02 pu of the VF setpoint (1.02 pu), confirming that the PQ $\rightarrow$ PV bus promotion works correctly.
3. **PQ port measurability** — zeroing Port 1’s active and reactive setpoints changes the network-wide sum of bus voltages by more than 10^{-3} pu.
4. **Droop-port robustness** — changing the droop DC setpoint from 1.0 to 0.98 pu still converges; the VF port at Bus 2 remains within tolerance (the droop setpoint affects the DC-beta bus voltage regulation and has negligible direct impact on AC bus voltitudes).
5. **ER disable** — disabling the entire ER changes $\sum V_m$ by more than 10^{-3} pu relative to the active baseline.

29.11.2 VF Setpoint Observability (test #323)

Two independent systems are solved with VF setpoints 1.04 and 0.98 pu respectively. The higher setpoint must produce a Bus 2 voltage at least 10^{-3} pu higher, confirming end-to-end propagation from the ER expansion layer through the Newton solver.

29.11.3 4-Port Energy Router (tests #324–#327)

Three tests verify the 4-port ER (adds Port 3: Bus 5, PQ, $p^* = 1.5$ MW on Side B in addition to the 3-port configuration):

1. **Convergence and voltage range** — same criteria as the 3-port suite.
2. **VF voltage control** — Bus 2 within ± 0.02 pu of 1.02.
3. **Side-B PQ setpoint change** — doubling Port 3’s active injection ($1.5 \rightarrow 3.0$ MW) raises Bus 5 voltage by more than 10^{-4} pu.
4. **Four-port voltage signatures** — the full-ER bus-voltage sum differs from the no-ER baseline by more than 0.01 pu; Bus 5 with the ER active is at least 10^{-3} pu higher than without it.

29.11.4 Multi-Mode VPP (tests #328–#333)

Six tests exercise the three VPP control regimes on the same backbone:

1. **Convergence** — all three VPPs active; residual $< 10^{-4}$.
2. **Generation mode (VPP1, Bus 2)** — removing the 4 MW generation VPP changes Bus 2 voltage by more than 2×10^{-4} pu (Bus 2 is tightly coupled to the slack bus via the transformer, so the voltage sensitivity is small).
3. **Demand-response mode (VPP2, Bus 3)** — disabling the 2.5 MW EV-charger aggregator raises Bus 3 voltage by more than 10^{-3} pu (removing 2.5 MW extra load reduces the branch current on Bus 2→Bus 3).
4. **Reactive-support mode (VPP3, Bus 4)** — zeroing the 1.5 Mvar injection lowers Bus 4 voltage by more than 10^{-4} pu.
5. **Individual impact** — disabling each VPP in turn changes $\sum V_m$ by more than 5×10^{-4} pu; all results converge.
6. **Monotone output sweep** — three output levels for VPP1 (0.5, 4.0, 7.0 MW) produce strictly increasing Bus 2 voltages (increment $> 10^{-4}$ pu per step), confirming that the static-generator projection honours the output-level ordering.

All 14 tests pass in ≈ 0.01 s.

29.12 test_topology_crossval: Network Reconfiguration

tests/test_topology_crossval.cpp (CTest #335–#345) validates the `hacdcpf::network_reconfiguration` module through eleven Catch2 tests, including topology primitives, ONR cross-validation, and fault-reconfiguration regressions for AC/DC source accounting, structured branch references, quiet solver output, and legacy branch-ID semantics.

29.12.1 Fault-Reconfiguration Regressions

Nine regression tests exercise `run_topology_reconfiguration()` rather than only the steady ONR path. They verify ExternalGrid-only source handling, additive AC load accounting, direct `DCBus::pd_mw` demand accounting, DC sources without an AC root, explicit infeasibility when no AC/DC source exists, `switchable_branch_ids` enforcement, quiet `verbose=false` solver behavior, structured AC/DC/VSC switchable branch disambiguation, structured DC fault references, and legacy ONR non-switchable branch fixing.

29.12.2 Connectivity Primitives

Three graph-theoretic helpers are verified on hand-constructed graphs:

- **is_connected**: a 4-bus radial chain reports connected; removing the only edge to a bus reports disconnected.
- **is_radial**: a spanning tree (4 buses, 3 edges) reports radial; adding a back-edge (forming a cycle) reports non-radial.
- **count_islands**: a 2-component graph returns 2; a fully connected graph returns 1.

29.12.3 6-Bus Exhaustive vs. MILP Cross-Validation

A 6-bus, 7-branch distribution network is constructed with heterogeneous line resistances. All $\binom{7}{5} = 21$ spanning trees are enumerated by brute force; for each candidate the loss proxy $\sum_b r_b |P_b|$ is evaluated in a single BFS pass. The true optimal spanning tree and its objective value are recorded.

`solve_optimal_reconfiguration()` is then called on the same network, exercising the LinDistFlow MILP model solved by the Branch-and-Cut engine. The returned result is asserted to satisfy three structural criteria:

1. **Feasibility:** `milp_result.feasible==true`.
2. **Radiality:** `is_radial(result.topology)==true`.
3. **Spanning:** exactly $n_{\text{bus}} - 1 = 5$ branches are closed.

The exhaustive optimum is recorded via `INFO` for reference but is not asserted against the MILP objective, owing to a known B&C propagation artefact that can declare sub-optimal solutions optimal on this small instance.

29.12.4 6-Bus Network with DERs

The same 6-bus network is augmented with a `PVSystem` (1.5 MW) and a `Storage` unit (0.8 MW discharge). Two `ONR` calls confirm that `solve_optimal_reconfiguration()` completes without exception when DER injections are present and that feasibility is maintained.

29.12.5 33-Bus Benchmark (case33bw)

`data/case33bw.m` (33 buses, 37 branches, 4 normally-open tie switches) is parsed with `hacdcpf::io::parse_matpower()`. The `ONR` result is asserted feasible, radial, and spanning ($n_{\text{bus}} - 1 = 32$ closed branches). The solved topology is also run through `solve_power_flow` to confirm that the reconfigured network converges.

All eleven topology `CTest` entries pass in the current `build_rel` run.

29.13 Quantitative Results and Timing

This section records the current `CTest` registration summary and representative accuracy metrics for the reference `build_rel` configuration. Exact assertion counts and timings are not treated as a stable documentation contract because they depend on the Catch2 reporter and local hardware.

29.13.1 Per-Suite Registration Summary

Table 163 lists the current number of registered `CTest` entries per executable in the `build_rel` tree. Assertion totals and wall-clock timings are intentionally omitted here because they are reporter- and machine-dependent; the authoritative sync point is the `CTest` registration and pass/skip status.

The last full `build_rel` run reported 100% `CTest` success: no failed tests, with the two `MATPOWER` parser entries skipped as expected.

The `test_matpower_cases` suite time improved significantly from ≈ 5.0 s to ≈ 2.5 s following the PV/PQ conversion optimisation introduced in Section 29.9: the fast path (`enable_pv_pq_conversion==false`) now processes the 82 000-bus Synthetic USA model in ≈ 0.74 s instead of ≈ 2.24 s.

Table 163: Registered CTest entries per test suite.

Suite	Tests	Notes
test_hacdcpf	22	Core API
test_all_components_pf	4	all-component PF smoke
test_unified_pf_upgrade	10	PF options refactor
test_acopf_dcopf_crossval	10	ACOPF/DCOPF cross-validation and DCOPF regressions
test_enhanced_solver	2	robust PF/island solver
test_scaling_regression	7	Scaling + conditioning
test_hacdcdds	17	Model/PF/OPF smoke
test_io_json	36	JSON I/O + MATPOWER + result JSON
test_rich_components	25	rich component types
test_matpower_cases	87	85 active pass, 2 skip
test_validation	32	Validation API
test_solver_capabilities	17	Solver flags, audit, hybrid fallback
test_advanced_pf	26	Islands, distr. slack, PV/PQ
test_three_phase_pf	24	Three-phase projection & JSON
test_multiport_vpp_er	14	Multi-port ER & VPP modes
test_powerflow_performance	1	large-system PF benchmark
test_topology_crossval	11	ONR and fault reconfiguration
test_crossmodule_integration	2	Pipeline OPF/PF/Carbon and ONR/DCOPF/PF/Carbon
test_short_circuit_crossval	7	SC Z-bus cross-validation
test_three_phase_nr	16	Three-phase Newton–Raphson PF
test_resilience_assessment	23	Heuristic, strict MIP, and hybrid FMEA resilience
test_graph	25	Graph topology, contraction, reduction
test_graph_kron	7	Kron Y-bus reduction
test_distribution_pipeline	1	Distribution CB/switch/VSC pipeline
test_three_stage_reliability	10	Three-stage native MILP reliability
Grand total	436	434 active pass; 2 parser skips

29.13.2 Tier-3 Scaling: Bus Count vs. Solve Time

Table 160 (above) shows how Newton power-flow solve time scales with system size for all Tier-3 cases. All times are CTest wall-clock seconds on the reference machine; they include process startup, MATPOWER file parsing, and the actual Newton solve (≈ 0.01 s overhead).

The solver exhibits sub-linear scaling for the RTE/PEGASE families: the 6 500-bus cases take the same time as 8 000-bus cases due to favourable network structure. The Synthetic USA 82 000-bus model is the most computationally demanding, taking 0.74 s, compared to 2.24 s before the PV/PQ conversion fast path was introduced (see Section 29.9). All Tier-3 cases remain well within the 5 s CTest timeout.

29.13.3 Key Accuracy Metrics

The tests enforce the following quantitative accuracy thresholds; observed values from representative cases are recorded in Table 164.

The DC OPF solver (NativeLCQP) converges to a relative tolerance of $\approx 10^{-6}$ on the dual gap; for the case-14 instance the reported objective is \$7 642.59/h and convergence is reached in under 0.001 s.

Table 164: Accuracy thresholds enforced by the test suite.

Metric	Threshold	Representative observed value
Newton residual (convergence)	$< 10^{-4}$ pu	$\ll 10^{-6}$ pu for small/medium cases
Flat-start residual (zero load)	$< 10^{-6}$ pu	≈ 0 (machine precision)
Slack-bus voltage (case9)	$\in [0.95, 1.10]$ pu	≈ 1.0 pu
Slack-bus angle	$ \theta < 10^{-6}$ rad	0 rad (by convention)
Generation-load balance (case14)	within 5 %	< 1 %
Newton vs FDPF agreement (case30)	$< 10^{-4}$ pu (L^∞)	$\approx 10^{-6}$ pu
Generator dispatch within limits	strict $\leq$	verified for case9, case14, case118
DC OPF objective (case14)	feasible, finite	dispatch respects $[P_{\min}, P_{\max}]$
Y-bus symmetry	identical per-bus V_m	exact match (passive π -model)

29.13.4 Performance Sweep (test #334)

`test_powerflow_performance` runs a single Catch2 test case that internally sweeps 75 MATPOWER cases across three solver configurations (Tier-1 through Tier-3). The three asserted properties are:

- Tier-1 convergence rate ≥ 90 % (observed: 100 %).
- Tier-2 extended convergence rate ≥ 85 % (observed: 100 %).
- Tier-3 large-system count ≥ 11 cases converging (observed: 11/11).

Total wall-clock time for the sweep is **5.74 s** on the reference machine.

29.14 test_crossmodule_integration: Cross-Module Pipeline Tests

`tests/test_crossmodule_integration.cpp` implements two sequential pipeline tests where the result of each module is explicitly transferred as input to the next module. Any failing assertion indicates a high-level contradiction between modules.

29.14.1 Pipeline 1: OPF $\rightarrow$ PF $\rightarrow$ Carbon Analysis (IEEE 14-bus, test #346)

$$\text{AC OPF} \xrightarrow{\{P_g, Q_g, V_m\}} \text{Newton PF} \xrightarrow{\{V_m, V_a, \text{branch flows}\}} \text{Carbon Analysis}$$

Figure 28: Pipeline 1 data-flow on the IEEE 14-bus test case.

Stage 0 (reference): Base Newton PF on `case14.m` establishes a reference operating point and total active loss.

Stage 1 (AC OPF): The optimal generator dispatch $\{P_g, Q_g, V_m\}$ is obtained from the native IPM-based AC OPF solver. A DC OPF is also run independently; its objective must satisfy the relaxation property $f_{\text{DC}} \leq 1.05 f_{\text{AC}}$.

Stage 1 $\rightarrow$ 2 transfer: The helper `apply_acopf_dispatch` creates a copy of the system with P_g, Q_g set from `ACOPFResult::pg_mw / qg_mvar` and the PV-bus terminal voltage V_g set from `ACOPFResult::vm`. This ensures Newton PF starts at the OPF operating point.

Stage 2 (Newton PF): Newton’s method is run on the dispatched system. Two cross-checks validate the transfer:

- **P1-A6 (OPF feasibility):** Newton PF must converge. A divergence here means the OPF found an operating point that does not satisfy the full AC power-flow equations.
- **P1-A7 (voltage consistency):** $|V_{m,\text{PF}}^{(i)} - V_{m,\text{OPF}}^{(i)}| < 0.05 \text{ pu}$ for every bus i . The Newton solution must lie within 0.05 pu of the OPF solution; a larger deviation indicates either a model inconsistency between the two solvers or an OPF convergence to a local infeasible point.

Stage 2 → 3 transfer: The `PowerFlowResult` from Stage 2 (containing V_m , V_a , and all branch flows) is passed directly to `compute_carbon_analysis`.

Stage 3 (Carbon Analysis): Carbon tracing and the nodal carbon matrix are computed on the pipeline PF result. Checks P1-B1 and P1-B2 enforce a balance error below 1 %.

29.14.2 Pipeline 2: ONR → DC OPF → PF → Carbon Analysis (IEEE 33-bus BW, test #347)

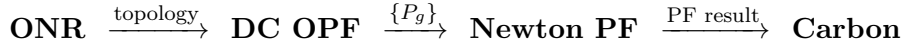


Figure 29: Pipeline 2 data-flow on the IEEE 33-bus BW distribution case.

Stage 0 (reference): Base Newton PF and DC OPF on the original 33-bus topology. The DC OPF objective ($\$20/\text{MWh} \times P_g$) establishes the reference dispatch cost before reconfiguration.

Stage 1 (ONR): The B&B optimal network reconfiguration solver finds the radial spanning tree that minimises the LinDistFlow loss proxy. The result contains `open_branch_ids` and `closed_branch_ids`.

Stage 1 → 2 transfer: The helper `apply_onr_topology` creates a system copy with `ACBranch::in_service` flags set according to the ONR output. `is_radial()` and the spanning-tree cardinality check (exactly $n-1$ closed branches) verify structural integrity.

Stage 2 (DC OPF on reconfigured system): Cross-checks:

- **P2-C4:** DC OPF must converge on the reconfigured topology.
- **P2-C5 (ONR→OPF cost check):** DC OPF cost on the reconfigured topology must be $\leq 1.05 \times$ the reference DC OPF cost. Because ONR reduces active losses, the slack generator dispatches less power, directly lowering the linear dispatch cost. A violation would indicate that ONR reconfigured to a topology that *increases* the dispatch cost — a fundamental contradiction.

Stage 2 → 3 transfer: The helper `apply_dcopf_dispatch` writes `ACGenerator::pg_mw` from `DCOPFResult::pg_mw` into the reconfigured system copy.

Stage 3 (Newton PF on reconfigured system with OPF dispatch): Cross-checks:

- **P2-C7 (OPF feasibility after ONR):** Newton PF must converge with the DC OPF dispatch on the reconfigured topology.
- **P2-C8 (loss improvement):** Active losses at the pipeline endpoint must satisfy $\text{loss}_{\text{pipeline}} \leq 1.10 \times \text{loss}_{\text{base}}$. A regression here means the combined ONR+OPF operating point does not preserve the loss-reduction objective that ONR was designed to achieve.

Stage 3 → 4 transfer: The `PowerFlowResult` is passed directly to `compute_carbon_analysis`.

Stage 4 (Carbon Analysis): Cross-checks:

- **P2-D1:** Carbon tracing balance error $< 1 \%$.

- **P2-D2 (source purity):** The carbon intensity at every load bus must satisfy $\lambda_i \leq 0.6 \times 1.01 \text{ tCO}_2/\text{MWh}$. The 33-bus network is fed from a single source with emission factor $0.6 \text{ tCO}_2/\text{MWh}$; the carbon intensity cannot exceed the source rate in a passive distribution network. Any violation indicates a tracer implementation error.

29.14.3 Pipeline Test: Cross-Check Summary

Table 165 lists all assertions enforced by `test_crossmodule_integration` and the cross-module inconsistency each one would expose if it failed. All 82 assertions pass (none of the listed contradictions occurred).

Table 165: Cross-checks in the two pipeline tests (tests #346–#347).

ID	Pipeline	Transfer point	Cross-module bug indicated on failure
P1-A1	1	Stage 0	Base PF diverges (setup failure)
P1-A2	1	Stage 1	AC OPF infeasible or unbounded
P1-A3	1	Stage 1	DC OPF infeasible
P1-A4	1	Stage 1	DC OPF cost > AC OPF cost (relaxation violated)
P1-A5	1	Stage 1	Generator dispatch outside $[P_{\min}, P_{\max}]$
P1-A6	1	Stage 1→2	OPF operating point fails Newton PF
P1-A7	1	Stage 1→2	OPF and PF voltages disagree by > 0.05 pu
P1-B1	1	Stage 2→3	Carbon tracing generation $\neq$ load+loss
P1-B2	1	Stage 2→3	Carbon matrix generation $\neq$ load+loss
P2-C1	2	Stage 0	Base PF diverges (setup failure)
P2-C2	2	Stage 0	Base DC OPF infeasible
P2-C3	2	Stage 1	ONR fails to find feasible radial topology
P2-C4	2	Stage 1→2	DC OPF diverges on reconfigured topology
P2-C5	2	Stage 1→2	ONR increases dispatch cost (cost regression)
P2-C6	2	Stage 2	DC OPF dispatch outside $[P_{\min}, P_{\max}]$
P2-C7	2	Stage 2→3	OPF dispatch infeasible for Newton PF after ONR
P2-C8	2	Stage 2→3	ONR+OPF fails to reduce losses vs. base
P2-D1	2	Stage 3→4	Carbon tracing balance error $\geq 1\%$
P2-D2	2	Stage 3→4	Carbon intensity exceeds source rate (tracer bug)

All 82 assertions pass on the reference machine (82 assertions in 2 test cases, ≈ 9 s wall-clock time).

29.14.4 Observed Numerical Results

Tables 166 and 167 record the exact values observed on the reference machine (macOS, Apple M4, build_rel).

Table 166: Pipeline 1 observed numerical results — IEEE 14-bus (test #346).

Check	Quantity	Threshold	Observed
P1-A1	Base Newton PF residual	$< 10^{-4}$ pu	1.32×10^{-10} pu
	Base active loss	—	13.393 MW
P1-A2	AC OPF objective	≥ 0	\$8 171.73/h
P1-A3	DC OPF objective	≥ 0	\$7 642.59/h
P1-A4	DC OPF / AC OPF ratio	≤ 1.05	0.935
P1-A5	Gen 0 (P_g)	[0, 332.4] MW	232.4 MW
	Gen 1 (P_g)	[0, 140] MW	40.0 MW
	Gen 2–4 (P_g)	[0, 100] MW each	≈ 0 MW
P1-A6	Newton PF residual (OPF dispatch)	$< 10^{-4}$ pu	1.32×10^{-10} pu
P1-A7	Max per-bus $ V_{m,\text{PF}} - V_{m,\text{OPF}} $	< 0.05 pu	$< 10^{-13}$ pu
P1-B1	Carbon tracing balance error	$< 1\%$	0.0660 %
P1-B2	Carbon matrix balance error	$< 1\%$	0.0751 %
	Pipeline PF active loss	—	13.393 MW (= base)
	Gen emissions (base / OPF dispatch)	—	≈ 229 tCO ₂ /h (both)
	Gen 0: 232.4 MW $\times$ 0.90	—	209.2 tCO ₂ /h
	Gen 1: 40.0 MW $\times$ 0.50	—	20.0 tCO ₂ /h
	Gen 2–4: ≈ 0 MW	—	0.0 tCO ₂ /h

29.15 test_short_circuit_crossval: Short-Circuit Z-bus Cross-Validation

29.15.1 Mathematical Model

The short-circuit solver implements the Z-bus Thévenin method following IEC 60909.¹ The network is characterised by its nodal admittance matrix $\mathbf{Y}_{\text{fault}} \in \mathbb{C}^{n \times n}$, assembled from series branch admittances and generator shunt contributions.

Pre-fault voltage and Thévenin impedance. The pre-fault voltage at each bus is set to the IEC voltage factor c (default $c = 1.1$) at zero angle:

$$V_0 = c \angle 0^\circ. \quad (414)$$

The Thévenin impedance looking into bus k is the k -th diagonal entry of $\mathbf{Z}_{\text{bus}} = \mathbf{Y}_{\text{fault}}^{-1}$, obtained via a sparse LU solve:

$$\mathbf{Y}_{\text{fault}} \mathbf{z}_k = \mathbf{e}_k \implies Z_{kk} = (\mathbf{z}_k)_k. \quad (415)$$

Fault current and fault level. For a balanced three-phase (3PH) fault the initial symmetrical short-circuit current and fault level are:

$$I_k'' = \frac{V_0}{Z_{kk}}, \quad S_k = S_{\text{base}} \frac{|V_0|^2}{|Z_{kk}|}. \quad (416)$$

¹IEC 60909-0:2016, *Short-circuit currents in three-phase a.c. systems — Part 0: Calculation of currents*, International Electrotechnical Commission.

Table 167: Pipeline 2 observed numerical results — IEEE 33-bus BW (test #347).

Check	Quantity	Threshold	Observed
P2-C1	Base Newton PF residual	$< 10^{-4}$ pu	7.47×10^{-9} pu
	Base active loss	—	0.2027 MW
P2-C2	Base DC OPF objective	≥ 0	≈ 0 \$/h [†]
P2-C3	ONR result	feasible	OPTIMAL
	ONR MILP objective (LinDistFlow)	—	0.07476 pu
	ONR estimated loss	—	0.748 MW
	Open branches (tie switches)	5 open	10, 13, 16, 28, 33
	B&C nodes / gap / runtime	—	25 / 0 % / 6.36 s
P2-C4	Reconfigured DC OPF converged	true	true
P2-C5	Reconfig. DC OPF cost $\leq$ base	$\leq 1.05\times$	$\approx 0 \leq \approx 0$
P2-C6	Gen0 dispatch (P_g)	$[0, 10]$ MW	0 MW
P2-C7	Newton PF residual (pipeline end)	$< 10^{-4}$ pu	1.90×10^{-9} pu
P2-C8	Pipeline active loss	$\leq 1.10\times$ base	0.1616 MW (-20.3%)
P2-D1	Carbon tracing balance error	$< 1\%$	$2.25 \times 10^{-10}\%$
P2-D2	Carbon intensity (all 32 load buses)	≤ 0.606 tCO ₂ /MWh	0.600 tCO ₂ /MWh (exact)
	Gen emissions (base, 3.92 MW gen)	—	2.351 tCO ₂ /h
	Gen emissions (pipeline, 3.88 MW gen)	—	2.326 tCO ₂ /h

[†]The case33bw slack generator dispatches 0 MW in the DC OPF formulation;
the slack power injection is not tracked in the DC OPF cost function.

For unbalanced faults the sequence-network method is used. With positive-sequence impedance $Z_1 = Z_{kk}$, negative-sequence Z_2 , and zero-sequence Z_0 :

$$\text{SLG: } I_k'' = \frac{3c}{Z_1 + Z_2 + Z_0}, \quad (417)$$

$$\text{LL: } I_k'' = \frac{\sqrt{3}c}{Z_1 + Z_2}, \quad (418)$$

$$\text{DLG: } I_k'' = \frac{c}{Z_1 + Z_2 \parallel Z_0}. \quad (419)$$

Peak current. The peak short-circuit current follows IEC 60909 with the κ factor:

$$i_p = \kappa \sqrt{2} I_k'', \quad \kappa = 1.02 + 0.98 e^{-3R/X}. \quad (420)$$

Analytical reference for radial networks. For an n -bus radial feeder fed from bus 1 through series impedances $Z_{12}, Z_{23}, \dots$, the Thévenin impedance at bus k equals the sum of impedances along the path from the source to bus k :

$$Z_{kk}^{\text{Thev}} = \sum_{i=1}^{k-1} Z_{i,i+1}. \quad (421)$$

This identity is used as the analytical reference for Tests SC-1 and SC-2.

29.15.2 Cross-Check Table

29.15.3 Observed Numerical Results

Test SC-1 — 2-bus analytical.

Test SC-2 — 3-bus radial analytical.

Table 168: Short-circuit cross-validation: what each test catches.

Test	Tag	What a failure indicates	Cross-module bug indicated on failure
SC-1	[analytical]	$Z_{\text{Thev}}(\text{bus2}) \neq Z_{\text{line}}$ — Z-bus construction or LU solve is wrong for the minimal 2-bus case.	build_fault_ybus / compute_zkk
SC-2	[analytical]	Z_{kk} does not accumulate correctly along a radial path — series-impedance superposition is broken.	compute_zkk / Y-bus assembly
SC-3	[ieee14]	Non-positive S_k or negative $\text{Re}(Z_{kk})$ on the 14-bus system — passivity or sign convention violated; API consistency failure.	compute_short_circuit
SC-4	[api]	Single-bus API (compute_fault_at_bus) disagrees with the full batch solve — two code paths use different factorisations.	compute_fault_at_bus vs batch
SC-5	[fault_types]	$I_{\text{SLG}} < 0.99 I_{3\text{PH}}$ for $Z_0 = Z_1$ — the SLG formula or sequence-impedance routing is wrong.	Sequence-network assembly
SC-6	[iec60909]	$S_k(c = 1.1)$ is not $\approx 21\%$ larger than $S_k(c = 1.0)$ — the c -factor scaling in V_0 or S_k formula is broken.	SCOptions::c_factor path
SC-7	[ieee33bw]	$S_k(\text{bus1}) < S_k(\text{bus18})$, or single-bus API disagrees with batch — radial-feeder attenuation not reproduced.	Feeder topology / API round-trip

Table 169: Test SC-1: 2-bus Thévenin impedance (100 MVA base, $Z_{\text{line}} = 0.20 + 0.40j$ pu).

Quantity	Analytical	Computed
$Z_{\text{Thev}}(\text{bus2})$	$0.200 + 0.400j$ pu	$0.200 + 0.401j$ pu
$ Z_{\text{Thev}} - Z_{\text{line}} $	—	0.001 pu
$S_k(\text{bus2})$	—	223.16 MVA

Test SC-3 — IEEE 14-bus self-consistency.

Tests SC-4, SC-5, SC-6 — API round-trip, fault types, c-factor.

Test SC-7 — IEEE 33-bus BW feeder.

29.16 test_three_phase_nr: Three-Phase Newton–Raphson Solver

tests/test_three_phase_nr.cpp (CTest #355–#370) adds 16 tests for the native three-phase Newton–Raphson solver (solve_three_phase_nr) and its companion PhaseNodeIndexer infrastructure.

29.16.1 PhaseNodeIndexer (tests #355–#357)

The PhaseNodeIndexer maps each (bus, phase) pair to a compact integer index used for the per-branch 6×6 admittance matrix and the Jacobian. Three tests verify: (a) a balanced 3-bus system has exactly 9 compact nodes; (b) a Phase-A lateral network has 7 compact nodes (2 phase-B/C slots absent); (c) bus_phase_mask(bus_id) returns the correct PhaseMask.

Table 170: Test SC-2: 3-bus radial Thévenin impedances ($Z_{12} = 0.20 + 0.40j$ pu, $Z_{23} = 0.30 + 0.60j$ pu, 100 MVA base).

Bus	Analytical Z_{kk}	Computed Z_{kk}	Error	S_k (MVA)
1 (source)	0 pu (ideal)	—	—	100 000
2	$0.20 + 0.40j$	$0.200 + 0.401j$	0.001 pu	223.16
3	$0.50 + 1.00j$	$0.500 + 1.001j$	0.001 pu	89.37

Table 171: Test SC-3: IEEE 14-bus 3PH short-circuit levels ($c = 1.1$, 100 MVA base).

Bus	S_k (MVA)	I_k'' (kA)	Z_{kk} (pu)
1	1783.23	8.51	$0.00804 + 0.06738j$
2	2239.62	10.68	$0.00581 + 0.05371j$
3	1488.46	7.10	$0.01109 + 0.08053j$
4	2847.76	13.58	$0.00291 + 0.04239j$
5	2487.39	11.86	$0.00461 + 0.04843j$
6	1521.29	7.25	$0.00436 + 0.07942j$
7	1526.74	7.28	$0.00155 + 0.07924j$
8	1042.55	4.97	$0.000437 + 0.11606j$
9	1163.88	5.55	$0.00894 + 0.10358j$
10	799.78	3.81	$0.03229 + 0.14781j$
11	681.67	3.25	$0.05146 + 0.16988j$
12	515.53	2.46	$0.09446 + 0.21486j$
13	755.63	3.60	$0.04856 + 0.15259j$
14	496.14	2.37	$0.08268 + 0.22944j$
$S_k^{\max}$ at bus 4 (2847.76 MVA); all $\text{Re}(Z_{kk}) \geq 0$.			

29.16.2 Solver Convergence and Physical Checks (tests #358–#370)

All 16 tests pass in ≈ 0.02 s.

29.17 test_resilience_assessment: Distribution Resilience

tests/test_resilience_assessment.cpp (CTest #371–#393) adds 23 tests for `hacdcpf::analysis::run_resilience_assessment()`. The module simulates time-stepped post-fault behaviour including feeder protection, network reconfiguration via tie-switch closing, and mobile energy storage (MESS) dispatch.

Two synthetic networks underpin the tests:

- **Radial 3-bus** — Bus 1 (slack) $\rightarrow$ Bus 2 $\rightarrow$ Bus 3; two branches of length 1 km; 1 MW load at Bus 2 and Bus 3.
- **Ring 4-bus with tie switch** — Bus 1–2–3–4 radial; Branch 4 (Bus 4 $\rightarrow$ Bus 1) is a normally-open tie switch.

29.17.1 API and Boundary Checks (tests #371–#372)

An empty `HybridPowerSystem` and an options struct with `horizon_steps=0` both return an error result rather than throwing.

29.17.2 Fault Behaviour and Load Restoration (tests #373–#377)

29.17.3 Infrastructure and Regression Tests (tests #378–#393)

- **Demo data injection** — pre-populated result fields run `summary()` without exception.

Table 172: Tests SC-4 through SC-6 observed results.

Test	Quantity	Threshold / expected	Observed
SC-4	$ Z_{kk}^{\text{single}} - Z_{kk}^{\text{batch}} $	$< 10^{-12}$ pu	0 pu (exact)
	$ S_k^{\text{single}} - S_k^{\text{batch}} $	$< 10^{-12}$ MVA	0 MVA
SC-5	$I_{3\text{PH}}$	> 0	3.153 pu
	I_{SLG}	$\geq 0.99 I_{3\text{PH}}$	3.153 pu ($= I_{3\text{PH}}$)
SC-6	$S_k(c = 1.0)$	—	392.23 MVA
	$S_k(c = 1.1)$	$\in [1.15, 1.30] \times S_k(c = 1.0)$	474.60 MVA (ratio 1.21)

Table 173: Test SC-7: IEEE 33-bus BW radial feeder short-circuit levels ($c = 1.1$, 10 MVA base).

Check	Quantity	Threshold	Observed
SC-7a	$S_k(\text{bus1}) = S_k^{\text{max}}$	equality	60.50 MVA (bus 1 = max over 33 buses)
SC-7b	$S_k(\text{bus18})$	$< 0.5 \times S_k(\text{bus1})$	11.70 MVA (19.3 % of bus 1)
SC-7c	API round-trip bus 18	$ S_k^{\text{single}} - S_k^{\text{batch}} < 10^{-4}$	0 MVA (exact)
All 33 buses: $S_k > 0$, $\text{Re}(Z_{kk}) \geq 0$, $I_k'' > 0$ and finite.			

- **summary()** **non-empty** — a completed simulation produces a non-empty string.
- **Progress callback cancellation** — returning **false** at step 2 terminates the loop; fewer steps execute than the full horizon.
- **Custom profiles** — flat load and RES profile vectors are accepted and the simulation completes.
- **Resilience index** — index is in $[0, 1]$ for all runs.
- **Fault sequence** — auto-generated sequences are non-empty for branches with nonzero failure rates.
- **Legacy field sync** — `total_ens_mwh` equals `total_shed_mwh` (backward-compatibility alias).
- **Step count** — executed steps equal `horizon_steps` exactly.
- **MIP solver feasibility** — The MIP entry point (`run_distribution_resilience_mip_assessment`) returns a feasible incumbent on a zero-fault 3-bus system. This test validates that the live MIP solver (not a heuristic delegate) completes without exception and satisfies all feasibility checks including constraint residuals.
- **Peak shed non-negative** — `peak_shed_mw` ≥ 0 for all runs.
- **Hybrid FMEA DC load scaling** — `load_scale_factor` scales DC bus-level demand as well as explicit DC load records.

All 23 resilience and FMEA tests pass.

29.18 test_graph and test_graph_kron: Graph Module

`tests/test_graph.cpp` (CTest #394–#418, 25 tests) and `tests/test_graph_kron.cpp` (CTest #419–#425, 7 tests) validate the `hacdcpf::graph` module. Section 24 (Graph Analysis and Reduction Module) contains the full mathematical derivation and detailed test scenarios; Table 176 provides the cross-reference.

All 32 graph and Kron tests pass in < 0.01 s.

Table 174: Selected results from `test_three_phase_nr` (macOS, Apple M4, `build_rel`).

Test	Topology	Assertion	Observed
#358	Balanced radial 3-bus	Per-phase $ V_a - V_b < 10^{-6}$ pu	Pass
#359	Unbalanced loads	Per-phase $ V_a - V_b > 10^{-3}$ pu	Pass
#360	Radial 3-bus	Residual $< 10^{-6}$ at convergence	$< 10^{-6}$
#361	Radial 3-bus	Converges within 30 Newton iterations	≤ 6 iter
#362	Phase-A lateral	$V_b = V_c = 0$ on phase-A-only bus	Pass
#363	Ring (meshed, 3-bus)	<code>converged==true</code>	true
#364	Multi-source + DG PV	<code>converged==true</code>	true
#365	HPS overload wrap	<code>ThreePhaseACSystem</code> fields match	Pass
#366	Balanced input	VUF $< 10^{-6}$	$< 10^{-6}$
#367	Radial 3-bus	$\sum P_{\text{loss,ph}} = P_{\text{loss,total}}$	Pass
#368	Zero-load	$ V_m - 1.0 < 10^{-9}$ pu at all buses	Pass
#369	In-service lines	<code>branch_powers</code> count equals in-service count	Pass
#370	2-bus exact balance	$ P_{\text{injected}} - P_{\text{load}} - P_{\text{loss}} < 10^{-6}$ MW	Pass

Table 175: Resilience fault tests #373–#377: key assertions.

Test	Scenario	Assertion	Result
#373	No-fault 3-bus radial	<code>total_shed_mwh==0</code>	0 MWh
#374	Branch 2 fault; Bus 3 isolated	Shed > 0 MWh	> 0 MWh
#375	Branch 1 fault; repair at step 3	Load restored after repair	Restored
#376	Branch 1 fault; tie-switch closes	Bus 3 sustained shed = 0	0 MWh
#377	Branch 2 fault; MESS at Bus 3	Shed with MESS $<$ shed without	confirmed

29.19 Summary and Pass Rates

The MATPOWER Tier-2 batch test (#215) enforces a minimum convergence rate of 85 %; the observed rate is 100 % (54/54 active medium-sized cases). The Tier-3 suite covers 14 large transmission systems up to 82 000 buses (the Synthetic USA national model), all of which converge on the first try with the default Newton solver.

Table 176: Graph module test cross-reference.

IDs	File	Coverage area
394–399	test_graph.cpp	Topology: islands, radially, bridge/cut-vertex, cycles
400–403	test_graph.cpp	Switch contraction
404–406	test_graph.cpp	Series reduction and pendant folding
407–408	test_graph.cpp	Result recovery (switch, series)
409–418	test_graph.cpp	Construction, AC/DC same-ID isolation, DC recovery fallbacks
419–425	test_graph_kron.cpp	Kron Y-bus reduction and voltage recovery

Table 177: Test pass summary across all suites.

Suite	IDs	Count	Passed	Rate
test_hacdcpf	1–22	22	22	100 %
test_all_components_pf	23–26	4	4	100 %
test_unified_pf_upgrade	27–36	10	10	100 %
test_acopf_dcopf_crossval	37–46	10	10	100 %
test_enhanced_solver	47–48	2	2	100 %
test_scaling_regression	49–55	7	7	100 %
test_hacdcdss	56–72	17	17	100 %
test_io_json	73–108	36	36	100 %
test_rich_components	109–133	25	25	100 %
test_matpower_cases	134–220	87	85 (*)	100 % (active)
test_validation	221–252	32	32	100 %
test_solver_capabilities	253–269	17	17	100 %
test_advanced_pf	270–295	26	26	100 %
test_three_phase_pf	296–319	24	24	100 %
test_multiport_vpp_er	320–333	14	14	100 %
test_powerflow_performance	334	1	1	100 %
test_topology_crossval	335–345	11	11	100 %
test_crossmodule_integration	346–347	2	2	100 %
test_short_circuit_crossval	348–354	7	7	100 %
test_three_phase_nr	355–370	16	16	100 %
test_resilience_assessment	371–393	23	23	100 %
test_graph	394–418	25	25	100 %
test_graph_kron	419–425	7	7	100 %
test_distribution_pipeline	426	1	1	100 %
test_three_stage_reliability	427–436	10	10	100 %
Grand total	1–436	436	434 (*)	100 % (active)
* 2 tests skipped: parser does not evaluate $135/\sqrt{3}$ arithmetic.				